Quantization of Large Language Models

Weights, Activations, KV Cache, and Training-Time Low-Precision Formats

A monograph-style survey covering rate–distortion foundations, second-order and activation-aware PTQ, rotations, vector quantization, quantization-aware training, micro-scaling floating-point formats, kernels, and deployment practice

Paper 1 of the LLM compression series

Literature cutoff: October 2026

Claude (Anthropic)
29 October 2026

All scores, forecasts and rankings in this document are the author's subjective judgments informed by cited evidence. Numbers labelled "own simulation" were produced for this document on synthetic data; numbers labelled "approximate" are typical figures from memory of the cited literature and were not re-measured.

Abstract

Quantization is the dominant compression technique for deployed large language models (LLMs): it shrinks weights, activations, optimizer state and the key–value (KV) cache from 16 or 32 bits to 8, 4, or fewer, converting a memory-bandwidth-bound decoding workload into one that moves a fraction of the bytes. This monograph surveys the field from its information-theoretic foundations to the formats and kernels that ship in production in 2026. We derive the rate–distortion limits and the high-resolution distortion of uniform quantizers; develop the second-order (Optimal Brain Surgeon to GPTQ) view of post-training quantization with full proofs of the key update rules; explain outlier phenomena, activation-aware scaling (AWQ), smoothing (SmoothQuant) and rotation/incoherence processing (QuIP, QuaRot, SpinQuant); treat vector, trellis and lattice quantization (AQLM, QuIP#, QTIP); analyse mixed-precision bit allocation; review quantization-aware training, ternary and binary networks (BitNet b1.58) and their scaling laws; cover floating-point and micro-scaling formats (FP8, MXFP4/6/8, NVFP4), the hardware that supports them, and 8-bit and 4-bit *training*; and analyse KV-cache quantization, MoE and reasoning-model quantization, kernels (Marlin-style mixed-precision GEMM), and evaluation pitfalls. The document includes a year-by-year timeline to October 2026, capability tables with sourced numbers, a risk register, a deployment decision guide, and an explicit, rubric-based and openly subjective priority assessment of every sub-topic. We also include original small-scale simulations (clearly labelled) that illustrate the effects discussed. Our central conclusions are that (i) 8-bit is a solved problem and 4-bit weight-only is a near-solved one for large dense models, while 4-bit *activation* quantization depends on rotations and block-scaled formats; (ii) the frontier has moved to 2–3 bit weights with vector/trellis codes and QAT, 4-bit training, and micro-scaled hardware formats; (iii) average perplexity is an unreliable guide to reasoning, long-context and safety behaviour; and (iv) recipes are increasingly hardware-format-specific, which raises lock-in and reproducibility risk.

Contents

Abstract ii

1 Executive Summary of Findings 1

1.1 The ten findings . . . 1
1.2 State of the art at a glance . . . 3
1.3 Reading guide . . . 3

2 Introduction and Motivation 5

2.1 Why quantize at all . . . 5
2.2 Decoding is memory-bandwidth bound . . . 5
2.2.1 A worked example: decoding a 70B model . . . 7
2.2.2 Where the batch-size crossover happens . . . 7
2.2.3 The KV cache: the second memory stream . . . 8
2.2.4 Energy . . . 8
2.3 Scope and organizing principles . . . 9
2.4 A short history . . . 9
2.5 Notation conventions . . . 9

3 Mathematical Foundations 11

3.1 Rate–distortion limits . . . 11
3.1.1 The rate–distortion function . . . 11
3.1.2 Reverse water-filling and transform coding . . . 12
3.1.3 Gap of practical quantizers . . . 12
3.2 Uniform affine quantization . . . 13
3.2.1 Definition . . . 13
3.2.2 Error decomposition and the 6.02 dB rule . . . 14
3.2.3 Clipping versus rounding . . . 14
3.3 Granularity of scales . . . 14
3.3.1 Pseudocode for group quantization . . . 15
3.4 Non-uniform scalar quantizers . . . 16
3.4.1 Lloyd–Max . . . 16
3.4.2 Non-uniform grids in practice . . . 17
3.5 Rounding rules . . . 17

3.6 The layer-wise objective and the Hessian proxy . . . 17
3.7 Error propagation through depth . . . 18

4 A Taxonomy of LLM Quantization 20
4.1 Axis 1: the target . . . 20
4.2 Axis 2: the representation . . . 21
4.3 Axis 3: the fitting procedure . . . 21
4.4 Axis 4: data requirements . . . 22
4.5 Axis 5: the deployment regime . . . 22
4.6 A map of the methods . . . 23
4.6.1 Shorthand for bit-width settings . . . 23

5 Second-Order Post-Training Quantization: From OBS to GPTQ and Successors 25
5.1 Round-to-nearest as the baseline . . . 25
5.2 Optimal Brain Surgeon . . . 25
5.3 GPTQ . . . 26
5.3.1 Three simplifications . . . 26
5.3.2 The algorithm . . . 28
5.3.3 Why the gain is so large: a controlled simulation . . . 28
5.4 Successors and variants . . . 29
5.4.1 Asymmetric calibration: GPTAQ . . . 29
5.4.2 Error-correcting variants . . . 29
5.4.3 Preconditioning the weights . . . 29
5.4.4 Gradient-based rounding search: SignRound and AutoRound . . . 30
5.4.5 Format-aware GPTQ: MR-GPTQ . . . 30
5.4.6 Targeted weaknesses . . . 30
5.4.7 Pruning and quantization share the machinery . . . 30
5.5 Sensitivity to calibration data . . . 30
5.6 Reported results (approximate) . . . 31

6 Outliers, Smoothing, and Activation-Aware Quantization 32
6.1 The outlier phenomenon . . . 32
6.2 Integer matrix multiplication and why scales must factor . . . 33
6.3 LLM.int8(): vector-wise quantization and mixed-precision decomposition . . . 33
6.4 SmoothQuant: equivalent per-channel rescaling . . . 34
6.5 Other activation-quantization designs . . . 35
6.6 AWQ: activation-aware weight quantization . . . 36

6.7 Activation quantization experiments: a controlled simulation . . . 37
6.8 Practical guidance from this chapter . . . 38

7 Rotations and Incoherence Processing **39**
7.1 Why rotations help . . . 39
7.1.1 Invariance . . . 39
7.1.2 Concentration of rotated coordinates . . . 39
7.1.3 Fast transforms . . . 40
7.2 QuIP: incoherence and LDLQ . . . 40
7.3 QuIP#: randomized Hadamard transform and lattice codebooks . . . 42
7.4 QuaRot: rotating the whole network . . . 42
7.5 SpinQuant: learned rotations . . . 43
7.6 Other transformation-based methods . . . 43
7.7 When do rotations hurt or help? A decision analysis . . . 44

8 Vector, Additive, Lattice, and Trellis Quantization **45**
8.1 Why vectors beat scalars . . . 45
8.1.1 Granular and shaping gain . . . 45
8.1.2 Codebook size and decode cost . . . 46
8.2 GPTVQ: k-means codebooks with Hessian-aware updates . . . 46
8.3 Additive and residual quantization . . . 47
8.3.1 Additive quantization (AQ) . . . 47
8.3.2 AQLM . . . 47
8.3.3 VPTQ and residual VQ . . . 48
8.4 Fine-tuning compressed weights: PV-Tuning . . . 48
8.5 Lattice codebooks: QuIP# and E_8 . . . 48
8.6 Trellis-coded quantization and QTIP . . . 49
8.6.1 Trellis-coded quantization . . . 49
8.6.2 The bitshift trellis . . . 49
8.7 Comparing VQ-family methods . . . 50
8.8 Practical considerations . . . 50

9 Mixed Precision and Bit Allocation **52**
9.1 Formulation . . . 52
9.2 The high-resolution optimum . . . 52
9.3 Sensitivity measures . . . 54
9.4 Granularity of mixed precision . . . 54

9.5 Any-precision and nested quantization . . . 55
9.6 Learned and joint allocation . . . 56
9.7 Worked example: allocating bits across a 32-layer model . . . 56
9.8 Pitfalls . . . 56

10 Quantization-Aware Training, Distillation, and Extreme Low-Bit Models 58
10.1 The mechanics of QAT . . . 58
10.1.1 Fake quantization and the straight-through estimator . . . 58
10.1.2 Learned step size (LSQ) . . . 58
10.1.3 What the STE does (and does not) guarantee . . . 59
10.2 QAT for LLMs: recipes . . . 59
10.2.1 LLM-QAT: data-free distillation . . . 59
10.2.2 EfficientQAT: block-wise then end-to-end . . . 60
10.2.3 Distillation-based QAT . . . 60
10.2.4 Parameter-efficient variants . . . 60
10.2.5 Low-data VQ-QAT . . . 60
10.3 Industrial QAT and native low-precision releases . . . 61
10.4 Scaling laws for quantized models . . . 61
10.5 Extreme low-bit: binary and ternary networks . . . 63
10.5.1 Pre-LLM roots . . . 63
10.5.2 BitNet and BitNet b1.58 . . . 63
10.5.3 Follow-ups . . . 64
10.6 When to use QAT . . . 65

11 Floating-Point and Micro-Scaling Formats, Hardware, and Low-Precision Training 67
11.1 Floating-point basics . . . 67
11.1.1 The formats . . . 68
11.2 Block-scaled (micro-scaling) formats . . . 68
11.2.1 OCP MX . . . 68
11.2.2 NVFP4 . . . 69
11.2.3 Why block-scaled formats are hardware-friendly . . . 69
11.3 Analytical and simulated comparison of 4-bit block formats . . . 70
11.4 PTQ for micro-scaled formats . . . 71
11.5 Hardware support . . . 72
11.6 Low-precision training . . . 73
11.6.1 Mixed-precision lineage . . . 73
11.6.2 FP8 training . . . 73

11.6.3 FP4 training . . . 74
11.7 Integer versus floating point: the live debate . . . 75

12 KV-Cache Quantization **77**
12.1 Why the KV cache is a distinct problem . . . 77
12.2 Statistics of keys and values . . . 77
12.2.1 Error in attention logits: per-token versus per-channel . . . 78
12.3 Method families . . . 79
12.3.1 Group quantization with a full-precision residual window: KIVI and SKVQ 79
12.3.2 Non-uniform, sensitivity-weighted, outlier-aware: KVQuant, QAQ, GEAR 79
12.3.3 Exploiting redundancy: AQUA-KV . . . 80
12.3.4 Rotation- and sketch-based methods . . . 80
12.4 Long-context and reasoning fragility . . . 81
12.5 Practical guidance . . . 82

13 Error Analysis and Evaluation Methodology **83**
13.1 Layer-level error bounds . . . 83
13.2 Composition of weight and activation noise . . . 84
13.3 Outliers: why energy-based metrics can mislead . . . 84
13.4 From error to perplexity . . . 85
13.5 Evidence of where average metrics mislead . . . 85
13.6 Evaluation methodology: a protocol . . . 85
13.6.1 Statistical power . . . 86
13.6.2 Pitfalls . . . 86
13.7 Predictive diagnostics . . . 87

14 Kernels, Systems, and Frameworks in Practice **89**
14.1 Anatomy of a mixed-precision GEMM . . . 89
14.1.1 A simple throughput model . . . 89
14.2 Marlin and its successors . . . 90
14.3 System-level view: which format for which serving regime . . . 92
14.3.1 End-to-end arithmetic including the KV cache and Amdahl effects . . . 92
14.3.2 Tensor parallelism, MoE, and quantized collectives . . . 93
14.4 Formats and frameworks . . . 93
14.4.1 GGUF and llama.cpp . . . 93
14.4.2 Other frameworks . . . 94

15 Quantization of MoE Models, Reasoning Models, and the Bits-versus-Parameters

Frontier **96**
15.1 MoE models: anatomy and systems regime . . . 96
15.2 What is different about quantizing MoE models . . . 97
15.2.1 Experts are robust, shared experts and attention are not . . . 97
15.2.2 Calibration imbalance . . . 97
15.2.3 Routing sensitivity . . . 98
15.2.4 Mixed-precision across experts . . . 98
15.3 Reasoning models . . . 99
15.3.1 Evidence . . . 99
15.3.2 A simple error-accumulation model . . . 100
15.3.3 Practical implications . . . 100
15.4 The bits-versus-parameters frontier . . . 101

16 Recent Advances: A Timeline from 2022 to October 2026 **103**
16.1 Four eras . . . 103
16.2 2022: the outlier problem and second-order rounding . . . 103
16.3 2023: weight-only 4-bit goes mainstream . . . 104
16.4 2024: rotations, codes, KV cache, ternary weights, systems . . . 105
16.5 2025: reasoning models, MX/NVFP4 formats, and low-precision training . . . 106
16.6 2026 to October: hardware-native 4-bit, QAD, and theory . . . 107
16.7 Summary table . . . 107
16.8 Cross-cutting trends . . . 107

17 Capabilities: What Quantization Delivers in Practice **109**
17.1 Memory arithmetic . . . 109
17.2 Weight-only post-training quantization . . . 109
17.3 Weight and activation quantization . . . 110
17.4 KV-cache quantization . . . 110
17.5 Training-time low precision . . . 110
17.6 Fine-tuning and the long tail of use cases . . . 110
17.7 Capability matrix . . . 111
17.8 Accuracy retention: what can be said in general . . . 111
17.9 Quantization in released models . . . 113
17.10 Summary of capabilities . . . 113

18 Pros, Cons, and Risks **115**
18.1 Pros and cons by technique family . . . 115

18.2 Risk register . . . 115
18.3 Silent degradation: why average metrics are not enough . . . 115
18.4 Calibration sensitivity . . . 116
18.5 Safety, trustworthiness, and security . . . 116
18.6 Systems risks: kernels, formats, and lock-in . . . 117
18.7 MoE and reasoning model risks . . . 117
18.8 Training-time risks . . . 117
18.9 Reproducibility and evidence quality . . . 118
18.10 Risk-informed rules of thumb . . . 118

19 Deployment Decision Guide **121**
19.1 Principle: start from the binding resource . . . 121
19.2 Decision flowchart . . . 121
19.3 Worked sizing example (analytic) . . . 121
19.4 Recommendations by hardware class . . . 123
19.5 Recommendations by workload and model type . . . 123
19.6 Choosing the method within a format . . . 123
19.7 Frameworks and operational notes . . . 123
19.8 Cost reasoning . . . 123

20 Priority Assessment of Quantization Sub-Topics **127**
20.1 What is being prioritised, and for whom . . . 127
20.2 Rubric . . . 127
20.3 Scored table . . . 127
20.4 Final ranking with confidence . . . 128
20.4.1 Reading the ranking in tiers . . . 129
20.5 Stability and what would change the ranking . . . 130
20.6 Limitations of the assessment . . . 131

21 Open Problems and Conclusions **132**
21.1 Open problems . . . 132
21.1.1 A theory of quantization error for end-to-end behaviour . . . 132
21.1.2 Why do some models quantize better than others? . . . 132
21.1.3 Robust 4-bit activation quantization without training . . . 132
21.1.4 Sub-3-bit weights at scale with fast kernels . . . 133
21.1.5 Quantization-aware training for modern post-training pipelines . . . 133
21.1.6 Scaling laws for quantization, including MoE and training-time formats 133

21.1.7 Long-context and agentic robustness . . . 133
21.1.8 Safety, security and verification . . . 133
21.1.9 Evaluation science . . . 134
21.1.10 Hardware–algorithm co-design and portability . . . 134
21.1.11 Low-precision training at frontier scale . . . 134
21.1.12 Quantization together with other compression . . . 134
21.2 Conclusions . . . 134

A Notation **136**

B Supplementary Derivations **138**
B.1 Stochastic rounding: unbiasedness and variance . . . 138
B.2 Optimal scale for fixed ternary pattern . . . 138
B.3 Effective bits per weight of common formats . . . 139
B.4 Scale rounding in two-level (NVFP4-style) scaling . . . 139
B.5 E2M1 grid and rounding error . . . 140
B.6 Hadamard transforms: orthogonality and cost . . . 140
B.7 Error propagation through a quantized linear layer . . . 140
B.8 Lloyd–Max conditions . . . 141

C Glossary **142**

Bibliography **145**

Chapter 1

Executive Summary of Findings

This chapter condenses the document into the findings a practitioner or researcher most needs. Every claim is developed, with derivations or citations, in the chapter indicated. The scope is quantization of LLMs: weights, activations, the KV cache, and the number formats used for training. Pruning, distillation, low-rank factorisation and KV-cache eviction are the subject of companion papers and appear only where they interact with quantization.

1.1 The ten findings

Finding 1: decoding is a memory problem, so bytes are the currency

At small batch sizes the token-by-token decode of a dense transformer reads every weight once per step and performs about two floating-point operations per weight byte-pair. On every current accelerator the ratio of peak compute to memory bandwidth is hundreds of FLOP per byte, so decode sits far to the left of the roofline ridge point and its latency is proportional to bytes moved (Chapter 2). Moving from 16-bit to 4-bit weights therefore buys, to first order, a $3\times$ to $4\times$ latency reduction *if* the kernel dequantizes at full bandwidth. Compute-bound regimes (prefill, large batches) benefit only when the tensor cores consume the low-precision format directly (W8A8, W4A4, FP8, FP4).

Finding 2: 8-bit is solved; 4-bit weight-only is mature

LLM.int8() and SmoothQuant established that 8-bit weights and activations are essentially lossless once outliers are handled [93, 169]. For weight-only quantization, 4 bits with a group size of 64 to 128 and a calibration-based method (GPTQ, AWQ) typically costs a small fraction of the model's perplexity and downstream accuracy [16, 73]. A large controlled study of the Llama-3.1 family reported that W8A8-FP is lossless and W8A8-INT and W4A16-INT retain most accuracy on the open LLM benchmarks used there [20]; reasoning-model studies reach compatible conclusions for W8A8 and W4A16 but warn about lower widths [141].

Finding 3: second-order information is the workhorse of PTQ

Layer-wise reconstruction with the activation Hessian $\boldsymbol{H} = \boldsymbol{X}^\top \boldsymbol{X}$ (OBS $\rightarrow$ OBQ $\rightarrow$ GPTQ) remains the backbone of calibration-based methods in 2026 [2, 73, 117]. We prove (Chapter 5) that the GPTQ update is the exact optimal compensation for a single rounded coordinate under the quadratic proxy, and we show in our own simulation that on a correlated synthetic layer GPTQ reduces output error by more than an order of magnitude relative to round-to-nearest at 3 and 4 bits. Newer variants (GPTAQ, Qronos, MR-GPTQ, SignRoundV2) change the objective or the rounding search while keeping this skeleton

[71, 110, 142, 164].

Finding 4: outliers dictate the design of activation quantization

A handful of channels and tokens ("massive activations", attention sinks) carry magnitudes orders of magnitude above the bulk [34, 101, 165]. Per-tensor INT8 or INT4 activation quantization fails without remedy. The remedies fall into three classes: mixed-precision decomposition [93], equivalent re-scaling (SmoothQuant, AWQ, OmniQuant) [16, 118, 169], and orthogonal rotations that spread outliers evenly (QuaRot, SpinQuant, QuIP) [146, 150, 173]. Rotations make W4A4 practical; in our simulation a randomized Hadamard transform lowered 4-bit per-token activation quantization error by more than a factor of ten on a synthetic outlier-channel layer (Chapter 7).

Finding 5: below 4 bits, codebooks and training win

At 2–3 bits per weight, scalar grids waste capacity. Vector, additive and trellis codebooks combined with incoherence processing (AQLM, QuIP#, QTIP, VPTQ, PV-Tuning) reach usable quality at 2 bits for large models [10, 132, 144, 149, 181], and lattice/trellis methods approach the rate–distortion bound for Gaussian sources [115, 144]. Quantization-aware training (EfficientQAT, ParetoQ, BitDistiller, LC-QAT) further closes gaps at 2 bits, and training from scratch in ternary form (BitNet b1.58) reaches parity at small scale in the authors' reports [24–26, 46, 89, 129]. A ParetoQ study placed a "learning transition" between 2 and 3 bits: at 3 bits and above, fine-tuning stays near the pre-trained solution; at 2 bits and below it must re-learn representations [129].

Finding 6: micro-scaled FP4 is the new hardware default but is not an automatic upgrade over INT4

NVIDIA Blackwell and later parts execute block-scaled 4-bit floating-point matrix multiplies natively; the Open Compute Project MX formats (MXFP4, MXFP6, MXFP8) and NVIDIA's NVFP4 (16-element blocks with an E4M3 scale) are the two main variants [114, 206]. A 2025 study concluded that FP4 "is not an automatic upgrade over INT4": NVFP4's small blocks weaken traditional outlier-mitigation, MXFP4's power-of-two scale adds error, and specialized methods (MR-GPTQ) are needed to realise the accuracy and the reported kernel speedups of up to 3.6× layer-wise and 2.2× end-to-end on B200 [110]. Our simulation reproduces the ordering qualitatively: on Gaussian data, INT4 with a high-precision scale beats FP4 with the same group size, while NVFP4 beats MXFP4 by about 1.5 dB (Chapter 11). A separate comprehensive study of fine-grained formats reaches related conclusions for INT versus FP [81].

Finding 7: training in 8 and 4 bits is now credible

FP8 mixed-precision training is used in frontier-scale models [43, 63, 64]. NVIDIA reported a 12B-parameter model trained on 10T tokens in NVFP4 with loss and downstream accuracy comparable to an FP8 baseline, using random Hadamard transforms, two-dimensional block scaling, stochastic rounding, and a few high-precision layers [116]. Academic recipes (Quartet, Quartet II, "FP4 All the Way") report competitive quality at smaller scales and kernel speedups on Blackwell [58, 147, 148]. Whether FP4 training is robust beyond the

reported scales and across architectures (MoE, long context, RL post-training) remains open.

Finding 8: the KV cache needs its own recipe

Keys have channel-structured outliers and values do not; hence per-channel key and per-token value quantization (KIVI) and rotation-based methods [85, 87, 157, 178]. Reported results reach 2–4 bits per cache element with limited quality loss on moderate-length tasks, but long-context, multi-hop and reasoning evaluations are far less forgiving (Chapter 12). In our simulation, which of per-token and per-channel key grouping wins flips depending on whether queries amplify the outlier channels, a reminder that conclusions are workload-dependent.

Finding 9: perplexity is necessary but not sufficient

Average perplexity on WikiText-2 or C4 hides degradations in long-context retrieval, multi-step reasoning, code, multilingual text, calibration and safety behaviour [19, 41, 48, 109, 141]. Evaluation must include task suites sensitive to the capability being deployed, long-generation tests (errors accumulate over thousands of tokens of chain-of-thought), and paired safety checks. Calibration-set sensitivity and quantization-conditioned attacks are documented risks [36, 50].

Finding 10: the community priority has shifted from algorithms to formats, kernels and QAT

Our subjective priority assessment (Chapter 20) ranks (1) weight-only 4-bit PTQ and FP8 inference as the production default, (2) micro-scaled FP4 for weights and activations with matching training recipes, (3) KV-cache quantization, (4) QAT and distillation-based recovery, and (5) sub-3-bit vector/trellis quantization, with MoE- and reasoning-aware methods rising fastest. Early open-weight releases that ship *natively* quantized checkpoints (MXFP4 MoE weights in gpt-oss, INT4 QAT in Kimi K2 Thinking, QAT checkpoints from Gemma 3) show that post-training quantization is being absorbed into the training pipeline [72, 192, 201].

1.2 State of the art at a glance

Table 1.1 gives a one-page map of the maturity of each method family as of October 2026. "Maturity" is the author's qualitative judgement (production-ready, emerging, or research), and "typical" accuracy statements refer to large ($\geq$ 30B) dense models; small models and MoE models are generally less tolerant.

1.3 Reading guide

Chapters 2–4 give motivation, mathematics and a taxonomy. Chapters 5–9 cover post-training weight and activation methods. Chapters 10–12 cover training-time methods, number formats and the KV cache. Chapters 13–15 treat error analysis, systems, MoE and reasoning models. Chapters 16–21 give the timeline, capabilities, risks, a deployment guide, the priority

Table 1.1: Maturity map of quantization method families (author's qualitative judgement, informed by the cited literature).

Family	Representative methods	Typical regime	Maturity
RTN + groups	bitsandbytes NF4, GGUF Q*n*_K	4–8 bit weights; weak below 4	Production
Second-order PTQ	GPTQ, OBQ, GPTAQ, Qronos	3–4 bit weights	Production
Activation-aware	AWQ, OmniQuant, SignRound	3–4 bit weights	Production
Smoothing	LLM.int8(), SmoothQuant, ZeroQuant	W8A8	Production
Rotation	QuaRot, SpinQuant, QuIP, MR-GPTQ	W4A4, W4A8, FP4	Emerging/production
Vector/trellis	AQLM, QuIP#, QTIP, VPTQ	2–3 bit weights	Research/emerging
Mixed precision	HAWQ-style, SliM-LLM, EvoPress, GEMQ	Average 2.5–4.5 bits	Emerging
QAT/distill.	LLM-QAT, EfficientQAT, ParetoQ	2–4 bit	Emerging/production (vendor)
Ternary/binary	BitNet b1.58, TriLM/Spectra	1.58 bit native training	Research
FP/MX formats	FP8, MXFP4, NVFP4	W4A4, W8A8 on new GPUs	Emerging/production
Low-precision training	FP8, NVFP4 recipes	8/4-bit GEMMs	FP8 production; FP4 emerging
KV quantization	KIVI, KVQuant, RotateKV, TurboQuant	2–8 bit cache	Emerging

assessment and open problems. Appendices collect notation, additional derivations, and a glossary.

How to read the numbers in this document

Where a number is taken from a paper it is cited. Numbers marked "own simulation" come from small NumPy experiments run for this document on synthetic data and are illustrative of mechanisms, not predictions of LLM accuracy. Numbers marked "approximate" are widely reported figures that the author recalls but did not re-verify against the original table; they should be treated as indicative to within a few percent. Vendor performance claims (for example for Blackwell or Rubin) are marketing figures unless independently benchmarked.

Chapter 2

Introduction and Motivation

2.1 Why quantize at all

A decoder-only transformer with P parameters stored at b bits per parameter occupies $Pb/8$ bytes. A 70-billion-parameter model in bfloat16 therefore needs 140 GB for weights alone, more than any single accelerator offered until recently carries, and a 405-billion-parameter model needs 810 GB [91]. Quantization attacks three costs at once.

Capacity.
Fewer bytes per weight let a model fit on fewer devices, on a single consumer GPU, or in the RAM of a laptop. Table 2.1 lists exact footprints.

Bandwidth.
Autoregressive decoding reads (nearly) all weights and the whole KV cache once per generated token. Fewer bytes per weight and per cache entry reduce the time of every step in direct proportion when the step is bandwidth-bound.

Compute and communication.
When the tensor cores natively execute 8-bit or 4-bit operands, peak throughput rises by a factor of two or four relative to 16-bit. Quantized collectives (for tensor- or expert-parallel all-reduce and all-to-all) also reduce interconnect traffic.

The economics are stark. A server-class GPU carries on the order of 80 to 200 GB of high-bandwidth memory (HBM); inference providers pay for GPU-hours, and weight memory determines how many accelerators a replica needs. Reducing a 70B model from 140 GB to roughly 37 GB moves it from a two-GPU tensor-parallel replica to a single device, which both cuts cost and removes inter-GPU communication from the critical path.

2.2 Decoding is memory-bandwidth bound

We make the claim quantitative. Consider a single linear layer $\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x}$ with $\boldsymbol{W} \in \mathbb{R}^{n \times m}$ applied to a batch of B token vectors, as in the matrix multiplications that dominate transformer inference (the attention score and value products behave similarly, with the KV cache playing the role of $\boldsymbol{W}$).

Definition 2.1 (Arithmetic intensity)**.** The arithmetic intensity of an operation is $I = \text{FLOPs/bytes moved from main memory}$.

A weight matrix of nm entries at b bits costs $nmb/8$ bytes to read once. The product performs $2Bnm$ floating-point operations (a multiply and an add per weight per token). Ignoring

Table 2.1: Weight footprint in gigabytes (10^9 bytes) as a function of average bits per weight (bpw). The 4.25 and 4.5 columns include typical group-scale overheads (for example a 16-bit scale per group of 64 weights adds 0.25 bit; a 16-bit scale and zero-point per group of 64 adds 0.5 bit; MXFP4 costs exactly 4.25 bpw and NVFP4 about 4.5 bpw). Parameter counts are the publicly stated totals. Arithmetic is exact; it ignores embeddings kept at higher precision.

Model (params)	16	8	4.5	4.25	3	2
7B	14.0	7.0	3.9	3.7	2.6	1.8
70B	140.0	70.0	39.4	37.2	26.3	17.5
405B (Llama 3.1)	810.0	405.0	227.8	215.2	151.9	101.3
671B (DeepSeek-V3 total)	1342.0	671.0	377.4	356.5	251.6	167.8

Table 2.2: Approximate vendor-stated peak figures (dense, no sparsity) for selected NVIDIA accelerators, and the implied ridge points Π/β. These are specification-sheet values recalled by the author and rounded; real kernels reach a fraction of peak. The Rubin entries are NVIDIA's pre-release claims as reported in trade press [193]: roughly 50 PFLOP/s of NVFP4 inference throughput, 35 PFLOP/s NVFP4 training throughput and 22 TB/s of HBM4 bandwidth. For Rubin NVFP4 the 50 PFLOP/s figure is the marketing "inference" number achieved with adaptive precision, so the implied ridge is indicative only.

Device	HBM (GB)	BW (TB/s)	BF16 (TFLOP/s)	Low-prec. peak (TFLOP/s)	Ridge BF16 / low (FLOP/B)
A100 80GB SXM	80	2.0	312	624 (INT8)	156 / 312
H100 SXM	80	3.35	989	1979 (FP8)	295 / 591
H200	141	4.8	989	1979 (FP8)	206 / 412
B200 (approx.)	180–192	8	2250	9000 (FP4)	281 / 1125
Rubin (claimed)	–	22	–	50000 (NVFP4 inf.)	– / 2270

activation traffic (small when $B \ll n, m$),

$$I(B, b) = \frac{2Bnm}{nmb/8} = \frac{16\,B}{b} \text{ FLOP/byte.} \tag{2.1}$$

For bfloat16 weights $I = B$; for 4-bit weights $I = 4B$.

Proposition 2.1 (Roofline latency bound)**.** *Let a device have peak throughput Π (FLOP/s) and memory bandwidth β (byte/s). The time of one layer is bounded below by $T \geq \max(T_{\text{comp}}, T_{\text{mem}})$ with $T_{\text{comp}} = 2Bnm/\Pi$ and $T_{\text{mem}} = nmb/(8\beta)$. The layer is memory-bound iff $I < \Pi/\beta$, the* ridge point.

Proof. Compute and memory transfers can overlap but neither can finish faster than its own bound, so the time is at least the larger. The two bounds are equal when $2Bnm/\Pi = nmb/(8\beta)$, i.e. $16B/b = \Pi/\beta$, which is $I = \Pi/\beta$. Below the ridge, $T_{\text{mem}} > T_{\text{comp}}$. □

Table 2.2 lists approximate vendor-specification ridge points. They are of order 10^2 to 10^3 FLOP/byte, while single-stream decoding has $I = B = 1$ and typical serving batches reach B of 16 to 256. The ridge point is higher for narrower formats because peak throughput rises faster than bandwidth.

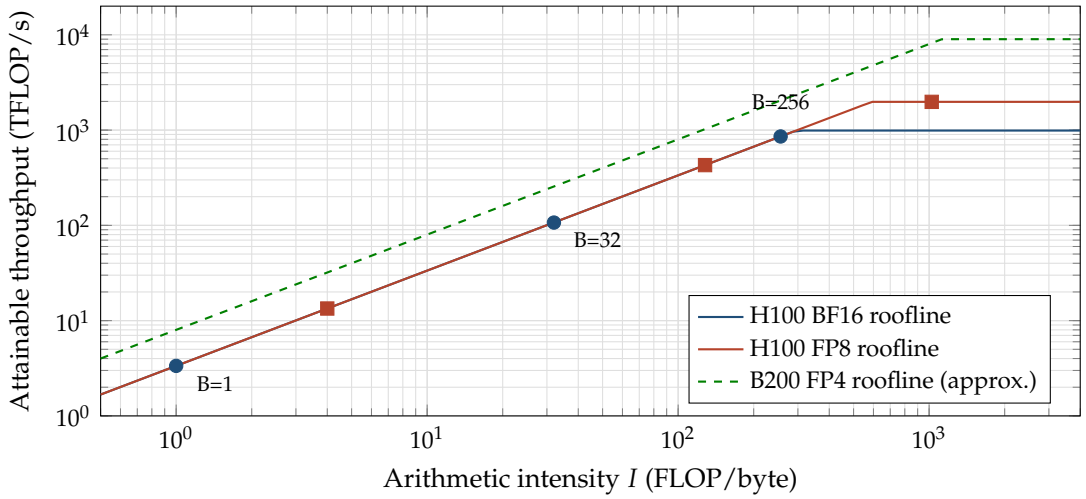


Figure 2.1: Roofline model from (2.1) and Proposition 2.1 for approximate H100 specifications (analytic curves, not measurements). Circles: BF16-weight decoding at batch sizes $B = 1, 32, 256$ ($I = B$). Squares: 4-bit-weight decoding with BF16 compute at $B = 4, 128, 1024$ chosen to show the same operating points shifted right by $4\times$ in intensity. Weight-only quantization moves the operating point rightward along the bandwidth slope (higher throughput at the same batch); only low-precision compute raises the flat ceiling.

2.2.1 A worked example: decoding a 70B model

Take $P = 70 \times 10^9$ parameters, $\beta = 3.35$ TB/s, and ignore the KV cache and the (small) non-linear-layer traffic. If the weights were stored in bfloat16 across k GPUs with ideal tensor parallelism, each step must read 140 GB in total, so the time per decoded token (one stream, $B = 1$) is bounded below by

$$T_{\text{mem}} = \frac{140 \text{ GB}}{k \cdot 3.35 \text{ TB/s}} = \frac{41.8 \text{ ms}}{k}. \tag{2.2}$$

With $k = 2$ this is 20.9 ms per token, or about 48 tokens/s as an upper bound. At an effective 4.25 bits per weight (a 4-bit group-quantized model with a 16-bit scale per 64 weights) the weights occupy 37.2 GB, fit on a single GPU, and the single-GPU bound is 11.1 ms per token or about 90 tokens/s, an improvement of $1.9\times$ in the single-stream upper bound while using half the hardware. Per *GPU*, the improvement in tokens per second per GPU is $3.8\times$. These are upper bounds: real kernels reach perhaps 60 to 90 percent of peak bandwidth, and dequantization overhead, attention, sampling and communication add time.

2.2.2 Where the batch-size crossover happens

With 4-bit weights but BF16 tensor-core arithmetic after dequantization, the intensity at batch B is $4B$ relative to the 4-bit bytes while the ridge is unchanged. The memory-bound regime therefore ends at $B^* = \Pi/(4\beta)$ rather than Π/β, i.e. at roughly 74 instead of 295 on an H100. Beyond B^* the kernel becomes compute-bound and the dequantization instructions compete with the tensor cores; this is why mixed-precision GEMM kernels (Marlin and its successors, Chapter 14) are engineered to keep the speedup up to batch sizes of 16 to 32 or more [100]. Beyond that, the only way to gain is to use quantized *activations* and low-precision tensor-core

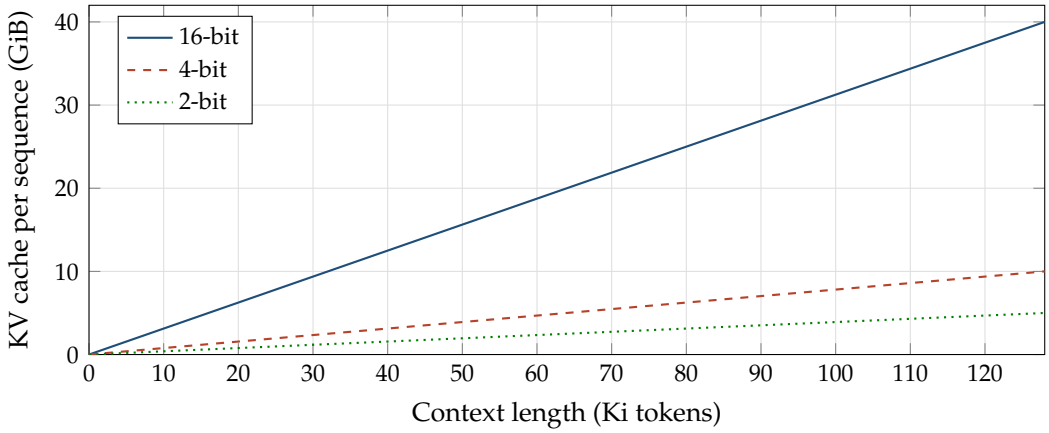


Figure 2.2: KV-cache memory per sequence for a 70B-class GQA configuration ($L = 80$, $h_{kv} = 8$, $d_h = 128$; 320 KiB/token at 16 bits) from (2.3), ignoring scale/zero-point overhead (which adds roughly 0.25 to 1 bit per element for small groups, so ideal 4-bit and 2-bit curves are optimistic). Analytic curves.

instructions (W8A8, W4A4, FP8, FP4), which is why activation quantization (Chapters 6 and 7) matters for prefill and large-batch serving despite being harder.

2.2.3 The KV cache: the second memory stream

For a transformer with L layers, h_{kv} key–value heads of dimension d_h, and cache element width b_{kv} bits, each cached token costs

$$M_{\text{tok}} = 2\, L\, h_{kv}\, d_h\, \frac{b_{kv}}{8} \text{ bytes.} \tag{2.3}$$

For Llama-2-7B ($L = 32$, $h_{kv} = 32$, $d_h = 128$) and 16-bit cache this is $2 \cdot 32 \cdot 32 \cdot 128 \cdot 2 = 524{,}288$ bytes, i.e. 512 KiB per token, so a 4096-token context costs 2 GiB per sequence. Grouped-query attention reduces it: for a 70B-class model with $L = 80$, $h_{kv} = 8$, $d_h = 128$ the cost is $2 \cdot 80 \cdot 8 \cdot 128 \cdot 2 = 327{,}680$ bytes (320 KiB) per token, so a 128K-token context costs about 40 GiB per sequence at 16 bits, comparable to the weights. In long-context or high-concurrency serving the KV cache can exceed the weight memory, and *per-step* bandwidth includes reading the entire cache of every active sequence. This is why KV quantization is an independent axis (Chapter 12); it composes with weight quantization.

2.2.4 Energy

Data movement also dominates energy. Widely quoted 45-nm process figures attribute roughly two orders of magnitude more energy to an off-chip DRAM access than to a floating-point multiply-add of the same operand width [198]. Modern HBM is more efficient than the DRAM of that study, but the qualitative lesson (bytes cost more than arithmetic) persists, and 4-bit arithmetic units are also much smaller and lower-energy than 16-bit ones. Quantization therefore improves tokens per joule roughly in proportion to bytes saved when memory-bound.

2.3 Scope and organizing principles

We use the word *quantization* for any mapping of a real-valued tensor to a finite set of representable values with a bit-cost smaller than the original. This includes scalar rounding to integer or floating-point grids, vector codebooks, and trellis codes. We distinguish three questions that any quantization proposal must answer.

Q1. What is quantized? Weights ($\boldsymbol{W}$), activations ($\boldsymbol{X}$), KV-cache entries, gradients, optimizer states, or communication buffers.

Q2. With which representation? Integer grid, floating-point grid, non-uniform codebook, vector or trellis code; with which granularity of scales (per-tensor, per-channel, per-group, per-block) and which scale format.

Q3. How are the parameters chosen? Round-to-nearest, second-order (Hessian-aware) rounding, activation-aware scaling, learned rotations, or end-to-end gradient training (QAT).

Each of these three axes forms part of the taxonomy in Chapter 4. The order of exposition in the document follows increasing sophistication of Q3 (post-training methods first, training-time methods later) while interleaving Q2 (formats) where they enable the methods.

Three regimes of benefit

(i) Capacity-bound: the model does not fit; any bit reduction matters, and the operating point is set by what accuracy is tolerable at a given footprint (bits per parameter vs. number of parameters, Chapter 15). **(ii) Bandwidth-bound**: decoding at small to medium batch; speedup $\approx$ byte reduction if the dequantization kernel is efficient. **(iii) Compute-bound**: prefill and large-batch serving; speedup requires low-precision *compute*, hence quantized activations and native FP8/FP4/INT4 tensor-core operands.

2.4 A short history

Quantization of neural networks predates LLMs by decades: binary and ternary networks [23, 33, 184], integer-arithmetic-only inference [82, 86], learned clipping and step sizes [97, 127], and Hessian-aware mixed precision [78] were all developed for convolutional networks. The 2022 inflection point was the discovery that, beyond roughly 6.7B parameters, systematic outlier features emerge in transformer activations that break naive 8-bit quantization [93], followed within months by GPTQ [73] (second-order one-shot weight quantization to 3–4 bits for models up to 175B parameters) and SmoothQuant [169]. Chapters 5 to 8 trace the subsequent development; the full timeline to October 2026 is in Chapter 16.

2.5 Notation conventions

Matrices are bold capitals ($\boldsymbol{W}, \boldsymbol{X}$); vectors are bold lowercase. $\mathcal{Q}(\cdot)$ denotes a quantizer; $\widehat{\boldsymbol{W}} = \mathcal{Q}(\boldsymbol{W})$ the dequantized weights. For a linear layer $\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x}$ with calibration inputs stacked as columns of $\boldsymbol{X} \in \mathbb{R}^{m \times N}$, the layer-wise loss is $\left\| \boldsymbol{W}\boldsymbol{X} - \widehat{\boldsymbol{W}}\boldsymbol{X} \right\|_F^2$ and $\boldsymbol{H} = \boldsymbol{X}\boldsymbol{X}^\top \in \mathbb{R}^{m \times m}$ is the

(proxy) Hessian. Bits per weight include the amortized cost of scales and zero-points unless stated otherwise. A full notation table is in Appendix A.

Chapter 3

Mathematical Foundations

This chapter develops the mathematics used throughout the document: the information-theoretic limits of lossy compression, the distortion of uniform and non-uniform scalar quantizers, the parametrization of affine quantizers with scales and zero-points at various granularities, rounding rules, and the quadratic proxy loss that underlies post-training quantization.

3.1 Rate–distortion limits

3.1.1 The rate–distortion function

Let X be a source with a distribution p and let $d(x, \hat{x})$ be a distortion measure. The *rate–distortion function* is

$$R(D) = \min_{p(\hat{x}|x):\ \mathbb{E}[d(X,\hat{X})] \le D} I(X; \hat{X}), \tag{3.1}$$

and the source coding theorem with a fidelity criterion states that no code of rate below $R(D)$ bits per sample can achieve expected distortion D in the limit of long blocks, and that rates arbitrarily close to $R(D)$ are achievable [190, 213]. For a memoryless Gaussian source $X \sim \mathcal{N}(0, \sigma^2)$ with squared error,

$$R(D) = \max\Big\{0,\ \tfrac{1}{2}\log_2 \tfrac{\sigma^2}{D}\Big\}, \qquad D(R) = \sigma^2\, 2^{-2R}. \tag{3.2}$$

Equivalently, each additional bit reduces the minimum distortion by a factor of four, i.e. improves the signal-to-quantization-noise ratio (SQNR) by $10 \log_{10} 4 = 6.02$ dB.

Proposition 3.1 (Gaussian rate–distortion)**.** *For $X \sim \mathcal{N}(0, \sigma^2)$ and squared error, $D(R) = \sigma^2 2^{-2R}$.*

Proof sketch. The converse follows from $I(X; \hat{X}) = h(X) - h(X - \hat{X} \mid \hat{X}) \ge h(X) - h(X - \hat{X}) \ge \frac{1}{2}\log(2\pi e\sigma^2) - \frac{1}{2}\log(2\pi e D)$, using that the Gaussian maximizes entropy for a given second moment, so the error's entropy is at most $\frac{1}{2}\log(2\pi e D)$. Achievability uses the test channel $X = \hat{X} + Z$ with $\hat{X} \sim \mathcal{N}(0, \sigma^2 - D)$ and $Z \sim \mathcal{N}(0, D)$ independent, for which $I(X; \hat{X}) = \frac{1}{2}\log(\sigma^2/D)$; random coding over long blocks realizes this rate [213]. □

Real weight tensors are not i.i.d. Gaussian, but two facts make the Gaussian bound the right reference. First, weights of trained transformer layers are well approximated, after normalizing for per-channel scale, by roughly bell-shaped (sometimes heavy-tailed) distributions. Second, among all sources with a given variance the Gaussian has the *largest* rate–distortion function under squared error [213]; thus $\frac{1}{2}\log_2(\sigma^2/D)$ is a conservative (worst-case) rate for any source of that variance. Incoherence processing (Chapter 7) deliberately makes weights look Gaussian so that this bound is attainable by codes designed for the Gaussian.

3.1.2 Reverse water-filling and transform coding

For a vector of independent Gaussian components with variances σ_i^2 and total distortion budget $D = \sum_i D_i$, the optimal allocation is *reverse water-filling*: $D_i = \min(\theta, \sigma_i^2)$ with θ chosen so that $\sum_i D_i = D$, and the rate is $R = \sum_i \frac{1}{2} \log_2^+(\sigma_i^2/\theta)$ [213]. Components with variance below the water level θ receive zero bits.

The same principle governs quantization with a *weighted* error. Suppose a weight vector $\boldsymbol{w} \in \mathbb{R}^m$ is approximated by $\hat{\boldsymbol{w}}$ and the loss is $(\boldsymbol{w} - \hat{\boldsymbol{w}})^\top \boldsymbol{H} (\boldsymbol{w} - \hat{\boldsymbol{w}})$ with $\boldsymbol{H} \succeq 0$ having eigen-decomposition $\boldsymbol{H} = \boldsymbol{U} \boldsymbol{\Lambda} \boldsymbol{U}^\top$. In the eigenbasis, $\tilde{\boldsymbol{w}} = \boldsymbol{U}^\top \boldsymbol{w}$, the loss is $\sum_i \lambda_i (\tilde{w}_i - \hat{\tilde{w}}_i)^2$, a weighted sum.

Proposition 3.2 (Bit allocation under a Hessian-weighted loss)**.** *Let the rotated weight coordinates be independent $\mathcal{N}(0, s^2)$ and let the loss be $\sum_i \lambda_i D_i$ with D_i the squared error of coordinate i. For a total rate $R = \sum_i R_i$ the minimizing rates are*

$$R_i = \max\Big\{0,\ \tfrac{1}{2} \log_2 \frac{\lambda_i s^2}{\theta}\Big\}. \tag{3.3}$$

At the high-rate optimum all active coordinates have equal weighted distortion $\lambda_i D_i = \theta$, i.e. coordinates with large Hessian eigenvalue receive $\frac{1}{2} \log_2(\lambda_i/\lambda_j)$ more bits.

Proof. Minimize $\sum_i \lambda_i s^2 2^{-2R_i} + \mu(\sum_i R_i - R)$ over $R_i \geq 0$. The stationarity condition $-2 \ln 2\, \lambda_i s^2 2^{-2R_i} + \mu = 0$ yields $\lambda_i s^2 2^{-2R_i} = \theta := \mu/(2 \ln 2)$, equal for all active coordinates; solve for R_i. Coordinates with $\lambda_i s^2 < \theta$ would receive negative rate and are clipped to zero (they are quantized to the mean). □

This is the information-theoretic justification of two empirical practices: (a) assign more bits (or finer groups) to the input channels with large activation energy (AWQ, SpQR, OWQ), and (b) mixed-precision allocation by Hessian trace or Fisher information (Chapter 9). It also explains why the rank of $\boldsymbol{H}$ matters: directions with tiny eigenvalue can be quantized almost arbitrarily coarsely, which is the effect exploited by GPTQ's error-compensation (Chapter 5).

3.1.3 Gap of practical quantizers

A scalar quantizer followed by ideal entropy coding loses a constant in the high-resolution limit: $\frac{1}{2} \log_2(\pi e/6) \approx 0.254$ bit per sample, equivalent to 1.53 dB. Vector quantizers in dimension n close the gap as $n \to \infty$; lattice and trellis-coded quantizers recover a substantial part of it (the E_8 lattice quantizer used in QuIP# in 8 dimensions, and trellis codes with hundreds to thousands of states as in QTIP) [144, 149, 195, 199]. In LLM weight compression, however, *fixed-rate* codes are used (no entropy coding): every weight group costs the same number of bits so that kernels can index weights in constant time. Fixed-rate scalar quantization of a Gaussian at R bits per sample has a larger gap than the entropy-coded one, especially at low rates, as our simulation shows below.

Two practical conclusions follow. (1) At ≤ 4 bits the clipping range is a first-order design variable; this is the reason the best calibration methods search or learn clip ratios (OmniQuant's learnable clipping, AWQ's clipping search, and optimal-range choices in the earliest post-training methods [4, 16, 118]). (2) Fixed-rate scalar quantizers at 2 bits lose about 2.8 dB relative to the bound (about 0.47 bit); vector quantization at the same rate gains part of this back, which is the motivation of Chapter 8.

Table 3.1: SQNR (dB) of b-bit fixed-rate scalar quantizers on i.i.d. $\mathcal{N}(0,1)$ data (400k samples; own simulation). "Max-abs" sets the range to the sample maximum (what per-tensor absmax scaling does on a large tensor); "Optimal clip" searches the symmetric clipping threshold; "Lloyd–Max" is a non-uniform 1-D codebook trained on 200k samples with 60 iterations (only reported for $b \le 4$ where it converged); "R–D bound" is $6.02\,b$ dB from (3.2).

Bits	Max-abs uniform	Optimal-clip uniform (clip, in σ)	Lloyd–Max	R–D bound
1	−12.0	4.39 (0.8)	4.39	6.02
2	0.15	9.24 (1.5)	9.28	12.04
3	8.18	14.25 (2.05)	14.58	18.06
4	14.79	19.35 (2.5)	20.15	24.08
5	21.10	24.57 (2.95)	–	30.10
6	27.26	29.84 (3.3)	–	36.12
8	39.41	40.66 (3.9)	–	48.16

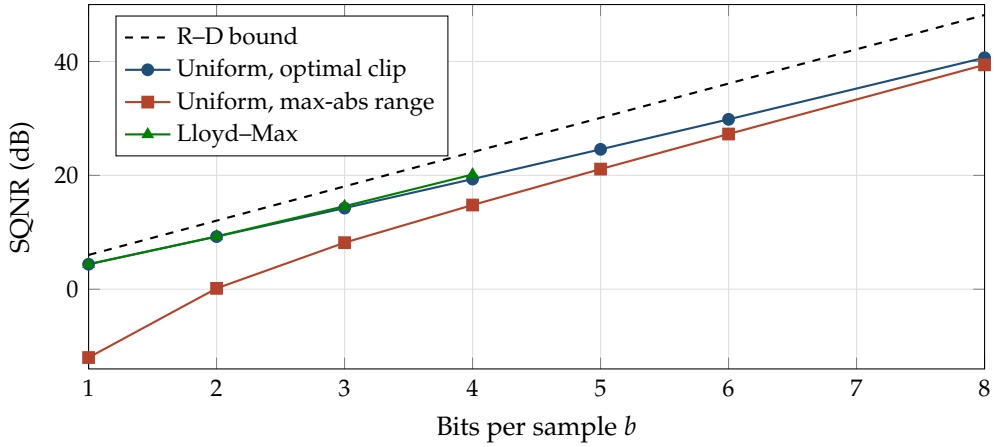


Figure 3.1: SQNR versus bits for Gaussian data (own simulation; data of Table 3.1). Clipping matters enormously at low rates: at 3 bits, max-abs scaling loses 6 dB relative to an optimized clip, equal to a full 1 bit. The gap to the rate–distortion line is 3.7 dB at 3 bits for optimal-clip uniform and 3.3 dB for Lloyd–Max.

3.2 Uniform affine quantization

3.2.1 Definition

Definition 3.1 (Uniform affine quantizer)**.** A b-bit uniform affine quantizer with scale $s > 0$ and integer zero-point z maps $x \in \mathbb{R}$ to an integer code

$$q = \operatorname{clip}\big(\operatorname{round}(x/s) + z,\; q_{\min},\; q_{\max}\big), \qquad \hat{x} = s\,(q - z), \tag{3.4}$$

with $[q_{\min}, q_{\max}] = [0, 2^b - 1]$ (unsigned) or $[-2^{b-1}, 2^{b-1} - 1]$ (signed). It is *symmetric* if $z = 0$ and *asymmetric* otherwise.

For a tensor with observed range $[\alpha, \beta]$ the min–max (asymmetric) calibration is

$$s = \frac{\beta - \alpha}{2^b - 1}, \qquad z = \operatorname{round}\Big(q_{\min} - \frac{\alpha}{s}\Big), \tag{3.5}$$

and the symmetric absmax calibration is $s = \max|x|/q_{\max}$ with $q_{\max} = 2^{b-1} - 1$ (giving the grid $\{-2^{b-1}, \dots, 2^{b-1} - 1\}$ with the code -2^{b-1} used only if the range is extended asymmetrically; many implementations restrict the symmetric grid to $\pm(2^{b-1} - 1)$ to keep it symmetric, at the price of one wasted code).

Remark 3.1. Asymmetric quantization spends one extra parameter per group (the zero-point) but represents skewed distributions much better. Weight distributions are approximately symmetric, so weight-only methods often use symmetric grids; activations after a non-linearity (e.g. post-GELU/SiLU gated outputs) and keys/values in the KV cache are often skewed, which motivates asymmetric grids (as in KIVI, Chapter 12).

3.2.2 Error decomposition and the 6.02 dB rule

Write the total error as $e = \hat{x} - x = e_{\text{round}} + e_{\text{clip}}$ where clipping error is nonzero only outside $[\, s(q_{\min} - z),\ s(q_{\max} - z)\,]$.

Proposition 3.3 (High-resolution uniform quantizer)**.** *Suppose x has a density f that is approximately constant across each quantization cell, and no clipping occurs. Then the rounding error is approximately uniform on $[-s/2, s/2]$, uncorrelated with x, and has $\mathbb{E}[e^2] = s^2/12$. For a signal uniformly distributed over the full range $[-A, A]$ with $s = 2A/2^b$,* $\text{SQNR} = 10\log_{10}(\mathbb{E}x^2/\mathbb{E}e^2) = 6.02\,b$ *dB.*

Proof. Within a cell the density is approximately flat, so the error $x - s\ \text{round}(x/s)$ is uniform on $[-s/2, s/2]$ and has variance $\int_{-s/2}^{s/2} u^2\,du/s = s^2/12$. For $x \sim U[-A, A]$, $\mathbb{E}x^2 = A^2/3$ and $s^2/12 = (2A)^2/(12 \cdot 4^b) = A^2/(3 \cdot 4^b)$; hence $\text{SQNR} = 4^b$, i.e. $20b\log_{10} 2 = 6.02b$ dB. □

The approximation (due to Bennett) fails when cells are not small relative to the density's variation, i.e. at low b, and when the signal is far from full-scale; for a Gaussian signal with the range set to $\pm k\sigma$, the loss relative to the uniform-source case is $10\log_{10}(k^2/3)$ dB, which explains the 6 dB offset between max-abs and optimal-clip curves at 3 bits in Figure 3.1 [3, 207].

3.2.3 Clipping versus rounding

For range parameter c (so $s = 2c/(2^b - 1)$ approximately) and Gaussian input, the mean-square error as a function of c is the sum of

$$D_{\text{round}}(c) \approx \frac{s^2}{12}\left[1 - 2Q(c/\sigma)\right], \qquad D_{\text{clip}}(c) = 2\int_c^\infty (x - c)^2 \phi_\sigma(x)\,dx, \tag{3.6}$$

with Q the Gaussian tail function. The first term increases with c (coarser grid), the second decreases; the optimum balances them. Numerically (Table 3.1) the optimal clip for a Gaussian is $\approx 0.8\sigma$ at 1 bit, 1.5σ at 2 bits, 2.5σ at 4 bits and 3.9σ at 8 bits. In neural-network practice, heavy tails make the optimal clip ratio smaller still, while the *functional* importance of the extreme values makes clipping dangerous (the outlier phenomenon of Chapter 6); thus a mean-square optimum is not always a task-loss optimum.

3.3 Granularity of scales

A single scale per tensor is hopeless below 8 bits because a few large values set s for all. *Granularity* refers to how many elements share one scale (and zero-point).

Table 3.2: Common scale granularities. For a weight matrix $\boldsymbol{W} \in \mathbb{R}^{n \times m}$ (output n, input m) with group size g along the input dimension.

Granularity	Definition	Scales per matrix
Per-tensor	one s for all elements	1
Per-channel (per-row)	one s per output channel (row of $\boldsymbol{W}$)	n
Per-group / per-block	one s per g consecutive input elements in a row	nm/g
Per-token (activations)	one s per token vector; computed dynamically	T
2-D block	one s per $g_1 \times g_2$ tile (used for training weights)	$nm/(g_1 g_2)$

Table 3.3: Absmax-scaled symmetric INT4 and INT3 on i.i.d. $\mathcal{N}(0,1)$ data as a function of group size g (own simulation, 20,000 groups each). $\mathbb{E}\max|z|$ is the mean group maximum; MSE is measured; "Pred." is $\mathbb{E}[a_g^2]/(12 q_{\max}^2)$ from Proposition 3.3.

g	$\mathbb{E}\max\lvert z\rvert$	INT4 MSE	INT4 pred.	INT4 SQNR (dB)	INT3 MSE	INT3 SQNR (dB)
16	2.07	0.00722	0.00770	21.4	0.0393	14.1
32	2.35	0.00943	0.00973	20.3	0.0515	12.9
64	2.59	0.01152	0.01170	19.4	0.0628	12.0
128	2.82	0.01373	0.01383	18.6	0.0748	11.3
256	3.04	0.01591	0.01598	18.0	0.0867	10.6
1024	3.44	0.02027	0.02028	16.9	0.1104	9.6
4096	3.80	0.02469	0.02470	16.1	0.1345	8.7

Storage overhead. If each group of g elements stores a scale of b_s bits and an optional zero-point of b_z bits, the effective bits per weight are

$$\text{bpw} = b + \frac{b_s + b_z}{g}. \tag{3.7}$$

For $b = 4$, $b_s = 16$, $b_z = 0$: $g = 128$ gives 4.125, $g = 64$ gives 4.25, $g = 32$ gives 4.5, $g = 16$ gives 5.0. With scale and zero-point both in 16-bit: $g = 128 \Rightarrow 4.25$; $g = 64 \Rightarrow 4.5$. For MXFP4, $b = 4$, $b_s = 8$ (E8M0), $g = 32$ yields 4.25; for NVFP4, $b_s = 8$ (E4M3), $g = 16$ yields 4.5 plus a negligible per-tensor FP32 scale [114, 206].

Why finer groups help. For absmax scaling, the step size within a group is $\Delta = a_g/q_{\max}$ with $a_g = \max_j |x_j|$ the group maximum. The expected maximum of g i.i.d. Gaussians grows like $\sigma\sqrt{2\ln(2g)}$, so smaller groups give a smaller a_g and hence a finer grid. Combining with Proposition 3.3, $D \approx \mathbb{E}[a_g^2]/(12\, q_{\max}^2)$, which our simulation confirms to within a few percent (Table 3.3).

Going from $g = 4096$ (nearly per-channel for a typical hidden dimension) to $g = 128$ gains 2.5 dB at 4 bits at a storage cost of 0.125 bit per weight, and to $g = 32$ gains 4.2 dB at 0.5 bit. By comparison, adding one bit buys 6 dB: group scaling is a cheap way to buy SQNR at 4 bits but its marginal return diminishes.

3.3.1 Pseudocode for group quantization

Example 3.1 (Worked example: one group)**.** Let $g = 8$, $b = 4$ and $\boldsymbol{w} = (0.12, -0.31, 0.05, 0.44, -0.02, 0.18, -0.51, 0.27)$. Then $q_{\max} = 7$, $s = 0.51/7 = 0.07286$. The quotients w_j/s are

Algorithm 1 Group-wise symmetric absmax quantization of a weight matrix

Require: $\boldsymbol{W} \in \mathbb{R}^{n\times m}$, bit width b, group size g ($g \mid m$)
1: $q_{\max} \leftarrow 2^{b-1}-1$
2: **for** each row $i = 1..n$ and group $k = 1..m/g$ **do**
3: $\quad \boldsymbol{w} \leftarrow \boldsymbol{W}[i, (k-1)g+1 : kg]$
4: $\quad s_{ik} \leftarrow \max(|\boldsymbol{w}|)/q_{\max}$ ▷ store in FP16
5: $\quad \boldsymbol{q}_{ik} \leftarrow \mathrm{clip}(\mathrm{round}(\boldsymbol{w}/s_{ik}), -q_{\max}-1, q_{\max})$ ▷ store as b-bit integers
6: **end for**
7: **return** integer codes $\boldsymbol{q}$ and scales $\boldsymbol{s}$; dequantization is $\widehat{\boldsymbol{W}}[i,j] = s_{ik}\, q_{ij}$

$(1.65, -4.25, 0.69, 6.04, -0.27, 2.47, -7.0, 3.71)$, so the codes are $(2, -4, 1, 6, 0, 2, -7, 4)$ and the dequantized values are $(0.146, -0.291, 0.073, 0.437, 0, 0.146, -0.510, 0.291)$. The squared errors are $(0.00068, 0.00036, 0.00052, 0.00005, 0.00040, 0.00103, 0, 0.00054)$ with mean 0.00045, close to the predicted $s^2/12 = 0.00044$. With a 16-bit scale the storage is $8 \cdot 4 + 16 = 48$ bits, or 6 bpw for this tiny group; at $g = 128$ it would be 4.125.

3.4 Non-uniform scalar quantizers

3.4.1 Lloyd–Max

The mean-square optimal K-level scalar quantizer for a density f satisfies two coupled conditions (Lloyd, Max): decision thresholds are midpoints of adjacent reproduction levels, and each reproduction level is the centroid of its cell [196, 210]:

$$t_k = \tfrac{1}{2}(c_k + c_{k+1}), \qquad c_k = \frac{\int_{t_{k-1}}^{t_k} x f(x)\,dx}{\int_{t_{k-1}}^{t_k} f(x)\,dx}. \tag{3.8}$$

Iterating these gives the 1-D k-means algorithm, a descent method converging to a local optimum. For a Gaussian, Lloyd–Max gives about 0.8 dB (at 4 bits: our simulation 20.15 versus 19.35 dB) over the best uniform quantizer.

Proposition 3.4 (Panter–Dite high-resolution law)**.** *For a density f and a fixed-rate quantizer with $K = 2^R$ levels with a locally optimal point density $\lambda(x) \propto f(x)^{1/3}$, the high-resolution mean-square distortion is*

$$D \approx \frac{1}{12K^2}\Big(\int f(x)^{1/3}dx\Big)^3. \tag{3.9}$$

For a Gaussian this equals $\frac{\sqrt{3}\pi}{2}\sigma^2 K^{-2} \approx 2.72\,\sigma^2 K^{-2}$.

Proof sketch. Let the cell width at x be $\delta(x) = 1/(K\lambda(x))$. Distortion is $\int f(x)\delta(x)^2/12\,dx = \frac{1}{12K^2}\int f/\lambda^2$. Minimizing $\int f\lambda^{-2}$ subject to $\int \lambda = 1$ by Hölder's inequality gives $\lambda \propto f^{1/3}$ and the stated value. □

The take-away is that non-uniform grids put resolution where the density is high. This is why NF4 ("NormalFloat" used by QLoRA [138]) defines 16 levels at quantiles of a standard normal (with an exact zero), scaled by a per-block absmax; why float formats (E2M1, E4M3) with exponentially spaced levels help for heavy-tailed data; and why SqueezeLLM learns *per-row*

k-means centroids weighted by Fisher information [175]. The uniform-versus-float comparison for the 4-bit micro-scaled formats is revisited with our simulation in Chapter 11.

3.4.2 Non-uniform grids in practice

Hardware prefers structure. Integer grids give the cheapest multiplier arrays. Non-uniform grids with lookup-table (LUT) dequantization are convenient for weight-only inference because dequantization is a register shuffle [56, 98]; they cost throughput in the compute-bound regime where integer or FP tensor-core operands are needed. Power-of-two levels have been proposed for shift-based arithmetic but are not used in mainstream LLM serving.

3.5 Rounding rules

Round-to-nearest (RTN). The default, $\text{round}(x/s)$, minimizes the elementwise error and is biased only in the sense that it is deterministic: errors are correlated with the input across structured data (e.g. duplicates round identically), and in dot products $\sum_j w_j x_j$ the errors e_j of different weights add up with signs determined by the data.

Stochastic rounding. $\text{SR}(u) = \lfloor u \rfloor + \mathbb{1}[\xi < u - \lfloor u \rfloor]$ with $\xi \sim U[0,1]$ is unbiased: $\mathbb{E}[\text{SR}(u)] = u$, with variance $\{u\}(1-\{u\}) \leq 1/4$ where $\{u\}$ is the fractional part. Variance is up to $3\times$ that of RTN, which has second moment $\leq 1/12$ on uniformly distributed fractional parts versus SR's average $1/6$. It is irreplaceable in training gradients, where bias accumulates, and in 4-bit training recipes it is applied to gradients but not to forward weights [116, 148]; a lower-variance unbiased scheme was proposed in Quartet II [147].

Learned rounding. Rather than nearest, one may choose between $\lfloor u \rfloor$ and $\lceil u \rceil$ to minimize a layer-wise or block-wise reconstruction loss: AdaRound, BRECQ, AdaQuant for convolutional nets [5, 6, 35], and OmniQuant and SignRound (signed-gradient descent on rounding offsets) for LLMs [15, 118, 164]. GPTQ's sequential rounding with error feedback is also, in effect, a learned rounding (Chapter 5).

3.6 The layer-wise objective and the Hessian proxy

Let $\mathcal{L}(\boldsymbol{\theta})$ be the task loss at a trained weight vector $\boldsymbol{\theta}^*$ with approximately zero gradient. A quantization perturbation $\boldsymbol{\delta} = \widehat{\boldsymbol{\theta}} - \boldsymbol{\theta}^*$ changes the loss by

$$\Delta\mathcal{L} \approx \tfrac{1}{2}\boldsymbol{\delta}^\top \boldsymbol{H}_{\mathcal{L}} \boldsymbol{\delta}, \tag{3.10}$$

where $\boldsymbol{H}_{\mathcal{L}}$ is the Hessian of the loss with respect to the weights [2, 215]. Because $\boldsymbol{H}_{\mathcal{L}}$ is intractably large, post-training methods restrict attention to a *layer* and replace the task loss with the squared error of that layer's output on calibration data.

Proposition 3.5 (Layer-wise proxy)**.** *For a linear layer $\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x}$ with calibration inputs $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]$, the squared output error of replacing $\boldsymbol{W}$ by $\widehat{\boldsymbol{W}}$ is*

$$\left\| (\boldsymbol{W} - \widehat{\boldsymbol{W}})\boldsymbol{X} \right\|_F^2 = \sum_{i=1}^{n} (\boldsymbol{w}_i - \hat{\boldsymbol{w}}_i)^\top \boldsymbol{H} (\boldsymbol{w}_i - \hat{\boldsymbol{w}}_i), \qquad \boldsymbol{H} = \boldsymbol{X}\boldsymbol{X}^\top \in \mathbb{R}^{m \times m}, \tag{3.11}$$

with $\boldsymbol{w}_i^\top$ the i-th row of $\boldsymbol{W}$. The problem decouples across rows, and each row has Hessian $2\boldsymbol{H}$ for the loss $\|\boldsymbol{w}^\top \boldsymbol{X} - \hat{\boldsymbol{w}}^\top \boldsymbol{X}\|^2$.

Proof. $\left\|(\boldsymbol{W} - \widehat{\boldsymbol{W}})\boldsymbol{X}\right\|_F^2 = \sum_i \|(\boldsymbol{w}_i - \hat{\boldsymbol{w}}_i)^\top \boldsymbol{X}\|_2^2 = \sum_i (\boldsymbol{w}_i - \hat{\boldsymbol{w}}_i)^\top \boldsymbol{X}\boldsymbol{X}^\top (\boldsymbol{w}_i - \hat{\boldsymbol{w}}_i)$. The second derivative of $\|\boldsymbol{w}^\top \boldsymbol{X} - \boldsymbol{c}^\top\|^2$ in $\boldsymbol{w}$ is $2\boldsymbol{X}\boldsymbol{X}^\top$. □

For a transformer layer, the Gauss–Newton approximation of $\boldsymbol{H}_{\mathcal{L}}$ with respect to a weight matrix $\boldsymbol{W}$ is $\mathbb{E}[\boldsymbol{g}_{\boldsymbol{y}}\boldsymbol{g}_{\boldsymbol{y}}^\top] \otimes \mathbb{E}[\boldsymbol{x}\boldsymbol{x}^\top]$ (a Kronecker-factored form where $\boldsymbol{g}_{\boldsymbol{y}}$ is the output-side gradient), so dropping the output-side factor assumes the sensitivity of the loss to output errors is isotropic and equal across channels [183]. This assumption is the weakest link in the layer-wise proxy; it is partially relaxed by Fisher-weighted objectives such as SqueezeLLM's and by block-wise or end-to-end reconstruction (BRECQ, OmniQuant, EfficientQAT) [35, 46, 118, 175]. The decoupling over rows is exploited by GPTQ to process all rows with one shared inverse Hessian.

3.7 Error propagation through depth

Consider an L-layer residual network $\boldsymbol{h}_{\ell+1} = \boldsymbol{h}_\ell + f_\ell(\boldsymbol{h}_\ell)$ with each f_ℓ κ_ℓ-Lipschitz. If layer ℓ injects an additive perturbation $\boldsymbol{\epsilon}_\ell$ of norm $\leq \eta_\ell$ (the quantization error of that layer's outputs), the final-state perturbation obeys

$$\|\delta \boldsymbol{h}_L\| \leq \sum_{\ell=1}^{L} \eta_\ell \prod_{j>\ell} (1 + \kappa_j). \tag{3.12}$$

Proof. Induct on depth: $\delta \boldsymbol{h}_{\ell+1} = \delta \boldsymbol{h}_\ell + (f_\ell(\boldsymbol{h}_\ell + \delta \boldsymbol{h}_\ell) - f_\ell(\boldsymbol{h}_\ell)) + \boldsymbol{\epsilon}_\ell$, so $\|\delta \boldsymbol{h}_{\ell+1}\| \leq (1 + \kappa_\ell)\,\|\delta \boldsymbol{h}_\ell\| + \eta_\ell$; unrolling gives (3.12). □

The bound is loose (it ignores averaging of independent errors, which makes the root-mean-square error grow like $\sqrt{\sum \eta_\ell^2}$ under independence) but identifies two structural facts: errors injected early are amplified by all later layers, and residual connections prevent the contraction that could dampen errors. Layer-wise methods therefore commonly quantize sequentially, feeding the *quantized* model's activations as inputs to later layers when calibrating, so that later layers can compensate earlier errors [35, 71, 142]; GPTQ with its original ordering uses the full-precision inputs of each layer, a point addressed by asymmetric-calibration successors (Chapter 5). Chapter 13 returns to this topic with more refined bounds.

Takeaways of the chapter

- Gaussian $D(R) = \sigma^2 2^{-2R}$: each bit buys 6.02 dB at best; scalar fixed-rate quantizers lose 3–4 dB at 2–4 bits (own simulation, Table 3.1).
- Clipping range and group size are first-order design variables: group size 128 → 32 at 4 bits gains $\sim$ 1.7 dB for 0.375 bit (Table 3.3).
- Under a Hessian-weighted loss, optimal bit allocation is reverse water-filling over the Hessian spectrum (Proposition 3.2); this is the principle behind activation-aware and sensitivity-based mixed precision.

- The decoupled layer-wise proxy $\left\|(\boldsymbol{W}-\widehat{\boldsymbol{W}})\boldsymbol{X}\right\|_F^2$ is the basis of GPTQ and most PTQ methods; its Kronecker/isotropy assumptions are the main source of the gap to end-to-end training.

Chapter 4

A Taxonomy of LLM Quantization

The literature is large and its vocabulary inconsistent. This chapter fixes a vocabulary and shows where each method discussed later sits. We organize methods along five orthogonal axes: the *target* (what is quantized), the *representation* (grid and scale format), the *fitting procedure* (how parameters are chosen), the *data requirement*, and the *deployment regime* (what hardware executes).

4.1 Axis 1: the target

Weights. Weight-only quantization (W4A16, W3A16, W2A16) stores low-bit weights and dequantizes them on the fly to 16-bit for the matrix multiply. It requires no activation handling and is the easiest to make accurate; its speedup comes only from reduced memory traffic. Methods: RTN, GPTQ, AWQ, SpQR, SqueezeLLM, OmniQuant (weight-only), QuIP/QuIP#, AQLM, QTIP, VPTQ, GPTVQ.

Weights and activations. W8A8, W4A8 and W4A4 quantize both operands of the matmul so that integer or low-precision floating-point tensor cores can be used. Activation quantization is dynamic (computed per token at run-time) or static (calibrated). Methods: LLM.int8(), ZeroQuant, SmoothQuant, OmniQuant, Atom, QServe, QuaRot, SpinQuant, DuQuant, FlatQuant, PrefixQuant, and micro-scaled FP8/FP6/FP4 pipelines.

KV cache. Keys and values stored after each decoding step are quantized to reduce the capacity and the per-step read bandwidth of the attention. Methods: KIVI, KVQuant, GEAR, SKVQ, QAQ, RotateKV, TurboQuant, and rotation variants.

Training-time tensors. Forward weights/activations, backward activation gradients and weight gradients in the GEMMs, optimizer states (8-bit optimizers), and master weights. Methods: FP8 mixed precision (FP8-LM, DeepSeek-V3, COAT), MXFP4/NVFP4 training (Quartet, NVIDIA's NVFP4 recipe), and 8-bit optimizers [38, 43, 63, 116, 120].

Communication. Quantized all-reduce/all-to-all and quantized parameter or gradient sharding. These are outside the scope of this paper except where noted.

Table 4.1: Representation families used in LLM quantization.

Family	Description	Examples
Integer, uniform	Equally spaced levels with scale (and zero-point)	INT8, INT4, INT3, INT2; GPTQ, AWQ, RTN
Integer, non-uniform (LUT)	Per-tensor/group codebook of 2^b scalar levels	NF4, SqueezeLLM, GGUF IQ-quants
Floating point, scalar	Sign/exponent/mantissa grid, tensor- or channel-scaled	FP8 E4M3/E5M2 [61]
Micro-scaled (block) float	Small element format + shared per-block scale	MXFP8/6/4, NVFP4 [114, 206]
Vector / additive	Groups of d weights coded jointly by index into learned codebooks	AQLM, GPTVQ, VPTQ
Lattice / trellis	Structured codes with fast encode/decode	QuIP# (E_8), QTIP (trellis), NestQuant
Ternary / binary	$\{-1, 0, +1\}$ or $\{\pm 1\}$ with scalar scale	BitNet b1.58, BiLLM, OneBit
Mixed / sparse + dense	Few outliers in high precision, bulk in low bits	LLM.int8(), SpQR, SqueezeLLM-Dense&Sparse, OWQ

4.2 Axis 2: the representation

4.3 Axis 3: the fitting procedure

The fitting procedure answers: given the real-valued weights (and possibly data), how are the integer codes, scales, rotations and codebooks chosen?

(a) **Naive rounding (RTN).** Compute scales from the weight statistics and round. No data.

(b) **Scale search/clipping.** Choose scales, clip ratios, or per-channel equivalence factors to minimize a layer- or block-wise output error on calibration data (AWQ, OmniQuant, SmoothQuant, AutoRound).

(c) **Error-compensating rounding (second-order).** Round weights one at a time and update the remaining ones to compensate using $\boldsymbol{H}^{-1}$ (OBS $\rightarrow$ OBQ $\rightarrow$ GPTQ, GPTAQ, Qronos).

(d) **Transform-then-quantize.** Apply an invertible or orthogonal transform (smoothing scales, Hadamard or learned rotation, affine transform) that makes the tensors easier to quantize, then apply any of (a)–(c) (QuIP, QuaRot, SpinQuant, FlatQuant, MR-GPTQ).

(e) **Codebook learning.** Learn codebooks and assignments by k-means, EM or beam search under a Hessian-weighted objective, optionally followed by fine-tuning (AQLM, GPTVQ, QuIP#, QTIP).

(f) **Quantization-aware training (QAT).** Train or fine-tune weights, scales and optionally the full model with simulated quantization in the loop, using straight-through gradients or alternatives (LLM-QAT, EfficientQAT, BitDistiller, ParetoQ, BitNet).

(g) **Native low-precision pretraining.** Train from scratch in a low-precision regime so the model never has a high-precision version (BitNet b1.58 from scratch, FP8/NVFP4

Table 4.2: Data and compute cost of quantization families (orders of magnitude, author's characterization).

Family	Calibration data	Compute for a 70B model	Notes
RTN, data-free	none	seconds to minutes	lowest quality below 4 bits
GPTQ / AWQ	128–1024 sequences of 2K tokens	tens of minutes to hours on one GPU	the production default
Rotation + GPTQ	same, plus rotation fitting (SpinQuant: optimization on a small set)	hours	needs kernel support for online rotations
VQ/trellis PTQ (AQLM, QuIP#, QTIP)	similar calibration, plus fine-tuning	many GPU-hours to days	best at 2 bits
Block/E2E fine-tuning (OmniQuant, EfficientQAT)	10^7–10^9 tokens	hours to days on multiple GPUs	better at 2–3 bits
QAT with distillation	10^9–10^{11} tokens	days to weeks on a cluster	strongest accuracy at $\leq$3 bits
Native pretraining	full pretraining corpus	full pretraining budget	only for model developers

pretraining, native INT4/MXFP4 releases).

4.4 Axis 4: data requirements

4.5 Axis 5: the deployment regime

Which hardware executes the format determines which trade-offs matter. We distinguish (i) *weight-only dequant-in-kernel* regimes, where memory traffic is saved but arithmetic stays 16-bit; (ii) *integer tensor-core* regimes (INT8, INT4) on Ampere/Hopper-class GPUs and many edge NPUs; (iii) *floating-point low-precision* regimes (FP8 on Hopper/Ada/Blackwell, micro-scaled FP4/FP6/FP8 on Blackwell and later); (iv) *CPU/edge* regimes using lookup tables and SIMD integer dot products (llama.cpp GGUF formats, T-MAC) [177, 191]; and (v) *custom accelerators*. The choice constrains which formats are available at all: INT4 tensor-core instructions exist on Turing and Ampere but, to the author's knowledge, were not carried forward to Hopper, which offers INT8 and FP8; MXFP4/NVFP4 tensor-core instructions arrive with Blackwell (Chapter 11).

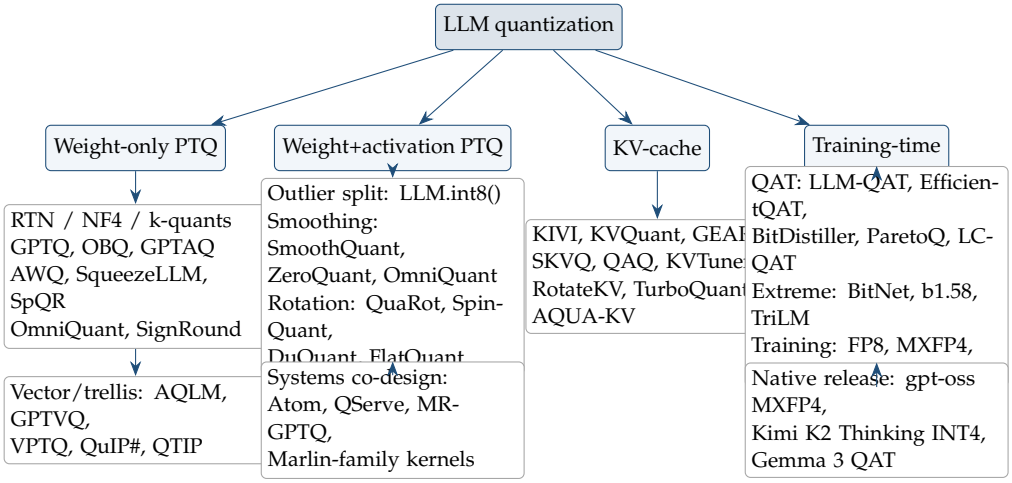


Figure 4.1: Taxonomy by target and representative methods (non-exhaustive). Cross-cutting techniques (mixed-precision allocation, rotations, scale search) appear under several branches.

4.6 A map of the methods

4.6.1 Shorthand for bit-width settings

We write WxAyKVz for x-bit weights, y-bit activations, z-bit KV cache, and append $-gg$ for group size g when relevant (for example W4A16-g128). "Bits" refer to element widths; effective bits include scale overhead (3.7). A *fake-quantization* (simulated) implementation quantizes and dequantizes in floating point to study accuracy; a *real* implementation packs low-bit integers and uses dedicated kernels. Many papers report only fake-quantized accuracy; throughput claims require real kernels (Chapter 14).

How to use the taxonomy

When evaluating a new quantization paper, locate it on the five axes, then ask: (1) which axis does the claimed improvement come from? (2) is the fitting procedure compared against the strongest baseline on the *same* representation and granularity? (3) are bits per weight (including scales) matched? (4) are the claimed speedups produced by real kernels at the stated batch size? Most apparent advances dissolve under these four questions.

Table 4.3: Typical operating points and the first methods to establish them (dates are first arXiv versions). The listing is chronological by first appearance of the setting in the LLM literature.

Setting	Year	Methods establishing it	Typical fidelity claim
W8A8	2022	LLM.int8(), SmoothQuant, ZeroQuant [93, 169, 186]	near-lossless
W4A16 (g128)	2022–23	GPTQ, AWQ [16, 73]	small loss
W3A16	2022–23	GPTQ, SqueezeLLM, SpQR [174, 175]	noticeable loss at small scale
W4A4	2023–24	Atom, QuaRot, SpinQuant [14, 146, 173]	small-to-moderate loss
W2A16	2023–24	QuIP, AQLM, QuIP#, QTIP [10, 144, 149, 150]	moderate loss, strong at large scale
KV 2-bit	2024	KIVI, KVQuant [85, 87]	moderate; task dependent
W1.58A8 (native)	2024	BitNet b1.58 [25]	parity at small scale (authors' claim)
FP8 train	2022–24	FP8 formats, FP8-LM, DeepSeek-V3 [43, 61, 63]	matches BF16
FP4 train	2025	NVFP4, Quartet [116, 148]	close to FP8

Chapter 5

Second-Order Post-Training Quantization: From OBS to GPTQ and Successors

5.1 Round-to-nearest as the baseline

Round-to-nearest (RTN) with group-wise scales (Algorithm 1) is the simplest post-training quantization (PTQ) method: no data, no iteration, seconds of compute. Its loss at 4 bits with group size 128 is modest for models above a few billion parameters, and RTN-based formats are the backbone of consumer inference tools: bitsandbytes' NF4 and 8-bit paths, and the GGUF `Qn_0`/`Qn_K` families in llama.cpp [138, 191, 194, 212]. Below 4 bits RTN collapses. In the original GPTQ evaluation, 3-bit RTN on OPT-175B reached a WikiText-2 perplexity in the thousands (approximately 7×10^3, from the GPTQ paper's table as the author recalls it) versus a perplexity of about 8.7 for GPTQ, while at 4 bits RTN degraded from roughly 8.3 to 10.5 and GPTQ stayed within 0.1 [73]. These numbers are quoted as approximate. The remainder of this chapter explains why second-order information changes the picture so dramatically.

5.2 Optimal Brain Surgeon

Optimal Brain Damage and Optimal Brain Surgeon (OBS) were introduced for pruning neural networks [2, 215]. OBS asks: given a trained weight vector $\boldsymbol{w}$ at a minimum of the loss, which single weight should be removed (set to zero) and how should the other weights be adjusted to minimize the increase in loss? Using the second-order model $\Delta\mathcal{L} = \frac{1}{2}\boldsymbol{\delta}^\top \boldsymbol{H}\boldsymbol{\delta}$ and the constraint that component q of $\boldsymbol{w} + \boldsymbol{\delta}$ be zero, $\boldsymbol{e}_q^\top \boldsymbol{\delta} + w_q = 0$:

Theorem 5.1 (Optimal Brain Surgeon). *Let $\boldsymbol{H} \succ 0$. The minimizer of $\frac{1}{2}\boldsymbol{\delta}^\top \boldsymbol{H}\boldsymbol{\delta}$ subject to $\boldsymbol{e}_q^\top \boldsymbol{\delta} = -w_q$ is*

$$\boldsymbol{\delta}^* = -\frac{w_q}{[\boldsymbol{H}^{-1}]_{qq}} \boldsymbol{H}^{-1}\boldsymbol{e}_q, \qquad \Delta\mathcal{L}^* = \frac{w_q^2}{2\,[\boldsymbol{H}^{-1}]_{qq}}. \tag{5.1}$$

Proof. Form the Lagrangian $\frac{1}{2}\boldsymbol{\delta}^\top \boldsymbol{H}\boldsymbol{\delta} + \mu(\boldsymbol{e}_q^\top \boldsymbol{\delta} + w_q)$. Stationarity gives $\boldsymbol{H}\boldsymbol{\delta} = -\mu \boldsymbol{e}_q$, i.e. $\boldsymbol{\delta} = -\mu \boldsymbol{H}^{-1}\boldsymbol{e}_q$. The constraint then reads $-\mu[\boldsymbol{H}^{-1}]_{qq} + w_q = 0$, so $\mu = w_q / [\boldsymbol{H}^{-1}]_{qq}$. Substituting, $\frac{1}{2}\boldsymbol{\delta}^\top \boldsymbol{H}\boldsymbol{\delta} = \frac{1}{2}\mu^2 \boldsymbol{e}_q^\top \boldsymbol{H}^{-1}\boldsymbol{e}_q = \frac{1}{2} w_q^2 / [\boldsymbol{H}^{-1}]_{qq}$. □

The *saliency* $w_q^2 / (2[\boldsymbol{H}^{-1}]_{qq})$ ranks weights: those with small diagonal entries of $\boldsymbol{H}^{-1}$ in the pivot position are the costliest to remove. The update direction $\boldsymbol{H}^{-1}\boldsymbol{e}_q$ distributes the correction over correlated weights.

From pruning to quantization. Replace "set w_q to zero" by "set w_q to quant(w_q)". The constraint becomes $\boldsymbol{e}_q^\top \boldsymbol{\delta} = \text{quant}(w_q) - w_q$, and the same derivation gives

$$\boldsymbol{\delta}^* = -\frac{w_q - \text{quant}(w_q)}{[\boldsymbol{H}^{-1}]_{qq}} \boldsymbol{H}^{-1}\boldsymbol{e}_q, \qquad \Delta\mathcal{L}^* = \frac{(w_q - \text{quant}(w_q))^2}{2[\boldsymbol{H}^{-1}]_{qq}}. \tag{5.2}$$

This is the core of Optimal Brain Quantization (OBQ), part of the Optimal Brain Compression framework, which applies OBS repeatedly: pick the weight with smallest $\Delta\mathcal{L}^*$ (greedy), quantize it, apply the update to the not-yet-quantized weights, delete the pivot row and column from $\boldsymbol{H}^{-1}$ by Gaussian elimination, and repeat [117]. For the layer-wise squared loss of Proposition 3.5 each row of $\boldsymbol{W}$ has the same Hessian $\boldsymbol{H} = \boldsymbol{X}\boldsymbol{X}^\top$ (up to a factor), but the greedy order differs per row, so OBQ costs $O(nm^3)$ for an $n \times m$ matrix, which is prohibitive for $m \approx 10^4$ and $n \approx 10^4$ in LLMs.

5.3 GPTQ

5.3.1 Three simplifications

GPTQ [73] achieves a 10^4-fold speedup over OBQ through three observations.

(1) Arbitrary order. Quantizing coordinates in a fixed (e.g. left-to-right) order, identical for all rows, loses little accuracy relative to greedy order, since the greedy rule tends to leave the hard weights for last when few free coordinates remain to absorb error, while a fixed order spreads error roughly evenly. Fixed order means the update matrix can be shared across all n rows, and $\boldsymbol{H}^{-1}$ need only be updated once per column, giving $O(m^3 + nm^2)$.

(2) Lazy batch updates. The update (5.2) touches all remaining columns after each quantized column: a rank-one update with arithmetic intensity ≈ 1 (memory-bound). Grouping $B = 128$ columns, applying updates *within* the block immediately and to the remaining columns once per block turns the dominant work into matrix–matrix products.

(3) Cholesky reformulation. Repeated Gaussian elimination of $\boldsymbol{H}^{-1}$ accumulates numerical error. GPTQ instead precomputes the Cholesky factor of $\boldsymbol{H}^{-1}$ once, whose rows contain exactly the information needed.

Lemma 5.1 (Cholesky rows are normalized Schur complements)**.** *Let $\boldsymbol{A} = \boldsymbol{H}^{-1} \succ 0$ have the Cholesky factorization $\boldsymbol{A} = \boldsymbol{U}^\top\boldsymbol{U}$ with $\boldsymbol{U}$ upper triangular with positive diagonal. Let $\boldsymbol{A}^{(q)}$ be the matrix obtained from $\boldsymbol{A}$ by eliminating coordinates $1, \dots, q-1$ via successive Schur complements (so $\boldsymbol{A}^{(q)}$ is the inverse of the Hessian restricted to coordinates $q, \dots, m$). Then for $j \geq q$,*

$$\frac{A^{(q)}_{qj}}{A^{(q)}_{qq}} = \frac{U_{qj}}{U_{qq}}. \tag{5.3}$$

Proof. Write $\boldsymbol{A} = \boldsymbol{U}^\top\boldsymbol{U}$ in block form after the first $q-1$ rows. Cholesky factorization is the symmetric Gaussian elimination: step k divides row k of the current Schur complement by the square root of its pivot, so $U_{kj} = A^{(k)}_{kj} / \sqrt{A^{(k)}_{kk}}$ for $j \geq k$, and the next Schur complement is $\boldsymbol{A}^{(k+1)} = \boldsymbol{A}^{(k)}_{k+1:,k+1:} - \boldsymbol{u}_k\boldsymbol{u}_k^\top$ with $\boldsymbol{u}_k = U^\top_{k,k+1:}$. Taking $k = q$: $U_{qj}/U_{qq} = A^{(q)}_{qj}/A^{(q)}_{qq}$ for all $j \geq q$. □

Algorithm 2 GPTQ with lazy batch updates, static groups and damping

Require: $\boldsymbol{W} \in \mathbb{R}^{n\times m}$, calibration activations $\boldsymbol{X} \in \mathbb{R}^{m\times N}$, bits b, group size g, block size B, damping λ

$\boldsymbol{H} \leftarrow 2\boldsymbol{X}\boldsymbol{X}^\top$ ▷ accumulated over calibration batches
$\boldsymbol{H} \leftarrow \boldsymbol{H} + \lambda\,\mathrm{mean}(\mathrm{diag}\,\boldsymbol{H})\,\boldsymbol{I}$ ▷ typically $\lambda = 0.01$
$\boldsymbol{U} \leftarrow \mathrm{chol}(\boldsymbol{H}^{-1})^\top$ ▷ upper-triangular Cholesky factor of the inverse
$\boldsymbol{Q} \leftarrow \boldsymbol{0}_{n\times m}$; $\boldsymbol{E} \leftarrow \boldsymbol{0}_{n\times B}$
for $i = 0, B, 2B, \ldots$ **do**
  for $j = i, \ldots, i + B - 1$ **do**
    if $j \bmod g = 0$ **then**
      compute scales $\boldsymbol{s}$ (and zero-points) from $\boldsymbol{W}_{:,j:j+g}$
    end if
    $\boldsymbol{Q}_{:,j} \leftarrow \mathcal{Q}_{\boldsymbol{s}}(\boldsymbol{W}_{:,j})$ ▷ round to the grid
    $\boldsymbol{e} \leftarrow (\boldsymbol{W}_{:,j} - \boldsymbol{Q}_{:,j})/U_{jj}$
    $\boldsymbol{W}_{:,j+1:i+B} \leftarrow \boldsymbol{W}_{:,j+1:i+B} - \boldsymbol{e}\,U_{j,j+1:i+B}$ ▷ in-block update
    $\boldsymbol{E}_{:,j-i} \leftarrow \boldsymbol{e}$
  end for
  $\boldsymbol{W}_{:,i+B:} \leftarrow \boldsymbol{W}_{:,i+B:} - \boldsymbol{E}\,U_{i:i+B,\,i+B:}$ ▷ lazy update of remaining columns
end for
return $\boldsymbol{Q}$ and scales

Combining (5.2) with Lemma 5.1 yields the compact GPTQ update. When column q of $\boldsymbol{W}$ (all rows at once) is quantized, with $\boldsymbol{w}_{:,q}$ the current column and $\boldsymbol{q}_{:,q}$ its quantized value, the remaining columns are updated by

$$\boldsymbol{W}_{:,\,q+1:} \;\leftarrow\; \boldsymbol{W}_{:,\,q+1:} - \frac{\boldsymbol{w}_{:,q} - \boldsymbol{q}_{:,q}}{U_{qq}}\,U_{q,\,q+1:}. \tag{5.4}$$

Proposition 5.1 (Optimality of the GPTQ step)**.** *Given the already-fixed coordinates* $1, \ldots, q-1$, (5.4) *is the minimizer of the quadratic proxy loss over the free coordinates* $q+1, \ldots, m$ *subject to coordinate* q *taking the value* $\boldsymbol{q}_{:,q}$*.*

Proof. Restrict the problem to coordinates $q, \ldots, m$; its Hessian is $\boldsymbol{H}^{(q)} = (\boldsymbol{A}^{(q)})^{-1}$ (the inverse of a Schur complement of the inverse equals the Hessian restricted to the unconstrained block after marginalizing the frozen coordinates). Apply Theorem 5.1 with pivot q: $\boldsymbol{\delta} = -\frac{w_q - \hat{w}_q}{A^{(q)}_{qq}}\boldsymbol{A}^{(q)}_{:,q}$. By Lemma 5.1 the normalization by $A^{(q)}_{qq}$ and the row of $\boldsymbol{A}^{(q)}$ equal those expressed with $\boldsymbol{U}$: $\boldsymbol{\delta}_j = -(w_q - \hat{w}_q)U_{qj}/U_{qq}$ for $j > q$ (the $j = q$ component is the rounding of w_q itself). □

Since the loss is quadratic, *greedy per-step optimality* makes each step optimal given the earlier decisions but not jointly optimal: the rounding of w_q to its nearest grid point is a heuristic that ignores the effect on later coordinates. QuIP shows (Chapter 7) that GPTQ is equivalent to a special case of an adaptive-rounding procedure (LDLQ) with a worst-case bound that depends on the incoherence of the Hessian [150].

5.3.2 The algorithm

Complexity and memory. Computing $\boldsymbol{H}$ costs $O(Nm^2)$ and is accumulated online. Inverting and Cholesky-factoring $\boldsymbol{H}$ costs $O(m^3)$. The column loop costs $O(nm^2)$. For $m = 8192$ the factorization takes seconds on a modern GPU, and a whole 70B model quantizes in tens of minutes to a few hours on a single GPU, with memory dominated by one layer's $\boldsymbol{H}$ and weights; this practicality, rather than any claim of optimality, is the reason GPTQ became ubiquitous.

Damping. $\boldsymbol{H}$ may be rank-deficient when $N < m$ (few calibration tokens) or contain dead inputs ($\operatorname{diag} \boldsymbol{H} = 0$). Adding $\lambda \operatorname{mean}(\operatorname{diag} \boldsymbol{H})\boldsymbol{I}$ regularizes the inverse; the damping can be interpreted as a Bayesian prior that the weights should not move too far in directions unseen in calibration, and as a trade-off between proxy-loss optimality and robustness to calibration-set shift (Chapter 18).

Act-order and groups. Quantizing columns in order of decreasing $\boldsymbol{H}_{jj}$ ("act-order" or "desc_act") handles high-energy input channels first, when most free coordinates remain to compensate. This improves perplexity but, with grouped scales, makes the group membership non-contiguous in the original order, which complicates kernels (one needs a permutation index, or static groups computed before reordering). Many deployments therefore use either act-order without groups or groups without act-order, or fold the permutation into the preceding layer.

Sequential versus parallel calibration. The original GPTQ quantizes a transformer block sub-layer by sub-layer using activations propagated through the already-quantized layers within the block, but each block's input comes from the quantized preceding blocks only in the sequential variant; implementations differ. Asymmetric successors (below) make the treatment explicit.

5.3.3 Why the gain is so large: a controlled simulation

Table 5.1 reports our own synthetic experiment. A 256×512 weight matrix with i.i.d. Gaussian entries is quantized with per-group-128 symmetric absmax scales; the calibration activations $\boldsymbol{X} \in \mathbb{R}^{8192\times 512}$ come from a 64-dimensional latent factor model plus noise and six outlier channels with scale 15–30 (so the Hessian is highly anisotropic, as in real transformer layers). The metric is the relative output error $\left\|(\boldsymbol{W} - \widehat{\boldsymbol{W}})\boldsymbol{X}^\top\right\|_F^2 / \left\|\boldsymbol{W}\boldsymbol{X}^\top\right\|_F^2$.

The mechanism is visible in (5.4): errors incurred on a weight whose input correlates with later inputs are pushed onto those later weights, which have the freedom to cancel it on the calibration distribution. When $\boldsymbol{H}$ is close to the identity (uncorrelated inputs), there is nothing to exploit and GPTQ degenerates to RTN. Real LLM layers lie in between, so GPTQ's practical gains over RTN at 4 bits are modest in perplexity but decisive at 3 bits and below.

Example 5.1 (Two coordinates)**.** Take two weights $\boldsymbol{w} = (0.44, 0.44)$ with Hessian $\boldsymbol{H} = \left(\begin{smallmatrix} 1 & 0.9 \\ 0.9 & 1 \end{smallmatrix}\right)$ and a uniform grid of step 0.1. Round-to-nearest maps both weights to 0.4, so $\boldsymbol{\delta} = (-0.04, -0.04)$ and the loss is $\boldsymbol{\delta}^\top \boldsymbol{H}\boldsymbol{\delta} = 0.0016\,(1 + 1 + 1.8) = 0.00608$ (the errors are positively

Table 5.1: Relative layer output error of 4-, 3- and 2-bit group-wise ($g = 128$) weight quantization on a synthetic layer (own simulation; single run, seed 7). The calibration data are strongly correlated and low-rank-plus-noise, which favors second-order compensation more than typical real layers do; the *ordering* of methods is the lesson, not the magnitude of the gap.

Bits	RTN	GPTQ	RTN / GPTQ
4	1.26×10^{-2}	4.4×10^{-4}	29×
3	6.84×10^{-2}	2.63×10^{-3}	26×
2	5.46×10^{-1}	4.28×10^{-2}	13×

correlated and the Hessian couples them, so they add). Here $\boldsymbol{H}^{-1} = \frac{1}{0.19}\left(\begin{smallmatrix} 1 & -0.9 \\ -0.9 & 1 \end{smallmatrix}\right)$, hence $[\boldsymbol{H}^{-1}]_{21}/[\boldsymbol{H}^{-1}]_{11} = -0.9$. GPTQ quantizes the first weight to 0.4 (residual $w_1 - q_1 = 0.04$) and updates the second by $-0.04 \times (-0.9) = +0.036$, giving $w_2 = 0.476$, which rounds to 0.5. The final error is $\boldsymbol{\delta} = (-0.04, +0.06)$ and the loss is $0.0016 + 0.0036 - 2(0.9)(0.04)(0.06) = 0.00088$, a 6.9× reduction: the second weight absorbed the first one's error because their inputs are highly correlated. If the grid were coarse (step 0.4) the second weight, moved to 0.476, would round back to 0.4 and nothing would be gained: compensation helps only when the remaining coordinates have enough resolution to express it, which is why GPTQ's advantage grows with the bit-width and shrinks at ≤ 2 bits.

5.4 Successors and variants

5.4.1 Asymmetric calibration: GPTAQ

GPTQ minimizes $\left\|\boldsymbol{W}\boldsymbol{X} - \widehat{\boldsymbol{W}}\boldsymbol{X}\right\|_F^2$ with the *same* $\boldsymbol{X}$ on both sides. After earlier layers are quantized, the input to the current layer in the quantized model is $\widetilde{\boldsymbol{X}} \neq \boldsymbol{X}$; a more faithful objective matches the full-precision *output*, $\left\|\boldsymbol{W}\boldsymbol{X} - \widehat{\boldsymbol{W}}\widetilde{\boldsymbol{X}}\right\|_F^2$. GPTAQ derives an efficient finetuning-free closed-form update for this asymmetric objective, reusing the machinery above [71]. The general principle (calibrate against the true full-precision output with the actual quantized inputs) recurs in block-wise methods [35, 118].

5.4.2 Error-correcting variants

Qronos revisits correcting accumulated quantization error from previously quantized layers while shaping the treatment of coordinates not yet quantized [142]. We describe it only at the level of its title and abstract; readers should consult the paper for the precise algorithm.

5.4.3 Preconditioning the weights

MagR minimizes the ℓ_∞ norm of the weights, subject to preserving the layer output, before quantization, thereby reducing the per-group range (and hence the step) without changing the function [99]. This composes with GPTQ and is conceptually related to the clipping/range-reduction logic of Section 3.2.

5.4.4 Gradient-based rounding search: SignRound and AutoRound

SignRound/AutoRound optimizes rounding offsets $\boldsymbol{v} \in [-\frac{1}{2}, \frac{1}{2}]$ and clip parameters per tensor with signed gradient descent on a block-wise reconstruction loss, using only a few hundred steps per block [15]. SignRoundV2 (December 2025) adds a layer-wise adaptive mixed-precision assignment using gradient information and quantization error, and stabilization steps; it reports near-lossless mixed-MXFP configurations with a gap of about 1% at an average of 4.5 bits and improved 2-bit weight-only results [164]. These methods are the main alternative to GPTQ-style compensation in the Intel Neural Compressor/AutoRound ecosystem.

5.4.5 Format-aware GPTQ: MR-GPTQ

For micro-scaled 4-bit formats, MR-GPTQ combines block-wise Hadamard rotation with GPTQ, choosing format-specific scale selection to cope with NVFP4's small block and MXFP4's power-of-two scales, and ships kernels fusing the rotation [110]. We return to it in Chapter 11.

5.4.6 Targeted weaknesses

D^{2}Quant observes that down-projection matrices (the MLP output projection) are a consistent weak point of sub-4-bit PTQ and that weight quantization shifts activation distributions; it proposes a dual-scale quantizer for down-projections and a LayerNorm mean-shift correction [40]. Similar findings of sensitive layers motivate mixed precision (Chapter 9).

5.4.7 Pruning and quantization share the machinery

The same OBS update drives SparseGPT, which prunes to 50–60% sparsity in one shot with GPTQ-like column sweeps [171]; joint sparsification and quantization are discussed in the companion papers. WoodFisher gave the first scalable approximations of the empirical Fisher inverse used in OBS-style compression [183].

5.5 Sensitivity to calibration data

GPTQ and its relatives fit the quantized weights to a small sample of activations. Results depend on the sample: the number of sequences (128 is conventional), their length (2048 tokens), and their distribution (C4 web text is the common default; WikiText-2 is sometimes used and can inflate WikiText perplexity numbers while hurting other domains). A systematic study of calibration data in post-training quantization and pruning found that the choice can change downstream results and that no single dataset dominates [36]. In practice one should calibrate on data resembling the deployment distribution (including code, multilingual text and chat templates if relevant) and report the calibration set.

Risk: calibration overfitting

Second-order PTQ minimizes error *on the calibration activations*. If the deployment inputs excite directions of $\boldsymbol{H}$ with tiny calibration eigenvalues, the compensated weights can be unusually poor there. Symptoms: perplexity on the calibration domain improves while

Table 5.2: Approximate WikiText-2 perplexity (sequence length 2048) of Llama-2-7B under group-128 weight-only quantization, as commonly reported in the AWQ and GPTQ-line literature. FP16 is ≈ 5.47. These are the author's recollection of published table entries and are provided only to convey ordering and rough magnitude; the third decimal is not meaningful and values vary between implementations [16, 73, 118].

Method (g128)	W4	W3	W2
FP16		5.47	
RTN	≈ 5.73	≈ 6.66	$\gg 10^2$
GPTQ	≈ 5.69	≈ 6.43	$\gg 10^2$
AWQ	≈ 5.60	≈ 6.24	$\gg 10^2$
OmniQuant (learned clipping + equivalent transform)	$\approx 5.74 - 5.8$	$\approx 6.5 - 6.6$	$\approx$ 9–11 (reported at W2A16, g64–g128)

out-of-domain perplexity or long-context behaviour degrades. Mitigations: more and more diverse calibration tokens, stronger damping, evaluation on out-of-domain data, and (for safety-critical use) re-evaluating alignment behaviour after quantization.

5.6 Reported results (approximate)

Takeaways of the chapter

- OBS gives the optimal single-coordinate compensation under a quadratic loss (Theorem 5.1); GPTQ makes it practical via fixed order, lazy batching and the Cholesky identity (Lemma 5.1).
- The gain of compensation over RTN is large when activation covariance is strongly correlated (our simulation: 13–29$\times$ lower layer error), small when it is near-identity.
- Successors modify the objective (asymmetric calibration), the scale and rounding search (SignRound), the weight preconditioning (MagR) or the format (MR-GPTQ) but keep the sequential error-feedback skeleton.
- Calibration data are a hidden hyperparameter; report them.

Chapter 6

Outliers, Smoothing, and Activation-Aware Quantization

Weight quantization is a problem of compressing a static tensor with known statistics. Activation quantization is harder because activations are data-dependent and exhibit *outliers*: a few channels and tokens whose magnitudes dwarf the rest. This chapter describes the outlier phenomenon, the three families of remedies (mixed-precision decomposition, equivalent re-scaling, and activation-aware weight scaling), and the machinery of integer matrix multiplication that constrains them.

6.1 The outlier phenomenon

Emergent feature outliers. Dettmers et al. observed that once transformer language models exceed roughly 6.7 billion parameters, a small set of hidden-state dimensions develops systematic large-magnitude values that are present across most layers and most tokens; naive 8-bit quantization of these activations destroys performance, whereas decomposing them away preserves it [93]. The phenomenon was seen earlier in smaller BERT-type models as "outlier dimensions" [18], and tied to LayerNorm and residual structure [34].

Massive activations and attention sinks. Sun et al. identified *massive activations*: a handful of scalar activations, at specific feature dimensions and specific tokens (typically the first token and some delimiter tokens), whose magnitude exceeds the median by up to four to five orders of magnitude, nearly constant across inputs, and functioning like implicit bias terms [101]. They are closely related to *attention sinks*: tokens that attract disproportionate attention regardless of content [165]. Bondarenko et al. argued that such outliers emerge because attention heads learn to "do nothing" by placing mass on low-information tokens while requiring a near-zero update, which in a softmax attention requires extremely large pre-softmax logits or values, and they proposed architectural fixes (clipped softmax and gated attention) that remove outliers at training time [145].

Consequences. Outliers affect quantization in three ways. (1) A per-tensor scale is set by the outlier, leaving the bulk with only a few effective levels. (2) A per-token scale is robust for tokens without massive activations but wastes range for those with them. (3) Quantization noise on a massive activation is large in absolute terms and, because those values act as biases and attention keys for sinks, can corrupt the model's attention pattern more than the noise on the bulk; IntactKV and PrefixQuant respectively keep the key–value entries of pivot tokens in full precision and prepend fixed tokens that absorb massive activations [80, 131].

Where outliers live

- **Channels**: fixed hidden dimensions with large values for most tokens (the LLM.int8() features); they dominate per-tensor ranges. A per-*channel* activation scale would fix them but cannot be used in an integer GEMM (Proposition 6.1).
- **Tokens**: a few special positions with massive activations; per-token dynamic scaling handles them if applied before the matmul.
- **Layers**: the inputs of the MLP down-projection and the attention output projection are the worst offenders in many models; D^2Quant also finds down-projections to be a main weak point of weight-only quantization [40].

6.2 Integer matrix multiplication and why scales must factor

Let $\boldsymbol{X} \in \mathbb{R}^{T\times m}$ (tokens by input channels) and $\boldsymbol{W} \in \mathbb{R}^{n\times m}$. Suppose activations are quantized with a scale per token, s_t^x, and weights with a scale per output channel, s_i^w: $X_{tj} \approx s_t^x q_{tj}^x$, $W_{ij} \approx s_i^w q_{ij}^w$ with integer q. Then

$$Y_{ti} = \sum_j X_{tj} W_{ij} \approx s_t^x \, s_i^w \sum_j q_{tj}^x q_{ij}^w, \tag{6.1}$$

so the inner sum is an integer GEMM accumulated in INT32, and the scales are applied once per output element in the epilogue.

Proposition 6.1 (Which scale granularities factor out of the dot product)**.** *A scale that depends on the* reduction index *j (a per-input-channel scale for activations or weights) cannot be factored out of $\sum_j(\cdot)$ and therefore prevents the use of a single low-precision integer accumulation. Scales that depend only on the output indices (t, i) factor out. Group scales along j factor out only within each group, so the GEMM must be split into m/g partial integer sums, each rescaled, which costs extra epilogue work.*

Proof. If $X_{tj} = s_j q_{tj}$ then $\sum_j X_{tj} W_{ij} = \sum_j s_j q_{tj} W_{ij}$ with s_j varying inside the sum; no scalar multiple can be pulled out. A scale constant over a group $\mathcal{G}_k$ gives $\sum_k s_k \sum_{j\in\mathcal{G}_k} q_{tj} q'_{ij}$. □

This fact structures the whole subject: outlier *channels* are the problem, but the natural fix (per-channel scales) is not computable by integer hardware. Remedies therefore either (i) move the scale into a place where it *can* be absorbed (SmoothQuant moves it into the weights and the previous normalization layer), (ii) handle outlier channels separately (LLM.int8(), Atom), or (iii) remove the channel structure with an orthogonal transform (Chapter 7). Micro-scaled formats solve it with small groups along j (block sizes of 16 or 32) and hardware that applies group scales inside the tensor-core (Chapter 11).

6.3 LLM.int8(): vector-wise quantization and mixed-precision decomposition

LLM.int8() [93] combines two ideas.

Vector-wise quantization. Use (6.1) with per-token activation scales and per-output-channel weight scales: each inner product has its own normalization constants.

Mixed-precision decomposition. Identify outlier feature dimensions $\mathcal{O} = \{j : \max_t |X_{tj}| \geq \tau\}$ with threshold $\tau = 6$; compute $\boldsymbol{X}_{:,\mathcal{O}}\boldsymbol{W}_{:,\mathcal{O}}^\top$ in 16-bit and the rest in INT8:

$$\boldsymbol{Y} = \boldsymbol{X}_{:,\mathcal{O}}\boldsymbol{W}_{:,\mathcal{O}}^\top + \boldsymbol{X}_{:,\overline{\mathcal{O}}}\boldsymbol{W}_{:,\overline{\mathcal{O}}}^\top \text{ (INT8, dequantized).} \tag{6.2}$$

Fewer than about 0.1% of input dimensions are outliers at the scales studied, so the 16-bit part is cheap in flops, though the scatter/gather and the two-kernel structure make LLM.int8() *slow* in practice compared with 16-bit for small models, a point repeatedly noted in practice: the method was designed for memory reduction (fitting OPT-175B on fewer GPUs) rather than speed [93, 212].

6.4 SmoothQuant: equivalent per-channel rescaling

SmoothQuant [169] makes W8A8 (and with extensions lower) integer inference efficient by migrating quantization difficulty from activations to weights using an *equivalent transformation*.

Proposition 6.2 (Equivalence)**.** *For any positive diagonal* $\boldsymbol{S} = \mathrm{diag}(s_1, \ldots, s_m)$, $\boldsymbol{Y} = \boldsymbol{X}\boldsymbol{W}^\top = (\boldsymbol{X}\boldsymbol{S}^{-1})(\boldsymbol{W}\boldsymbol{S})^\top$.

Choose $\boldsymbol{s}$ so that $\hat{\boldsymbol{X}} = \boldsymbol{X}\boldsymbol{S}^{-1}$ has milder channels. Define channel statistics $a_j = \max_t |X_{tj}|$ and $w_j = \max_i |W_{ij}|$ and set

$$s_j = \frac{a_j^\alpha}{w_j^{1-\alpha}}, \qquad \alpha \in [0, 1]. \tag{6.3}$$

The smoothed channel maxima are then $a_j/s_j = a_j^{1-\alpha} w_j^{1-\alpha}$ for the activations and $w_j s_j = a_j^\alpha w_j^\alpha$ for the weights.

Proposition 6.3 (Range equalization at $\alpha = \frac{1}{2}$)**.** *With* $\alpha = \frac{1}{2}$, $\max_t |\hat{X}_{tj}| = \max_i |\hat{W}_{ij}| = \sqrt{a_j w_j}$ *for every channel* j.

Proof. At $\alpha = \frac{1}{2}$, $s_j = \sqrt{a_j / w_j}$; thus $a_j/s_j = \sqrt{a_j w_j}$ and $w_j s_j = \sqrt{a_j w_j}$. □

Hence both operands have equal range in every channel and the dynamic range across channels in $\hat{\boldsymbol{X}}$ shrinks from $\max_j a_j / \min_j a_j$ to its square root (at $\alpha = \frac{1}{2}$ and ignoring w_j variation). Larger α moves more difficulty to the weights; models with severe activation outliers need a larger α, and the original paper tunes it per model.

Implementation. $\boldsymbol{S}^{-1}$ is folded into the preceding LayerNorm/RMSNorm weight (for QKV and the first MLP projections), and $\boldsymbol{S}$ into the weights offline. For the attention output projection and MLP down-projection, whose inputs are not directly produced by a normalization layer, SmoothQuant keeps those in per-token dynamic quantization or applies the scale through the preceding linear layer where possible. Three efficiency levels (O1: per-token dynamic activation and per-channel weight; O2: per-tensor dynamic; O3: per-tensor static) trade accuracy for speed. Reported results show W8A8 close to FP16 for OPT, BLOOM and later Llama-family models up to hundreds of billions of parameters [169].

Algorithm 3 SmoothQuant calibration and folding for one linear layer fed by a norm layer

Require: calibration activations $\boldsymbol{X}$ (tokens $\times$ m), weights $\boldsymbol{W} \in \mathbb{R}^{n \times m}$, migration strength α, norm weight $\boldsymbol{\gamma}$

1: $a_j \leftarrow \max_t |X_{tj}|$; $w_j \leftarrow \max_i |W_{ij}|$ for $j = 1..m$
2: $s_j \leftarrow a_j^{\alpha} / w_j^{1-\alpha}$ (clamped to a sane range)
3: $\boldsymbol{W} \leftarrow \boldsymbol{W}\ \mathrm{diag}(\boldsymbol{s})$ ▷ scale input columns of the weight
4: $\boldsymbol{\gamma} \leftarrow \boldsymbol{\gamma} \oslash \boldsymbol{s}$; $\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} \oslash \boldsymbol{s}$ ▷ fold $\boldsymbol{S}^{-1}$ into the preceding norm
5: quantize $\boldsymbol{W}$ per output channel to INT8; at run-time quantize activations per token (or per tensor) to INT8

Limits. Smoothing is a per-channel *diagonal* transform; it equalizes channel maxima but cannot remove outliers that are token-specific (massive activations), cannot make W4A4 accurate (the weights become harder), and the optimal α is layer-dependent. OmniQuant learns the smoothing scales (and shifts) by gradient descent on a block-wise loss, and AffineQuant extends to full affine matrices [7, 118]; SmoothQuant+ extends the idea to 4-bit weight quantization [168]. Outlier Suppression is an earlier line of work on suppressing outliers in transformer language models by modifying normalization and clipping [124].

6.5 Other activation-quantization designs

ZeroQuant and ZeroQuant-V2. ZeroQuant combines fine-grained group-wise weight quantization with token-wise dynamic activation quantization and layer-by-layer knowledge distillation, with kernel fusion to hide quantization overhead [186]; ZeroQuant-V2 studies post-training quantization comprehensively across scales and proposes low-rank compensation [188]; ZeroQuant-FP explores W4A8 with floating-point formats [187].

RPTQ. Reorder-based quantization groups channels with similar activation ranges so that each cluster gets its own scale [159], an early approach to the channel-heterogeneity problem that reappears as "channel reordering" in Atom.

Atom. Atom targets W4A4 with three ingredients: keep a small number of outlier channels in INT8 (found by calibration and moved to the end by reordering), use group quantization (e.g. $g = 128$) for the rest, and quantize the KV cache to 4 bits, with dedicated kernels [14]. It is a mixed-precision decomposition in the spirit of LLM.int8() at 4 bits, trading kernel complexity for accuracy.

QServe and QQQ. QServe's W4A8KV4 uses "progressive" two-level weight quantization (4-bit weights with INT8 intermediate scales so that dequantization to INT8 happens in registers and the main GEMM runs on INT8 tensor cores), plus SmoothAttention for the KV cache, and emphasizes system co-design on NVIDIA GPUs [143]. QQQ similarly pairs 4-bit weights with 8-bit activations with Hessian-based adaptive smoothing and dedicated kernels [140].

Learned transforms. DuQuant uses block-diagonal rotations and permutations to distribute outliers, FlatQuant learns per-layer affine transforms (Kronecker-factored) to flatten distribu-

Algorithm 4 AWQ scale search for one linear layer (weight-only, group quantization)

Require: $\boldsymbol{W} \in \mathbb{R}^{n\times m}$, calibration activations $\boldsymbol{X}$, bits b, group size g, grid $\{\alpha_k\}_{k=1}^{20} \subset [0,1]$

$\bar{a}_j \leftarrow \text{mean}_t |X_{tj}|$

for $k = 1..20$ **do**

  $\sigma_j \leftarrow \bar{a}_j^{\alpha_k}$; normalize $\boldsymbol{\sigma} \leftarrow \boldsymbol{\sigma} / \sqrt{\max \boldsymbol{\sigma} \min \boldsymbol{\sigma}}$

  $\widehat{\boldsymbol{W}}_k \leftarrow \mathcal{Q}_{b,g}(\boldsymbol{W} \operatorname{diag} \boldsymbol{\sigma}) \operatorname{diag}(\boldsymbol{\sigma})^{-1}$

  $\ell_k \leftarrow \left\| \boldsymbol{W}\boldsymbol{X}^\top - \widehat{\boldsymbol{W}}_k \boldsymbol{X}^\top \right\|_F^2$

end for

$k^* \leftarrow \arg\min_k \ell_k$; fold $\operatorname{diag}(\boldsymbol{\sigma}_{k^*})^{-1}$ into the previous operator (norm or linear layer)

optionally search a clipping ratio per group; **return** $\mathcal{Q}_{b,g}(\boldsymbol{W} \operatorname{diag} \boldsymbol{\sigma}_{k^*})$ and $\boldsymbol{\sigma}_{k^*}$

tions and fuses them, and OstQuant optimizes orthogonal and scaling transformations with a distribution-fitting prior [44, 53, 123]. These belong to the rotation family of Chapter 7.

6.6 AWQ: activation-aware weight quantization

AWQ [16] observes that not all weights matter equally: weights multiplied by large-magnitude activation channels are *salient*. Keeping $\sim 1\%$ of such channels in FP16 recovers much of the accuracy lost by RTN in weight-only quantization, but a mixed-precision weight matrix has awkward kernels. AWQ obtains a similar effect through an equivalent scaling.

Derivation. Consider one weight w in a group with step size Δ (determined by the group's range) and input activation x. The output error from RTN is $e_{\text{out}} = \text{Err}(\mathcal{Q}(w))\, x$, where Err is the rounding error with variance approximately $\Delta^2/12$. Scale the weight up by $\sigma > 1$ and the activation down by σ: the output error becomes

$$e'_{\text{out}} = \text{Err}\big(\mathcal{Q}(\sigma w)\big)\frac{x}{\sigma}, \qquad \mathbb{E}\big[e'^2_{\text{out}}\big] \approx \frac{\Delta'^2}{12}\frac{x^2}{\sigma^2}, \tag{6.4}$$

where Δ' is the step size after scaling. If scaling a single channel does not change the group maximum (the channel is not the group's extremum), $\Delta' \approx \Delta$ and the output-error variance of that channel is reduced by $1/\sigma^2$, i.e. by $20\log_{10}\sigma$ dB. As more channels are scaled, group maxima increase and $\Delta' > \Delta$, hurting non-salient channels: the trade-off is regulated by choosing σ_j as a power of activation magnitude,

$$\sigma_j = \bar{a}_j^{\,\alpha}, \qquad \bar{a}_j = \mathbb{E}_t|X_{tj}| \ \text{(mean activation magnitude)}, \qquad \alpha \in [0,1], \tag{6.5}$$

with α found by a grid search over, say, 20 values minimizing the block output error $\left\|\boldsymbol{W}\boldsymbol{X} - \mathcal{Q}(\boldsymbol{W}\boldsymbol{\Sigma})\boldsymbol{\Sigma}^{-1}\boldsymbol{X}\right\|$ [16]. The search optionally also tunes a per-group clipping ratio. No backpropagation or weight updates are needed, which makes AWQ's accuracy less sensitive to the calibration set than reconstruction-based methods; the original paper reports that it generalizes better to out-of-distribution calibration data than GPTQ (in their experiments), with a small calibration set [16].

AWQ versus GPTQ. The two exploit different structure: AWQ reduces the error of salient channels by scaling (a data-dependent but update-free transformation), GPTQ compensates

Table 6.1: Relative layer output error of group-128 weight-only RTN versus an AWQ-style scaling search (activation power $\alpha \in \{0, 0.1, \ldots, 1\}$, per-layer best) on the synthetic layer of Table 5.1 (own simulation). The selected α is 0.3–0.4. Scaling reduces error by 2–3$\times$ relative to plain RTN but does not match GPTQ's error compensation on this strongly correlated layer.

Bits	RTN	AWQ-like RTN	GPTQ	selected α
4	1.26×10^{-2}	5.26×10^{-3}	4.4×10^{-4}	0.3
3	6.84×10^{-2}	2.82×10^{-2}	2.63×10^{-3}	0.3
2	5.46×10^{-1}	1.88×10^{-1}	4.28×10^{-2}	0.4

Table 6.2: Relative mean-square error $\mathbb{E}\,\|\boldsymbol{X} - \mathcal{Q}(\boldsymbol{X})\|^2 / \mathbb{E}\,\|\boldsymbol{X}\|^2$ of per-token symmetric activation quantization (own simulation, same synthetic activations as before). Smoothing uses (6.3) with $\alpha = 0.5$ and statistics from the same activations; the rotation is a random-sign Hadamard transform over the 512 channels.

Activation bits	Per-token (plain)	+ SmoothQuant ($\alpha = 0.5$)	+ Hadamard rotation
8	1.23×10^{-3}	2.47×10^{-4}	3.7×10^{-5}
6	2.04×10^{-2}	4.14×10^{-3}	6.2×10^{-4}
4	1.56×10^{-1}	8.11×10^{-2}	1.21×10^{-2}

errors via correlations. They are complementary and combinations (AWQ scaling then GPTQ) appear in toolchains. In practice reported accuracies of the two are close at 4 bits and AWQ is simpler and faster; GPTQ is often better at 3 bits and with act-order, and AWQ's kernels (TinyChat, and Marlin-compatible formats) are widely supported [16, 100, 214]. AWQ's authors also contributed TinyChat for on-device deployment [16].

Related methods. OWQ keeps a small set of "weak" columns, identified by Hessian sensitivity, in FP16 [126]; SpQR isolates outlier weights in a sparse high-precision component and uses small-group two-level quantization, achieving near-lossless 3–4 bit compression [174]; SqueezeLLM combines sensitivity-based non-uniform codebooks with dense-and-sparse decomposition [175]. All instantiate Proposition 3.2's message: spend precision where the Hessian (activation energy times output sensitivity) is large.

6.7 Activation quantization experiments: a controlled simulation

Table 6.2 quantizes the synthetic activations of Table 5.1 (6 outlier channels of scale 15–30 among 512) per token to INT8, INT6 and INT4, with and without smoothing and with a randomized Hadamard rotation (anticipating Chapter 7).

Plain per-token INT4 on activations with persistent outlier channels loses about 16% of the signal energy; smoothing halves it; rotation reduces it by 13$\times$. The ranking (plain $<$ smoothing $<$ rotation) mirrors published findings that rotations enable W4A4 where smoothing does not [146, 173]. We caution that smoothing here has no accompanying weight-side cost accounted for, and that real activation distributions have token-level massive activations not modelled in the simulation.

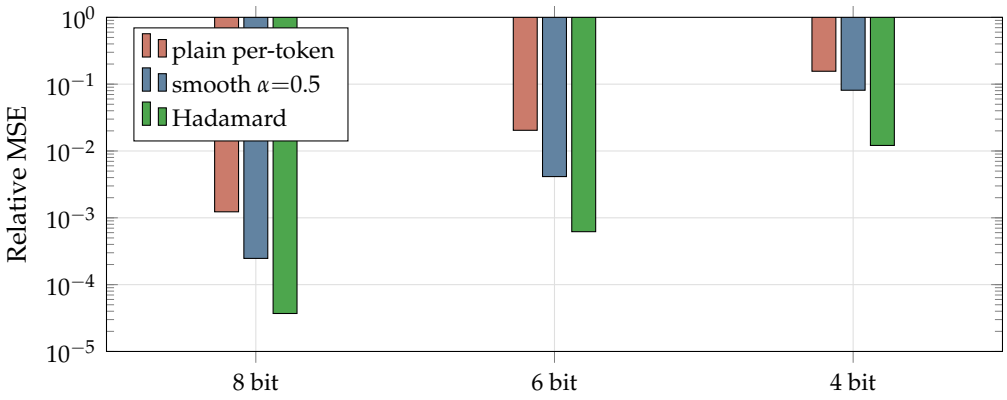


Figure 6.1: Data of Table 6.2 (own simulation on synthetic activations).

6.8 Practical guidance from this chapter

1. For W8A8, per-token activation and per-channel weight scales with SmoothQuant-style smoothing suffice for most dense models; FP8 (E4M3) with per-tensor scaling is usually even easier and now hardware-native on Hopper and later.
2. For weight-only W4A16, group-128 AWQ or GPTQ is the default; if accuracy at 3 bits matters use GPTQ with act-order or learned-rounding methods.
3. For W4A4 or FP4 activations, smoothing alone is insufficient; use rotations (Chapter 7) or micro-scaled formats (Chapter 11).
4. Treat the first token and delimiter tokens specially in KV-cache and static-scale schemes.

Takeaways of the chapter

- Outlier channels and massive-activation tokens are structural and emerge with scale; any activation-quantization design must address them.
- Integer GEMM constrains scales to factor over output indices (Proposition 6.1); SmoothQuant moves per-channel scales into weights and norms; AWQ applies a similar scaling to protect salient weights; LLM.int8()/Atom keep outliers in higher precision.
- Scaling transforms are diagonal; rotation transforms are dense and mix channels, which is stronger at the cost of kernel complexity.

Chapter 7

Rotations and Incoherence Processing

Diagonal scaling (Chapter 6) equalizes channel *maxima*. An orthogonal rotation does something qualitatively stronger: it mixes all channels so that the energy of any outlier coordinate is spread over every output coordinate, making the rotated tensor look Gaussian with coordinates of nearly equal magnitude. This is the idea of *incoherence processing*, introduced for LLM quantization in QuIP and developed by QuIP#, QuaRot, SpinQuant and a large family of successors [146, 149, 150, 173].

7.1 Why rotations help

7.1.1 Invariance

Proposition 7.1 (Orthogonal invariance of linear maps)**.** *For an orthogonal $\boldsymbol{R} \in \mathbb{R}^{m\times m}$ ($\boldsymbol{R}\boldsymbol{R}^\top = \boldsymbol{I}$), $\boldsymbol{W}\boldsymbol{x} = (\boldsymbol{W}\boldsymbol{R})(\boldsymbol{R}^\top\boldsymbol{x})$. Quantizing $\boldsymbol{W}\boldsymbol{R}$ and $\boldsymbol{R}^\top\boldsymbol{x}$ instead of $\boldsymbol{W}$ and $\boldsymbol{x}$ leaves the full-precision function unchanged.*

Rotation preserves Euclidean norms and the Frobenius norm of the layer output error: if $\widehat{\boldsymbol{W}}' = \mathcal{Q}(\boldsymbol{W}\boldsymbol{R})$ then $\left\|(\boldsymbol{W}\boldsymbol{R} - \widehat{\boldsymbol{W}}')\boldsymbol{R}^\top\boldsymbol{X}\right\|_F = \left\|(\boldsymbol{W} - \widehat{\boldsymbol{W}}'\boldsymbol{R}^\top)\boldsymbol{X}\right\|_F$, so the proxy loss of Proposition 3.5 is *unchanged in form* but the quantizer sees different tensors: $\boldsymbol{W}' = \boldsymbol{W}\boldsymbol{R}$ with Hessian $\boldsymbol{H}' = \boldsymbol{R}^\top\boldsymbol{H}\boldsymbol{R}$.

7.1.2 Concentration of rotated coordinates

Proposition 7.2 (Randomly rotated vectors have flat coordinates)**.** *Let $\boldsymbol{u} \in \mathbb{R}^d$ be fixed with $\|\boldsymbol{u}\|_2 = 1$ and let $\boldsymbol{R}$ be a random orthogonal matrix distributed according to the Haar measure, or $\boldsymbol{R} = d^{-1/2}\boldsymbol{H}\boldsymbol{D}$ where $\boldsymbol{H}$ is a $d \times d$ Hadamard matrix and $\boldsymbol{D}$ a diagonal of i.i.d. uniform signs. Then for every coordinate i and $t > 0$,*

$$\Pr\big[\,|(\boldsymbol{R}\boldsymbol{u})_i| > t\,\big] \le 2\exp\!\big(-dt^2/2\big), \tag{7.1}$$

and therefore, with probability at least $1-\delta$,

$$\|\boldsymbol{R}\boldsymbol{u}\|_\infty \le \sqrt{\frac{2\ln(2d/\delta)}{d}}\;\|\boldsymbol{u}\|_2\,. \tag{7.2}$$

Proof (randomized Hadamard case). Coordinate i equals $d^{-1/2}\sum_j h_{ij}\epsilon_j u_j$ with $h_{ij} \in \{\pm 1\}$ and independent Rademacher ϵ_j. By Hoeffding's inequality, a sum of independent symmetric bounded variables $\sum_j a_j\epsilon_j$ is sub-Gaussian with $\Pr[|\sum_j a_j\epsilon_j| > t] \le 2\exp(-t^2/(2\sum_j a_j^2))$. Here $a_j = d^{-1/2}h_{ij}u_j$ and $\sum_j a_j^2 = \|\boldsymbol{u}\|_2^2/d = 1/d$, which gives the tail bound. A union bound over the d coordinates with $t = \sqrt{2\ln(2d/\delta)/d}$ yields (7.2). For Haar rotations, the coordinates are

those of a uniformly random point on the sphere S^{d-1} and the same sub-Gaussian tail follows from Lévy's concentration inequality [208]. □

The RMS of the entries of $\boldsymbol{Ru}$ is $1/\sqrt{d}$. Hence the ratio of maximum to RMS after rotation is at most $\sqrt{2\ln(2d/\delta)}$, which is ≈ 4.6 for $d = 4096$ and $\delta = 0.01$, regardless of how spiky $\boldsymbol{u}$ was. Since an absmax-scaled quantizer's error scales with the max-to-RMS ratio squared (Table 3.3), rotation turns an arbitrarily bad ratio into a bounded one. For the extreme case of one dominant coordinate, $\boldsymbol{u} = \boldsymbol{e}_1$, the Hadamard rotation yields coordinates all of magnitude exactly $d^{-1/2}$.

Gaussianization

A random rotation maps *any* fixed vector to one with approximately i.i.d. sub-Gaussian coordinates of variance $\|\boldsymbol{u}\|^2/d$. After rotation, the optimal (Gaussian-shaped) codebooks and clip ratios of Chapter 3 apply, and the rate–distortion bound for Gaussians is the right target for the codebook design in Chapter 8.

7.1.3 Fast transforms

A dense $d \times d$ rotation costs $O(d^2)$ per vector. The (normalized) Walsh–Hadamard transform costs $O(d \log d)$ via the fast Walsh–Hadamard transform (FWHT), with only additions and subtractions and one final scaling by $d^{-1/2}$; with random signs it is the randomized Hadamard transform (RHT), the core of the fast Johnson–Lindenstrauss transform [202]. Hadamard matrices of order 2^k exist by Sylvester's construction; for dimensions of the form $2^k p$ with a known small Hadamard matrix of order p one can use Kronecker products $\boldsymbol{H}_{2^k} \otimes \boldsymbol{H}_p$ (the width $11008 = 64 \cdot 172$ of Llama-2-7B's MLP requires a Hadamard matrix of order 172, which exists but is not a power-of-two construction), and otherwise implementations pad or rotate blocks separately. Many implementations also use *block-diagonal* Hadamard rotations (e.g. blocks of size 16, 32, 64 or 128), which still concentrate within each block and match the block size of micro-scaled formats [110].

7.2 QuIP: incoherence and LDLQ

QuIP [150] defines two incoherence notions and an adaptive rounding method that exploits them.

Definition 7.1 (Incoherence)**.** A weight matrix $\boldsymbol{W} \in \mathbb{R}^{n\times m}$ is *μ-incoherent* if $\max_{ij} |W_{ij}| \le \mu \, \|\boldsymbol{W}\|_F / \sqrt{nm}$. A symmetric positive semidefinite $\boldsymbol{H}$ with eigen-decomposition $\boldsymbol{H} = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^\top$ is μ-incoherent if $\max_{ij} |Q_{ij}| \le \mu/\sqrt{m}$.

Weight incoherence says no entry is much larger than the RMS (the range-to-RMS ratio governing absmax quantization). Hessian incoherence says the eigenvectors, i.e. the directions in which rounding error matters most, are not aligned with coordinate axes; as a consequence the rounding errors of different coordinates cannot be systematically correlated with the sensitive directions.

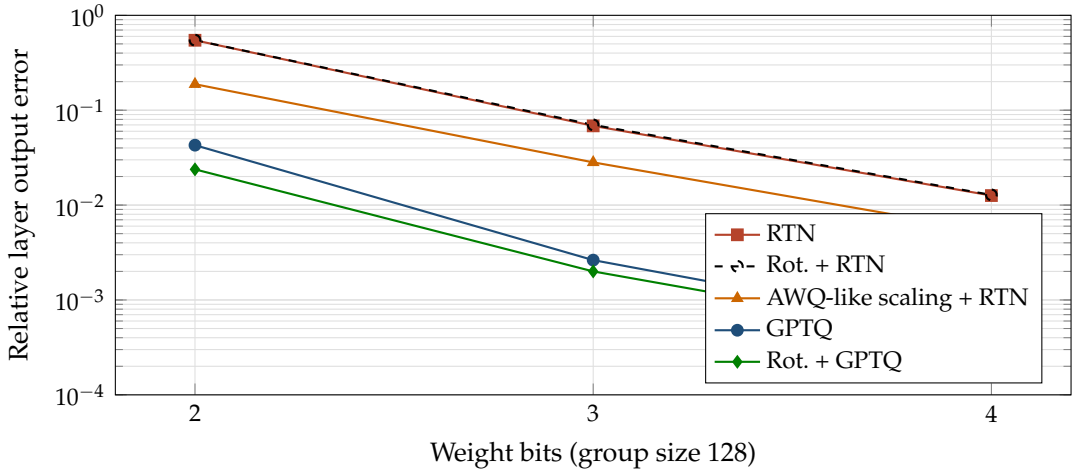


Figure 7.1: Relative layer output error versus weight bit-width on the synthetic layer of Table 5.1 (own simulation). Rotation does not help RTN on Gaussian weights (the weights are already incoherent) but helps GPTQ by making the Hessian less axis-aligned (18–44% lower error here); error compensation matters far more than scaling at low bits on this strongly correlated layer. Rotation is more valuable for *activations* (Table 6.2).

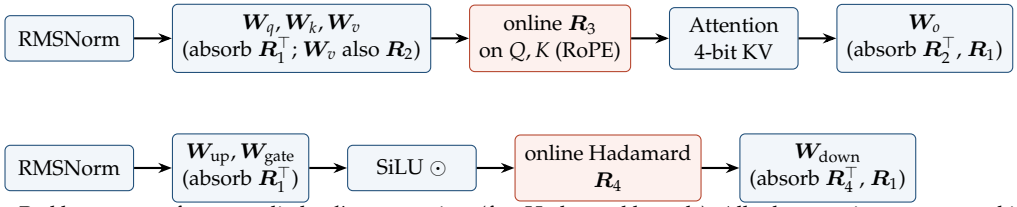
Processing. QuIP multiplies $\boldsymbol{W} \mapsto \boldsymbol{U}\boldsymbol{W}\boldsymbol{V}^\top$ and $\boldsymbol{H} \mapsto \boldsymbol{V}\boldsymbol{H}\boldsymbol{V}^\top$ with random orthogonal $\boldsymbol{U} \in \mathbb{R}^{n\times n}$, $\boldsymbol{V} \in \mathbb{R}^{m\times m}$ (originally Kronecker products of small random orthogonal matrices for speed), which makes both incoherent with high probability, quantizes, and undoes the transform at inference.

LDLQ. Given the Hessian proxy, QuIP's adaptive rounding quantizes columns sequentially with linear feedback of past errors,

$$\widehat{\boldsymbol{w}}_k = \mathcal{Q}\Big(\boldsymbol{w}_k + \sum_{j<k} (\boldsymbol{w}_j - \widehat{\boldsymbol{w}}_j)\, a_{jk}\Big), \tag{7.3}$$

where the coefficients a_{jk} come from the LDL decomposition $\boldsymbol{H} = (\boldsymbol{U}_s + \boldsymbol{I})\boldsymbol{D}(\boldsymbol{U}_s + \boldsymbol{I})^\top$ with $\boldsymbol{U}_s$ strictly upper triangular: a_{jk} are the entries of $\boldsymbol{U}_s$. QuIP proves that LDLQ is optimal within the class of adaptive rounding procedures with linear feedback, in both worst-case and average-case senses, that its proxy loss involves $\mathrm{tr}(\boldsymbol{D})$, and that for incoherent $\boldsymbol{H}$ it satisfies a bound on $\mathrm{tr}(\boldsymbol{D})$ in terms of $\mathrm{tr}(\boldsymbol{H}^{1/2})^2/m$ [150]. Since $\mathrm{tr}(\boldsymbol{H}^{1/2})^2 \le m\,\mathrm{tr}(\boldsymbol{H})$ by Cauchy–Schwarz, this is never worse than the $\mathrm{tr}(\boldsymbol{H})$ governing nearest rounding, and is far smaller for Hessians with rapidly decaying spectra. Crucially, QuIP also shows that OPTQ (GPTQ) is equivalent to LDLQ, so its theory applies to GPTQ [150].

Remark 7.1 (Link to our Cholesky lemma). Lemma 5.1 expresses the GPTQ update through the upper Cholesky factor of $\boldsymbol{H}^{-1}$, whose normalized rows are the OBS update directions. The LDL decomposition of $\boldsymbol{H}$ with reversed coordinate order is the same object in different clothing, which is why the two procedures coincide.



Red boxes: transforms applied *online* at run time (fast Hadamard kernels). All other rotations are merged into weights offline.

Figure 7.2: Schematic of rotation placement in QuaRot [146] (our redrawing). $\boldsymbol{R}_1$ rotates the residual stream and is fully absorbed; $\boldsymbol{R}_2$ is a per-head Hadamard inside value/output projections and is absorbed; $\boldsymbol{R}_3$ (after RoPE, on queries and keys) and $\boldsymbol{R}_4$ (before the down-projection) must be applied online but are cheap Hadamard transforms.

7.3 QuIP#: randomized Hadamard transform and lattice codebooks

QuIP# [149] replaces QuIP's Kronecker random orthogonal matrices with the randomized Hadamard transform: it is faster ($O(d \log d)$), has a better theoretical incoherence guarantee, and (via Proposition 7.2) the rotated weights have sub-Gaussian entries of near-equal magnitude. The second and third ingredients are vector quantization to exploit the ball-shaped distribution of incoherent weights using codebooks based on the E_8 lattice (the densest sphere packing in 8 dimensions [195]), and fine-tuning. These are covered in Chapter 8. QTIP uses the same incoherence processing with trellis-coded quantization [144].

7.4 QuaRot: rotating the whole network

QuaRot [146] targets the harder W4A4KV4 setting by using *computational invariance*: any orthogonal transform of the residual stream can be absorbed into the adjacent weight matrices. Consider a pre-norm transformer whose layer inputs are RMSNorm outputs. RMSNorm without learned scale is rotation-equivariant: $\mathrm{RMSNorm}(\boldsymbol{x}\boldsymbol{R}) = \mathrm{RMSNorm}(\boldsymbol{x})\boldsymbol{R}$ because the RMS is invariant to orthogonal transforms; the learned scale $\boldsymbol{\gamma}$ is first folded into the following linear layers. With $\boldsymbol{x} \mapsto \boldsymbol{x}\boldsymbol{R}_1$ on the residual stream, every weight that writes into the stream ($\boldsymbol{W}_o$, $\boldsymbol{W}_{\text{down}}$, embeddings) is replaced by $\boldsymbol{W}\boldsymbol{R}_1$ and every weight that reads from it ($\boldsymbol{W}_q, \boldsymbol{W}_k, \boldsymbol{W}_v, \boldsymbol{W}_{\text{up}}, \boldsymbol{W}_{\text{gate}}$, LM head) by $\boldsymbol{R}_1^\top \boldsymbol{W}$. The network function is unchanged and every activation entering a linear layer is now rotated and outlier-free.

The remaining activations (the attention-internal values and the input of the down-projection) are not directly after a norm layer, so they need *online* Hadamard transforms: an online Hadamard at the input of $\boldsymbol{W}_{\text{down}}$ (whose input has severe outliers in many models), a head-wise Hadamard on values merged into $\boldsymbol{W}_v / \boldsymbol{W}_o$, and online rotations of queries and keys after RoPE to flatten the KV cache. QuaRot reports that the 4-bit quantized Llama-2-70B loses at most 0.47 WikiText-2 perplexity and retains 99% of zero-shot performance, and that RTN with QuaRot rotations gives lossless 6- and 8-bit models without calibration data [146]; weights use GPTQ in the 4-bit setting.

7.5 SpinQuant: learned rotations

Random rotations are not equally good: SpinQuant found that different random rotations produce up to a 13-point difference in zero-shot reasoning accuracy, and optimizes the rotation matrices on the Stiefel manifold of orthogonal matrices using Cayley SGD (a Riemannian optimizer [37]) with the quantized-network loss and fixed weights [173]. With W4A4KV4 it reports narrowing the zero-shot gap to full precision to 2.9 points on Llama-2 7B, and improving on QuaRot, with the largest relative improvement on Llama-3 8B which is hard to quantize [173]. Meta's quantized Llama 3.2 release (1B and 3B) used SpinQuant for post-training and a QLoRA-style QAT variant [200]. The rotations that must be applied online remain Hadamard; the learned rotations are the absorbable ones ($\boldsymbol{R}_1$, $\boldsymbol{R}_2$).

7.6 Other transformation-based methods

DuQuant. Uses block-wise rotations constructed from outlier-dimension priors, a zigzag permutation to balance outliers across blocks, and a second rotation, targeting both ordinary and massive outliers at W4A4 [44].

FlatQuant. Learns per-layer affine transforms constrained to Kronecker products of two small matrices, calibrated in hours, with all operations fused into a single kernel; it reports less than 1% accuracy drop for W4A4 on Llama-3-70B, surpassing SpinQuant by 7.5% in the authors' evaluation, and up to 2.3× prefill and 1.7× decoding speedup over FP16 [53]. It argues that Hadamard-transformed distributions can still be steep and dispersed, and flatness of both weights and activations matters.

OstQuant, AffineQuant, OmniQuant. OstQuant combines learnable orthogonal and scaling transformations with a prior that targets distribution fit; AffineQuant extends equivalent transformations to affine matrices; OmniQuant learns clipping and diagonal equivalence transforms [7, 118, 123].

PrefixQuant and IntactKV. Handle massive tokens directly: PrefixQuant prefixes the input with fixed tokens whose KV pairs absorb the outliers and enables static activation quantization; IntactKV keeps the key–value entries of pivot tokens lossless [80, 131]. They are complementary to rotations.

Micro-rotations for micro-scaled formats. For MXFP4 and NVFP4 the relevant block size is only 16 or 32 elements, so block-diagonal Hadamard rotations of that size suffice and are cheap; MR-GPTQ fuses them into the weights and applies them to activations with fused kernels [110]. The same block-Hadamard idea appears in the NVFP4 pretraining recipe, which uses random Hadamard transforms to limit block-level outliers [116], and in Quartet [148]. The benefit *depends on the format*: a study of FP4 post-training quantization reported that NVFP4's small group size undermines traditional outlier-mitigation techniques while MXFP4's power-of-two scales add error, and proposed block-wise rotations combined with format-specific choices

Table 7.1: Rotation, pros and cons (author's assessment).

Advantages	Disadvantages
Removes channel outliers without per-channel scales; compatible with integer/FP tensor-core GEMM; theoretical guarantees (QuIP)	Online transforms ($\boldsymbol{R}_3, \boldsymbol{R}_4$) add runtime cost and need custom kernels
Data-free variants (RTN + rotation gives lossless W8/W6 per QuaRot)	Non-power-of-two dimensions complicate Hadamard use
Composable with GPTQ, scale search, VQ and QAT	Learned rotations (SpinQuant, FlatQuant) need optimization data and extra engineering
Whitens distributions so Gaussian-optimal codebooks apply	Provides little benefit (or hurts) for small blocks with well-behaved data (NVFP4 weights)
Fast ($O(d \log d)$) with Hadamard	Merging into weights is only exact for architectures with rotation-equivariant norms (RMSNorm); LayerNorm needs mean-centering folded in

as a remedy; it concluded that FP4 "is not an automatic upgrade over INT4" [110]. This is consistent with our simulation (Chapter 11).

7.7 When do rotations hurt or help? A decision analysis

Proposition 7.3 (Rotation helps when the max-to-RMS ratio is high and hurts when the data are already flat)**.** *Consider absmax-scaled uniform quantization with block size g and b bits. The expected block error is approximately $\mathbb{E}[a_g^2]/(12q_{\max}^2)$ with a_g the block maximum. If the block's coordinates are i.i.d. Gaussian, a_g is already close to its minimum possible ratio to the RMS and rotation cannot reduce $\mathbb{E}[a_g^2]$ materially; if the block contains a heavy-tailed or spiky component with* $\max/\mathrm{RMS} = \rho \gg \sqrt{2\ln 2g}$*, rotation reduces it to at most about $\sqrt{2\ln(2g/\delta)}$, a reduction of the error by a factor of up to $\rho^2/(2\ln(2g/\delta))$.*

This justifies two empirical regularities: (i) rotation is most useful for *activations* and for *large blocks* (per-token or per-channel scale), whose spikes are severe; (ii) for small blocks of a micro-scaled format applied to already well-behaved weights, rotation gains are small and sometimes negative. Our simulation of 4-bit formats supports this: with a fixed block size of 16 or 32, rotating Gaussian data changes SQNR by < 0.05 dB, whereas for data with sparse $20\times$ outliers INT4 with a block of 32 improves from 16.4 dB to 21.7 dB with rotation (Table 11.2).

Takeaways of the chapter

- Orthogonal rotations preserve the function and the proxy loss while flattening coordinate magnitudes: after a randomized Hadamard transform the max-to-RMS ratio is bounded by $\sqrt{2\ln(2d/\delta)}$ for any input (Proposition 7.2).
- QuIP gives theory (incoherence, LDLQ optimality, GPTQ equivalence); QuIP# and QTIP combine it with lattice and trellis codes; QuaRot and SpinQuant extend it end-to-end to W4A4KV4 by absorbing rotations into weights and adding a few online Hadamards.
- Rotation helps most where outliers are severe and blocks are large; for small micro-scaled blocks the gain is format- and data-dependent.

Chapter 8

Vector, Additive, Lattice, and Trellis Quantization

Scalar quantizers round each weight independently. At 4 bits that is nearly adequate; at 2–3 bits it is wasteful, because independent rounding cannot exploit the geometry of the joint distribution of several weights. This chapter develops the theory of vector quantization (VQ) and the methods that bring it into LLM compression: k-means codebooks (GPTVQ), additive and residual codebooks (AQLM, VPTQ), lattice codebooks (QuIP# with E_8), and trellis-coded quantization (QTIP).

8.1 Why vectors beat scalars

8.1.1 Granular and shaping gain

An n-dimensional fixed-rate quantizer of rate R bits per dimension has 2^{nR} codewords. At high resolution, Zador's theorem expresses the mean-square distortion per dimension as

$$D \;\approx\; G_n \left(\int f^{\,n/(n+2)}\right)^{(n+2)/n} 2^{-2R}, \tag{8.1}$$

where G_n is the *normalized second moment* of the optimal quantization cell in dimension n [1, 207]. For $n = 1$ cells are intervals and $G_1 = 1/12 \approx 0.0833$; for $n \to \infty$, $G_n \to 1/(2\pi e) \approx 0.0585$. The ratio, $10\log_{10}\!\left(\frac{1/12}{1/(2\pi e)}\right) = 1.53$ dB, is the *space-filling (granular) gain* and equals $\frac{1}{2}\log_2(\pi e/6) \approx 0.254$ bit per dimension: the best conceivable improvement from replacing cubic cells by round ones with entropy coding. The E_8 lattice has $G \approx 0.0717$ (about 0.65 dB better than the cubic lattice) and the Leech lattice in 24 dimensions $G \approx 0.0658$ (about 1.03 dB) [195].

Fixed-rate quantization of a bell-shaped source has an additional *shaping (boundary) gain*: a scalar quantizer's cubic cell array fills a cube, whereas the source's probability mass concentrates in a ball of radius about $\sqrt{n}\sigma$; the ratio of cube to ball volume grows with dimension. For Gaussian sources the combination of both gains at low rates exceeds the 1.53 dB number; the practical consequence is that at 2 bits per weight the gap between a scalar quantizer and the rate–distortion bound is about 2.8 dB (Table 3.1), and VQ, lattices and trellis codes recover a large part.

Remark 8.1. Incoherence processing (Chapter 7) is what makes this theory applicable to LLM weights. After a randomized Hadamard transform the weights of a block are approximately i.i.d. sub-Gaussian; the optimal codebook for the resulting "ball-shaped" distribution can be designed once, offline, for a Gaussian source, and need not be learned per matrix [149].

Algorithm 5 Hessian-weighted VQ of a weight group with GPTQ-style feedback (GPTVQ schematic)

Require: weights $\boldsymbol{W} \in \mathbb{R}^{n \times m}$, Hessian $\boldsymbol{H}$, vector dimension d, codebook size K, columns per block B

initialize K centroids per group by Hessian-weighted k-means on sub-vectors $\boldsymbol{W}_{i,jd:(j+1)d}$
for column block c of width d (left to right) **do**
  for each row i **do**
    assign sub-vector $\boldsymbol{v}_i = \boldsymbol{W}_{i,c}$ to centroid $\boldsymbol{\mu}_{k_i}$ minimizing $(\boldsymbol{v}_i - \boldsymbol{\mu}_k)^\top \boldsymbol{H}_{cc}^{-1-1} (\boldsymbol{v}_i - \boldsymbol{\mu}_k)$
  end for
  $\boldsymbol{E} \leftarrow \boldsymbol{W}_{:,c} - \boldsymbol{\mu}_{\boldsymbol{k}}$ ▷ quantization error for the d columns
  update remaining columns: $\boldsymbol{W}_{:,c'>c} \leftarrow \boldsymbol{W}_{:,c'>c} - \boldsymbol{E}\,(\boldsymbol{H}_{cc}^{-1})^{-1} \boldsymbol{H}_{c,c'}^{-1}$ ▷ block OBS update
end for
refine centroids to minimize the Hessian-weighted error; quantize the codebook to 8 bits; **return** indices and codebooks

8.1.2 Codebook size and decode cost

A VQ of dimension d at R bits per weight needs a codebook of 2^{dR} vectors, each with d entries. For $d = 8$ and $R = 2$ that is 65,536 entries of 8 weights, i.e. 1 MiB at 16-bit entries, which fits in GPU shared memory only with care and certainly not in registers. Decoding is a table lookup per d weights: attractive for memory-bound kernels (a few cycles per group), but the table traffic competes for cache and bandwidth, and the encoder must search the codebook (nearest-neighbour search for each group, or beam search under a Hessian-weighted metric). The "curse of dimensionality" on the codebook size is the central engineering trade-off of VQ and is the motivation for structured codebooks (additive, lattice, trellis) that decouple the effective quantization dimension from the table size.

8.2 GPTVQ: k-means codebooks with Hessian-aware updates

GPTVQ [74] extends GPTQ's column sweep to vector quantization. The weights of a group (several rows by several columns) are partitioned into d-dimensional sub-vectors; a codebook of 2^{dR} centroids is initialized by a Hessian-weighted EM (data-aware k-means), and quantization proceeds by interleaving quantization of d columns at a time with GPTQ-style updates of the remaining columns using the inverse Hessian; codebooks are then updated and compressed (integer quantization and SVD-based compression of the codebook itself). The authors report that increasing the quantization dimensionality improves the size versus accuracy trade-off, that a Llama-2-70B model is processed on a single H100 in 3 to 11 hours depending on the setting, and that VQ decompression on a mobile CPU improved latency over a 4-bit integer format [74].

8.3 Additive and residual quantization

8.3.1 Additive quantization (AQ)

Additive (multi-codebook) quantization represents a g-dimensional weight group as the *sum* of M codewords, one from each of M codebooks $\boldsymbol{C}_1, \ldots, \boldsymbol{C}_M \in \mathbb{R}^{2^B \times g}$:

$$\hat{\boldsymbol{w}} = \sum_{m=1}^{M} \boldsymbol{C}_m[\, b_m \,], \qquad b_m \in \{0, \ldots, 2^B - 1\}. \tag{8.2}$$

The number of representable vectors is 2^{BM}, exponential in M, but the stored codebooks are only $M 2^B g$ numbers, so the representable set is rich while the tables are small. The bit rate per weight, ignoring codebook overhead, is

$$R = \frac{MB}{g}. \tag{8.3}$$

For example, $M = 1$, $B = 16$, $g = 8$ gives 2 bits per weight with a 65,536-entry codebook (the "1×16" scheme), while $M = 2$, $B = 8$, $g = 8$ gives the same 2 bits with two 256-entry codebooks ("2×8"), which are $128\times$ smaller tables (and less accurate: the sum structure represents only a subset of the effective space). Residual quantization is the special case where codebook m is trained on the residual after codebooks $1..m-1$.

8.3.2 AQLM

AQLM [10] adapts AQ to LLMs with two innovations: (i) *input-adaptive* learned additive quantization, meaning the code assignment and codebooks minimize the layer-output error $\left\| (\boldsymbol{W} - \widehat{\boldsymbol{W}})\boldsymbol{X} \right\|_F^2$ rather than the weight reconstruction error, and (ii) *joint optimization of codebook parameters across each transformer block* by fine-tuning on calibration data. The layer-wise step alternates between (a) beam search over discrete codes for each group given the codebooks (a discrete optimization over M indices with a Hessian-weighted objective), and (b) gradient/least-squares updates of the codebooks and scales given the codes, followed by block-wise fine-tuning. The authors claim AQLM is the first scheme that is Pareto-optimal in accuracy versus model size below 3 bits per parameter, that it significantly improves the 2-bit regime, and that its GPU and CPU kernels match or outperform optimized FP16 implementations for token generation while using much less memory [10]. The price is encoding time: the beam-search and block-wise fine-tuning stages are expensive.

Example 8.1 (Bits per weight of an AQLM-style configuration with codebook overhead)**.** Take $M = 1$, $B = 16$, $g = 8$ for a 4096×4096 matrix (16,777,216 weights). The codes cost 2 bits per weight, i.e. 4 MiB. The single codebook has $65{,}536 \times 8$ FP16 entries, i.e. 1 MiB, which is another 0.5 bit per weight if the codebook belongs to this matrix alone, giving 2.5 bits per weight. If one codebook were shared by seven equally sized matrices the overhead would fall to $0.5/7 \approx 0.07$ bit. Per-row scales add $4096 \cdot 16$ bits, a further 0.004 bit. Honest rate accounting therefore requires stating whether codebooks are shared or quantized.

8.3.3 VPTQ and residual VQ

VPTQ formulates VQ weight quantization via second-order optimization, adds channel-independent second-order refinement for finer granularity, a simple codebook initialization derived from decomposing the optimization problem, and extends to residual and outlier quantization [181]. The authors report perplexity reductions at 2 bits over prior state of the art of 0.01–0.34 on Llama-2, 0.38–0.68 on Mistral-7B and 4.41–7.34 on Llama-3 [181].

8.4 Fine-tuning compressed weights: PV-Tuning

Methods such as QuIP# and AQLM fine-tune the compressed representation (codebooks, scales, sign vectors) with straight-through estimators (STE) for the discrete codes. PV-Tuning argues that the STE is suboptimal in this regime and proposes an alternating scheme that separately optimizes continuous parameters (codebooks, scales) and discrete assignments, with convergence guarantees in restricted cases; applied to 1–2 bit vector quantization it reports first Pareto-optimal quantization for the Llama-2 family at 2 bits per parameter [132]. We return to STE and its limitations in Chapter 10.

8.5 Lattice codebooks: QuIP# and E_8

After randomized Hadamard processing the weights in groups of 8 are close to i.i.d. sub-Gaussian. QuIP# quantizes groups of $g = 8$ with a codebook derived from the E_8 lattice [149]. The E_8 lattice consists of integer or half-integer vectors in $\mathbb{R}^8$ whose coordinate sum is even, and gives the densest sphere packing in 8 dimensions [195]. At 2 bits per weight an 8-dimensional codebook has 2^{16} codewords. QuIP#'s "E8P" codebook is built to be hardware-friendly: as we understand the construction, it stores a small (2^8 entries) source codebook of E_8 points and generates the full 2^{16}-point codebook by applying sign flips (with the eighth sign determined by a parity constraint) and a half-lattice shift, so that decoding needs only a small table and bit manipulation [149]. For 3 and 4 bits QuIP# uses residual vector quantization, adding a second stage of E_8-based or scalar codebook on the residual. Combined with incoherence processing and fine-tuning, it was the first method to show that 2-bit weights can be competitive for sufficiently large models, enabling new "scaling behaviours" in PTQ, and it supports fast inference kernels [149].

NestQuant. A related theory-driven line quantizes with *nested* lattices (Gosset/E_8-based), proving near-optimal rates for matrix products: NestQuant applies self-similar nested-lattice quantization for weights, activations and KV caches, building on information-theoretic results on optimal quantization for matrix multiplication [115, 121]. Ordentlich and Polyanskiy analyse the rate–distortion limit for approximating a matrix product directly, as opposed to quantizing each operand separately for minimal reconstruction error, which is the right information-theoretic target for weight–activation quantization and relates to the weighted-loss view of Section 3.1 [121].

Algorithm 6 Viterbi encoding on a bitshift trellis (L-bit states, k bits per step)

Require: sequence $x_1..x_T$, code table $c[0..2^L-1]$, bits per step k

1: $J_1(s) \leftarrow (x_1 - c[s])^2$ for all states s ▷ initial state is free (costs L bits once)
2: **for** $t = 2..T$ **do**
3: **for** each state s' **do**
4: predecessors: $P(s') = \{\, (s' \gg k) \mid (j \ll (L-k)) :\ j = 0..2^k - 1 \}$
5: $p_t(s') \leftarrow \arg\min_{s \in P(s')} J_{t-1}(s)$; $J_t(s') \leftarrow J_{t-1}(p_t(s')) + (x_t - c[s'])^2$
6: **end for**
7: **end for**
8: trace back from $s^* = \arg\min_s J_T(s)$ using p_t; the code bits are the low k bits of each state along the path
9: **return** bit stream and MSE $J_T(s^*)/T$

8.6 Trellis-coded quantization and QTIP

8.6.1 Trellis-coded quantization

Trellis-coded quantization (TCQ) [199] achieves the granular gain of high-dimensional VQ with encode/decode complexity that grows only linearly in the dimension. The quantizer's decoder is a finite-state machine: a sequence of T weights $x_1, \dots, x_T$ is described by kT bits, which drive a state machine; at each step the machine emits a reconstruction value determined by its current state. The encoder runs the Viterbi algorithm to find the bit sequence whose emitted values minimize the squared error. With 2^L states the effective quantization dimension is T (arbitrarily large), while the codebook is the table of 2^L emitted values: *the codebook size is decoupled from the dimension and the bitrate*. QTIP [144] observes that conventional VQ limits the dimension to ≤ 8, because a codebook has 2^{dR} entries, which bounds quality (the shaping gain increases with dimension), and uses TCQ to reach ultra-high-dimensional quantization.

8.6.2 The bitshift trellis

QTIP uses a hardware-efficient "bitshift" trellis: the state is an L-bit word; each step shifts in k new bits (the code stream) and shifts out k old bits,

$$s_t = \big((s_{t-1} \ll k)\ \&\ (2^L - 1)\big) \mid b_t, \qquad b_t \in \{0, \dots, 2^k - 1\}, \qquad \hat{x}_t = \mathrm{code}(s_t). \tag{8.4}$$

The decoder is just a sliding window of L bits over the stored bit stream followed by a function $\mathrm{code} : \{0,1\}^L \to \mathbb{R}$; with k bits per step, the rate is exactly k bits per weight with no overhead apart from the initial state. QTIP introduces a family of codes from pure lookup tables (for small L) to *computed* lookup-free codes that generate approximately Gaussian values from the state with a few integer instructions (hash-like functions), so that no large table is needed even for $L = 16$ (65,536 states) [144]. With the randomized Hadamard transform, the sequence to be quantized is approximately i.i.d. Gaussian, so a pseudo-random Gaussian code is nearly optimal. This is the connection between incoherence and trellis codes: the former makes the source Gaussian, the latter then approaches its rate–distortion bound.

Encoding costs $O(T 2^L 2^k)$ operations, trivial next to the other costs of calibration for the $L \leq 16$ used in practice; decoding costs $O(1)$ per weight and is parallel across independent sequences

Table 8.1: Trellis-coded quantization of i.i.d. $\mathcal{N}(0,1)$ data on a bitshift trellis with a random Gaussian code table (own simulation, $T = 768$–1024 weights per sequence, 2–3 random codes per setting, Viterbi encoding; a structured code would do somewhat better and the numbers carry sampling noise of order 0.1 dB). SQNR in dB; the bound is $6.02\,k$ dB. For reference, optimal 1-D scalar quantization (Lloyd–Max) gives 9.28 dB at $k = 2$ and 14.58 dB at $k = 3$ (Table 3.1).

k (bits/weight)	$L = 6$	$L = 10$	$L = 16$	R–D bound
2	9.68	11.21	11.82	12.04
3	14.19	16.42	17.84	18.06

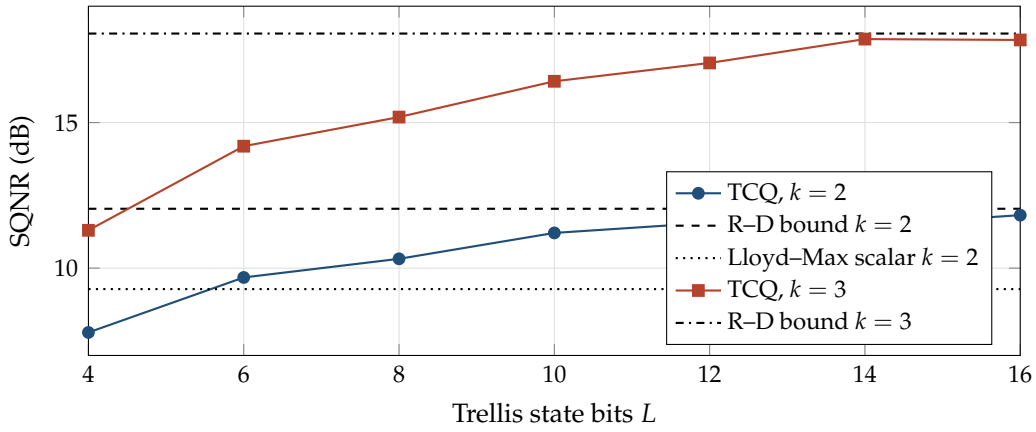


Figure 8.1: TCQ performance on Gaussian data versus the number of trellis state bits (own simulation, data of Table 8.1 and intermediate L; noisy). With $L = 16$, TCQ at 2 bits is within roughly 0.3 dB of the bound and 2.5 dB better than the best scalar quantizer; at 3 bits it is within about 0.2 dB.

(e.g. tiles of the weight matrix).

Real QTIP results: the authors report state-of-the-art quality and inference speed relative to prior VQ-based PTQ, with lookup-only to lookup-free trellis codes [144]. The simulation above illustrates *why*: ultra-high effective dimension with a modest table captures almost all of the available gain over scalar quantization.

8.7 Comparing VQ-family methods

8.8 Practical considerations

Kernels. VQ and trellis decoders are table lookups and integer bit manipulation on CUDA cores. They perform well in the memory-bound regime (batch 1–8) but cannot feed tensor cores directly, so for larger batches the dequantization cost (to FP16/BF16 followed by a tensor-core GEMM) becomes the limiter; AQLM and QuIP# papers report speed at batch size one [10, 149]. FLUTE and LUT-GEMM develop general lookup-table kernels for non-uniform and VQ-style formats on GPUs [56, 98], and T-MAC does so on CPUs [177].

Table 8.2: Vector/trellis-family methods (claims are those of the cited abstracts; "Pareto" refers to accuracy versus bits against the baselines in each paper).

Method	Representation	Fitting	Reported claim
GPTVQ [74]	d-dim k-means codebooks with integer-quantized codebooks	Hessian-weighted EM + GPTQ-style block updates	better size/accuracy with higher dimension; 3–11 h for Llama-2-70B on one H100
AQLM [10]	Additive (multi-codebook), $g = 8$	learned codes by beam search; block-wise joint fine-tuning	Pareto-optimal below 3 bits; kernels match or beat FP16 speed
VPTQ [181]	Vector + residual + outlier	second-order VQ, channel-independent refinement	PPL gains at 2 bits over SOTA
QuIP# [149]	RHT + E_8 lattice codebooks, RVQ	fine-tuning of codebook-adjacent parameters	SOTA at ≤ 4 bits; fast inference
QTIP [144]	RHT + bitshift trellis	Viterbi encoding, fine-tuning	SOTA quality and speed vs VQ-based PTQ
PV-Tuning [132]	representation-agnostic fine-tuning	alternating continuous/discrete updates	Pareto at 2 bits for Llama-2 family
NestQuant [115]	nested lattices	theory-driven	W+A+KV quantization with near-optimal rates

Encoding cost. Compared with GPTQ (minutes to hours), VQ methods need hours to days of GPU time including fine-tuning (the AQLM and QuIP# pipelines are reported to need considerably longer for the largest models, on the order of GPU-days, so check the papers for exact budgets; GPTVQ lists 3–11 hours for 70B). For a model provider amortizing over millions of requests this is negligible; for a practitioner quantizing a custom fine-tune it is a real cost.

Ecosystem. As of October 2026, scalar 4-bit formats (AWQ, GPTQ, GGUF, FP8, NVFP4) dominate production. VQ/trellis checkpoints exist for popular open models and run in dedicated runtimes, but support in mainstream serving stacks is thinner (Chapter 14).

Takeaways of the chapter

- Vector quantization gains the space-filling and shaping gain of high dimension: up to 1.53 dB from cell shape plus boundary gain, worth roughly half a bit per weight at 2 bits.
- Structured codebooks decouple effective dimension from table size: additive codebooks (AQLM) sum small tables; lattices (QuIP# E_8) give geometry for free; trellises (QTIP) reach dimension in the hundreds with $O(1)$ decoding.
- Incoherence processing is the enabler: it makes weights approximately Gaussian, so a universal Gaussian-optimal code works. Our simulation shows TCQ within ~ 0.3 dB of the Gaussian rate–distortion bound at 2–3 bits.
- Cost: long encoding time and kernels that run on CUDA cores, favouring small batch sizes.

Chapter 9

Mixed Precision and Bit Allocation

Not all parts of a network are equally sensitive to quantization. Embeddings, the first and last layers, attention value projections, and MLP down-projections are repeatedly reported to be more sensitive than the bulk; some input channels and some groups of weights matter more than others. *Mixed-precision quantization* spends a fixed bit budget non-uniformly. This chapter formulates the allocation problem, derives the closed-form optimum under a high-resolution noise model, reviews sensitivity measures (Hessian trace, Fisher information, activation energy) and search algorithms, and describes how production formats such as GGUF k-quants implement mixed precision.

9.1 Formulation

Partition the model into N units (layers, tensors, channels, or groups) with parameter counts p_i. Choose bit-widths b_i from an allowed set $\mathcal{B}$ to minimize the loss increase subject to an average-bit budget:

$$\min_{b_i \in \mathcal{B}} \sum_{i=1}^{N} \Delta_i(b_i) \quad \text{s.t.} \quad \sum_i p_i b_i \le P\,\bar{b}, \qquad P = \textstyle\sum_i p_i. \tag{9.1}$$

Here $\Delta_i(b)$ is the (estimated) loss increase from quantizing unit i at b bits with all others at full precision. The additive form assumes independence of units, which is an approximation (see the discussion of EvoPress below). Problem (9.1) is a multiple-choice knapsack problem: NP-hard in general but solved exactly by dynamic programming for the small N and $|\mathcal{B}|$ of practice, or by integer linear programming (ILP), and approximated by greedy marginal-return rules.

9.2 The high-resolution optimum

Model the loss increase as quadratic in the quantization noise. If unit i's weights are perturbed by independent noise of variance σ_i^2, then to second order

$$\Delta_i \approx \tfrac{1}{2}\,\sigma_i^2\,\operatorname{tr}(\boldsymbol{H}_i), \tag{9.2}$$

where $\boldsymbol{H}_i$ is the Hessian of the loss with respect to unit i's parameters: $\mathbb{E}[\tfrac{1}{2}\boldsymbol{\delta}^\top \boldsymbol{H}_i \boldsymbol{\delta}] = \tfrac{1}{2}\sigma_i^2 \operatorname{tr} \boldsymbol{H}_i$ for isotropic noise. This is the idea behind HAWQ-V2's trace-based sensitivity [76]. With a uniform quantizer, $\sigma_i^2 = s_i^2/12$ and the step $s_i \propto r_i 2^{-b_i}$ with r_i the range, so

$$\Delta_i(b_i) \approx c_i\, 2^{-2b_i}, \qquad c_i \propto \operatorname{tr}(\boldsymbol{H}_i)\, r_i^2. \tag{9.3}$$

Table 9.1: Gain of optimal mixed precision over uniform allocation under the model (9.3) for $N = 64$ equal-size units with log-normal sensitivities of log-std ς (own simulation with random draws; "continuous" is (9.4) ignoring integrality; "greedy" restricts bits to $\{2, 3, 4, 5, 6, 8\}$ with a marginal-return rule). Gains in dB of total distortion reduction at average budgets 3 and 4 bits; $\varsigma = 2$ is shown but its continuous solution would assign negative bits to some units, which is infeasible.

ς	AM/GM	Continuous gain (dB)	Greedy gain, avg 3 b (dB)	Greedy gain, avg 4 b (dB)
0.5	1.12	0.49	0.13	0.13
1.0	1.41	1.50	1.11	1.18
1.5	2.08	3.18	2.71	2.83
2.0	4.68	6.70	5.66	6.16

Proposition 9.1 (Optimal real-valued bit allocation)**.** *Under* (9.3)*, ignoring integrality and the nonnegativity constraint $b_i \geq 0$, the minimizer of $\sum_i c_i 2^{-2b_i}$ subject to $\sum_i p_i b_i = P\bar{b}$ is*

$$b_i^* = \bar{b} + \tfrac{1}{2}\log_2 \frac{c_i/p_i}{\bar{\kappa}}, \qquad \bar{\kappa} = \exp\Big(\sum_i \tfrac{p_i}{P} \ln \frac{c_i}{p_i}\Big) \ \ (\textit{parameter-weighted geometric mean of } c_i/p_i). \tag{9.4}$$

The resulting minimum is $\sum_i c_i 2^{-2b_i^*} = P\,\bar{\kappa}\,2^{-2\bar{b}}$, and the gain over uniform allocation $b_i \equiv \bar{b}$ is the ratio of the arithmetic to the geometric mean (AM/GM): for $p_i \equiv p$,

$$\frac{D_{uniform}}{D_{opt}} = \frac{\frac{1}{N}\sum_i c_i}{\big(\prod_i c_i\big)^{1/N}} \geq 1. \tag{9.5}$$

Proof. Lagrangian: $\sum_i c_i 2^{-2b_i} + \mu \sum_i p_i b_i$. Setting the derivative to zero gives $-2\ln 2\, c_i 2^{-2b_i} + \mu p_i = 0$, i.e. $c_i 2^{-2b_i}/p_i = \theta$ for all i: equal marginal distortion per parameter. Hence $b_i = \frac{1}{2}\log_2 \frac{c_i}{p_i\theta}$. The budget constraint $\sum_i p_i b_i = P\bar{b}$ fixes $\log_2 \theta = \sum_i \frac{p_i}{P}\log_2 \frac{c_i}{p_i} - 2\bar{b}$, giving (9.4). The minimum value is $\sum_i p_i\theta = P\theta = P\bar{\kappa}2^{-2\bar{b}}$. For equal p_i, uniform allocation gives $D = 2^{-2\bar{b}}\sum_i c_i$ and the ratio is AM/GM, which is ≥ 1 with equality iff all c_i are equal. □

Proposition 9.1 is the discrete-unit analogue of reverse water-filling (Proposition 3.2): units with higher curvature-times-range-squared receive $\frac{1}{2}\log_2(c_i/c_j)$ more bits. Each factor of 4 in sensitivity buys one bit. The gain depends on the dispersion of sensitivities. If $\ln c_i$ is Gaussian with standard deviation ς across units, then AM/GM $= e^{\varsigma^2/2}$, equivalent to $\frac{\varsigma^2}{4\ln 2}$ bits per weight.

Table 9.1 shows the point: when layer sensitivities span a factor of $e^{\pm 1.5}$ (a plausible dispersion for transformer layers according to Hessian-trace studies on CNNs and the LLM sensitivity maps below, though we have not measured it for LLMs here), optimal allocation is worth about 3 dB, equivalent to 0.5 bit per weight; for nearly homogeneous layers, little. A discrete greedy allocation captures most of the continuous gain but, with few allowed bit-widths, none of the low-gain regime.

9.3 Sensitivity measures

Hessian-based (HAWQ family). HAWQ ranks layers by the top eigenvalue of the layer Hessian; HAWQ-V2 argues that the *average Hessian trace* weighted by the squared quantization perturbation is a better indicator (as in (9.3)); HAWQ-V3 formulates integer-only mixed precision as an ILP including hardware constraints [76–78]. Traces are computed by Hutchinson's stochastic estimator using Hessian–vector products. HAQ uses reinforcement learning with hardware feedback in the loop [75].

Fisher information. For models trained by maximum likelihood, the expected Hessian at the optimum equals the Fisher information $F = \mathbb{E}[\nabla\ell\nabla\ell^\top]$; its diagonal, computed from squared gradients on calibration data, is a cheap sensitivity. SqueezeLLM uses diagonal Fisher entries both to weight k-means centroids (sensitivity-based non-uniform quantization) and to decide which weights go to a sparse high-precision component [175]. SpQR and OWQ use the layer-wise Hessian diagonal from calibration activations ($\sum_t X_{tj}^2$, *activation energy*) to find sensitive columns [126, 174]; this equals the GPTQ-proxy Hessian diagonal, tying allocation to the objective of Proposition 3.5.

Linearity theorem. The HIGGS paper establishes a "linearity theorem": the perplexity increase due to quantization is, to first order, a sum of layer-wise ℓ_2 reconstruction errors weighted by per-layer coefficients, which justifies layer-wise objectives and enables an optimal solution of non-uniform per-layer bit-width assignment under a total budget by dynamic programming; the same work proposes a data-free method (HIGGS) using Hadamard rotations and MSE-optimal grids that outperforms NF4 [79].

Empirical perturbation. Quantize one layer at a time (or leave one layer at high precision) and measure the loss or KL divergence on a calibration set. This "leave-one-out" profile has cost proportional to N forward passes per candidate bit-width and is the simplest sensitivity map; it ignores interactions.

Interactions and EvoPress. The additive model (9.1) assumes layer errors are independent. EvoPress points out that this independence does not generally hold for LLM compression and proposes an evolutionary search over per-block compression levels that evaluates *whole-model* candidates against a global threshold, outperforming importance-score methods in the authors' tests across quantization and sparsity [49]. The lesson is that sensitivity profiles from single-layer perturbations are a good starting point but not the final answer.

9.4 Granularity of mixed precision

Allocation can act at several levels, with increasing flexibility but also increasing kernel difficulty.

(1) Layer/tensor level. Different tensors use different bit-widths (e.g. 8-bit embeddings and output head, 6-bit value and down-projections, 4-bit elsewhere). Each GEMM has

uniform bit-width, so existing kernels apply. This is the standard in production (GGUF `K_M` schemes, vLLM per-layer configs, ModelOpt recipes).

(2) Block/group level. Different groups within a matrix use different widths, e.g. SliM-LLM allocates bits at the group level by salience with a structured partition that keeps hardware efficiency close to uniform quantization [167].

(3) Channel level. Keep $< 1\%$ of channels in higher precision: OWQ, SpQR, Atom (outlier channels in INT8) [14, 126, 174]. Requires reorder-and-split GEMM kernels.

(4) Element level. Sparse outlier storage (SpQR, SqueezeLLM) with COO/CSR corrections; slower in the dense GEMM pipeline.

GGUF k-quants and importance matrices. In llama.cpp, the k-quant formats organize weights into super-blocks of 256 values with sub-blocks of 16 or 32 values, each with its own quantized scale (and minimum), the scales themselves stored with 6 bits or less relative to a per-super-block FP16 scale, giving effective rates around 4.5 bits per weight for `Q4_K`, 5.5 for `Q5_K`, and 6.56 for `Q6_K` [191, 194]. The `Q4_K_M` recipe is mixed-precision at the tensor level: as documented, it uses the higher-precision `Q6_K` for roughly half of the attention value and feed-forward down-projection tensors and `Q4_K` elsewhere, reflecting the sensitivity ordering noted above. An *importance matrix* (`imatrix`) computed from calibration activations weights the rounding error per input channel by the mean squared activation, which is exactly the diagonal of the GPTQ proxy Hessian. Community "dynamic" quantizations of very large MoE models extend the idea by assigning very different bit-widths to different tensor classes (for instance very low-bit routed experts and higher-precision attention and shared experts), which exploits the observation (Chapter 15) that routed experts dominate parameters but each is rarely used.

9.5 Any-precision and nested quantization

A different notion of flexibility is to store *one* checkpoint that can be run at several bit-widths.

- **Any-Precision LLM** overlays models of different bit-widths on shared storage via a bit-plane layout, with a serving engine that reads only the planes needed for the chosen precision [9].
- **Matryoshka Quantization (MatQuant)** exploits the nested structure of integer types: int4 and int2 live in the most significant bits of int8; it trains a single model whose int8 weights give accurate int4 and int2 slices, so one set of weights serves multiple quality/latency trade-offs [102].
- **BitStack** is a training-free decomposition that allows megabyte-granular memory-versus-quality trade-offs and dynamic adjustment of model size at run time [32].

These approaches target deployments with variable memory (on-device) and speculative decoding with draft models of different precision.

9.6 Learned and joint allocation

Gradient-based allocation within PTQ. SignRoundV2 assigns bit-widths layer by layer using gradient information and quantization error, adaptively within a mixed-MXFP setting; the authors report near-lossless results (about a 1% gap) at an average of 4.5 bits [164].

MoE-specific allocation. For MoE models, allocation at the *expert* level is natural because experts differ in activation frequency and importance. GEMQ proposes global expert-level mixed-precision quantization; MxMoE co-designs mixed-precision quantization with accuracy and performance for MoE kernels; AlphaQ proposes calibration-free bit allocation for MoE quantization (all as 2025–26 preprints, see Chapter 15) [8, 68, 112].

Training-integrated. Differentiable mixed precision (learned bit-widths via gradient descent with relaxation) is well established for CNNs; for LLMs it is rarer because of training cost. ParetoQ and related QAT studies instead compare uniform bit-widths across scales to expose the Pareto front (Chapter 10) [129].

9.7 Worked example: allocating bits across a 32-layer model

Suppose sensitivities (per-parameter, normalized) for a hypothetical 32-layer transformer are $c_\ell = 1$ for 24 layers, $c_\ell = 4$ for six layers (e.g. down-projection-heavy layers), and $c_\ell = 16$ for two layers (the first and last). With equal parameter counts and average budget $\bar{b} = 3$ bits, the geometric mean is $\bar{\kappa} = (1^{24}4^{6}16^{2})^{1/32} = 2^{(12+8)/32} = 2^{0.625} = 1.54$. The optimal allocation (9.4) is $b = 3 + \frac{1}{2}\log_2(c/1.54)$, i.e. 2.69 bits for the 24 baseline layers, 3.69 for the six, and 4.69 for the two. Rounding to the hardware-supported $\{2, 3, 4, 8\}$ is crude; a greedy discrete rule would give, for example, 3-bit for baseline layers and 4-bit for the eight sensitive layers, with average $(24 \cdot 3 + 8 \cdot 4)/32 = 3.25$ bits. Distortion relative to uniform 3 bits: uniform is $\sum c \cdot 4^{-3} = (24 + 24 + 32)/64 = 1.25$; the 3.25-bit mixed solution gives $(24 \cdot 4^{-3} + 6 \cdot 4 \cdot 4^{-4} + 2 \cdot 16 \cdot 4^{-4})/1 = (24/64) + (24/256) + (32/256) = 0.375 + 0.094 + 0.125 = 0.594$, a 2.1× (3.2 dB) reduction for an 8% budget increase. (Uniform 3.25 bits would give $80 \cdot 4^{-3.25} = 0.88$.)

9.8 Pitfalls

Risks of mixed precision

(i) Sensitivity measured by perplexity can mis-rank layers relevant to reasoning or long-context behaviour, which are not reflected in short-context perplexity. **(ii)** Kernel support is uneven: per-channel or per-group heterogeneity can erase the speedup; many serving stacks support only per-tensor bit-width. **(iii)** Budget accounting: include scale/zero-point and codebook overhead, and the cost of keeping embeddings and the head at high precision, which for small models can be a large share of parameters. **(iv)** Interactions between layers (Section 3.7) mean single-layer sensitivity underestimates joint damage when many layers are lowered.

Takeaways of the chapter

- Optimal bit allocation equalizes marginal distortion per parameter: $b_i = \bar{b} + \frac{1}{2}\log_2(c_i/p_i/\bar{\kappa})$; the gain over uniform is AM/GM of sensitivities (Proposition 9.1).
- Sensitivity can be estimated by Hessian trace, Fisher diagonal, activation energy, or leave-one-out perturbation; the linearity theorem links layer-wise ℓ_2 error to perplexity, but interactions mean whole-model search (EvoPress) can still beat additive scores.
- In production, tensor-level mixing (GGUF `K_M` recipes, 8-bit embeddings/head) is common; finer granularity requires kernel support.

Chapter 10

Quantization-Aware Training, Distillation, and Extreme Low-Bit Models

Post-training methods fit quantized weights to a frozen model using a few hundred calibration sequences. Quantization-aware training (QAT) instead lets the weights *adapt* to the quantizer by training with quantization in the forward pass. At 4 bits and above the gain over good PTQ is modest; at 2–3 bits and below it is large, and for ternary and binary weights it is the only route. This chapter covers the mechanics of QAT (straight-through estimation and learned step sizes), the efficient QAT recipes designed for LLMs, distillation, scaling laws for quantized training and inference, and extreme low-bit models from BinaryConnect to BitNet b1.58.

10.1 The mechanics of QAT

10.1.1 Fake quantization and the straight-through estimator

QAT maintains full-precision *master weights* $\boldsymbol{W}$ and, in the forward pass, uses $\widehat{\boldsymbol{W}} = \mathcal{Q}_s(\boldsymbol{W})$ (and similarly for activations). The rounding function has zero gradient almost everywhere, so one replaces its derivative in the backward pass by a surrogate: the *straight-through estimator* (STE) [17],

$$\frac{\partial\ \mathrm{round}(u)}{\partial u} := 1, \qquad \frac{\partial\ \mathrm{clip}(u,a,b)}{\partial u} := \mathbb{1}[a \le u \le b]. \tag{10.1}$$

With $\widehat{w} = s\,\mathrm{clip}(\mathrm{round}(w/s), q_{\min}, q_{\max})$, the STE gradient with respect to w is $\partial\mathcal{L}/\partial\widehat{w} \cdot \mathbb{1}[q_{\min} \le w/s \le q_{\max}]$: the loss gradient at the quantized weight is applied to the master weight, zeroed outside the clipping range. The update $w \leftarrow w - \eta\,\partial\mathcal{L}/\partial\widehat{w}$ then moves the master weight; it changes the integer code only when w/s crosses a rounding boundary.

10.1.2 Learned step size (LSQ)

LSQ [97] makes the step size s a trainable parameter and derives its gradient through the quantizer. For $\widehat{w} = s\,\bar{q}$ with $\bar{q} = \mathrm{clip}(\mathrm{round}(w/s), q_{\min}, q_{\max})$,

$$\frac{\partial\widehat{w}}{\partial s} = \begin{cases} \mathrm{round}(w/s) - w/s, & q_{\min} \le w/s \le q_{\max} \\ q_{\min} \text{ or } q_{\max}, & \text{otherwise} \end{cases} \tag{10.2}$$

by the chain rule with the STE applied to the rounding only. The first branch is the signed rounding residual of the code; the second says that clipped elements push the step up (to widen the range). LSQ scales the step-size gradient by $g = 1/\sqrt{N q_{\max}}$ (N = number of weights sharing s) so that relative updates of weights and step sizes are of similar magnitude, which was necessary for stable training [97]. PACT introduced learned clipping thresholds for activations

Algorithm 7 One QAT training step with fake quantization and STE

Require: master weights $\boldsymbol{W}$ (BF16/FP32), quantizer parameters (step sizes $\boldsymbol{s}$), batch (x, y), learning rate η

1: $\widehat{\boldsymbol{W}} \leftarrow \boldsymbol{s} \odot \text{clip}(\text{round}(\boldsymbol{W}/\boldsymbol{s}), q_{\text{min}}, q_{\text{max}})$ ▷ forward uses quantized weights
2: (optionally) $\widehat{\boldsymbol{X}} \leftarrow \mathcal{Q}_{\text{act}}(\boldsymbol{X})$ for activations; run forward pass to obtain loss $\mathcal{L}$
3: backward: obtain $\nabla_{\widehat{\boldsymbol{W}}}\mathcal{L}$ ▷ gradients w.r.t. dequantized weights
4: $\nabla_{\boldsymbol{W}}\mathcal{L} \leftarrow \nabla_{\widehat{\boldsymbol{W}}}\mathcal{L} \odot \mathbb{1}[q_{\text{min}} \leq \boldsymbol{W}/\boldsymbol{s} \leq q_{\text{max}}]$ ▷ STE
5: $\nabla_{\boldsymbol{s}}\mathcal{L} \leftarrow \sum \nabla_{\widehat{\boldsymbol{W}}}\mathcal{L} \odot \big(\text{round}(\boldsymbol{W}/\boldsymbol{s}) - \boldsymbol{W}/\boldsymbol{s}$ inside range; q_{min} or q_{max} outside$\big)$ ▷ LSQ
6: update $\boldsymbol{W}, \boldsymbol{s}$ with the optimizer (Adam), keeping master weights in high precision

[127], and DoReFa-Net low-bitwidth gradients and weights [42]. The LLM-specific recipes below inherit these components.

10.1.3 What the STE does (and does not) guarantee

The STE gradient is a *biased* surrogate. Yin et al. analyse coarse-gradient descent for training activation-quantized networks and show that, for a suitable choice of surrogate derivative (e.g. the derivative of a clipped ReLU), the expected coarse gradient correlates positively with the true population gradient, so descent is meaningful, while a poorly chosen surrogate can fail [185]. At extreme low bit-widths the weights move across rounding boundaries frequently and the STE becomes noisy; PV-Tuning shows that, in the setting of fine-tuning vector-quantized LLMs, alternating updates of continuous and discrete variables beats STE [132]. In the Adam optimizer the effective step on the latent weight is independent of gradient scale, which interacts with step size s: if $\eta \gg s$ the codes flip randomly every step, if $\eta \ll s$ they never change. The practical rule is to choose learning rates such that typical latent-weight updates are a fraction of one quantization step per epoch; ParetoQ finds that at ≤ 2 bits the training enters a different regime from ≥ 3 bits (see below).

Risk: QAT ruins what it does not measure

QAT on a small, narrow corpus (a few hundred million tokens) moves all weights. It can silently degrade capabilities absent from that corpus (long context, multilinguality, code, safety behaviour, tool use). Distillation from the original model on a broad distribution mitigates this, but the evaluation must include out-of-distribution tests (Chapter 13).

10.2 QAT for LLMs: recipes

10.2.1 LLM-QAT: data-free distillation

LLM-QAT [94] addresses the first obstacle to QAT on pretrained LLMs: the training data are often unavailable, and generic corpora do not match the output distribution of a post-trained chat model. Its solution is a data-free distillation: generate sequences from the pre-trained model itself and train the quantized student to match the teacher's output distribution on them. It quantizes weights, activations and the KV cache, evaluated on Llama 7B, 13B and 30B at levels down to 4 bits, with large improvements over training-free methods at the lowest bit-widths [94].

10.2.2 EfficientQAT: block-wise then end-to-end

Full-model QAT of a 70B model needs a training cluster. EfficientQAT [46] splits the problem in two phases: *Block-AP* trains all parameters of one transformer block at a time (weights *and* quantization parameters) to reconstruct the block's full-precision output, which enlarges the solution space relative to PTQ methods that train only clipping or scale parameters; and *E2E-QP* then trains only the quantization parameters (step sizes) end-to-end to account for interactions between blocks, with the integer weights frozen. The authors report that it outperforms prior quantization methods on base, instruction-tuned and multimodal models from 7B to 70B at 2–4 bits [46]. The memory cost is that of a single block plus step-size gradients, which makes 70B tractable on a single GPU in the authors' setting.

10.2.3 Distillation-based QAT

BitDistiller [24] combines QAT with self-distillation: it uses a tailored asymmetric quantization with clipping to preserve weight fidelity, and a *confidence-aware* KL objective (CAKLD) that interpolates between forward and reverse KL according to the teacher's confidence, reporting significant gains over prior methods at 3 and 2 bits on general language-understanding and reasoning benchmarks with less data and compute [24].

Why distillation. For a teacher distribution $p_T(\cdot \mid x)$ and a student p_S, the standard objectives are the forward KL $\mathrm{KL}(p_T \| p_S)$ (mode-covering) and reverse KL $\mathrm{KL}(p_S \| p_T)$ (mode-seeking). Gradient of the forward KL with respect to the student's logits z_S is $p_S - p_T$ per position; matching the whole distribution carries far more information per token than the one-hot label of cross-entropy, so fewer tokens suffice, and it preserves the *teacher's* behaviour (including post-training alignment) irrespective of the quality of the training corpus. In QAT of RLHF-tuned models, self-distillation with sampled or teacher-generated data is the default.

10.2.4 Parameter-efficient variants

QLoRA fine-tunes LoRA adapters on top of a frozen 4-bit NF4-quantized base with double quantization of scales and paged optimizers; it is parameter-efficient fine-tuning *of* a quantized model, not QAT, but it democratized fine-tuning of 65B models on a single 48 GB GPU [138]. LoftQ chooses the quantization and the LoRA initialization jointly so that $\mathcal{Q}(\boldsymbol{W}) + \boldsymbol{AB}$ approximates $\boldsymbol{W}$ (reducing the initial gap), and QA-LoRA makes the adapters quantization-aware so they can be merged into the quantized weights without re-quantizing [95, 134].

10.2.5 Low-data VQ-QAT

LC-QAT (2026) represents quantized weights through a learned affine mapping over discrete vectors, which provides a strong post-training initialization and permits fully differentiable training *without codebook lookups in the training forward pass*; the authors report outperforming leading QAT methods while using only 0.1% to 10% of the training data for 2-bit weight-only models [89]. This points to a trend: the best 2-bit results combine VQ-style representations with short, data-efficient QAT.

Table 10.1: Fine-tuning then quantizing gpt-oss-20b (LMSYS/NVIDIA ModelOpt blog [197]): pass rates on two validation sets. Post-training quantization (PTQ, MXFP4) after supervised fine-tuning (SFT) loses a lot on the second task; QAT recovers it. Numbers as reported in the post; single run on small validation sets.

Model	Multilingual val.	FalseReject val.
Original gpt-oss-20b (MXFP4)	16%	30%
SFT, BF16	99%	98%
SFT + PTQ (MXFP4)	89%	59%
SFT + QAT (MXFP4)	100%	97%

10.3 Industrial QAT and native low-precision releases

By 2025–2026 several model developers shipped natively quantized checkpoints, i.e. the quantized model is the released artifact and training included quantization:

- **gpt-oss**: OpenAI released two open-weight MoE models with the MoE weights in MXFP4 (4-bit micro-scaled floating point), so that the larger model runs on a single 80 GB GPU and the smaller one within 16 GB of memory [72]. The LMSYS/NVIDIA ModelOpt write-up notes that the MXFP4 checkpoint has limited training support and describes a QAT workflow (BF16 master weights, MXFP4 fake-quantization, small learning rates of about 10^{-5} to 10^{-4}) [197].
- **Kimi K2 Thinking**: Moonshot AI released a 1-trillion-parameter-scale MoE reasoning model whose model card describes native INT4 weight quantization obtained by quantization-aware training during post-training, aimed at faster low-latency generation [84, 201].
- **Gemma 3 QAT** and **Llama 3.2 quantized**: Google released QAT checkpoints of Gemma 3 aimed at running on consumer GPUs, and Meta released 1B/3B Llama 3.2 models using SpinQuant and a QLoRA-style QAT variant [67, 192, 200].
- **DeepSeek-V3**: trained with FP8 mixed precision at 671B-parameter scale (Chapter 11) [43].

Table 10.1 is one of the cleaner public demonstrations that, for *4-bit formats and fine-tuned behaviours*, PTQ after fine-tuning can lose much more than aggregated perplexity suggests, and that QAT (the fine-tune performed with the quantizer in the loop) removes most of the loss. It is a single benchmark pair, so we treat it as an existence proof, not a rate.

10.4 Scaling laws for quantized models

How should one trade parameters against bits? How does quantization damage change with model size and training data? Several empirical studies establish regularities.

***k*-bit inference scaling (Dettmers and Zettlemoyer).** Over more than 35,000 experiments with 3–8 bit weights across BLOOM, OPT, Pythia/NeoX and GPT-2 at scales from 19M to 176B parameters, they found that at a fixed total number of weight bits, 4-bit precision is almost universally optimal for zero-shot accuracy, that improving the bit-level trade-off is hard, and

that small block sizes and the data type (integer versus float) are the only things that helped [83]. This is the origin of the "4-bit is the sweet spot" rule (for PTQ of 2022-era methods and models).

Low-bit quantization favours undertrained LLMs. A study of more than 1500 quantized checkpoints found that larger models and models trained on fewer tokens suffer less quantization-induced degradation (QiD), while small models trained on many tokens suffer more; fitted laws predicted that low-bit quantization of future models trained with 100T tokens may be undesirable [179].

Scaling laws for precision. Kumar et al. propose precision-aware laws: training in lower precision reduces the model's "effective parameter count"; post-training quantization degradation *increases* with the amount of pre-training data, to the point that additional data can become harmful; and training larger models in lower precision may be compute-optimal. They fit 465 pretraining runs and validate up to 1.7B parameters and 26B tokens [162]. A related study fits scaling laws specifically for floating-point quantization training [65].

Scaling law for QAT. Chen et al. (268 QAT experiments) report that QAT quantization error decreases with model size but *increases* with more training tokens and with coarser quantization granularity, decompose W4A4 error into weight and activation parts (weight error grows faster with tokens), identify outliers in the FC2 (MLP down-projection) layer's activations as the primary W4A4 bottleneck, and show that mixed precision for that layer substantially reduces the error [136].

ParetoQ. A unified comparison of 1-, 1.58-, 2-, 3- and 4-bit QAT within a single framework: the authors find a *learning transition* between 2 and 3 bits (at ≥ 3 bits fine-tuned weights stay near the pre-trained distribution; at ≤ 2 bits representations change drastically), that ternary, 2-bit and 3-bit quantization offer comparable size-accuracy trade-offs and generally outperform 4-bit and binary, that a ternary 600M model beats the previous state-of-the-art ternary 3B model with one-fifth the parameters, and that 2-bit shows strong potential for memory reduction and speedup given hardware constraints [129].

Reconciling the studies. The "4 bits is optimal" result (PTQ, one-shot, scalar) and the "ternary/2/3-bit are competitive" result (QAT) are not contradictory: the former measures what rounding of a fixed model can achieve; the latter measures what retraining can achieve. Moving the target from 4 to 2–3 bits requires training. The undertrained-models result adds the caveat that as pretraining token budgets grow, post-hoc quantization gets harder (the weights carry more information), making QAT or native low-precision training increasingly important for the best models.

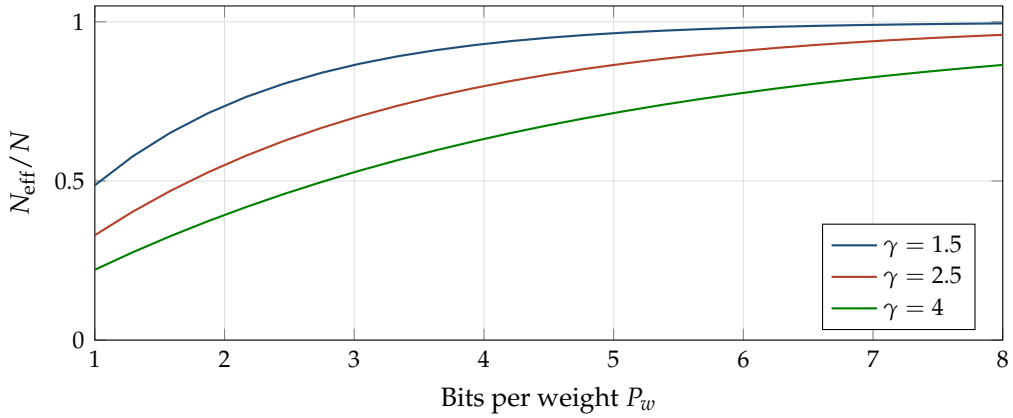


Figure 10.1: **Illustrative, not fitted.** A saturating-exponential form $N_{\text{eff}} = N(1 - e^{-P_w/\gamma})$ for the effective parameter fraction versus weight bit-width, which we recall as the functional form used for weights in the "scaling laws for precision" work [162]; the values of γ here are arbitrary examples chosen to show the qualitative shape (diminishing returns past 4–6 bits), not the paper's fitted constants.

10.5 Extreme low-bit: binary and ternary networks

10.5.1 Pre-LLM roots

BinaryConnect trained networks with binary weights using real-valued latent weights and the STE; Binarized Neural Networks extended to binary activations; XNOR-Net used binary weights and activations with scaling factors so that convolutions reduce to XNOR and pop-count operations [23, 33, 184]. Ternary variants add zero, which provides sparsity and better representation.

10.5.2 BitNet and BitNet b1.58

BitNet replaced linear layers by `BitLinear`: weights binarized to ± 1 after centering with a per-tensor scale $\beta = \frac{1}{nm}\sum |W_{ij}|$, activations quantized to 8 bits (absmax per tensor/token) after layer normalization, with an STE and latent full-precision weights; trained from scratch [31]. **BitNet b1.58** uses ternary weights $\{-1, 0, +1\}$ with the *absmean* quantizer

$$\gamma = \frac{1}{nm}\sum_{ij} |W_{ij}|, \qquad \widetilde{W}_{ij} = \text{clip}\big(\text{round}(W_{ij}/(\gamma + \epsilon)), -1, 1\big), \tag{10.3}$$

which carries $\log_2 3 \approx 1.58$ bits per weight; it adopts a LLaMA-like architecture (RMSNorm, SwiGLU, no biases) and 8-bit activations. The authors report that, trained from scratch, it matches the full-precision (FP16/BF16) Transformer of the same size and token count in perplexity and end-task performance beyond roughly the 3B scale in their comparisons, with lower latency, memory, throughput cost and energy [25].

Proposition 10.1 (Why ternary matmul is cheap)**.** *With weights in $\{-1, 0, 1\}$ and a scale γ, $\boldsymbol{y} = \gamma\, \widetilde{\boldsymbol{W}}\boldsymbol{x} = \gamma\,(\boldsymbol{W}^{+}\boldsymbol{x} - \boldsymbol{W}^{-}\boldsymbol{x})$ where $\boldsymbol{W}^{\pm}$ are the 0/1 indicator matrices of ± 1 weights. Hence the layer needs only additions and subtractions of activations (plus one scale), no multiplications, and the zero weights are skipped.*

Table 10.2: Qualitative summary of scaling-law studies of quantization (findings as stated in the cited abstracts).

Study	Setting	Headline finding
Dettmers & Zettlemoyer [83]	PTQ zero-shot, 19M–176B, 3–8 bit	4-bit almost universally optimal at fixed total bits; small blocks and data type help
Ouyang et al. [179]	$>$ 1500 quantized checkpoints	larger/undertrained models degrade less; future 100T-token models may quantize poorly
Kumar et al. [162]	465 pretraining runs, $\leq$ 1.7B/26B tok.	lower precision reduces effective parameters; PTQ damage grows with data
Chen et al. [136]	268 W4A4 QAT runs	error falls with size, rises with tokens and group size; FC2 activations dominate
Liu et al. (ParetoQ) [129]	QAT, 1–4 bit	learning transition at 2–3 bits; ternary/2/3-bit beat 4-bit and binary

This holds for any activation precision but pays off most when the adders operate on 8-bit activations: hardware could replace multipliers by adders, which is the energy argument of the BitNet papers [25]. On today's GPUs, however, there are no ternary tensor-core instructions: implementations pack ternary weights into 2 bits (or ≈ 1.6 bits per weight) and unpack to INT8 for the INT8 tensor-core GEMM, or use table lookups on CPUs (bitnet.cpp, T-MAC) [28, 177]. The memory benefit (roughly $8\times$ versus FP16 at 2 bits) is real, while the arithmetic-energy benefit materializes only on dedicated hardware.

10.5.3 Follow-ups

- **BitNet b1.58 2B4T**: a ternary-weight model of about 2B parameters trained on 4T tokens with open weights and inference code for GPU and CPU, reported as comparable to similarly sized full-precision open models while being more efficient in memory, energy and decoding latency [26].
- **BitNet a4.8 and v2**: extend to 4-bit activations (a4.8: hybrid quantization and sparsification to handle outlier activations; v2: native 4-bit activations using Hadamard transformation to tame outliers, echoing Chapter 7) [27, 30].
- **Spectra/TriLM**: a suite of ternary and FP16 language models from 99M to 3.9B parameters trained on the same data to study scaling of ternary pretraining; **BitNet b1.58 Reloaded** studies smaller networks [29, 172].
- **Binarized LLMs**: FBI-LLM trains fully binarized LLMs from scratch via autoregressive distillation; OneBit, PB-LLM, BiLLM and ARB-LLM binarize pre-trained models (post-training or with light training) at about 1 to 1.1 bits per weight with salient-weight handling or alternating refined binarizations [12, 22, 52, 119, 130].

Table 10.3: Decision aid: PTQ versus QAT (author's guidance).

Situation	Prefer	Reason
$\geq$ 4-bit weights, large dense model, generic use	PTQ (AWQ/G-PTQ)	loss already small; QAT cost not justified
W4A4 / FP4 activations	rotation PTQ; QAT if loss still too high	QAT scaling law: FC2 outliers dominate; mixed-precision FC2
2–3 bit weights	VQ/trellis PTQ + short fine-tune, or efficient QAT	scalar PTQ insufficient; transition at 2–3 bit
Post-trained/chat model, aligned behaviour must survive	distillation QAT from the FP model	preserves distribution beyond the calibration set
You are the model developer	native low-precision training or in-training QAT	only route to best $\leq$4-bit quality (gpt-oss, Kimi K2 Thinking pattern)
Edge device with CPU/NPU	ternary or 2-bit models with dedicated kernels	compute and memory savings both realized

Risks of the ternary/binary route

(i) Ternary models are *trained* as ternary; one cannot cheaply convert an existing FP16 model to b1.58 without heavy retraining (post-hoc ternarization of pretrained models loses a lot, as ParetoQ and the binarization literature show). (ii) Claims of parity are at modest scales (up to a few billion parameters at the time of the cited reports); behaviour at frontier scale (hundreds of billions of parameters, long context, reasoning RL) remains untested in public literature as of this writing to the best of our knowledge. (iii) Realized speedups on commodity GPUs are limited by the lack of native low-bit arithmetic; the full efficiency claims need CPUs, NPUs or custom silicon. (iv) The comparison baseline matters: a ternary model should be compared against the best 2–4 bit quantization of a similar-quality FP model at equal memory, which ParetoQ suggests is roughly on par.

10.6 When to use QAT

Takeaways of the chapter

- QAT = fake quantization + STE (+ learned step sizes); the STE is biased but effective at $\geq$ 3 bits and becomes fragile at $\leq$ 2 bits.
- Efficient QAT for LLMs (EfficientQAT, BitDistiller, LC-QAT) uses block-wise training, distillation and short budgets; distillation from the full-precision model is the default for aligned models.
- Scaling laws: PTQ damage grows with training tokens; QAT error shrinks with model

size; there is a learning transition between 2 and 3 bits; 4-bit is the PTQ sweet spot but ternary/2/3-bit QAT is competitive per bit.
- Native low-precision releases (gpt-oss MXFP4, Kimi K2 Thinking INT4) show quantization moving into the training pipeline; the fine-tune-then-PTQ failure documented in Table 10.1 is a practical reason.

Chapter 11

Floating-Point and Micro-Scaling Formats, Hardware, and Low-Precision Training

Until about 2022 the vocabulary of LLM quantization was integer: INT8, INT4 with scales. Since then, floating-point formats with 8, 6 and 4 bits, and *block-scaled* ("micro-scaling") variants, have moved from research to silicon. This chapter explains the formats (FP8 E4M3 and E5M2, FP6, FP4 E2M1, E8M0 and E4M3 scales, MX and NVFP4), compares them analytically and by simulation with integer formats, surveys hardware support, and reviews low-precision *training* with 8-bit and 4-bit arithmetic.

11.1 Floating-point basics

A binary floating-point format with e exponent bits and m mantissa bits (plus a sign bit) represents normal numbers $(-1)^s\, 2^{E-\text{bias}}\,(1 + M/2^m)$ with $E \in \{1, \ldots, 2^e - 2\}$ (or up to $2^e - 1$ in formats that reclaim the infinity encodings), and subnormals $(-1)^s 2^{1-\text{bias}}(M/2^m)$ for $E = 0$. The spacing of representable values doubles with each binade, so *relative* precision is roughly constant: a value x in binade $[2^k, 2^{k+1})$ has spacing 2^{k-m} and thus relative rounding error at most 2^{-m-1}.

Proposition 11.1 (Relative versus absolute error)**.** *Under round-to-nearest, the rounding error of an integer grid with step Δ has variance $\Delta^2/12$ regardless of the magnitude of x, whereas a floating-point format with m mantissa bits has error variance $\approx 2^{2(k-m)}/12$ for x in binade k, i.e. proportional to x^2 up to a factor in $[1/4, 1]$. Hence INT is better for values near the block maximum (the INT step at maximum range A with b bits is $\approx A/2^{b-1}$, finer than the FP spacing at the top binade) and FP is better for values far below the maximum.*

Proof. A uniform grid with step Δ gives $\Delta^2/12$ for each element (Proposition 3.3). For FP, the step in binade k is 2^{k-m} and $x \in [2^k, 2^{k+1})$; the same argument gives variance $2^{2(k-m)}/12$, and $2^{2k} \le x^2 < 2^{2k+2}$ gives the stated proportionality. For the comparison with INT: with the same block maximum A, the top binade of an FP format with e exponent bits has spacing $A\,2^{-m-1}$ (for A near the top of the binade), compared to the INT spacing $A/2^{b-1}$ with $b = e + m + 1$; for typical $e \ge 2$, $2^{-m-1} > 2^{-(b-1)}$ so the INT step is smaller near the top. □

Consequently the best format depends on the *crest factor* (maximum over RMS) of the block, a point confirmed by our simulation below and by comparative studies [62, 81].

Table 11.1: Floating-point and related element formats used in LLM inference and training. Maximum finite values and minimum subnormals are for the OCP variants [61, 206]. E4M3 follows the "FN" convention (no infinities; one NaN pattern) which extends the range to 448. AMD's MI300-series FP8 uses "FNUZ" variants with a different bias and maximum (240 for E4M3) as the author recalls; the OCP forms are now the standard interchange formats.

Format	Bits	e	m	Max finite	Min subnormal	Notes
FP16 (binary16)	16	5	10	65504	6.0×10^{-8}	IEEE
BF16	16	8	7	3.4×10^{38}	9.2×10^{-41}	FP32 range
FP8 E4M3	8	4	3	448	2^{-9}	forward activations, weights
FP8 E5M2	8	5	2	57344	2^{-16}	gradients
FP6 E3M2	6	3	2	28	2^{-4}	OCP MX
FP6 E2M3	6	2	3	7.5	2^{-3}	OCP MX
FP4 E2M1	4	2	1	6	0.5	values $\{0, 0.5, 1, 1.5, 2, 3, 4, 6\}$ with sign
E8M0 (scale only)	8	8	0	2^{127}	–	power-of-two scale
INT8, INT4	8, 4	–	–	127, 7	1	scale per block

11.1.1 The formats

FP8. The FP8 proposal of Micikevicius et al. defines E4M3 (more precision, smaller range) and E5M2 (more range, less precision) [61]. The recommended hybrid usage is E4M3 for forward-pass weights and activations and E5M2 for gradients (which have a wider dynamic range). Since a single FP8 tensor has limited range, practical systems add scaling factors (per tensor, per row, or per block) so that the tensor's maximum maps near the format's maximum.

FP4 E2M1. Sixteen codes: a sign and magnitudes $\{0, 0.5, 1, 1.5, 2, 3, 4, 6\}$. Eight magnitudes give a dynamic range of $12\times$ among non-zero values and non-uniform spacing (0.5, 0.5, 0.5, 0.5, 1, 1, 2). Without scaling, FP4 is useless for typical tensors; with a per-block scale, it is a compact non-uniform 4-bit grid.

11.2 Block-scaled (micro-scaling) formats

11.2.1 OCP MX

The Open Compute Project MX specification (version 1.0, September 2023) defines formats consisting of a block of $k = 32$ elements in a narrow element type sharing one 8-bit power-of-two scale (E8M0) [114, 206]. Element types include FP8 (E5M2, E4M3), FP6 (E3M2, E2M3), FP4 (E2M1) and INT8, giving MXFP8, MXFP6, MXFP4 and MXINT8. The effective bit-widths including the shared scale are $8 + 8/32 = 8.25$, 6.25 and 4.25 bits per element. The MX paper evaluates these formats on over two dozen benchmarks as drop-in replacements for FP32 for inference and training and reports the first training of generative language models with sub-8-bit weights, activations and gradients with minimal accuracy loss [114]; an earlier paper on shared microexponents introduced a two-level variant [113].

Scale rule. With block maximum $A = \max_i |x_i|$ and the element format's largest exponent $e_{\max}$ (so that its maximum finite value is about $2^{e_{\max}} \times$ a mantissa factor; $e_{\max} = 2$ for E2M1),

Algorithm 8 Quantization of a vector to MXFP4 (OCP rule) and NVFP4

Require: vector $\boldsymbol{x} \in \mathbb{R}^d$ (reshaped to blocks)
procedure MXFP4($\boldsymbol{x}$) ▷ block $k = 32$
  for each block B **do**
    $A \leftarrow \max_{i \in B} |x_i|$; $e \leftarrow \lfloor \log_2 A \rfloor - 2$; $X \leftarrow 2^e$ ▷ store e in E8M0
    $q_i \leftarrow \mathrm{round}_{E2M1}(x_i / X)$ (saturating at ± 6)
  end for
end procedure
procedure NVFP4($\boldsymbol{x}$) ▷ block $k = 16$
  $s_t \leftarrow \max_i |x_i| / (6 \cdot 448)$
  for each block B **do**
    $s_B \leftarrow \mathrm{round}_{E4M3}\big(\max_{i \in B} |x_i| / (6 s_t)\big)$
    $q_i \leftarrow \mathrm{round}_{E2M1}\big(x_i / (s_B s_t)\big)$
  end for
end procedure

the OCP reference rule sets the shared exponent $\lfloor \log_2 A \rfloor - e_{\max}$, i.e. $X = 2^{\lfloor \log_2 A \rfloor - e_{\max}}$, and elements are $\mathrm{round}_{\mathrm{FP}}(x_i / X)$ with saturation. For MXFP4 this means $A/X \in [4, 8)$, so values above 6 saturate: elements in the block with $A/X \in (6, 8)$ are clipped. Alternative rules round the scale up ($X = 2^{\lceil \log_2 (A/6) \rceil}$), avoiding clipping at the cost of coarser resolution (we compare both below).

11.2.2 NVFP4

NVIDIA's NVFP4 uses E2M1 elements with a block of $k = 16$ and a scale in FP8 E4M3 (finer than a power of two) plus a per-tensor FP32 scale to keep block scales in E4M3's range [116, 203]. Writing $s_t = \max |\boldsymbol{x}| / (6 \cdot 448)$ for the tensor scale and $s_b = \mathrm{round}_{E4M3}\big(A_b / (6\, s_t)\big)$ for block b,

$$\hat{x}_i = \mathrm{round}_{E2M1}\big(x_i / (s_b s_t)\big) \cdot s_b s_t. \tag{11.1}$$

The effective cost is $4 + 8/16 = 4.5$ bits per element plus a negligible amount for the tensor scale. Two properties distinguish it from MXFP4: smaller blocks (16 versus 32), which track local magnitude better, and a scale with 3 mantissa bits, which loses far less range-fitting precision than a power-of-two scale.

11.2.3 Why block-scaled formats are hardware-friendly

Proposition 6.1 showed that scales along the reduction dimension cannot be applied after an integer accumulation. Micro-scaled tensor-core instructions solve this by taking the two block-scale vectors as operands: the hardware multiplies the element products within a block, accumulates, and applies the product of the two block scales before adding to the accumulator, so that per-block (along K) scaling costs essentially no throughput. This is the central architectural point of MX/NVFP4 tensor cores on Blackwell and of AMD's CDNA4 MX support, and is why small block sizes (16 or 32) are feasible in hardware while they would be intractable in a software dequantization loop.

Table 11.2: SQNR (dB) of 4-bit block formats *without* rotation, by block size (own simulation; synthetic sources; higher is better). G = Gaussian, T = Student-t_3, O = sparse outliers. Column pairs are block size 16 / 32.

	G16	G32	T16	T32	O16	O32
INT4, FP16 scale	21.4	20.3	18.9	16.8	19.1	16.4
INT4, power-of-two scale	17.6	16.8	15.6	13.7	16.8	14.5
E2M1, FP16 scale	20.5	19.9	21.0	19.5	21.0	18.7
MXFP4 (E8M0, OCP rule)	18.6	18.8	17.4	17.0	16.7	15.8
MXFP4 (E8M0, round-up rule)	18.9	18.7	17.5	16.4	17.2	15.6
NVFP4 (E4M3 scale, FP32 tensor)	20.4	19.9	20.8	19.4	20.8	18.6

Table 11.3: As Table 11.2 but *with* a random-sign Hadamard rotation over rows of 256 elements applied before quantization and inverted afterwards (own simulation).

	G16	G32	T16	T32	O16	O32
INT4, FP16 scale	21.4	20.3	21.6	20.6	22.6	21.7
INT4, power-of-two scale	17.6	16.8	17.9	17.0	18.9	18.1
E2M1, FP16 scale	20.5	19.9	20.5	19.9	20.2	19.7
MXFP4 (E8M0, OCP rule)	18.6	18.8	18.6	18.8	18.9	19.0
MXFP4 (E8M0, round-up rule)	19.0	18.8	19.0	18.8	19.2	19.1
NVFP4 (E4M3 scale, FP32 tensor)	20.4	19.9	20.4	19.8	20.1	19.7

11.3 Analytical and simulated comparison of 4-bit block formats

We simulate the quantization error (SQNR, dB) of 4-bit block formats on three synthetic sources, 1M samples each: i.i.d. $\mathcal{N}(0,1)$; Student-t with 3 degrees of freedom (heavy tails); and a "sparse outlier" source ($\mathcal{N}(0,1)$ with 0.5% of entries multiplied by 20). The "rotation" columns apply a random-sign Hadamard transform along rows of 256 before block quantization and invert it after, so that errors are measured in the original domain. Formats: INT4 symmetric with an FP16 scale or a power-of-two scale; E2M1 with an FP16 scale; MXFP4 (power-of-two E8M0 scale, the OCP rule and the round-up variant); NVFP4 (E4M3 block scale, FP32 tensor scale). Block sizes 16 and 32.

The simulation supports six observations.

1. **On Gaussian data, INT4 with a precise scale wins** (21.4 dB at block 16 versus 20.5 for E2M1 and 20.4 for NVFP4). This follows from Proposition 11.1: a block of 16 Gaussians has a small crest factor ($\approx$ 2.1 standard deviations), so absolute precision near the maximum matters more than dynamic range.
2. **On heavy-tailed or outlier-rich blocks, E2M1 wins by 2–3 dB** (e.g. T16: 21.0 versus 18.9; O32: 18.7 versus 16.4) because the non-uniform grid resolves small values while handling the maximum.
3. **A power-of-two scale costs 3–4 dB with INT4** (17.6 versus 21.4 on Gaussian at block 16) and about 1.8 dB with E2M1 (MXFP4 vs. FP4 with FP16 scale), so MXFP4 is 1.1–1.8 dB behind NVFP4 on Gaussian data and 3–4 dB on outlier-rich data. The OCP rule and the round-up rule differ by < 0.6 dB, with neither dominant.
4. **NVFP4 nearly equals ideal E2M1 with an FP16 scale** ($\leq$ 0.2 dB loss), i.e. the E4M3 scale

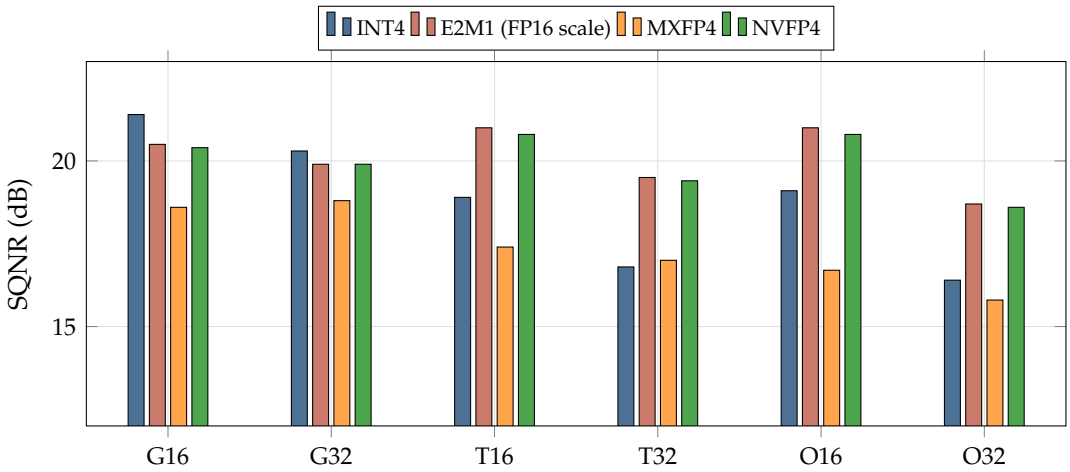


Figure 11.1: Data of Table 11.2 (own simulation, synthetic sources, no rotation). INT4 uses an FP16 scale. The crossover between INT and FP depends on the crest factor of the data.

is almost as good as an unquantized one.

5. **Rotation helps INT4 substantially on heavy-tailed data and barely helps (or slightly hurts) FP4 formats**: with rotation the INT4-FP16-scale rows on T and O exceed all E2M1-based rows (e.g. O32: 21.7 versus 19.7 for NVFP4), echoing a study that found NVINT4 can surpass NVFP4 once Hadamard rotation is applied while FP4 formats hold the advantage without it [81].
6. **Smaller blocks help most when the scale is precise**: 16 versus 32 gains about 1.1 dB for INT4/FP16 scale on Gaussian data, but nothing for MXFP4 on Gaussian (coarse scale dominates).

Caveat on interpreting the simulation

These are *signal-level* SQNR figures on synthetic distributions with absmax scaling and RTN. Real tensors have structured outliers; GPTQ-style error compensation changes the picture; hardware may not support some combinations (e.g. INT4 with FP16 scales at block 16 is not a native tensor-core format on Blackwell). The ranking is a mechanism-level guide, not a prediction of end-task accuracy. For measured end-task results see the MR-GPTQ study, which reports that NVFP4 and MXFP4 PTQ both fall short of their promise without format-specific methods and that MR-GPTQ narrows the MXFP4–NVFP4 gap [110].

11.4 PTQ for micro-scaled formats

Applying GPTQ or RTN to FP4 blocks is straightforward in principle but reveals format-specific issues. The MR-GPTQ study identified that NVFP4's small group size undermines traditional outlier-mitigation techniques while MXFP4's power-of-two scale quantization introduces high error; MR-GPTQ adds block-wise Hadamard transforms and format-specific optimizations to GPTQ, and with custom GPU kernels that fuse the rotation into the weights it reports speedups over FP16 of up to 3.6× layer-wise and 2.2× end-to-end on NVIDIA B200, and up to 6× layer-wise and 4× end-to-end on RTX 5090 [110]. Four Over Six (4/6) proposes adaptive block scaling for NVFP4: scaling some blocks so that their maximum maps to a

Table 11.4: Native low-precision tensor-core support by accelerator family, as understood by the author at the time of writing (October 2026). Vendor roadmaps change; consult current documentation. Entries with "announced" reflect public statements that the author could not verify against shipping products.

Platform	Integer	Floating point
NVIDIA Ampere (A100)	INT8, INT4 tensor cores	TF32, FP16, BF16
NVIDIA Hopper (H100/H200)	INT8	FP8 (E4M3, E5M2), BF16/FP16; no hardware FP4
NVIDIA Ada (L40S, RTX 40)	INT8, INT4 (legacy)	FP8
NVIDIA Blackwell (B200/GB200, RTX 50)	INT8	FP8, FP6, FP4: MXFP8/6/4 and NVFP4 tensor-core GEMM (confirmed for RTX 5090 and B200 by [110, 160])
NVIDIA Rubin (announced/early)	INT8	NVFP4 with adaptive-precision tensor cores; claimed $\sim$50 PFLOP/s NVFP4 inference per GPU [193]
AMD Instinct MI300X/MI325X	INT8	FP8 (FNUZ-style variants), FP16/BF16
AMD Instinct MI350-series (announced)	INT8	FP8; MXFP6/MXFP4 support announced
Intel Gaudi 2/3	INT8	FP8 (used for FP8/FP4-simulated training in [58, 64])
Google TPU (v5e/v6e/v7)	INT8 (and INT4 on some)	BF16; FP8 support announced for the latest generation
Mobile/edge NPUs	INT8/INT4 widely	varies; FP16 common

smaller FP4 value, making the representable grid effectively more uniform near the block maximum, where E2M1's spacing is widest (steps of 2 between 4 and 6); it was evaluated in NVFP4 training on a 30B-A3B-architecture model [57]. SignRoundV2 reports near-lossless mixed-MXFP configurations [164]. LLM-FP4 earlier studied PTQ to 4-bit floating point with searched exponent bits and clipping and per-channel activation scales reparameterized as exponent biases [92], and ZeroQuant-FP studied W4A8 with floating-point formats [187]. SageAttention3 uses Blackwell's FP4 tensor cores for attention, reporting 1038 TOPS on an RTX 5090 (about 5$\times$ faster than the fastest FlashAttention on that GPU) [160]; FlashAttention-3 and SageAttention develop 8-bit attention with block quantization and incoherent processing [51, 161].

11.5 Hardware support

Three consequences follow. (1) *Format fragmentation*: the same bits mean different things on different silicon (OCP versus FNUZ FP8; MX versus NVFP4). (2) *Software emulation*: on hard-

ware without native support (Hopper for FP4), FP4 formats can still save memory (weight-only dequantization to FP8 or BF16) but not compute. (3) *Lock-in*: a model released in NVFP4 is optimally served on Blackwell and later; a model released in INT4 with groups runs acceptably everywhere through weight-only kernels. This hardware coupling is a central risk in Chapter 18.

11.6 Low-precision training

11.6.1 Mixed-precision lineage

Mixed-precision training with FP16 compute and FP32 master weights plus loss scaling [108], and BF16 training thanks to its FP32-like exponent range [21], made 16-bit training standard. The next steps are 8-bit and 4-bit matrix multiplications with master weights and accumulation in higher precision.

11.6.2 FP8 training

FP8 GEMMs on Hopper-class hardware run at 2× the FP16 rate. The key design questions are *scaling granularity* (per tensor with delayed or just-in-time scaling versus per block) and which tensors use which format. Reports at scale include:

- **FP8-LM**: an FP8 automatic mixed-precision framework in which gradients, optimizer states and distributed communication progressively use FP8; on GPT-175B with H100 GPUs it reports a 39% reduction in memory and 75% faster training than a BF16 Megatron-LM baseline and 37% faster than NVIDIA's Transformer Engine [63].
- **DeepSeek-V3**: a 671B-parameter MoE trained with FP8 mixed-precision, using fine-grained scaling: as we recall the report, activations are quantized in 1×128 tiles and weights in 128×128 blocks, E4M3 is used for all GEMM operands, and partial sums are promoted to higher-precision accumulators periodically to control accumulation error; the authors report that the relative loss error versus a BF16 baseline stayed below a small fraction of a percent [43].
- **Scaling FP8 training to trillion-token LLMs**: runs of up to 2T tokens exposed instabilities traced to outlier amplification by SwiGLU appearing only over prolonged training (weight alignment); Smooth-SwiGLU removes it without changing the function; FP8 quantization of both Adam moments is demonstrated; a 7B model on 256 Gaudi2 chips matches BF16 with about 34% higher throughput [64].
- **COAT**: compresses optimizer states (dynamic range expansion) and activations (mixed-granularity quantization) for FP8 training, reporting 1.54× memory reduction and 1.43× speedup over BF16 [38].
- **Unit scaling** designs models so that tensors have unit scale by construction, reducing the need for dynamic scaling [180]; **8-bit optimizers** use block-wise quantization of Adam states [120].

Lesson from the trillion-token FP8 study

Instabilities in low-precision training may appear only after a very long training horizon. Short pilot runs can pass while long runs fail due to slowly developing outliers (here, SwiGLU weight alignment). Low-precision training recipes must be validated at the target token budget or with diagnostics that track outlier growth.

11.6.3 FP4 training

FP4 training with 16-value grids is much harder: quantization noise rivals gradient signal. The recipes that work share ingredients:

1. **Small blocks with precise scales**: NVFP4 (blocks of 16 with E4M3 scale) was found optimal among the formats examined in *FP4 All the Way* [58]; MXFP4 training uses blocks of 32 with power-of-two scales [111].
2. **Unbiased rounding for gradients**: stochastic rounding (SR) in backward and update passes and RTN in the forward pass [58]; SR with variance bounded by random Hadamard transforms [111]; lower-variance unbiased estimators (MS-EDEN in Quartet II, with more than $2\times$ lower quantization error than SR) [147].
3. **Outlier control**: random Hadamard transforms before quantization; outlier clamping and compensation [59, 116].
4. **Selective high precision**: a few layers (typically the first and last, plus numerically sensitive ones) remain in BF16 or FP8 [116].
5. **Consistent forward/backward quantization**: NVIDIA uses two-dimensional (16×16) block scaling for weights so that the forward and backward passes see the same quantized weights [116].

Stochastic rounding and why it is needed. For a weight update $w \leftarrow w - \eta g$ where the weight is stored or used in low precision, RTN of small updates results in *stagnation*: updates smaller than half a grid spacing are rounded away, so weights never move. Stochastic rounding is unbiased, $\mathbb{E}[\mathrm{SR}(u)] = u$, so small updates accumulate in expectation. For a dot product $y = \sum_i w_i x_i$ with SR applied to w on a grid of step Δ, the error is $\sum_i x_i e_i$ with independent zero-mean e_i with $\mathrm{Var}(e_i) \le \Delta^2/4$, so $\mathrm{Var}(\text{error}) \le \Delta^2 \|\boldsymbol{x}\|^2 / 4$: the variance is controlled by the grid step set by the block maximum, which is why random Hadamard transforms (reducing the maximum) reduce SR variance [111]. The FP4-All-the-Way threshold result adds: when the gradient norm falls below about $\sqrt{3}$ times the quantization noise, quantized training becomes less effective [58], suggesting precision should rise (or the learning rate decay be accompanied by a precision schedule) late in training.

Unresolved questions about FP4 training

(i) Public evidence extends to 12B parameters/10T tokens for one organization's recipe and to $\le$ 13B parameters in academic work; behaviour at frontier scale and in RL post-training is not documented publicly to our knowledge. (ii) FP4 training reduces effective parameter counts [65, 162]; the net compute-optimal trade-off at scale is open. (iii) Recipes are tied to Blackwell-class formats and kernels. (iv) Numerics may differ between the training kernel and the serving kernel, creating train–inference mismatch in RL pipelines.

Table 11.5: Reported 4-bit training results (claims as stated in the cited abstracts; none independently reproduced by the author).

Work	Scale	Claim
Wang et al. (FP4 framework) [59]	up to 13B params, 100B tokens	accuracy comparable to BF16/FP8 with minimal degradation
Tseng et al. (MXFP4) [111]	GPT up to 6.7B	near-lossless; $> \frac{1}{2}$ of training FLOPs in MXFP4; est. $> 1.3\times$ over FP8, $> 1.7\times$ over BF16 in backprop
Quartet [148]	Llama-style, Blackwell kernels	fully FP4 linear layers competitive with FP8/FP16 per its scaling law
Chmiel et al. (FP4 All the Way) [58]	7B params, 200B tokens (Gaudi2)	NVFP4 best; comparable to BF16; gradient-norm vs. noise threshold ($\approx \sqrt{3}$)
NVIDIA (NVFP4 pretraining) [116]	12B params, 10T tokens	loss/accuracy comparable to FP8 baseline; "longest publicly documented" 4-bit run
Quartet II [147]	up to 1.9B params, 38B tokens	improved unbiased gradients; kernels up to $4.2\times$ over BF16 on Blackwell
Four Over Six [57]	30B-A3B-architecture pretraining	closer to BF16 loss than prior NVFP4 recipes

11.7 Integer versus floating point: the live debate

An older whitepaper from Qualcomm AI Research concluded that FP8 may be good for training but that INT8 was sufficient and cheaper for on-device inference, stating that FP formats are 50–180% less efficient in dedicated compute hardware [62]. The 2025 fine-grained study finds a nuanced *crossover*: FP wins at coarse granularity, but at block-wise granularity MXINT8 is superior to MXFP8 in accuracy and hardware efficiency; for 4-bit formats FP often holds an accuracy advantage, yet NVINT4 can surpass NVFP4 when Hadamard rotation is applied; the authors argue that a one-size-fits-all FP trajectory is suboptimal [81]. Our simulation agrees on direction. Meanwhile, the empirical Llama-3.1 study found FP8 W8A8 effectively lossless and well-tuned INT8 W8A8 within 1–3% accuracy degradation [20].

Takeaways of the chapter

- FP formats have constant relative precision, INT constant absolute precision; the best choice depends on block crest factor. Block scaling with small blocks makes both viable at 4–8 bits.
- MXFP4 (32-element blocks, E8M0 scale, 4.25 bit) pays 1.5–4 dB for power-of-two scales relative to NVFP4 (16-element blocks, E4M3 scale, 4.5 bit); NVFP4 is within 0.2 dB of ideal E2M1 with an exact scale (own simulation).
- Hardware: FP8 since Hopper/Ada/MI300/Gaudi2; FP4/FP6 MX and NVFP4 since Blackwell; AMD MI350 and NVIDIA Rubin continue the trend. Format fragmentation

and lock-in are real.
- FP8 training is production-grade (DeepSeek-V3, FP8-LM); FP4 training is credible up to 12B/10T tokens with RHT, SR, 2D scaling and selective high precision.

Chapter 12

KV-Cache Quantization

This chapter treats only the *quantization* of the key–value (KV) cache: representing cached keys and values with fewer bits. Eviction, merging, sparse attention and other attention-compression techniques belong to a companion paper and appear here only where a quantization idea is used inside them.

12.1 Why the KV cache is a distinct problem

In Chapter 2 we computed the KV size per token (2.3) and noted that at long context or high concurrency it rivals or exceeds the weights. There is a second, sharper reason to treat it separately.

Proposition 12.1 (Attention decoding is always memory-bound)**.** *Consider decoding one token for one sequence, with T cached tokens, head dimension d_h, and g query heads sharing each KV head (grouped-query attention, $g = 1$ for multi-head attention). For one KV head, the attention reads $2Td_h$ cached elements of b bits and performs $4gTd_h$ FLOPs (two multiply-adds per element per query head: $QK^\top$ and PV). The arithmetic intensity is*

$$I_{\text{attn}} = \frac{4gTd_h}{2Td_hb/8} = \frac{16\,g}{b}\ \textit{FLOP/byte}, \tag{12.1}$$

independent of the context length T and of the batch size.

Proof. Direct computation. The batch size does not matter because each sequence has its own cache, so unlike weights the cache bytes are not amortized across sequences. □

With $b = 16$ and $g = 8$ (typical for modern GQA models) the intensity is 8 FLOP/byte, two orders of magnitude below the ridge points of Table 2.2. *Weight* traffic is amortized by batching, but KV traffic scales with batch and context; hence at large batch sizes and long contexts decoding time is dominated by reading the KV cache, and quantizing it yields near-linear speedups of attention with the byte reduction. This is the foundation of the FP8 KV cache's near-universal adoption in production stacks (vLLM, TensorRT-LLM offer FP8 KV caches as a switch [204, 214]) and of research on 4-bit and 2-bit caches.

12.2 Statistics of keys and values

Studies of KV cache element distributions found asymmetric behaviour: *key* tensors exhibit a few channels (feature dimensions) with consistently large magnitude across tokens, whereas *value* tensors do not show strong channel structure but vary more across tokens [85, 87]. KIVI's analysis led to the recommendation to quantize keys *per channel* (groups along the token

dimension within a channel) and values *per token* [85]. KVQuant likewise adopts per-channel key quantization and adds pre-RoPE key quantization (rotary position embedding mixes channel pairs by position-dependent rotation, scrambling the stable per-channel statistics, so keys are quantized before RoPE and rotated after dequantization), non-uniform sensitivity-weighted datatypes, and per-vector dense-and-sparse outlier handling, reporting under 0.1 perplexity degradation at 3 bits on WikiText-2 and C4 for several Llama and Mistral models [87].

12.2.1 Error in attention logits: per-token versus per-channel

Let a key be $\boldsymbol{k} \in \mathbb{R}^d$ and a query $\boldsymbol{q}$. The attention logit is $\boldsymbol{q}^\top \boldsymbol{k}/\sqrt{d}$. Suppose each coordinate of $\boldsymbol{k}$ is quantized with step Δ_c (so rounding noise of variance $\Delta_c^2/12$, independent across channels). Then the logit-error variance is

$$\mathrm{Var}\big[\delta\ell\big] = \frac{1}{12\,d}\sum_{c=1}^{d} q_c^2\,\Delta_c^2. \tag{12.2}$$

Proposition 12.2 (Logit-error decomposition)**.** *For independent rounding errors of variance $\Delta_c^2/12$ on key channel c, the logit-error variance is* (12.2)*. The choice of grouping affects the result only through the steps $\{\Delta_c\}$: per-token groups of g channels give every channel in a group the step $\Delta_G^{\mathrm{tok}} \propto \mathrm{range}_G(\textit{this token})$, while per-channel groups of g tokens give channel c the step $\Delta_c^{\mathrm{ch}} \propto \mathrm{range}_c(\textit{these tokens})$.*

Proof. $\delta\ell = \sum_c q_c\,\delta k_c/\sqrt{d}$ with independent zero-mean δk_c of variance $\Delta_c^2/12$; the variance of the sum is $\sum_c q_c^2\Delta_c^2/(12d)$. □

Remark 12.1 (Which grouping wins; informal). (i) If the query has comparable energy on all channels, the error is dominated by the *largest steps*. A per-token group that contains an outlier channel gives every channel of that group, including the ordinary ones, a step set by the outlier, so per-channel grouping, in which each channel has its own step, wins by roughly the ratio of the mean squared steps. (ii) If the query concentrates its energy on the outlier channels themselves (as when query and key outliers are tied), the error is dominated by the outlier channels' own steps. There, a per-token step reflects the outlier value *at that token*, typically about one standard deviation of the channel, whereas a per-channel step reflects the *extreme* of the channel over the token group (about 2–2.5 standard deviations for 32 tokens), roughly 2× larger, so per-token can win. Real models lie between the two cases.

Table 12.1 shows a simulation (own) with $T = 2048$ tokens, $d = 128$, six outlier channels with scale 8–16× the rest, 32-element groups, and asymmetric min–max quantization. We report the relative RMS error of attention logits (keys) or attention outputs (values) under two query models: isotropic queries and queries whose scale follows the key channel scale.

This illustrates why the literature's per-channel-key recommendation is robust in practice but not universal: in real transformers, query and key outlier channels are related (both are linear functions of the same residual stream, and RoPE rotates pairs), and empirical studies find per-channel key quantization superior at low bits [85, 87]; rotation-based methods avoid the question by removing the channel structure (below).

Table 12.1: Relative RMS error of attention logits (K) and outputs (V) under per-token and per-channel asymmetric quantization with group size 32 (own simulation on synthetic keys with 6 outlier channels; values i.i.d. Gaussian). Which grouping wins for keys depends on the query model (Proposition 12.2). Value results are insensitive to grouping, as expected without channel structure.

	Keys, isotropic $\boldsymbol{q}$		Keys, outlier-scaled $\boldsymbol{q}$		Values	
Bits	per-token	per-channel	per-token	per-channel	per-token	per-channel
4	0.107	**0.068**	**0.017**	0.068	0.081	0.078
3	0.237	**0.145**	**0.037**	0.145	0.164	0.166
2	0.528	**0.340**	**0.085**	0.340	0.387	0.388

Algorithm 9 KIVI-style decoding step with per-channel key and per-token value quantization (schematic)

Require: new key $\boldsymbol{k}_t$, value $\boldsymbol{v}_t$; quantized caches K_q, V_q with scales; FP16 residual buffers K_r, V_r (length $< R$); group size g, residual limit R

append $(\boldsymbol{k}_t, \boldsymbol{v}_t)$ to residual buffers
if $|K_r| = R$ **then**
    keys: for each channel c and each group of g tokens, compute min/max over the g tokens; quantize to b bits (per-channel, asymmetric); append to K_q
    values: for each token and each group of g channels, compute min/max; quantize (per-token); append to V_q
    clear residual buffers
end if
attention: logits against K_q (dequantized on the fly) and K_r (FP16); softmax; output $= P_q V_q + P_r V_r$

12.3 Method families

12.3.1 Group quantization with a full-precision residual window: KIVI and SKVQ

KIVI is a tuning-free asymmetric 2-bit scheme: keys per channel, values per token, group size g (typically 32), with a *residual* of the most recent R tokens kept in FP16 [85]. The residual is needed because per-channel key quantization groups along the token axis: a new token cannot be quantized until a full group has accumulated. As a side benefit the most recent tokens, which carry the highest attention mass in most tasks, stay exact. SKVQ makes the window explicit ("sliding-window KV quantization"), reorders channels so that channels in a group have similar statistics, applies clipped dynamic quantization at group level, and keeps the most recent window in high precision; it reports quantizing keys to 2 bits and values to 1.5 bits with minimal accuracy loss in its evaluation [166].

12.3.2 Non-uniform, sensitivity-weighted, outlier-aware: KVQuant, QAQ, GEAR

KVQuant (above) adds non-uniform quantization with layer-wise sensitivity-weighted datatypes [87]. QAQ shows theoretically that key and value caches have different quantization

sensitivities and designs separate non-uniform schemes with dedicated outlier handling and an attention-aware method, reporting up to 10× compression with little loss [135]. GEAR quantizes most entries (of similar magnitude) to ultra-low precision, approximates the quantization error with a low-rank matrix and corrects outlier entries with a sparse matrix, with the motivation that autoregressive decoding compounds per-step errors [66]. KVTuner searches layer-wise mixed-precision key/value precision pairs offline using sensitivity analysis, noting that the key cache is generally more important than the value cache [88].

12.3.3 Exploiting redundancy: AQUA-KV

AQUA-KV observes dependencies between keys and values across layers and uses compact adapters to predict the KV of a layer from earlier ones, quantizing only the unpredictable part (the residual), with high compression at maintained accuracy [11]. It is an instance of predictive coding: the rate–distortion benefit of conditioning on side information (the earlier layers).

12.3.4 Rotation- and sketch-based methods

- **QuaRot/SpinQuant** rotate queries and keys (online, after RoPE) and values (merged into projections), enabling 4-bit KV with the rest of W4A4 [146, 173].
- **RotateKV** proposes outlier-aware rotations (channel reordering to adapt Hadamard rotations to per-channel outlier distributions) and pre-RoPE grouped-head rotation for accurate 2-bit KV quantization [157].
- **OSCAR and OptR (2026)** refine rotation-based INT2 KV: OSCAR estimates attention-aware covariance structures offline to derive fixed rotations and clipping thresholds, ships an INT2 attention kernel compatible with paged KV serving in SGLang and vLLM, and evaluates reasoning models with traces up to 32k tokens; OptR learns per-head orthogonal corrections minimizing the post-W_O attention-output error with an attention-equivalent key reparameterization, reporting consistent improvements over QuaRot and OSCAR across reasoning and coding benchmarks and stronger long-context retrieval [122, 125].
- **QJL** uses a Johnson–Lindenstrauss projection followed by 1-bit sign quantization of keys, with an asymmetric unbiased inner-product estimator in which the query is projected without quantization; because no scales or zero-points are stored, the usual 1–2 bits of overhead vanish [137].
- **TurboQuant** (Zandieh et al., April 2025) rotates each vector randomly, applies an optimal scalar quantizer to each (now concentrated) coordinate, and corrects the inner-product bias of the MSE-optimal quantizer with a 1-bit QJL transform on the residual; it proves distortion within a factor of about 2.7 of the information-theoretic lower bound, and on KV caches reports quality neutrality at 3.5 bits per channel and marginal degradation at 2.5 bits [178].
- **RaBitQCache (2026)** uses randomized rotated binary quantization not to store the cache but as a cheap proxy to estimate attention weights for adaptive top-p sparse retrieval, an example of quantization used inside attention-selection methods (covered in the companion paper) [154].

Table 12.2: KV-cache quantization methods and their headline claims (as stated in the cited abstracts).

Method	Key ideas	Claim
KIVI [85]	2-bit asymmetric; K per-channel, V per-token; FP16 residual window	tuning-free; hardware-friendly implementation
KVQuant [87]	per-channel and pre-RoPE K; non-uniform; dense-and-sparse	$<$ 0.1 PPL degradation at 3 bits
SKVQ [166]	channel reordering, clipped dynamic group quant, sliding FP window	2-bit K, 1.5-bit V with high accuracy
QAQ [135]	separate K/V strategies, outliers, attention-aware	up to 10$\times$ compression
GEAR [66]	quantization + low-rank + sparse error correction	near-lossless high-ratio compression
KVTuner [88]	layer-wise mixed-precision K/V pairs by offline search	nearly lossless at throughput gains
AQUA-KV [11]	predict KV from earlier layers; quantize residual	improved compression at maintained accuracy
RotateKV [157]	outlier-aware rotations	accurate, robust 2-bit
TurboQuant [178]	random rotation + scalar quantizer + 1-bit QJL residual	neutral at 3.5 bits/channel; within $\sim 2.7\times$ of bound
OSCAR [122], OptR [125]	covariance- or output-aware rotations for INT2	INT2 for 32k-token reasoning traces

12.4 Long-context and reasoning fragility

Short-context perplexity is a poor guide for KV quantization because errors in cached entries affect every subsequent token and attention is a retrieval operation. A simple model illustrates the mechanism.

Proposition 12.3 (Retrieval margin under logit noise)**.** *Suppose a "needle" token has attention logit exceeding that of each of T distractor tokens by a margin m, and quantization perturbs every logit by independent $\mathcal{N}(0, \sigma^2)$ noise. Then retrieval (the needle remaining the argmax) fails with probability approaching $1/2$ once $m \lesssim \sigma\sqrt{2\ln T}$ (the typical maximum of the T noise terms), and the probability that the needle remains the* softmax-dominant *token requires $m \gtrsim \sigma\sqrt{2\ln T} + \ln T$-type terms when the distractor mass is large.*

Heuristic argument. The maximum of T i.i.d. $\mathcal{N}(0, \sigma^2)$ variables concentrates at $\sigma\sqrt{2\ln T}$. For the needle's perturbed logit to remain above the perturbed distractor maximum, its margin must exceed that scale. For softmax mass, the T distractors collectively contribute $\approx Te^{-m}$ relative to the needle's 1, so m must also exceed $\ln T$ plus a safety factor for the output to be dominated by the needle. □

The qualitative conclusion is that the tolerable logit-noise level decreases as context grows (like $1/\sqrt{\ln T}$) and shrinks further as the task requires precise retrieval from distractors (e.g. multi-key needle, aggregation, multi-hop). Short, redundant contexts tolerate more noise. This matches empirical caution: studies of quantized reasoning models evaluate weights,

activations and the KV cache across math, science and coding benchmarks and find near-lossless behaviour at moderate bit-widths but significant risks at lower ones, with model size, model origin and task difficulty as major factors [141]; and the 2026 INT2 KV papers specifically evaluate reasoning traces up to 32k tokens and long-context retrieval, because earlier perplexity-based evaluations had missed degradations [122, 125].

KV quantization risks

(i) Error accumulates over generation: each decoded token attends to previously quantized entries, and errors perturb the hidden state and thus later keys (GEAR notes that autoregressive decoding compounds error [66]). **(ii)** Long chain-of-thought and multi-hop retrieval are the most sensitive; test them. **(iii)** Attention sinks and massive-activation tokens have unusual KV statistics; keep them (and a window of recent tokens) in high precision, as IntactKV, KIVI and SKVQ do [80, 85, 166]. **(iv)** Kernel and paged-cache compatibility: per-channel key quantization with residual windows complicates paged cache layouts; check that the serving stack supports your scheme before relying on speedups.

12.5 Practical guidance

1. Default: **FP8 (E4M3) KV cache** with per-tensor or per-head scales. It halves cache memory with negligible loss in most reports, is natively supported on Hopper and Blackwell, and needs no calibration beyond scales.
2. If memory or bandwidth still dominate: **INT4 / 4-bit with group size 32–128**, asymmetric, with a recent-token FP16 window. Test on long-context retrieval and your hardest generation tasks.
3. 2–3 bits: use rotation/outlier-aware methods (KVQuant, RotateKV, OSCAR, TurboQuant-style), validate on reasoning traces of the actual target length, and monitor for degenerate loops or truncation.
4. Combine with weight quantization carefully: weight and KV quantization errors add and interact through attention.

Takeaways of the chapter

- Attention decode has intensity $16g/b$ FLOP/byte: always memory-bound, so KV quantization gives near-linear speedup with the byte reduction and matters more as batch and context grow.
- Keys have channel-structured outliers; values do not. Per-channel key and per-token value grouping (KIVI), pre-RoPE keys (KVQuant), and rotations (QuaRot, RotateKV, OSCAR, TurboQuant) are the main tools; the winner of a grouping choice depends on how queries load on outlier channels (Proposition 12.2).
- FP8 KV is the safe default; 4-bit is mature; 2-bit needs rotations and careful long-context and reasoning validation.

Chapter 13

Error Analysis and Evaluation Methodology

How much damage does a given quantization cause, where does it come from, and how should it be measured? This chapter develops the error theory that complements the method chapters (noise propagation, composition of weight and activation noise, the role of outliers), connects perplexity to KL divergence, surveys the evidence on where average metrics mislead (long context, reasoning, multilingual, safety), and gives an evaluation protocol and a list of methodological pitfalls.

13.1 Layer-level error bounds

Let a layer have weights $\boldsymbol{W} \in \mathbb{R}^{n\times m}$ and calibration inputs $\boldsymbol{X} \in \mathbb{R}^{m\times N}$, with quantization error $\boldsymbol{E} = \widehat{\boldsymbol{W}} - \boldsymbol{W}$.

Proposition 13.1 (Output error of a quantized linear layer)**.** *(a)* $\|\boldsymbol{EX}\|_F \le \|\boldsymbol{E}\|_2 \|\boldsymbol{X}\|_F \le \|\boldsymbol{E}\|_F \|\boldsymbol{X}\|_F$.
(b) $\|\boldsymbol{EX}\|_F^2 = \operatorname{tr}(\boldsymbol{EHE}^\top) \le \lambda_{\max}(\boldsymbol{H}) \|\boldsymbol{E}\|_F^2$ *with* $\boldsymbol{H} = \boldsymbol{XX}^\top$.
(c) If $\boldsymbol{E}$ *has independent zero-mean entries of variance* σ^2 *(idealized RTN noise), then* $\mathbb{E}\,\|\boldsymbol{EX}\|_F^2 = n\,\sigma^2 \operatorname{tr}(\boldsymbol{H}) = n\sigma^2 \|\boldsymbol{X}\|_F^2$.

Proof. (a) Submultiplicativity of the spectral and Frobenius norms. (b) $\operatorname{tr}(\boldsymbol{EHE}^\top) \le \lambda_{\max}(\boldsymbol{H}) \operatorname{tr}(\boldsymbol{EE}^\top)$. (c) $\mathbb{E}\operatorname{tr}(\boldsymbol{EHE}^\top) = \sum_i \mathbb{E}[\boldsymbol{e}_i^\top \boldsymbol{H}\boldsymbol{e}_i] = \sum_i \sigma^2 \operatorname{tr}(\boldsymbol{H}) = n\sigma^2 \operatorname{tr}(\boldsymbol{H})$ because the entries are uncorrelated with variance σ^2. □

Part (c) shows that for *unstructured* noise, the layer output error depends on $\boldsymbol{H}$ only through $\operatorname{tr}(\boldsymbol{H}) = \|\boldsymbol{X}\|_F^2$, the total activation energy, independent of correlations. Error-compensating methods (Chapter 5) deliberately create *structured* errors $\boldsymbol{E}$ that lie in the near-null directions of $\boldsymbol{H}$, so that $\operatorname{tr}(\boldsymbol{EHE}^\top)$ is far smaller than $n\sigma^2 \operatorname{tr}\boldsymbol{H}$: this is the mechanism of GPTQ's 10–30× gains in our simulation (Table 5.1). The more rapidly the eigenvalues of $\boldsymbol{H}$ decay, the larger the potential gain.

Proposition 13.2 (Lower bound from the Hessian spectrum, informal)**.** *For quantizers that round each coordinate to a grid with step* Δ *and are free to choose the rounding direction, the best achievable weighted error is of order* $\Delta^2 \left(\sum_i \lambda_i^{1/2}\right)^2 / m$ *rather than* $\Delta^2 \sum_i \lambda_i$*, when the Hessian eigenvectors are incoherent (QuIP's analysis [150]).*

Cauchy–Schwarz gives $(\sum \sqrt{\lambda_i})^2/m \le \sum_i \lambda_i$ with equality iff all eigenvalues are equal, so the gain is controlled by the dispersion of the Hessian spectrum, in the same spirit as the AM/GM gain of bit allocation (Proposition 9.1).

Table 13.1: Combined SQNR (dB) of weight and activation quantization from the harmonic composition (13.1), assuming for illustration that both operands behave like absmax-scaled Gaussian data with group size 128: SQNR $\approx$ 18.6 dB at 4 bits and 11.3 dB at 3 bits (measured in Table 3.3), extended by 6.02 dB per extra bit (42.7 dB at 8 bits, 30.7 dB at 6 bits). This is a stylized model: real activations are harder than Gaussian (outliers) and real weights easier than Gaussian after GPTQ.

	A8	A6	A4	A3
W8	39.7	30.4	18.6	11.3
W6	30.4	27.7	18.3	11.3
W4	18.6	18.3	15.6	10.6
W3	11.3	11.3	10.6	8.3

13.2 Composition of weight and activation noise

For a layer with both weights and activations quantized, $\boldsymbol{y} = (\boldsymbol{W} + \boldsymbol{E}_w)(\boldsymbol{x} + \boldsymbol{e}_x) \approx \boldsymbol{W}\boldsymbol{x} + \boldsymbol{E}_w\boldsymbol{x} + \boldsymbol{W}\boldsymbol{e}_x$ to first order. If the noise terms are independent and zero-mean, the output error powers add:

$$\mathbb{E}\,\|\delta\boldsymbol{y}\|^2 \approx \underbrace{n\sigma_w^2\,\|\boldsymbol{x}\|^2}_{\text{weight noise}} + \underbrace{\sigma_x^2\,\|\boldsymbol{W}\|_F^2}_{\text{activation noise}} . \tag{13.1}$$

Dividing by the signal power $\|\boldsymbol{W}\boldsymbol{x}\|^2$, relative errors add: $r_{\text{total}} \approx r_w + r_a$, or in terms of SQNR, $1/\text{SQNR}_{\text{tot}} = 1/\text{SQNR}_w + 1/\text{SQNR}_a$ (harmonic combination). The larger error dominates, a *bottleneck principle*: lowering the activation precision below the weight precision (or vice versa) buys nothing.

Table 13.1 makes the bottleneck concrete: W4A8 (18.6 dB) is essentially W4A16 (also $\approx$ 18.6 dB, ignoring activation noise), and W4A4 loses 3 dB relative to either. Its practical consequence is that 8-bit activations are "free" next to 4-bit weights while 4-bit activations are not; and that "W8A4" is wasteful. It also explains why W4A8 (QServe, QQQ) is attractive: INT8 tensor-core throughput with 4-bit weights, at an accuracy cost near W4A16 [140, 143].

13.3 Outliers: why energy-based metrics can mislead

Consider a vector $\boldsymbol{x} \in \mathbb{R}^d$ with $d-1$ entries $\sim \mathcal{N}(0,1)$ and one outlier of magnitude a. Its energy is $\approx d - 1 + a^2$. Per-tensor absmax quantization at b bits has step $\Delta = 2a/(2^b - 1)$, so the bulk entries (of unit scale) see rounding noise $\Delta^2/12$ each. The SQNR computed over the whole vector is $\approx \frac{d-1+a^2}{d\,\Delta^2/12}$, which is *inflated* by the outlier's energy: with $d = 4096$, $a = 1000$ and $b = 8$, $\Delta = 7.84$, $\Delta^2/12 = 5.1$ so the bulk SNR is $\approx 1/5.1 = -7$ dB (the bulk signal is destroyed), yet the whole-vector SQNR is $\approx (4095 + 10^6)/(4096 \cdot 5.1) = 48$, i.e. $+17$ dB. A relative error measured in energy terms thus looks fine while the information carried by the 4095 ordinary channels is lost; and since outliers in LLMs act as biases or attention sinks [101, 165], what the next layers need to read is exactly in the bulk. Two lessons: (i) report errors on the bulk (e.g. excluding top-k channels) or on downstream functional metrics, not just total energy; (ii) outlier *handling* (separation, smoothing, rotation) is more consequential than extra bits.

13.4 From error to perplexity

Proposition 13.3 (Perplexity change equals KL divergence for a well-calibrated model)**.** *Let p be the full-precision model's next-token distribution, $\hat{p}$ the quantized model's, and suppose the evaluation text is drawn from p (a good approximation for a well-calibrated model on in-distribution text). Then the per-token cross-entropy increase equals the KL divergence,*

$$\mathrm{CE}(\hat{p}) - \mathrm{CE}(p) = \mathbb{E}[\mathrm{KL}(p \,\|\, \hat{p})]\,, \qquad \frac{\mathrm{PPL}(\hat{p})}{\mathrm{PPL}(p)} = \exp\big(\mathbb{E}[\mathrm{KL}(p\|\hat{p})]\big). \tag{13.2}$$

For a perturbation $\boldsymbol{\delta}$ of the parameters, $\mathbb{E}[\mathrm{KL}] \approx \frac{1}{2}\boldsymbol{\delta}^\top \boldsymbol{F}\boldsymbol{\delta}$ with $\boldsymbol{F}$ the Fisher information.

Proof. $\mathrm{CE}(\hat{p}) - \mathrm{CE}(p) = \mathbb{E}_{x\sim p}[\log p(x) - \log \hat{p}(x)] = \mathrm{KL}(p\|\hat{p})$. The second-order expansion of KL around $\boldsymbol{\delta} = 0$ has Hessian equal to the Fisher information. □

This is the rigorous version of the heuristics of Section 3.6: the layer-wise proxy is a Gauss–Newton/Fisher quadratic form. It also gives conversion factors: Llama-2-7B going from perplexity 5.47 to 5.60 corresponds to $\mathbb{E}[\mathrm{KL}] \approx \ln(5.60/5.47) = 0.023$ nats per token. The HIGGS linearity theorem further asserts that, to first order, the perplexity increase is linear in the layer-wise ℓ_2 reconstruction errors with layer-dependent coefficients [79].

Limits of perplexity. (1) It averages over tokens; most tokens are easy and unaffected, while a few high-stakes tokens (numbers in a calculation, the choice among retrieved candidates) carry the error. (2) It is measured on teacher-forced text, so it ignores error compounding in free-running generation, which dominates long chain-of-thought. (3) It is relative to a distribution which may not match the deployment (e.g. post-trained chat models have low perplexity on WikiText only coincidentally). (4) It saturates: once KL is small, perplexity cannot discriminate methods that differ in behaviour on rare but important inputs.

13.5 Evidence of where average metrics mislead

The pattern is consistent: *8-bit is safe, 4-bit weight-only is usually safe, and everything below needs task-specific verification*; degradation concentrates in long-context inputs, non-English text, mathematical and multi-step reasoning, and in behaviour measured by human or distance metrics rather than accuracy.

13.6 Evaluation methodology: a protocol

E1. Distance metrics to the full-precision model. On a held-out set of representative text (including model-generated and chat-formatted text), report mean token-level KL divergence $\mathrm{KL}(p\|\hat{p})$ and top-1 token agreement; on multiple-choice tasks report accuracy and the flip rate between FP and quantized answers [55]. KL is a sensitive, label-free, low-variance indicator.

E2. Capability suites matching deployment. Knowledge (MMLU-style), instruction following, code (unit-test pass rate), math (GSM8K/MATH), multilingual (non-Latin scripts),

long context (needle, multi-hop, long-form output at the target length [96]), and reasoning (competition math/science/coding with long CoT [141]).

E3. Safety and alignment. Refusal and jailbreak resistance, toxicity, bias, calibration [19, 41]. Re-run after any quantization of an aligned model, in particular after QAT.

E4. Generation-level checks. Free-running generation with the production sampler: degenerate repetition, truncation, format failures, length distribution shift.

E5. Efficiency measured end to end. Time to first token, time per output token, tokens/s per GPU at the *target* batch size and sequence lengths, memory, on the target hardware with real (not fake-quant) kernels, against a well-optimized 16-bit baseline.

E6. Statistical rigor. See below.

13.6.1 Statistical power

An accuracy measured on N independent items has standard error $\sqrt{p(1-p)/N}$. For GSM8K's 1319 test problems and $p \approx 0.8$, the SE is 1.1 percentage points; a quantization "drop of 0.8%" is within noise for an unpaired comparison. A *paired* comparison (same items, e.g. McNemar's test on discordant pairs) is more powerful: the SE of the paired difference is $\sqrt{(f_{+-} + f_{-+})/N}$ where f are the discordant fractions, so with a flip rate of 5% ($f_{+-} + f_{-+} = 0.05$) and $N = 1319$, SE $= 0.62$ points, about half the unpaired value. For small benchmarks (AIME has 30 problems per year), one problem is 3.3 points, so a single greedy decoding is nearly useless: use many samples per problem (16–64) and report mean and confidence intervals [141]. The flips phenomenon [55] is a reminder that equal accuracy is compatible with many changed answers.

13.6.2 Pitfalls

Evaluation pitfalls specific to quantization

1. **Fake versus real quantization**: simulated quantization measures accuracy only; kernels may differ in accumulation precision, scale rounding, or fused operations, and results can differ slightly. Speed claims require real kernels.
2. **Perplexity protocol drift**: WikiText-2 versus C4, sequence length 2048 versus 4096, stride/sliding-window, and tokenizer differences change absolute perplexities by far more than the differences between methods; compare only within a protocol.
3. **Calibration leakage**: calibrating GPTQ/AWQ on the evaluation corpus (commonly WikiText-2) flatters perplexity on that corpus and says little about other domains [36].
4. **Bit accounting**: include scales, zero-points, codebooks, and layers kept in high precision (embeddings, LM head, outlier channels); compare methods at equal total memory, not equal nominal bit width.
5. **Weak or untuned baselines**: RTN without group scales, GPTQ without damping or act-order, AWQ without clipping search, different group sizes; many reported gains disappear against tuned baselines.
6. **Benchmark saturation, contamination and harness drift**: few-shot formatting, chat templates, harness versions, and prompt-sensitivity can swing scores by several points.

7. **Single-seed, small-benchmark comparisons**: especially for QAT and for competition-math reasoning.
8. **Throughput at batch size one only**: weight-only speedups shrink as batch grows (Chapter 2); report across batch sizes and context lengths.
9. **Baseline precision mismatch**: compare against BF16 when the model was trained in BF16, not against an upcast FP32 reference, and note FP8-native models.
10. **Reporting only averages**: report worst-category performance (languages, long-context lengths, task types).

13.7 Predictive diagnostics

Several inexpensive diagnostics predict trouble before running large evaluations: (i) *layer-wise relative error* (output error divided by output energy, bulk-only) to find hot spots (down-projections and value projections are typically worst); (ii) *top-k activation channel ratio* to detect outliers; (iii) *KL to FP on held-out text per layer-ablation* to build sensitivity maps (Chapter 9); (iv) *attention-pattern divergence* (change in attention maps) for KV quantization; (v) *token-position profile of KL* (does KL grow along the context or along the generation?), which reveals error accumulation.

Takeaways of the chapter

- For unstructured noise, expected layer output error is $n\sigma^2 \operatorname{tr}(\boldsymbol{H})$ regardless of correlations; structured, error-compensating rounding beats it by exploiting the Hessian spectrum.
- Weight and activation noise add in relative power: the larger one dominates (W4A8 $\approx$ W4A16; W4A4 loses $\sim$3 dB more).
- Perplexity increase equals KL divergence for in-distribution text; use KL and flip rates as sensitive, label-free metrics; add long-context, multilingual, reasoning, safety and free-generation tests because aggregate metrics hide the largest degradations.
- Evaluate against tuned baselines at equal total memory with real kernels and proper statistics.

Table 13.2: Documented degradations hidden by aggregate metrics (as stated in the cited abstracts).

Study	Setting	Finding
Mekala et al. [96]	9.7K long-context examples (>64K input and long outputs), 5 methods (FP8, GPTQ-int8, AWQ-int4, GPTQ-int4, BNB-nf4), 5 models	8-bit preserves accuracy (∼0.8% average drop); 4-bit methods lead to substantial losses, up to 59% on long inputs; worse for non-English; Qwen-2.5-72B robust under BNB-nf4 but Llama-3.1-70B drops 32% on the same task
Marchisio et al. [109]	multilingual LLMs; automatic, LLM-as-judge, human evals	a 1.7% average automatic drop in Japanese corresponded to a 16.0% drop in human evaluation; non-Latin-script languages hit worst; math reasoning degrades fastest
Liu et al. [141]	quantized reasoning models (R1-distilled Qwen/Llama 1.5B–70B, QwQ-32B, Qwen3-8B)	W8A8 and W4A16 near-lossless; lower widths risky; size, origin, and task difficulty matter; quantized models do not produce longer outputs
Dutta et al. [55]	compressed models, accuracy vs. answer flips	similar accuracy can hide many correct↔incorrect flips; MT-Bench shows significant degradation; propose KL divergence and flips
Hong et al. [41]	3 LLMs, 5 compression techniques, 8 trustworthiness dimensions	4-bit quantization retains trustworthiness while pruning degrades it; moderate-bit quantization can even improve some dimensions; 3-bit tends to reduce trustworthiness significantly
Xu et al. [19]	degeneration harm, representational harm, dialect bias	quantization mostly preserves bias while pruning degrades quickly; compression can alleviate degeneration harm yet exacerbate representational harm
Li et al. [48]	PTQ on weights, activations, KV across 11 model families; 5 task types (incl. trustworthiness, dialogue, long context)	systematic summary and recommendations by task type
Huang et al. [90], Zheng et al. [151]	LLaMA3, Qwen3	non-negligible degradation, particularly at ultra-low bit widths; Qwen3 competitive at moderate bit-widths

Chapter 14

Kernels, Systems, and Frameworks in Practice

A quantization scheme is only as good as its kernel. This chapter explains why low-bit weights do not automatically yield speedups, models the dequantization pipeline of mixed-precision GEMM, describes Marlin-style kernels and their successors, works through end-to-end speedup arithmetic including the KV cache and Amdahl effects, and surveys the formats and frameworks one actually deploys (GGUF/llama.cpp, bitsandbytes, vLLM, TensorRT-LLM and others).

14.1 Anatomy of a mixed-precision GEMM

For weight-only W4A16 quantization the matrix multiply $\boldsymbol{Y} = \boldsymbol{X}\boldsymbol{W}^\top$ with $\boldsymbol{X} \in \mathbb{R}^{B\times m}$ (batch of B tokens) in FP16 and $\boldsymbol{W}$ stored as packed 4-bit integers with group scales proceeds, per tile, as follows.

1. Load a tile of packed weights from HBM into shared memory (asynchronously, using multi-stage pipelines).
2. Load the matching group scales (and zero-points).
3. **Dequantize**: unpack integers (shifts and masks), convert to FP16 (often via bit tricks adding a magic constant), subtract zero-point, multiply by the scale: $\widehat{w} = s\,(q - z)$.
4. Feed the FP16 weight fragments and FP16 activation fragments to tensor-core matrix-multiply-accumulate (MMA) instructions.
5. Accumulate in FP32 across the K dimension; write the output.

Step 3 executes on the general-purpose (CUDA) cores and shares instruction-issue slots with the tensor-core instructions; step 1 is limited by HBM bandwidth. Each weight is dequantized *once per tile pass* but multiplied by B activation vectors.

14.1.1 A simple throughput model

Let a device have HBM bandwidth β, tensor-core throughput Π_{tc} (FLOP/s) for 16-bit operands, and CUDA-core throughput Π_{cc} (simple operations per second). Dequantizing one weight costs c_d simple operations (unpack, convert, scale; of order 2–4 with well-designed code). Per weight, the time lower bounds are

$$t_{\text{mem}} = \frac{b/8}{\beta}, \qquad t_{\text{dq}} = \frac{c_d}{\Pi_{\text{cc}}}, \qquad t_{\text{tc}}(B) = \frac{2B}{\Pi_{\text{tc}}}, \tag{14.1}$$

and an ideal fully-overlapped kernel takes $t = \max(t_{\text{mem}}, t_{\text{dq}}, t_{\text{tc}})$ per weight. For approximate H100 SXM values $\beta = 3.35$ TB/s, $\Pi_{\text{tc}} = 989$ TFLOP/s and $\Pi_{\text{cc}} \approx 60$ T simple operations/s

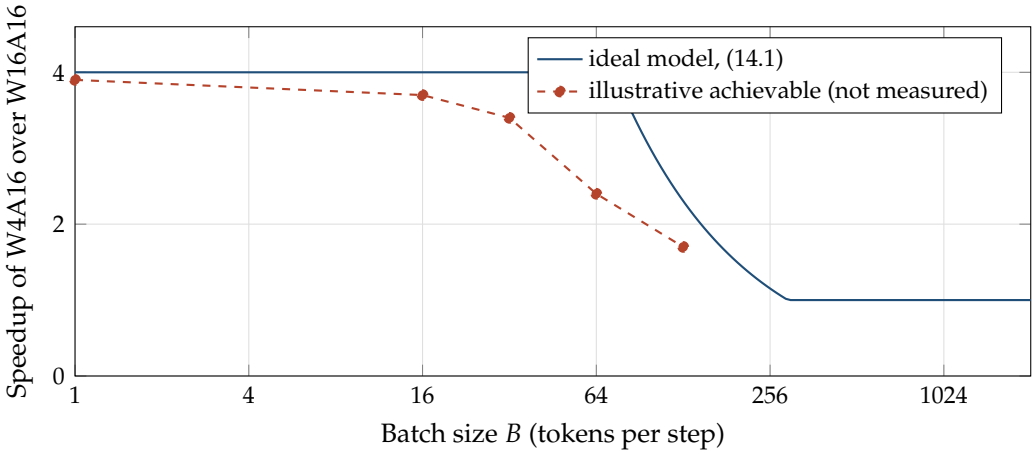


Figure 14.1: Ideal W4A16 versus W16A16 GEMM speedup under the model (14.1) with approximate H100 parameters (analytic, not measured). The dashed line is an *illustrative* shape consistent with the qualitative claim of the Marlin paper that batch sizes up to 16–32 retain close to the maximal 4× speedup and batch sizes 64–128 retain gradually decreasing but significant speedup [100]; the plotted points are our schematic, not measurements.

(the FP32 non-tensor peak is about 67 TFLOP/s), with $b = 4$ and $c_d = 3$: $t_{\text{mem}} = 1.5 \times 10^{-13}$ s, $t_{\text{dq}} = 5 \times 10^{-14}$ s, and $t_{\text{tc}}(B) = 2B/989 \times 10^{12}$. Then:

- For $B \leq 74$ ($t_{\text{tc}} \leq t_{\text{mem}}$) the kernel is memory-bound, and the speedup over FP16 weights ($t^{16}_{\text{mem}} = 6 \times 10^{-13}$ s, memory-bound until $B \approx 295$) is exactly $16/4 = 4\times$ if dequantization is overlapped: it is hidden because $t_{\text{dq}} < t_{\text{mem}}$.
- For $74 < B < 295$ the 4-bit kernel is compute-bound while the FP16 kernel is still memory-bound: speedup $= 295/B$.
- For $B > 295$ both are compute-bound: speedup $\to 1$ (or below 1 because of dequantization issue-slot contention).

Figure 14.1 plots this ideal curve and compares it to the Marlin claim.

The model has three lessons. (1) Dequantization is cheap relative to HBM time only if it is overlapped and the instruction count is small; poor kernels (extra conversions, unaligned loads, scale loads) lose the gain. QServe reports that existing INT4 methods incur 20–90% runtime overhead when dequantizing either weights or partial sums on GPUs, which motivated its progressive-quantization design for W4A8 [143]. (2) The throughput crossover ($B \approx 74$ here) is hardware-dependent and moves right with higher bandwidth-to-compute ratios. (3) Real kernels fall below the ideal curve at moderate batch sizes because the CUDA cores that dequantize also issue the MMA instructions, and shared-memory bandwidth and register pressure limit tile sizes.

14.2 Marlin and its successors

MARLIN (Mixed-precision Auto-Regressive LINear kernels) targets the regime where speedups from weight quantization were widely believed to vanish: batched inference with multiple parallel clients. The paper shows that with careful design the kernel stays practically memory-bound up to batch sizes 16–32 with close to the maximum 4× speedup for 4-bit weights,

and retains significant, gradually decreasing speedups up to batch sizes 64–128 [100]. The techniques it combines, in the paper's own list, include asynchronous memory access, complex task scheduling and pipelining, and carefully designed layouts; as we understand them these include:

- **Offline weight reordering** so that each thread reads consecutive 128-bit words that already hold the registers' worth of weights in the order the MMA fragments need, giving fully coalesced global loads and no shuffles.
- **Multi-stage `cp.async` pipelines** that prefetch weight tiles into shared memory while previous tiles are dequantized and consumed.
- **Fast dequantization** using bit-twiddling to convert packed INT4 to FP16 pairs with few instructions, and applying scales either to the dequantized weights or to the accumulator output (per-column scales commute with the sum).
- **Striped partitioning** of the GEMM across streaming multiprocessors, so that all SMs stay busy for the skinny shapes of decoding and reductions across partial results are done in a predictable order.
- **Activation reuse**: keeping the (small) activation tile resident in the L2 or shared memory across the weight sweep.

The Marlin kernel became the basis of efficient GPTQ/AWQ-style serving in vLLM and others, and variants extend it to other formats (FP8 weight-only on pre-Hopper GPUs, 2:4 sparse + quantized combinations, group scales with zero-points); newer mixed-input GEMMs for Hopper that use the asynchronous warp-group MMA instructions are documented in the vLLM project [214]. QQQ builds W4A8 kernels (4-bit weights, INT8 activations) on Marlin-like designs [140].

LUT-based and VQ kernels. For non-uniform and vector-quantized weights, dequantization is a lookup. LUT-GEMM, FLUTE and T-MAC develop table-driven kernels for non-uniform formats on GPUs and CPUs; AQLM and QuIP# ship dedicated kernels for additive and E_8 codebooks, evaluated at small batch sizes; QTIP's computed trellis codes avoid large tables [10, 56, 98, 144, 149, 177].

FP6 and odd bit-widths. FP6-LLM ("TC-FPx") provides tensor-core-compatible kernels for 6-bit floating-point weights, tackling the memory-alignment problem of non-power-of-two widths by bit-level pre-packing and ahead-of-time bit-layout transformation [60]. Odd bit-widths (3, 5, 6 bits) are attractive for accuracy-per-bit but are harder to pack with aligned loads; common solutions split a weight across two words (e.g. 2 bits + 1 bit planes) at a small cost.

Low-precision compute kernels. W4A4, W8A8 and FP8/FP4 pipelines require activation quantization fused with the preceding operation (for instance RMSNorm followed by per-token quantization in one kernel), scale handling in the epilogue or by the hardware (block-scaled MMA), and, for rotation methods, a fused fast Walsh–Hadamard transform. Atom reports end-to-end throughput up to 7.7× over FP16 and 2.5× over INT8 for W4A4 serving at matched latency [14]; FlatQuant reports up to 2.3× prefill and 1.7× decoding speedup over FP16 for W4A4 with a fused affine-transform kernel [53]; MR-GPTQ reports layer-wise speedups of up to 3.6× and end-to-end up to 2.2× on B200 and up to 6× layer-wise and 4× end-to-end on RTX 5090 for MXFP4/NVFP4 [110]; Quartet II reports training kernels up to 4.2× faster than

Table 14.1: Selected *reported* speedups. Each is the claim of the cited paper under its own conditions (hardware, batch size, baseline) and cannot be compared across rows.

Source	Setting	Reported speedup
Marlin [100]	W4A16 GEMM, batch $\leq$16–32	close to 4× (maximum); decreasing to batch 64–128
SpQR [174]	33B on a 24 GB consumer GPU, near-lossless	15% speedup (baseline as defined in the paper)
Atom [14]	W4A4 serving, throughput	up to 7.7× vs FP16, 2.5× vs INT8
FlatQuant [53]	W4A4, Llama-3 class	up to 2.3× prefill, 1.7× decode vs FP16
MR-GPTQ [110]	MXFP4/NVFP4 on B200	up to 3.6× layer-wise, 2.2× end-to-end vs FP16
MR-GPTQ [110]	MXFP4/NVFP4 on RTX 5090	up to 6× layer-wise, 4× end-to-end vs FP16
Quartet II [147]	NVFP4 training kernels, Blackwell	up to 4.2× vs BF16
FP8-LM [63]	FP8 training GPT-175B, H100	75% faster than BF16 Megatron-LM
COAT [38]	FP8 training	1.43× speedup vs BF16

BF16 on Blackwell [147]; SageAttention3 reaches 1038 TOPS on an RTX 5090 [160].

14.3 System-level view: which format for which serving regime

A large controlled study on the full Llama-3.1 family evaluated FP8, INT8 and INT4 in vLLM across over 500,000 evaluations and concluded that W4A16 is the most cost-efficient for *synchronous* (latency-oriented, low-concurrency) deployments, while W8A8 dominates in *asynchronous* continuous-batching deployments (high concurrency), with the optimum for mixed workloads depending on the use case [20]. This is exactly what the model of Section 14.1 predicts: low batch is memory-bound (4-bit weights win); high batch is compute-bound (8-bit tensor-core operands win).

14.3.1 End-to-end arithmetic including the KV cache and Amdahl effects

Decode step time is roughly the bytes moved divided by bandwidth, when memory-bound: $T \approx (M_W + M_{KV})/\beta$ plus overheads. Take a 70B model with 320 KiB of KV per token (Chapter 2), and compute bytes per decode step for B concurrent sequences with context T_c.

Table 14.2 contains the most important systems lesson for long-context, high-concurrency serving: once the KV cache dominates the bytes, weight quantization alone yields little (1.27× in the second scenario) and KV quantization is where the speedup lies. (Of course, a 4096-token context times 32 sequences, or 8192 times 128, requires the memory to hold the cache at all, 43 GB or 344 GB; KV quantization also determines how many sequences fit.) Conversely at short context and small batch weights dominate and KV quantization contributes little.

Amdahl's law. If a fraction f of the step time is spent in quantized operations that are sped up by S_q, the overall speedup is $1/\left(f/S_q + 1 - f\right)$. Components typically *not* accelerated: embedding lookups and LM head (often kept in 16 bit), normalization and elementwise operations, attention softmax and rotary embeddings, sampling, host-side scheduling, kernel

Table 14.2: Bytes read per decode step (GB) for a 70B-class model with weights at BF16 or 4.25 bpw (140 and 37.2 GB) and KV cache at 16, 8, 4 bits (320 KiB/token at 16 bits). Idealized speedup is the ratio to the BF16/KV16 row. Exact arithmetic; ignores activations, attention-score traffic, sampling and communication, and assumes a purely memory-bound step.

Scenario	Config	Weights	KV	Total	Ideal speedup
$B = 32$, $T_c = 4096$	W16, KV16	140.0	42.9	182.9	1.00×
	W4.25, KV16	37.2	42.9	80.1	2.28×
	W4.25, KV8	37.2	21.5	58.7	3.12×
	W4.25, KV4	37.2	10.7	47.9	3.82×
$B = 128$, $T_c = 8192$	W16, KV16	140.0	343.6	483.6	1.00×
	W4.25, KV16	37.2	343.6	380.8	1.27×
	W4.25, KV8	37.2	171.8	209.0	2.31×
	W4.25, KV4	37.2	85.9	123.1	3.93×

launch gaps (mitigated by CUDA graphs), and communication in tensor- or expert-parallel serving. With $f = 0.85$ and $S_q = 3.8$, the overall speedup is $1/(0.224 + 0.15) = 2.7\times$.

14.3.2 Tensor parallelism, MoE, and quantized collectives

Group-quantized weights must be sharded consistently with the quantization group structure: splitting the input dimension of a row-parallel layer across GPUs requires group sizes dividing the shard size, and GPTQ's act-order permutation complicates column/row sharding (permutations must be folded or applied consistently). For MoE models the dominant kernel is a grouped GEMM over experts with different token counts per expert, usually at very small per-expert batch (hence memory-bound), so weight-only quantization of the experts is attractive (Chapter 15). Quantized all-reduce and all-to-all (FP8 or INT8 activations in communication) reduce interconnect traffic but add quantization noise to partial sums, an under-explored risk.

14.4 Formats and frameworks

14.4.1 GGUF and llama.cpp

llama.cpp, with the GGML/GGUF format, is the dominant stack for CPU, Apple silicon and consumer-GPU inference [191]. Quantized tensor types are defined by blocks and super-blocks:

The "`_M`/`_S`/`_L`" suffixes denote mixed-precision recipes at the tensor level (Chapter 9) [194]. GGUF is a self-contained file format (tensors, metadata, tokenizer), loaded by memory-mapping. Its strengths: single-file distribution, broad hardware support, mature low-bit options and importance-matrix calibration. Weaknesses: formats designed for CPU SIMD dot products and not for tensor-core GEMM; GPU performance at large batch lags dedicated stacks.

Table 14.3: Representative GGUF quantization types (llama.cpp). Effective bits per weight include scales/minimums. Values are those documented by the project and as the author recalls them; the "i-quant" rows use importance-matrix-guided codebook-like grids. New types are added regularly.

Type	Family	Structure	bits/weight
`Q8_0`	legacy	blocks of 32, FP16 scale, INT8	8.5
`Q4_0`	legacy	blocks of 32, FP16 scale, 4-bit	4.5
`Q4_1`	legacy	blocks of 32, FP16 scale and min	5.0
`Q6_K`	k-quant	256-superblock, 16 sub-blocks, 6-bit	6.56
`Q5_K`	k-quant	256-superblock, 8 sub-blocks, 5-bit, scales+mins	5.5
`Q4_K`	k-quant	256-superblock, 8 sub-blocks of 32, 4-bit, 6-bit scales/mins	4.5
`Q3_K`	k-quant	256-superblock, 16 sub-blocks, 3-bit	3.44
`Q2_K`	k-quant	256-superblock, 2-bit with scales/mins	2.56
`IQ4_XS`, `IQ3_XXS`, `IQ2_XXS`, `IQ1_S`	i-quant	lattice/codebook-style grids, importance matrix	4.25, 3.06, 2.06, 1.56

14.4.2 Other frameworks

Checkpoint formats and interoperability. A model quantized by one pipeline is not necessarily loadable by another. The community has converged partly on the `compressed-tensors` convention in the vLLM ecosystem, GGUF for llama.cpp, and vendor formats (TensorRT-LLM engines, ModelOpt checkpoints) for NVIDIA; format conversion is lossy only if re-quantization is involved. When a model is released in a native low-bit format (gpt-oss MXFP4, Kimi K2 Thinking INT4), serving stacks add loaders and kernels for it quickly but not always on all hardware.

Kernel maturity risk

(i) A quantization format with an accuracy advantage but no optimized kernel on your hardware will be *slower* than FP16 (dequantization in Python, or generic kernels). (ii) New formats (NVFP4, MXFP4 on non-Blackwell GPUs, VQ/trellis codes) have uneven support across frameworks and versions; pin versions and test numerics. (iii) Fused kernels sometimes differ numerically from the reference quantizer (accumulation precision, scale rounding); validate end-to-end accuracy with the production kernel, not only with simulated quantization. (iv) Performance cliffs: kernels tuned for batch sizes 1–32 can degrade for 64–256; check your concurrency range.

Takeaways of the chapter

- Weight-only speedups are bandwidth speedups, hidden dequantization requires $c_d \lesssim 9$ simple ops per weight and an overlapped pipeline; beyond $B \approx 74$ (H100) 4-bit kernels become compute-bound.

Table 14.4: Quantization support in common frameworks as understood by the author in October 2026. Support changes quickly; this is a map, not a guarantee.

Framework	Weight formats	Notes
bitsandbytes [212]	LLM.int8(), NF4/FP4 4-bit, 8-bit optimizers	ease of use, QLoRA fine-tuning; not throughput-optimized
vLLM [128, 214]	GPTQ, AWQ (Marlin kernels), FP8, INT8, compressed-tensors, bitsandbytes, GGUF (partial), NVFP4/MXFP4 on supported GPUs	FP8 KV cache; broad MoE support; fast-moving
TensorRT-LLM / ModelOpt [204]	FP8, INT8 SmoothQuant, INT4 AWQ/GPTQ, NVFP4, FP8 KV	best on NVIDIA GPUs; engine build step; NVFP4 on Blackwell
SGLang [163]	FP8, AWQ, GPTQ, Marlin, MXFP4 for gpt-oss	MoE and gpt-oss focus
llama.cpp / Ollama [191]	GGUF k-/i-quants, ternary types	CPU, Metal, CUDA, Vulkan
Hugging Face Transformers	bitsandbytes, GPTQ, AWQ, FP8, other plug-ins	reference implementation; integrates with the above
torchao	INT4/INT8 weight-only, float8 training, QAT APIs	PyTorch-native
llm-compressor (vLLM project)	GPTQ, AWQ, SmoothQuant, FP8 recipes producing compressed-tensors checkpoints	calibration pipeline for vLLM
ExLlama family	EXL2 (variable bits per weight); newer versions use trellis-style quantization (QTIP-derived) as the author understands	consumer GPU, high accuracy per bit
MLX (Apple)	3-8 bit group quantization	Apple silicon

- Marlin-class kernels keep $\sim 4\times$ at batch $\leq$16–32; W4A16 wins in latency-oriented serving, W8A8 in high-concurrency continuous batching [20].
- At long context and high concurrency the KV cache dominates bytes per step; weight quantization alone then gives little (e.g. $1.27\times$ at B=128, 8K context), KV quantization gives the rest.
- Choose frameworks by hardware and format maturity; verify end-to-end accuracy with the production kernel.

Chapter 15

Quantization of MoE Models, Reasoning Models, and the Bits-versus-Parameters Frontier

The frontier of open-weight models in 2025–2026 is dominated by sparse Mixture-of-Experts (MoE) transformers and by *reasoning* models that generate thousands of tokens of chain-of-thought before answering. Both change the quantization problem. MoE models concentrate parameters in rarely used experts and add a discrete routing decision that quantization noise can flip. Reasoning models amplify small per-token errors over very long generations. This chapter analyses both and closes with the question of how to trade bits against parameters at a fixed memory budget.

15.1 MoE models: anatomy and systems regime

A sparse MoE layer replaces the dense MLP by E expert MLPs and a router; each token is processed by the top-k experts (plus, in some designs, one or more always-on *shared* experts), and the outputs are combined with gate weights. Public examples: DeepSeek-V3 has 671B total and 37B activated parameters per token [43]; Kimi K2 has 1T total and 32B activated [84]; the gpt-oss models are MoE transformers with MXFP4-quantized experts [72]; Qwen3 and GLM-4.5 families include MoE variants [70, 152]. In such models routed experts typically constitute well over 90% of all parameters, so quantizing the experts is where the capacity and bandwidth savings lie.

How many experts are read per decoding step? With uniform independent top-k routing over E experts, a batch of B tokens touches an expert with probability $1-(1-k/E)^B$.

Proposition 15.1 (Fraction of experts touched)**.** *Under uniform independent routing the expected number of distinct experts activated by a batch of B tokens is $E\left[1-(1-k/E)^B\right]$.*

For $k/E = 1/32$ (for example 8 of 256 routed experts, as in DeepSeek-V3-like configurations, or 4 of 128): $B=1$ touches 3.1% of experts, $B=8$ about 22.4%, $B=32$ about 63.8%, $B=128$ about 98.3%, and $B=512$ essentially all. Real routing is skewed (hot experts), so fewer distinct experts are touched at a given B, but the message stands: at moderate serving batch sizes nearly all expert weights are read in every step, so decode bandwidth is proportional to the *total* expert bytes, not the active parameters. Per-expert token counts are small (batch B tokens spread over E/k groups gives Bk/E tokens per expert on average, e.g. 4 tokens per expert at $B=128$), so expert GEMMs are deeply memory-bound and weight-only quantization of experts translates almost directly into speed (Chapter 14).

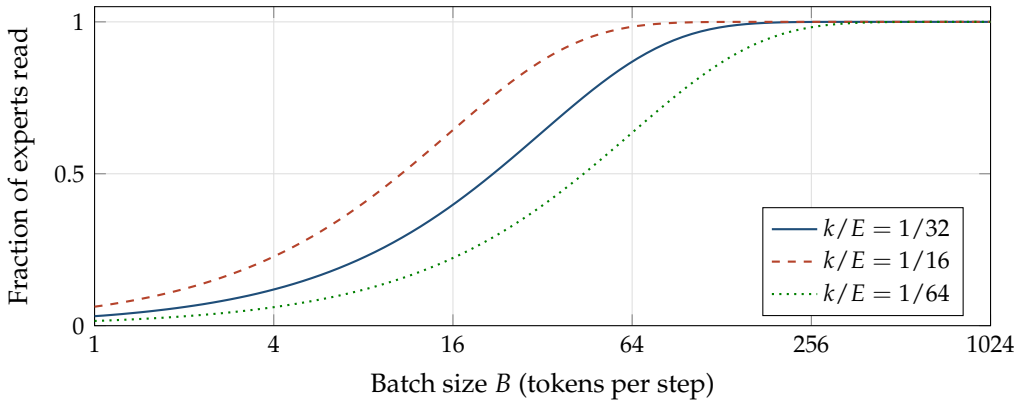


Figure 15.1: Expected fraction of routed experts touched in one step for batch size B under uniform routing (Proposition 15.1; analytic). Sparser routing delays but does not avoid saturation.

15.2 What is different about quantizing MoE models

15.2.1 Experts are robust, shared experts and attention are not

MoQE (weight-only quantization of expert weights down to 2 bits) found that expert layers are much more robust to quantization than dense feed-forward layers, and that MoE models with 2-bit expert weights can outperform a dense model trained on the same data at the same memory, a favourable interaction between sparsity and quantization [107]. QMoE compresses a 1.6-trillion-parameter SwitchTransformer to under 160 GB (0.8 bits per parameter, a 20× compression) with minor accuracy loss, in less than a day on a single GPU, using a custom format and decoding kernels [139]. QuantMoE-Bench argues that fine-grained precision should follow MoE structure: always-accessed shared experts likely need higher precision than rarely selected ones [106]. Production releases follow this: gpt-oss quantizes the expert weights to MXFP4 while keeping other components in higher precision [72, 197], and Kimi K2 Thinking's INT4 QAT targets the MoE components [201].

15.2.2 Calibration imbalance

Second-order and activation-aware methods need per-expert statistics. Expert e receives only the calibration tokens routed to it; its Hessian $\boldsymbol{H}_e = \boldsymbol{X}_e \boldsymbol{X}_e^\top$ is estimated from n_e tokens with $\mathbb{E}[n_e] = Nk/E$ for N calibration tokens. If $n_e < m$ (the input dimension), $\boldsymbol{H}_e$ is rank-deficient and the damped GPTQ solution degenerates toward RTN in unseen directions.

Example 15.1 (Calibration budget for a DeepSeek-V3-like layer)**.** With hidden size $m = 7168$, $E = 256$ routed experts and $k = 8$, requiring $n_e \geq 4m$ tokens per expert for a reasonably conditioned Hessian needs $N \geq 4mE/k = 4 \cdot 7168 \cdot 32 \approx 9.2 \times 10^5$ tokens under *uniform* routing. The conventional GPTQ calibration set of 128 sequences of 2048 tokens has 2.6×10^5 tokens, a factor ~ 3.5 short even in the idealized case, and routing skew makes the rarest experts receive far fewer. Expert-balanced calibration sampling or much larger calibration sets are therefore needed.

MoEQuant identifies *inter-expert imbalance* (uneven sample counts across experts, giving insufficient and biased calibration of rarely used experts) and *intra-expert imbalance* (different

degrees of correlation between samples and their assigned experts due to the aggregation mechanism), and proposes expert-balanced self-sampling and affinity-guided quantization [105]. EAQuant addresses activation outliers (smoothing aggregation across experts), routing consistency (aligning expert selection post-quantization) and calibration balance, reporting average accuracy improvements of 1.15–13.81% in extreme settings (W4A4, W3A4, W3A3, W2A4) across three MoE architectures [45].

15.2.3 Routing sensitivity

The router computes scores $\boldsymbol{z} = \boldsymbol{W}_r \boldsymbol{x}$ and selects the top k. Noise in $\boldsymbol{x}$ or $\boldsymbol{W}_r$ can change the selection.

Proposition 15.2 (Routing flips under logit noise; first-order analysis)**.** *Let $g \geq 0$ be the gap between the k-th and $(k{+}1)$-th router scores, with density ρ near 0, and let quantization perturb the two scores by independent $\mathcal{N}(0, \sigma^2)$ noise. Then the probability that the selected set changes is*

$$\Pr[\textit{flip}] \approx \rho\, \mathbb{E}\left[(-\Delta)_+\right] = \rho\, \sigma / \sqrt{\pi}, \qquad \Delta \sim \mathcal{N}(0, 2\sigma^2), \tag{15.1}$$

linear in σ. The expected output damage per token is $\Pr[\textit{flip}]$ times the change in the combined output caused by swapping experts. If gate weights vanish continuously at the selection boundary (softmax over all experts truncated without renormalization), that change is $O(g) = O(\sigma)$ at a flip and the expected damage is $O(\sigma^2)$. If, as in most current designs, gate weights are renormalized over the selected experts, the swapped expert carries $O(1)$ weight at the boundary and the expected damage is $O(\sigma)$, first order.

Sketch. A flip occurs when $g < (-\Delta)_+$ where Δ is the perturbation of the difference of the two scores; with density ρ of g near zero, $\Pr[g < t] \approx \rho t$, so $\Pr[\text{flip}] \approx \rho\, \mathbb{E}[(-\Delta)_+]$ and $\mathbb{E}[(-\Delta)_+] = \frac{1}{2}\mathbb{E}|\Delta| = \frac{1}{2} \cdot \sqrt{2}\sigma\sqrt{2/\pi} = \sigma/\sqrt{\pi}$. Damage at a flip is the gate weight of the swapped expert times the difference of expert outputs, which is $O(1)$ or $O(g)$ depending on the normalization. □

This heuristic explains why MoE-specific methods emphasize routing consistency: EAQuant aligns expert selection post-quantization; VSRAQ introduces value alignment (matching router logits or scores) and structure alignment (preserving expert ordering and top-k boundaries) as a PTQ objective with no inference-time overhead; GEMQ fine-tunes routers to adapt to quantized experts; MoBiE constrains error in the input null space to mitigate routing distortion in binarized experts [45, 68, 103, 158]. A cheap mitigation used in practice is to keep the router weights (a negligible fraction of parameters) and the router input norms in higher precision.

15.2.4 Mixed-precision across experts

Experts differ in training quality, frequency of use and sensitivity. Allocation methods: GEMQ's global linear-programming formulation over expert importance with progressive refinement [68]; MxMoE's co-design with GroupGEMM kernels supporting different precisions in parallel, reporting 2.4× lower WikiText-2 perplexity than GPTQ at 2.25 bits and up to 3.4× speedup over full precision [112]; an expert-wise strategy based on router-norm change during training with theoretical guarantees [104]; and AlphaQ, which allocates bits using heavy-tailed spectral statistics of expert weights, requiring no calibration data (useful when the training distribution is proprietary) [8].

Table 15.1: MoE-specific quantization methods and their contributions (claims from abstracts).

Method	Problem addressed	Idea
MoQE [107]	memory/bandwidth of experts	2-bit weight-only for experts; experts more robust than dense FFN
QMoE [139]	trillion-parameter MoEs	$<$ 1 bit/param via custom format and kernels; 20$\times$ compression of SwitchTransformer-c2048
MoEQuant [105]	inter-/intra-expert imbalance	expert-balanced sampling; affinity-guided quantization
EAQuant [45]	outliers, routing instability, sparse calibration	smoothing aggregation; routing-consistency alignment; calibration balance
VSRAQ [158]	routing flips	value + structure alignment of router scores
GEMQ [68]	layer-wise allocation, router shift	global LP over experts; router fine-tuning
MxMoE [112]	heterogeneous sensitivity and activation frequency	mixed-precision + GroupGEMM kernels
AlphaQ [8]	proprietary calibration data	calibration-free allocation from weight spectra
MoBiE [103]	binarizing MoE experts	joint SVD, gradient-enhanced Hessian, null-space error constraint

MoE quantization risks

(i) *Routing drift*: small perturbations flip experts; the damage is unevenly distributed (rare inputs, long-tail domains). (ii) *Rare-expert degradation*: calibration under-samples rarely used experts; benchmarks that exercise them (specific languages, code, niche domains) reveal losses that aggregate metrics do not. (iii) *Mixed formats*: experts in INT4/MXFP4 with attention in BF16 complicate kernels and TP sharding. (iv) *Expert-parallel communication* in quantized form (FP8 all-to-all) adds noise to the dispatch/combine path. (v) *Hardware coupling*: native MXFP4 expert checkpoints are optimally served on Blackwell-class parts.

15.3 Reasoning models

Reasoning models are trained with reinforcement learning to produce long chains of thought; a single answer may involve 10^3–10^5 generated tokens, each conditioned on all previous ones. Their quantization behaviour differs from that of short-answer chat models for three reasons: error accumulation, sensitivity to precise numerical and symbolic steps, and the long KV cache.

15.3.1 Evidence

The first systematic study quantized DeepSeek-R1-distilled Qwen and LLaMA models (1.5B to 70B), QwQ-32B and Qwen3-8B (weights, KV cache and activations at several bit-widths) and evaluated on AIME, MATH-500, GPQA and LiveCodeBench: W8A8 and W4A16 are near-lossless, lower bit-widths carry significant risk, model size, model origin and task difficulty are

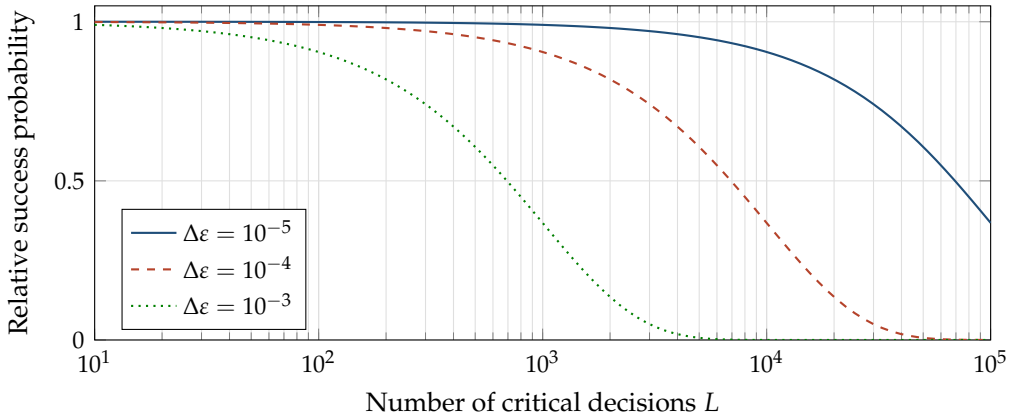


Figure 15.2: **Illustrative.** Relative success probability (15.2) (taking $1 - \varepsilon \approx 1$) versus the length of the chain of critical decisions for several per-step damage levels. The model is a deliberately simple independence calculation, not a fitted law.

major factors, quantized models do not produce longer outputs, and scaling up model size or reasoning steps can recover performance [141]. The 2026 INT2 KV-cache papers evaluate reasoning traces up to 32k tokens on Qwen3-4B-Thinking and Qwen3-8B, reflecting that long-trace evaluation is now the standard test [122, 125]. Industry practice, as visible in public releases, favors native 4-bit formats for reasoning MoE models (gpt-oss MXFP4, Kimi K2 Thinking INT4), trained or QAT-adapted with the quantizer in the loop, rather than post-hoc quantization [72, 201].

15.3.2 A simple error-accumulation model

Suppose a reasoning trace contains L "critical decisions" (steps whose correctness is necessary for the final answer), each independently correct with probability $1 - \varepsilon$ in the full-precision model, and quantization raises the failure probability by $\Delta\varepsilon$. The relative success probability is

$$\frac{(1-\varepsilon-\Delta\varepsilon)^L}{(1-\varepsilon)^L} = \left(1 - \frac{\Delta\varepsilon}{1-\varepsilon}\right)^L \approx e^{-L\,\Delta\varepsilon/(1-\varepsilon)}. \tag{15.2}$$

Per-step damage that is invisible in teacher-forced perplexity ($\Delta\varepsilon \sim 10^{-4}$) becomes decisive when $L \sim 10^4$ ($e^{-1} \approx 37\%$ relative success). The model is crude (steps are neither independent nor all critical; self-correction in reasoning can repair errors, which is why larger budgets recover performance), but it explains the qualitative difference between short-answer and long-chain tasks and why recovering performance by sampling more or reasoning longer is possible: it effectively reduces the per-chain failure probability.

15.3.3 Practical implications

1. Evaluate on multi-sample long-form reasoning (many samples per problem) with the production sampler and context length, not on perplexity or short MCQ.
2. Prefer W8A8/FP8 or W4A16 for reasoning models unless task-specific evidence supports lower bit-widths; treat 2–3 bit weights and 2-bit KV caches as research settings.
3. Include the KV cache in the test: long traces stress it most.

Table 15.2: Rules of thumb for fixed-memory deployment, with the supporting evidence and our confidence (author's judgement).

Rule	Evidence	Confidence
Prefer the largest model that fits at $\approx$ 4-bit (3.5–4.5 effective bits) with a calibrated PTQ method over a smaller model at 8-bit/16-bit	k-bit scaling laws; Llama-3.1 quantization study [20, 83]	High
Go to 2–3 bits only for large models ($\gtrsim$ 30B) with VQ/trellis codes plus fine-tuning, or with QAT	AQLM, QuIP#, QTIP, PV-Tuning, ParetoQ [10, 129, 132, 144, 149]	Medium
Small models ($\lesssim$ 3B) are hurt more by quantization; use 6–8 bits or QAT/native low-bit	undertrained-model scaling law; Llama-3 empirical study [90, 179]	Medium-high
For MoE models, quantize routed experts hardest, keep shared experts, attention and router at higher precision	MoQE, QuantMoE-Bench, release practice [72, 106, 107]	Medium-high
For reasoning workloads, stay at W8A8 or W4A16 unless validated	[141]	Medium-high
Do not extrapolate frontier numbers from 7B/13B studies to 400B+ MoE models	scale dependence of quantization-induced degradation [179]	Medium

4. For RL training and RL-based post-training with quantized rollouts, mismatch between the sampler's (quantized) distribution and the trainer's (higher-precision) distribution acts as an off-policy error; log-probability mismatch should be measured and corrected where used.
5. Re-check safety and format adherence after quantization.

15.4 The bits-versus-parameters frontier

A user with a fixed memory budget M must choose a model size N and a bit-width b with $N\,b \approx 8M$. Evidence on the Pareto frontier:

- **Dettmers & Zettlemoyer**: at fixed total weight bits, 4-bit is almost universally optimal for zero-shot accuracy among 3–8 bit PTQ methods of that era; better data types and small blocks improve the trade-off [83].
- **AQLM, QuIP#, PV-Tuning**: with additive/lattice codebooks and fine-tuning, 2-bit models become Pareto-optimal versus smaller models at higher precision for sufficiently large models; AQLM claims Pareto optimality below 3 bits, PV-Tuning claims the first Pareto-optimal 2-bit quantization for the Llama-2 family [10, 132, 149].
- **ParetoQ**: ternary, 2-bit and 3-bit QAT give comparable size–accuracy trade-offs and generally beat 4-bit and binary, with a learning transition at 2–3 bits [129].
- **Scaling laws for precision and QAT**: lower precision reduces effective parameters; PTQ damage increases with training tokens; QAT error decreases with model size [136, 162, 179].

Takeaways of the chapter

- MoE: experts hold most parameters, are robust to low-bit weights, but suffer calibration imbalance and routing flips; at moderate batch sizes nearly all experts are read per step, so expert-weight quantization maps directly to bandwidth savings.
- Routing flips cost $O(\sigma)$ per token when gate weights are renormalized over the selected experts; routing-consistency objectives and higher-precision routers mitigate.
- Reasoning: long generations amplify tiny per-step damage; W8A8 and W4A16 are safe, lower widths need long-trace, multi-sample validation.
- Frontier: 4 bits is the PTQ sweet spot; 2–3 bits are Pareto-optimal only with codebooks and/or training; model size and token budget shift the optimum.

Chapter 16

Recent Advances: A Timeline from 2022 to October 2026

This chapter places the methods of the preceding chapters on a calendar. The purpose is not completeness but a view of *why* the field moved in the order it did: which bottleneck was binding in each period, which idea relieved it, and which new bottleneck then became visible. Dates are those of the first public arXiv version or public release as recorded in the cited sources; where a date comes from a secondary news report this is stated. Claims about what a work "reports" paraphrase the abstract or model card of the cited item and were not independently reproduced.

16.1 Four eras

Reading guide

2022: making 8-bit and 4-bit *work* on LLMs at all. Outlier features (LLM.int8(), SmoothQuant) and layer-wise second-order rounding (OBC, GPTQ).
2023: weight-only 4-bit becomes standard; the first sub-4-bit ideas. AWQ, SpQR, SqueezeLLM, QuIP, QLoRA, first QAT for LLMs, the MX specification, BitNet, MoE compression.
2024: rotations, vector/trellis codes, KV cache, ternary, FP8 training, systems. QuaRot, SpinQuant, QuIP#, AQLM, QTIP, KIVI, KVQuant, BitNet b1.58, Marlin, QServe, DeepSeek-V3 FP8.
2025 to 2026: micro-scaled FP4 formats, 4-bit training, native low-precision releases, quantization of MoE and reasoning models, theory of bits versus data. MXFP4/NVFP4, gpt-oss, Kimi K2 Thinking, Quartet, NVFP4 pretraining, DeepSeek V4, QAD.

16.2 2022: the outlier problem and second-order rounding

Mid-2022. ZeroQuant demonstrated group-wise weight and token-wise activation quantization with layer-by-layer knowledge distillation for transformer inference [186]. LUT-GEMM (June 2022) proposed lookup-table kernels that make non-uniform and binary-coding weights efficient for generative models [98]. Outlier Suppression analysed the role of LayerNorm scale parameters in creating activation outliers in transformers [124]. These were preceded by the 2021 observations that transformer activations contain structured outliers that break 8-bit and lower post-training quantization [18, 34] (Chapter 6).

August to November 2022. Four results define the year. *LLM.int8()* introduced vector-wise quantization plus a mixed-precision decomposition that isolates the few outlier feature dimensions [93]. The Optimal Brain Compression (OBC) framework adapted Optimal Brain Surgeon to exact layer-wise post-training quantization [117]. *GPTQ* (October) made the OBS idea tractable at the scale of hundred-billion-parameter models by quantizing columns in a fixed order with a Cholesky-based lazy update [73] (Chapter 5). *SmoothQuant* (November) migrated activation difficulty into weights through a per-channel equivalent transformation to enable W8A8 [169] (Chapter 6). In September the 8-bit floating-point formats E4M3 and E5M2 were proposed for deep learning [61].

Remark 16.1. By the end of 2022 the three techniques that still anchor modern pipelines were in place: *error feedback in rounding order* (GPTQ), *equivalent transformations that move scale between operands* (SmoothQuant), and *an explicit high-precision escape for outliers* (LLM.int8()). Almost everything later is a refinement of one of these three ideas, or an attempt to remove the need for them by changing the basis (rotations) or the format (block-scaled floating point).

16.3 2023: weight-only 4-bit goes mainstream

Spring. QLoRA (May) combined a 4-bit NormalFloat data type with double quantization of scales and paged optimizers to fine-tune large models on a single GPU [138]. LLM-QAT (May) applied data-free quantization-aware training with generated data and, notably, also quantized the KV cache [94]. Systems such as FlexGen showed that low-bit weights and cache enable single-GPU throughput-oriented inference [54].

Summer. AWQ (June) observed that protecting a small fraction of salient weight channels, identified from *activation* statistics, suffices, and implemented this as an equivalent per-channel scaling rather than mixed precision [16]. SpQR, SqueezeLLM and OWQ (June) explored sparse outlier storage, sensitivity-weighted non-uniform codebooks, and outlier-aware column retention [126, 174, 175]. ZeroQuant-V2 studied PTQ systematically and introduced low-rank compensation [188]. QuIP (July) introduced *incoherence processing* with random orthogonal matrices and an adaptive rounding procedure later shown equivalent to GPTQ for the same ordering [150] (Chapter 7). OmniQuant (August) learned clipping and equivalent-transformation parameters by block-wise gradient descent [118]. PagedAttention gave the KV cache a virtual-memory abstraction that later enabled quantized cache pages [128]. The llama.cpp project introduced the GGML/GGUF ecosystem and the *k-quants* family, bringing block-wise low-bit formats to consumer hardware [191, 194].

Autumn. The OCP Microscaling (MX) specification (September) standardised block-scaled formats with a shared power-of-two scale [206]; the accompanying paper evaluated them for inference and training [114]. FP8-LM (October) trained GPT-style models with FP8 gradients, optimizer states and distributed communication [63]. BitNet (October) proposed training 1-bit-weight transformers from scratch [31]. Atom explored low-bit weight-activation serving with mixed precision and fine-grained groups [14]. QMoE compressed a 1.6-trillion-parameter MoE to under one bit per parameter with a dictionary-based scheme and custom decoding kernels [139]. LoftQ and QA-LoRA addressed quantization-aware LoRA initialization and

merging [95, 134]. StreamingLLM identified attention sinks, an observation later essential to KV-cache quantization [165].

16.4 2024: rotations, codes, KV cache, ternary weights, systems

January to March. AQLM (January) applied additive multi-codebook quantization with learned codebook updates to reach competitive 2-bit models [10]. KIVI and KVQuant (February and January) set the template for KV-cache quantization, namely per-channel keys and per-token values, with pre-RoPE key handling and non-uniform datatypes in KVQuant [85, 87] (Chapter 12). QuIP# (February) combined randomized Hadamard incoherence, E8-lattice codebooks and fine-tuning [149]. BitNet b1.58 (February) reported ternary-weight models that match a full-precision baseline at the scales studied [25]. GPTVQ and the study of massive activations clarified two further phenomena: vector quantization for LLM weights and the extreme-magnitude activations at specific tokens and channels [74, 101]. GEAR and QAQ proposed error-compensated and attention-aware KV compression [66, 135].

April to July. QuaRot (April) removed outliers by fusing Hadamard rotations into the weights and applying online Hadamard transforms where required, enabling end-to-end 4-bit weight, activation and KV quantization [146]. QServe (May) co-designed W4A8KV4 with progressive group quantization and kernel changes, reporting higher serving throughput on A100 and L40S GPUs [143]. SpinQuant (May) *learned* the rotations on the Stiefel manifold [173]. DuQuant (June) used rotation plus permutation [44]. QTIP (June) replaced vector codebooks by trellis-coded quantization with bitshift trellises, enabling very high effective dimension with fast decoding [144] (Chapter 8). QJL (June) gave a one-bit, zero-overhead sketch for keys [137]. EfficientQAT (July) split QAT into block-wise and end-to-end phases to make 2-bit QAT practical for large models [46]. The Llama 3 family was released in July [91]; earlier, an April 2024 empirical study had asked how well Llama 3 withstands quantization and concluded that degradation at low bit-widths is noticeable [90].

August to December. Marlin (August) provided a near-ideal W4A16 kernel that sustains close to the bandwidth-limited speedup up to moderate batch sizes [100] (Chapter 14). T-MAC and FLUTE explored table-lookup kernels for low-bit weights on CPUs and GPUs [56, 177]. VPTQ (September) scaled vector post-training quantization to extremely low bits [181]. PrefixQuant (October) isolated outlier tokens as a cached prefix to allow static per-tensor activation quantization [131]; FlatQuant learned affine transformations to flatten distributions [53]; and EvoPress treated the allocation of compression levels as a search problem with provable guarantees [49]. BitNet a4.8 combined ternary weights with 4-bit activations in much of the network [27]. Two foundational analyses appeared in November: "Scaling laws for precision" [162] and the study "Give me BF16 or give me death" on the accuracy–performance trade-offs of quantized Llama-3.1 models [20]. HIGGS gave a linearity theorem for per-layer error and a data-free Hadamard-plus-MSE-grid quantizer [79]. Finally, DeepSeek-V3 (December) reported large-scale FP8 mixed-precision training with fine-grained tile- and block-wise scaling [43] (Chapter 11).

Remark 16.2. The 2024 pattern is a *division of labour*. Rotations solved *activation* outliers; codebooks and trellises solved *weight rate* below 3 bits; Marlin-style kernels solved the *systems* gap

so that a 4-bit checkpoint translates into measured speedup; and FP8 training showed that quantization is also a *training-efficiency* technique, not only an inference one.

16.5 2025: reasoning models, MX/NVFP4 formats, and low-precision training

January to March. DeepSeek-R1 (January) launched the wave of open reasoning models [153], which immediately raised the question of whether long chain-of-thought generation is fragile under quantization (answered in Chapter 15). A study of FP4 training (January) proposed a differentiable quantization estimator and outlier clamping with compensation [59]. OSTQuant, RotateKV, AQUA-KV, KVTuner and NestQuant refined orthogonal-plus-scaling transformations, rotation for 2-bit KV caches, predictive KV compression, layer-wise KV precision search, and nested-lattice quantization of weights, activations and KV cache [11, 88, 115, 123, 157]. ParetoQ (February) provided a unified comparison of 1- to 4-bit QAT and reported a learning transition between 2 and 3 bits [129]. Work on MXFP4 training (February) analysed unbiased gradient estimation with stochastic rounding and Hadamard transforms [111].

April to June. Gemma 3 QAT checkpoints were released in April, with int4 variants targeted at consumer GPUs [67, 192]. TurboQuant (April) claimed near-optimal distortion for online vector quantization of KV cache and nearest-neighbour search [178]. BitNet b1.58 2B4T (April) released a native 1.58-bit open model trained on 4 trillion tokens [26]; BitNet v2 extended native 4-bit activations [30]. A systematic study of quantized reasoning models concluded that, within their evaluation, W8A8 and W4A16 are near-lossless [141]. Qwen3 was released in May together with an empirical quantization study [151, 152]. GPTAQ and Qronos refined the GPTQ objective to account for quantized inputs [71, 142]. A QAT scaling-law study (May) identified FC2 inputs as the W4A4 bottleneck [136]. The long-context quantization study (May) reported large degradations for some 4-bit methods on long-context tasks [96]. "FP4 All the Way" and Quartet (May) reported fully quantized FP4 training recipes for smaller models with Blackwell kernels [58, 148]; SageAttention3 (May) used FP4 microscaling for attention computation [160].

July to October. Kimi K2 (July) was released as a 1T-parameter open MoE [84]; GLM-4.5 followed in August [70]. *gpt-oss* (August) shipped open-weight MoE models whose expert weights are MXFP4, designed to fit a 120B-class model on a single 80 GB GPU [72]. MR-GPTQ and companion work (September) showed that MXFP4 and NVFP4 post-training quantization require format-specific treatment, with block-wise rotations and a dedicated kernel, and that "FP4 is not an automatic upgrade over INT4" [110]. NVIDIA reported pretraining a 12B model on 10 trillion tokens in NVFP4 with loss comparable to FP8 [116], and described the NVFP4 format publicly [203]. A study of INT versus FP formats at fine granularity (October) sharpened the debate [81].

November and December. Kimi K2 Thinking (November) was released with native INT4 weights obtained by quantization-aware training of the MoE components [201]. Four Over Six

(December) modified NVFP4 scaling to reduce error near block maxima [57], and SignRoundV2 (December) pushed rounding-search methods to extreme low-bit [164]. LMSYS and NVIDIA published a recipe for fine-tuning gpt-oss and recovering accuracy with MXFP4 QAT [197].

16.6 2026 to October: hardware-native 4-bit, QAD, and theory

January. At CES (6 January) NVIDIA made public claims for the Rubin platform, including NVFP4 inference and training throughput of the order of 50 and 35 petaflops per GPU and HBM4 bandwidth of 22 TB/s; these are vendor figures reported in the trade press [193] and we use them only as roadmap indications. Quartet II reported NVFP4 training with a new unbiased gradient estimator (MS-EDEN) and kernels up to 4.2× faster than BF16 on Blackwell [147]. *Quantization-aware distillation* (QAD) was presented as a way to recover NVFP4 accuracy of post-trained models (SFT, RL, merged) by matching a BF16 teacher's output distribution with a KL loss, rather than replaying the original training pipeline [133]; the authors report "near-BF16 accuracy" on five post-trained models, with no per-benchmark numbers in the abstract. ARCQuant added residual channels to keep NVFP4 uniform while compensating error inside the reduction dimension, with up to 3× speedup over FP16 reported on consumer and workstation Blackwell GPUs [13].

February to April. D2Quant targeted the down-projection layers and LayerNorm shift in sub-4-bit PTQ [40]. DeepSeek released V4 on 24 April 2026 (secondary report): 1.6T total (49B active) in the Pro variant and 284B total (13B active) in Flash, a 1M-token context, with MoE experts trained in FP4 and most other parameters in FP8 [205]. We could not verify from primary sources whether the released inference weights are further quantized; the secondary report does not say. A theoretical treatment of MoE quantization based on router-norm change was posted in April [104].

May to October. SOAR optimised block and tensor scales for NVFP4 in closed form with decoupled quantization and dequantization scales [170]. A set of MoE-specific proposals (GEMQ, AlphaQ, routing-aware quantization, MoBiE) studied router drift, mixed-precision allocation and binarized experts [8, 68, 103, 158]. LC-QAT and further QAT work targeted low-bit accuracy recovery [89]. KV-cache work moved towards rotation of output-relevant quantities and coupling quantization with sparse retrieval [122, 125, 154].

16.7 Summary table

16.8 Cross-cutting trends

From algorithms to formats. Through 2024 the frontier was *algorithmic* (how to round, how to rotate). From late 2025 a large share of work is *format-specific*: the optimum for MXFP4 (power-of-two scales, 32-element blocks) differs from NVFP4 (E4M3 scales, 16-element blocks) and from INT4 with group size 128, and published recipes now name their target format [57, 110, 170].

Table 16.1: Condensed timeline of representative results. "Binding bottleneck" is the author's characterization.

Year	Representative advances	Binding bottleneck	Representative citations
2022	LLM.int8(), OBC, GPTQ, SmoothQuant, FP8 formats	Activation outliers; scale of rounding	[61, 73, 93, 117, 169]
2023	AWQ, SpQR, QuIP, QLoRA, LLM-QAT, MX spec, BitNet, QMoE	Weight-only 4-bit quality; fine-tuning memory	[16, 31, 94, 138, 139, 150, 174, 206]
2024	QuaRot, SpinQuant, QuIP#, AQLM, QTIP, KIVI, KVQuant, Marlin, b1.58, DeepSeek-V3 FP8	W4A4 outliers; 2-bit rate; KV memory; kernel gap	[10, 25, 43, 85, 87, 100, 144, 146, 149, 173]
2025	gpt-oss MXFP4; K2 Thinking INT4 QAT; NVFP4 pretraining; MR-GPTQ; Quartet; reasoning-model studies	FP4 format-specific accuracy; training stability; long-CoT robustness	[72, 110, 116, 141, 148, 201]
2026	Quartet II; QAD; ARCQuant; DeepSeek V4; MoE theory; NVFP4 scale optimisation	Post-training recovery; MoE routing; scale design	[13, 104, 133, 147, 170, 205]

From post-training to training-time. The share of releases that are quantization-aware (Gemma 3 QAT, Kimi K2 Thinking, gpt-oss, Nemotron NVFP4) rose in 2025 to 2026; PTQ remains the default for third-party quantization of models whose training recipe is unavailable [72, 133, 192, 201].

From dense models to MoE and reasoning. The deployment-relevant models of 2025 to 2026 are mostly sparse and long-generation; this moves the binding constraint from per-layer error to router stability and error accumulation over thousands of tokens (Chapter 15).

From average benchmarks to distributional evidence. The accumulating studies of long-context, multilingual, safety and reasoning behaviour [41, 55, 96, 109, 141] converge on the message that perplexity and short-form multiple choice under-report damage (Chapter 13).

Chapter 17

Capabilities: What Quantization Delivers in Practice

This chapter collects, in tabular form, what the literature reports quantization can achieve. The tables have three kinds of content, kept visibly separate. (i) *Reported* figures are quoted from the abstract or model card of the cited work, on the hardware and models that work used; they are not comparable across rows because models, baselines, batch sizes and GPUs differ. (ii) *Analytic* figures are arithmetic that follows from the definitions (for example bytes per parameter) and carry no empirical uncertainty. (iii) *Qualitative ratings* are the author's synthesis and are marked as such. No number in this chapter was measured on new hardware by the author.

17.1 Memory arithmetic

The most reliable benefit of quantization is the reduction in bytes. For a model with P parameters stored at b effective bits per weight (including scales and zero-points, Chapter 3), weight memory is $Pb/8$ bytes.

For the KV cache the arithmetic is $2\, n_{\text{layers}}\, n_{\text{kv}}\, d_h\, b/8$ bytes per token (keys and values). For a Llama-3.1-70B-style configuration with 80 layers, 8 key–value heads and head dimension 128 [91], this is $2 \cdot 80 \cdot 8 \cdot 128 \cdot 2 = 327{,}680$ bytes (320 KiB) per token in 16-bit precision, hence about 42.9 GB for a single 131,072-token sequence. At an effective 4 bits (ignoring the scale overhead) the same sequence needs about 10.7 GB. This is why KV quantization (Chapter 12) matters most at long context and large batch, where the cache rivals or exceeds the weights.

Memory is the guaranteed gain; speed is conditional

Weight bytes shrink by exactly $16/b$ whatever the kernel. Latency improves only if the kernel keeps the GEMM memory-bound and the dequantization overhead is hidden (Chapter 14); throughput improves only if saved memory is converted into larger batches or longer contexts.

17.2 Weight-only post-training quantization

Reading across the table. All rows address the same regime (memory-bound decoding) with different trade-offs. Scalar group-wise methods (GPTQ, AWQ) are the practical default at 4 bits because kernels are mature. Sub-3-bit results (AQLM, QuIP#, QTIP, EfficientQAT) show that accuracy retention is possible, but at higher encoding cost and with narrower kernel support; they matter when the memory budget, not the engineering budget, binds.

Table 17.1: Weight memory in decimal gigabytes for P parameters at an *effective* bit-width b (analytic: $Pb/8$). Real checkpoints add 0.1 to 0.5 bits for scales and keep some tensors (embeddings, norms, sometimes the output head) at higher precision, so measured files are somewhat larger. For MoE models P is the *total* parameter count.

Parameters	16 bit	8 bit	4 bit	3 bit	2 bit
8B	16.0	8.0	4.0	3.0	2.0
70B	140	70	35	26.3	17.5
405B	810	405	202.5	151.9	101.3
671B (DeepSeek-V3 class)	1342	671	335.5	251.6	167.8
1T (Kimi K2 class)	2000	1000	500	375	250

17.3 Weight and activation quantization

Why W4A8KV4 and W4A4 differ. QServe's contribution is largely about dequantization overhead (20 to 90% in prior INT4 systems, as its authors report) on low-throughput CUDA cores, not about rounding error [143]. This is the system-level counterpart of the algorithmic rotation results: rotation makes W4A4 *accurate*, QServe-style design makes W4A8 *fast* on existing GPUs, and block-scaled FP4 hardware makes W4A4 *native* on newer ones (Chapter 11).

17.4 KV-cache quantization

The long-context caveat is essential. A broad study of quantization at long context reports that 8-bit methods lose about 0.8% on average while 4-bit methods can lose up to 59% depending on model, method and task, with some 4-bit settings losing 32% on Llama-3.1-70B [96]. Headline KV results at short context therefore do not transfer automatically (Chapter 13).

17.5 Training-time low precision

17.6 Fine-tuning and the long tail of use cases

QLoRA fine-tunes a 65B-parameter model on a single 48 GB GPU by backpropagating through a frozen 4-bit NF4 base into LoRA adapters; the authors report that their Guanaco models reach 99.3% of ChatGPT's score on the Vicuna benchmark [138]. Quantization is therefore also an *access* technology: it determines who can adapt large models at all. Quantization-aware distillation for NVFP4 extends this idea to post-trained models, where the authors report near-BF16 accuracy on five models [133].

Table 17.2: Headline claims of representative weight-only PTQ methods, as stated by their authors (reported; hardware, baselines and models differ between rows, so rows must not be ranked against each other).

Method	Headline claim	Caveat
LLM.int8() [93]	Int8 halves inference memory; a 175B checkpoint can be converted and used without degradation	Mixed-precision decomposition; latency benefit limited in the original implementation
GPTQ [73]	175B-parameter models quantized in about four GPU hours at 3 to 4 bits with negligible degradation; 3.25× (A100) and 4.5× (A6000) speedup over FP16 for generation	Speedup is for small-batch generation with custom kernels
AWQ [16]	Protecting about 1% salient channels reduces error; TinyChat gives more than 3× speedup over the Hugging Face FP16 implementation on desktop and mobile GPUs	Speedup relative to an unoptimised baseline
QuIP# [149]	State of the art at or below 4 bits per weight using Hadamard incoherence, E8 lattice codebooks and fine-tuning	Fine-tuning needs calibration compute
AQLM [10]	First scheme reported Pareto-optimal below 3 bits per parameter; GPU and CPU kernels match or beat optimised FP16 speed	Encoding is expensive compared with GPTQ
QTIP [144]	Trellis-coded quantization achieves ultra-high-dimensional quantization with a hardware-efficient bitshift trellis	Kernel support narrower than for scalar formats
EfficientQAT [46]	2-bit Llama-2-70B in 41 hours on one A100-80GB, with a 69.48 versus 72.41 average accuracy gap (under 3 points)	QAT-style training; accuracy is a zero-shot average

17.7 Capability matrix

17.8 Accuracy retention: what can be said in general

Studies that compare many models and methods allow some cautious generalisations. A large evaluation of quantized Llama-3.1 models (about 500,000 individual evaluations) concluded that FP8 weight-and-activation quantization is lossless across scales, INT8 loses 1 to 3%, and 4-bit weight-only quantization is competitive with 8-bit integer; W4A16 was most cost-effective for synchronous deployments and W8A8 for asynchronous deployments on high-end GPUs [20]. A study of quantized reasoning models reports that W8A8 and W4A16 are lossless or near-lossless and that lower precisions lose more, depending on the task difficulty and model size [141]. For MXFP4 and NVFP4 post-training quantization, MR-GPTQ reports that the best FP4 accuracy needs format-specific rotation and rounding, and that FP4 is not an automatic upgrade over INT4, with up to 3.6× layer-wise and 2.2× end-to-end speedup on B200 and 6×

Table 17.3: Reported results for weight-and-activation quantization (as stated by the authors).

Method	Setting	Reported outcome
SmoothQuant [169]	W8A8	Up to 1.56× speedup and 2× memory reduction with negligible accuracy loss; serves a 530B model in one node
QuaRot [146]	W4A4KV4, Llama-2-70B	At most 0.47 WikiText-2 perplexity loss and 99% of zero-shot performance retained; lossless 6 and 8 bit with round-to-nearest
SpinQuant [173]	W4A4KV4, Llama-2-7B	Zero-shot gap to full precision reduced to 2.9 points (19.1 better than LLM-QAT, 25.0 better than SmoothQuant); up to 13-point spread between random rotations; up to 45.1% smaller gap than QuaRot on Llama-3-8B
QServe [143]	W4A8KV4	Maximum serving throughput up to 1.2× (A100) and 1.4× (L40S) for Llama-3-8B and 2.4× and 3.5× for Qwen1.5-72B versus TensorRT-LLM; about 3× lower serving cost claimed

Table 17.4: Reported KV-cache quantization results.

Method	Setting	Reported outcome
KIVI [85]	2-bit, tuning-free, per-channel keys and per-token values	Almost unchanged quality for Llama, Falcon and Mistral; 2.6× less peak memory including weights; up to 4× larger batch; 2.35 to 3.47× throughput
KVQuant [87]	3-bit non-uniform with per-vector outlier isolation	Under 0.1 perplexity degradation on WikiText-2 and C4; 1M context for LLaMA-7B on one A100-80GB and 10M on 8 GPUs; up to about 1.7× matrix-vector speedup
TurboQuant [178]	Online, data-oblivious vector quantization	Reported quality-neutral at 3.5 bits per channel and within about 2.7× of the information-theoretic lower bound on distortion

and 4× on RTX 5090 reported [110].

Remark 17.1 (What the tables cannot say). No table here reports an accuracy figure measured by the author on a production model. Aggregated accuracy numbers move by one to several points depending on calibration data, prompt format and the benchmark set, so *differences below roughly one point between methods in different papers should not be interpreted*. The within-paper comparisons (same model, same evaluation) are informative; the across-paper comparisons are not.

Table 17.5: Reported training-time results for FP8 and FP4 (as stated by authors; not independently reproduced).

Work	Setting	Reported outcome
FP8-LM [63]	FP8 gradients, optimizer state, communication; GPT-175B on H100	39% less real memory; 75% faster than BF16 Megatron-LM; 37% faster than Transformer Engine
COAT [38]	FP8 optimizer states and activations	1.54× end-to-end memory reduction and 1.43× speedup versus BF16 with nearly lossless performance
DeepSeek-V3 [43]	FP8 mixed precision at 671B total parameters, 14.8T tokens	First validation of FP8 training at this scale as reported by the authors
NVFP4 pretraining [116]	12B model, 10T tokens	Loss and downstream accuracy comparable to an FP8 baseline
Quartet II [147]	NVFP4, 38B tokens	Kernels up to 4.2× faster than BF16; unbiased estimator with more than 2× lower error than stochastic rounding

17.9 Quantization in released models

17.10 Summary of capabilities

1. **8-bit** (FP8 or INT8 W8A8) is, for large models, near-lossless and broadly supported; the limiting factor is operational (kernels, calibration) rather than accuracy [20, 169].
2. **4-bit weight-only** (GPTQ, AWQ, group size 64 to 128) is the production default for memory-bound decoding on mainstream GPUs [16, 73, 100].
3. **4-bit weights and activations** is feasible with rotations or block-scaled formats, with accuracy that depends on model family [110, 146, 173].
4. **2 to 3 bits** is feasible with vector/trellis codes or QAT, trading accuracy margin for memory [10, 46, 129, 144].
5. **KV cache at 2 to 4 bits** is feasible at short and moderate context; long-context evidence is weaker [85, 87, 96].
6. **Training at FP8** is production practice; **FP4 training** has credible large-scale evidence from one source and promising academic recipes [43, 116, 147].

Table 17.6: Where each technique family applies. ✓ = primary use, (✓) = possible with caveats, ✗ = not applicable. Ratings of maturity are the author's qualitative judgement (High: widely deployed with mature kernels; Medium: public implementations, limited kernel or framework support; Low: research code).

Technique family	Weights	Acts	KV	Training	Maturity
Round-to-nearest, group-wise (INT8/INT4)	✓	✓	✓	✗	High
GPTQ-family (second-order)	✓	✗	✗	✗	High
AWQ / activation-aware scaling	✓	(✓)	✗	✗	High
SmoothQuant-style smoothing	(✓)	✓	✗	✗	High
Rotations (QuaRot, SpinQuant)	✓	✓	✓	✗	Medium
Vector / trellis codes	✓	✗	(✓)	✗	Medium–Low
Mixed-precision allocation	✓	(✓)	(✓)	✗	Medium
QAT / distillation	✓	✓	(✓)	(✓)	Medium (rising)
Ternary / binary native models	✓	(✓)	✗	✓	Low–Medium
FP8 formats	✓	✓	✓	✓	High
MXFP4 / NVFP4	✓	✓	(✓)	✓	Medium (rising)
KV-specific (KIVI, KVQuant)	✗	✗	✓	✗	Medium

Table 17.7: Released models with native low-precision weights or training, as documented in the cited sources.

Release	Precision	Notes
gpt-oss [72]	MXFP4 MoE weights	Larger model runs on one 80 GB GPU; smaller model within 16 GB
Kimi K2 Thinking [201]	Native INT4 (QAT)	Moonshot AI model card; aimed at low-latency generation
Gemma 3 QAT [192]	int4 QAT checkpoints	Aimed at consumer GPUs
Llama 3.2 1B/3B [200]	SpinQuant and QLoRA-style QAT variants	Meta's quantized releases for on-device use
BitNet b1.58 2B4T [26]	Native ternary weights	Trained on 4 trillion tokens
DeepSeek V4 [205]	FP4 experts in training, FP8 elsewhere (secondary report)	Inference-time quantization not stated in the source

Chapter 18

Pros, Cons, and Risks

The previous chapter listed what quantization delivers. This chapter lists what it costs and how it fails. We first give a structured pros-and-cons comparison of the main technique families, then a risk register with explicit, subjective likelihood and impact ratings, and then discuss the most consequential risks in depth: silent degradation hidden by averages, calibration sensitivity, safety and security, kernel and hardware lock-in, MoE and reasoning fragility, and reproducibility.

18.1 Pros and cons by technique family

18.2 Risk register

Each risk is rated on two scales: *likelihood* that a practitioner following common practice encounters the risk (Low, Medium, High) and *impact* if it occurs. These ratings are the author's judgements informed by the cited evidence; they are not measured probabilities.

18.3 Silent degradation: why average metrics are not enough

A quantized model can match a baseline's mean accuracy while changing many individual answers, as "Accuracy is Not All You Need" demonstrates: answers flip from correct to incorrect and vice versa in nearly equal proportion, flips and KL divergence correlate, and compressed models were significantly worse on free-form MT-Bench generation despite similar benchmark accuracy [55]. This has a simple statistical interpretation.

Proposition 18.1 (Equal accuracy does not imply equal behaviour)**.** *Let a baseline model and a quantized model answer N binary questions, with c_{10} answers correct only for the baseline and c_{01} correct only for the quantized model. The accuracy difference is $(c_{01} - c_{10})/N$, whereas the fraction of answers that changed correctness is $(c_{01} + c_{10})/N$.*

Proof. Both statements follow by counting: accuracy difference counts net changes, flips counts total changes. Setting $c_{01} = c_{10}$ yields zero accuracy difference with flip fraction $2c_{10}/N$ which can be arbitrarily large. □

Example 18.1 (Illustrative)**.** If a model's accuracy on 1000 questions is 70% before and after quantization, a flip count of 60 (30 each way) is fully consistent with the 0.0-point difference. Under the paired McNemar test the net difference is zero, but 6% of users would see a different answer. These numbers are hypothetical.

Consequence for practice. The evaluation protocol recommended in Chapter 13 reports KL divergence on held-out text, flip rate on a task suite, and at least one free-form generation benchmark, besides mean accuracy.

18.4 Calibration sensitivity

PTQ methods rely on a small calibration set to compute activation statistics or Hessians. Empirical work finds "substantial variations in downstream task performance" as the calibration data change, contrary to earlier suggestions of robustness [36]. Three mechanisms plausibly contribute (author's analysis): (i) the Hessian $\boldsymbol{H} = \boldsymbol{X}\boldsymbol{X}^\top$ in GPTQ estimates the activation covariance on the calibration domain, so directions rare in calibration are under-protected; (ii) activation outliers appear on specific tokens (massive activations) that may be absent from a short calibration sample [101, 165]; (iii) in MoE models some experts receive few or no calibration tokens (Chapter 15). Mitigations are cheap: more diverse calibration data, domain-matched samples, sequence lengths matched to deployment, and reporting variance over calibration seeds.

18.5 Safety, trustworthiness, and security

Trustworthiness. A study of three LLMs and five compression techniques across eight trustworthiness dimensions found that 4-bit quantization retained the trustworthiness of the original models while pruning degraded it even at 50% sparsity, that moderate quantization could even improve some dimensions such as ethics and fairness, but that extreme quantization to 3 bits tended to reduce trustworthiness significantly, a risk invisible to benign benchmarks [41]. A separate multi-dimensional safety evaluation of compression reaches the conclusion that perplexity is not a sufficient indicator [19]. The practical rule: any change in bit-width of a safety-tuned model needs a safety re-evaluation.

Quantization-conditioned attacks. Exploiting LLM Quantization shows that an adversary can fine-tune a malicious model, compute the set of full-precision weights that map to the same quantized model (a box constraint determined by the rounding cells), and use projected gradient descent to push the full-precision weights back toward benign behaviour inside that set [50]. The published full-precision model then looks benign but, once quantized with the targeted method, behaves maliciously. The attack is possible because rounding maps a set of full-precision models to one quantized model: the *quantization map is many-to-one*, so behaviour of the quantized model is not determined by tests on a single full-precision representative.

Operational implication

Evaluate the *quantized artifact that will be deployed*, not its full-precision parent, and obtain quantized checkpoints from sources whose conversion pipeline you can audit. This risk is rated low in likelihood here only because it requires a motivated adversary; its impact is high.

Multilingual and cultural loss. A study of quantization of multilingual LLMs found that automatic metrics understate the damage: for Japanese the authors report a 1.7% average automatic-metric drop versus a 16.0% drop in human evaluation, with non-Latin scripts and harder tasks affected more [109].

18.6 Systems risks: kernels, formats, and lock-in

Kernel maturity. A format is only as fast as its kernel. W4A16 has high-quality kernels for mainstream GPUs (Marlin reports close to 4× speedup up to batch 16–32 and 2.8× end-to-end in vLLM) [100]; novel formats such as trellis codes or FP4 on non-Blackwell hardware have fewer. Dequantization overhead on CUDA cores can erase gains at larger batch (QServe reports 20 to 90% overhead in earlier INT4 systems) [143].

Hardware and format lock-in. Native FP4 (MXFP4, NVFP4) depends on specific accelerator generations, and the best algorithms are format-specific: MR-GPTQ reports two distinct failure modes: NVFP4's small group size provably neutralizes traditional outlier-mitigation techniques, while MXFP4's power-of-two scales induce high error [110]. A checkpoint optimised for one format therefore does not transfer cleanly to another vendor or generation, which is a form of lock-in.

Framework drift. Two nominally identical formats (for example "INT4, group 128") differ across frameworks in scale storage, symmetric versus asymmetric zero-point, packing order, rounding mode, and which layers are excluded. The effect on accuracy is usually small but not zero; for reproducibility we recommend recording the quantizer version, group size, scale format, excluded layers and calibration set.

18.7 MoE and reasoning model risks

For MoE models, quantization noise can change the top-k routing decision itself; this is a discrete effect not captured by smooth error analysis, and MoE-specific work addresses calibration imbalance, expert-selection shift and router adaptation [45, 68, 158]. For reasoning models the key risk is accumulation: small per-token distribution shifts compound across thousands of generated tokens; the systematic study finds W8A8 and W4A16 lossless or near-lossless but lower bit-widths risky, with model size, origin and task difficulty as determinants [141]. The error accumulation model of Chapter 15 gives a stylised explanation (illustrative).

18.8 Training-time risks

Low-precision training adds optimization risk to representation risk: FP8 and FP4 recipes rely on fine-grained scaling, stochastic rounding, Hadamard transforms and selected high-precision layers [43, 116, 147]. The evidence for 4-bit pretraining at frontier scale comes mainly from a small number of reports, one with a 12B-parameter model on 10 trillion tokens [116]. Divergence, loss spikes and subtle quality loss may appear only at scales or token counts not

yet tested. We therefore treat FP4 pretraining at frontier scale as promising but unproven for general use.

18.9 Reproducibility and evidence quality

- **Benchmark mismatch.** Papers differ in model versions, tokenizer handling, sequence length for perplexity, zero-shot task sets and prompt templates, so cross-paper comparisons of small differences are unreliable.
- **Speedups are baseline-dependent.** Speedup claims depend on the baseline implementation (an optimised FP16 kernel versus a research baseline), batch size, sequence length, GPU and serving framework.
- **Preprint volume.** A sizable share of 2025 and 2026 sources used in this survey are preprints that have not completed peer review; we quote their claims as reported.
- **Vendor roadmaps.** Throughput and bandwidth figures for hardware announced but not shipping (for example Rubin [193]) are claims, not measurements.

18.10 Risk-informed rules of thumb

1. Treat 8-bit as default-safe and 4-bit weight-only as default-acceptable for large dense models, but still run a task-level regression suite.
2. Treat any sub-4-bit or 4-bit-activation deployment as a project requiring evaluation on the target task, language, context length and safety suite.
3. Evaluate the deployed artifact, record the quantizer configuration, and keep the full-precision parent for regression comparison.
4. For long-context, multilingual, reasoning and MoE workloads, do not transfer conclusions from short-context English benchmarks.

Table 18.1: Advantages and disadvantages of the main technique families (author's synthesis of the evidence in Chapters 5–15).

Family	Pros	Cons
Round-to-nearest group-wise	No calibration; trivially reproducible; supported everywhere; optimal enough at 8 bits	Visible loss at 4 bits and below; sensitive to outliers within a group
GPTQ-style second-order	Minutes to hours; data-light; strong 3 to 4 bit results; mature kernels	Calibration-set dependent [36]; layer-wise objective ignores later layers; collapses toward 2 bits
AWQ / scaling	Very fast; no backpropagation; robust across modalities; ubiquitous kernels	Mostly 4-bit weight-only; relies on activation statistics of the calibration set
SmoothQuant	W8A8 on INT8 tensor cores; training-free	Needs per-model α; limited at 4-bit activations
Rotations (QuaRot, SpinQuant)	Removes outliers from the residual stream; enables W4A4 and KV4; theory-backed	Online Hadamards add latency if not fused; learned rotations need optimisation; benefit depends on architecture; interacts with format (MXFP4/NVFP4) [110]
Vector / trellis	Best quality below 3 bits; approaches rate–distortion bound	Expensive encoding; custom kernels; limited framework integration
QAT / distillation	Best accuracy at the lowest bits; can adapt to format; native releases	Needs data, compute and the original training recipe; multi-stage post-trained models complicate QAT [133]
Native ternary	Large theoretical efficiency gains	Requires pretraining from scratch; evidence at frontier scale is absent; hardware lacks native ternary support
FP8	Near-lossless; hardware-native on Hopper and later; training and inference	Needs scaling-factor management; accuracy depends on recipe
MXFP4 / NVFP4	Hardware-native 4-bit with block scales; 4-bit activations	Accuracy format-specific; INT4 can be better in some settings [81, 110]; vendor-specific hardware
KV quantization	Scales batch and context; orthogonal to weight methods	Long-context risk [96]; requires fused attention kernels; quality depends on outlier handling

Table 18.2: Risk register (ratings are subjective).

Risk	Likelihood	Impact	Principal evidence and mitigation
Accuracy cliff at low bits (below 4)	High	High	Reported across studies; mitigate with QAT, mixed precision, task-level evaluation [20, 129]
Hidden degradation behind average metrics	High	High	Flips and trust studies [41, 55]; use KL and task suites
Long-context degradation	Medium	High	Up to 59% drops for some 4-bit methods [96]; test at target length
Calibration-set sensitivity	Medium	Medium	Substantial variance [36]; use diverse calibration, report seeds
Multilingual and low-resource loss	Medium	High	Human evaluation shows larger drops than automatic metrics [109]
Safety and trustworthiness shifts	Medium	High	Very low bits reduce trustworthiness [19, 41]; re-run safety evaluation
Quantization-conditioned attacks	Low	High	Benign full-precision model with malicious quantized behaviour [50]; verify quantized artifact
Reasoning-chain fragility	Medium	Medium	Lower bit-widths introduce accuracy risk [141]; test AIME/GPQA-class tasks
MoE routing drift and imbalance	Medium	Medium	Router flips and calibration imbalance [45, 158]; use expert-aware calibration
Kernel immaturity / format lock-in	High	Medium	FP4 formats tied to hardware generations; kernels vary [110]
Reproducibility and drift between frameworks	High	Medium	Same nominal format, different scale conventions, fused ops and rounding; pin versions
Training instability at FP4	Medium	High	Large-scale evidence only from one group so far [116]; use high-precision exceptions
Vendor claims not verified	High	Low–Medium	Roadmap numbers [193] are not measurements; plan with shipping hardware

Chapter 19

Deployment Decision Guide

This chapter turns the survey into a procedure. It is organised as a flowchart (Figure 19.1), a worked sizing example with analytic arithmetic, and a set of tables mapping hardware, workload and model type to recommended starting points. All recommendations are the author's synthesis; they encode defaults to try first, not guarantees, and every one must be validated with the evaluation protocol of Chapter 13 before shipping.

19.1 Principle: start from the binding resource

Quantization is a trade between four resources: memory capacity, memory bandwidth, compute throughput, and accuracy margin. The first question is therefore not "which algorithm" but "which resource binds". Using the roofline analysis of Chapter 14:

- If the model *does not fit*, capacity binds and any weight compression helps, even if kernels are slow.
- If the model fits but decoding at small batch is slow, bandwidth binds and weight-only quantization (W4A16, W8A16) gives close to $16/b$ speedup with good kernels [100].
- If serving is at large batch (compute-bound GEMMs), weight-only quantization stops helping and weight-and-activation formats (FP8, W8A8, W4A8, W4A4 on supporting hardware) are needed [20, 143].
- If long contexts or many concurrent sequences make the KV cache the largest tensor, KV quantization (or other cache reductions, covered in the companion paper) binds the design [85, 87].

19.2 Decision flowchart

19.3 Worked sizing example (analytic)

Consider serving a 70B-parameter dense model with a Llama-3.1-70B-style KV layout (80 layers, 8 KV heads, head dimension 128, so 320 KiB per token at 16 bits; Chapter 17) on a single 80 GB GPU. Ignoring activations, framework overhead and fragmentation (which in practice consume several GB), the number of KV tokens that fit after loading weights is $(80\,\text{GB} - \text{weights})/\text{bytes per token}$.

The table shows the typical sequence of reasoning: weight quantization is what makes the model fit at all; the KV precision then determines concurrency. With 8-bit weights the free memory holds less than one 32k-token sequence, so a deployment targeting long contexts would pair 4-bit weights with a lower-precision KV cache, subject to the long-context evaluation caveat [96].

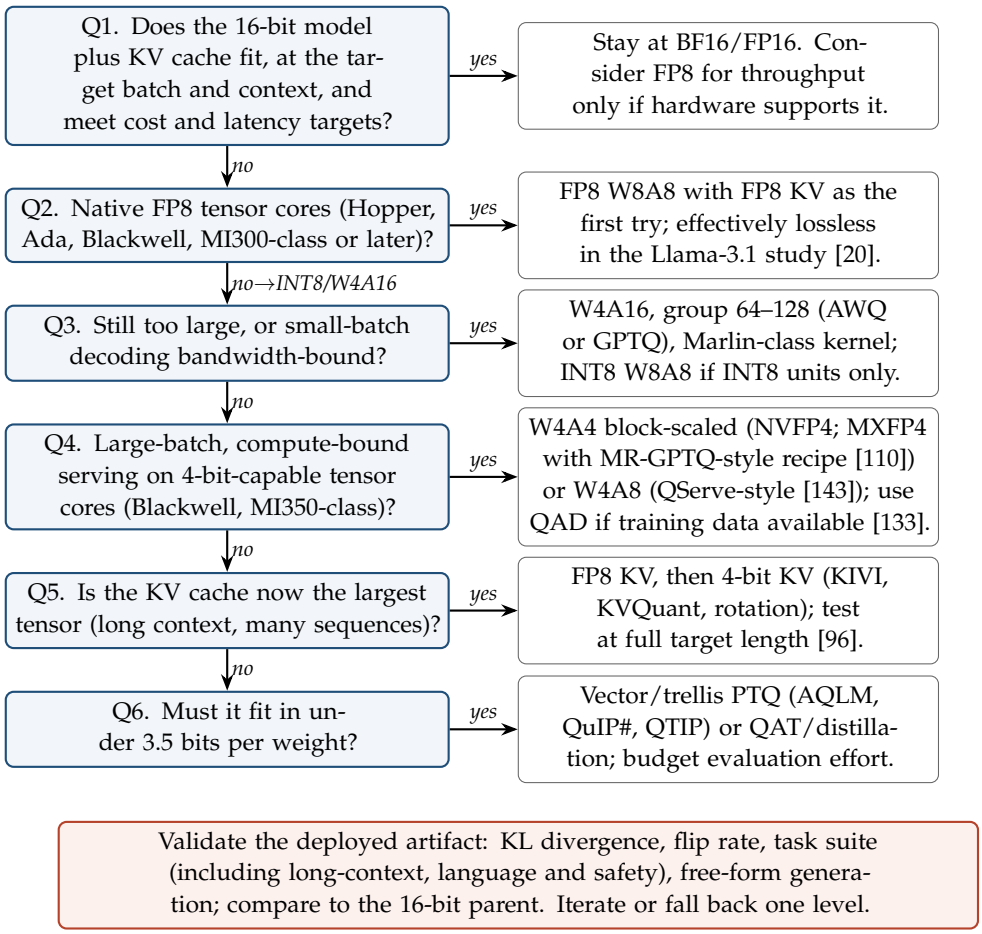


Figure 19.1: Decision flowchart for choosing a quantization configuration (author's synthesis). A "yes" answer gives the recommended *first try*; the questions are not mutually exclusive: a deployment may pass through several (for example FP8 weights and activations with a 4-bit KV cache). Every outcome ends in the validation step.

Bandwidth bound on decoding speed. At batch size one, each generated token reads all weights, so the weights-only upper bound on steps per second is bandwidth divided by weight bytes. With a nominal 3.35 TB/s of HBM bandwidth (approximate public specification of the H100 SXM), 35 GB of 4-bit weights gives at most about 96 steps/s and 70 GB of 8-bit weights about 48 steps/s, a ratio of two; real systems reach a fraction of these bounds because of attention reads, scale reads, kernel inefficiency and launch overhead. These numbers are analytic upper bounds, not benchmarks.

Table 19.1: KV capacity on one 80 GB accelerator for a 70B model (analytic, optimistic: no overheads). Weights at 16 bits do not fit. "Tokens" is the total over all concurrent sequences; "32k sequences" divides by 32,768.

Weights	KV precision	Free memory (GB)	KV tokens	32k sequences
BF16 (140 GB)	–	does not fit	–	–
8-bit (70 GB)	16-bit	10	30,500	0.9
4-bit (35 GB)	16-bit	45	137,300	4.2
4-bit (35 GB)	8-bit	45	274,700	8.4
4-bit (35 GB)	4-bit	45	549,300	16.8

19.4 Recommendations by hardware class

19.5 Recommendations by workload and model type

19.6 Choosing the method within a format

19.7 Frameworks and operational notes

Practical deployment depends on what the serving stack supports: vLLM, SGLang and TensorRT-LLM provide Marlin-style W4A16, FP8 and (on Blackwell) FP4 paths; llama.cpp provides GGUF k-quants for CPU and consumer GPUs; bitsandbytes provides NF4 and LLM.int8() for fine-tuning and prototyping [163, 191, 204, 212, 214] (Chapter 14). Because feature coverage changes rapidly, check release notes of the exact version in use.

1. **Pin versions** of quantizer, kernel library and framework; log the full quantization config.
2. **Keep sensitive tensors in higher precision**: embeddings, output head, norms, and the first and last blocks are the usual exceptions; MoE routers are commonly kept at high precision (common practice; see also the router-drift work in [158]).
3. **A/B test with real traffic** where possible; offline benchmarks cannot capture distribution shift.
4. **Plan a rollback path** to the previous precision level.

19.8 Cost reasoning

Quantization reduces cost through three levers: fewer GPUs per replica (capacity), higher tokens per second per GPU (bandwidth or compute) and larger batch per GPU (more KV capacity). The first and third are determined by bytes and are reliable; the second depends on kernels. For a rough first estimate of cost per token, divide the GPU price per hour by the sustained tokens per hour measured with the *actual* serving stack at the target concurrency; published speedups (for example 2.4 to 3.5$\times$ throughput over a baseline in QServe's reported setting [143]) should be read as upper indications of what is possible, not predictions for another stack.

Table 19.2: Starting points by hardware class (author's synthesis; "first try" means the configuration to evaluate first). Hardware feature statements follow Table 11.4; where the author is uncertain, check current vendor documentation.

Hardware	Large-batch serving	Small-batch / edge / local
Ampere-class (A100, A6000, RTX 30)	INT8 W8A8 (SmoothQuant-style) or W4A16 with Marlin-class kernels; W4A8 (QServe) if engineering budget allows	W4A16 (AWQ or GPTQ); GGUF k-quants for CPU-offload scenarios
Hopper and Ada (H100, L40S, RTX 40)	FP8 W8A8 and FP8 KV as default; W4A16 where memory binds	W4A16; FP8 where supported
Blackwell (B200, RTX 50)	NVFP4 or MXFP4 W4A4 with a format-specific recipe, or FP8; validate against FP8 first [110]	W4A16 or NVFP4 weights; consumer-GPU gains reported by MR-GPTQ and ARCQuant [13, 110]
AMD Instinct (MI300/MI350)	FP8; MXFP4 on MI350-class as documented by AMD	Framework-dependent; check support
CPU and Apple silicon	–	GGUF k-quants, MLX 4-bit [191]
Mobile / NPU	–	INT4 or INT8 weights with vendor toolchains; SpinQuant-style variants for small models [200]

Table 19.3: Starting points by workload and model type (author's synthesis).

Workload / model	First try	Watch out for
Chat, short context, dense model, memory-bound	W4A16 AWQ/GPTQ, group 128	Calibration data; safety regression [36, 41]
High-throughput API serving	FP8 W8A8 + FP8 KV; W4A8 or FP4 on supporting hardware	Kernel maturity; asynchronous vs synchronous mix [20]
Long-context (above 32k)	8-bit KV first; weights W4A16 or FP8	4-bit methods can degrade sharply [96]
Reasoning / chain-of-thought	W8A8 or W4A16; avoid lower precision without testing	Accumulation over long outputs [141]
MoE (large, many experts)	Expert weights 4-bit (W4A16); native releases (MXFP4, INT4 QAT) when available	Router drift; calibration imbalance [45, 72, 201]
Multilingual / low-resource	Higher bit-width; human or LLM-judge evaluation	Larger real drops than automatic metrics [109]
Safety-critical or regulated	8-bit, with full safety re-evaluation of the exact artifact	Quantization-conditioned attacks [50]
Fine-tuning on one GPU	QLoRA (NF4 base + LoRA) [138]	Merge-time rounding (QA-LoRA, LoftQ) [95, 134]
Pretraining / continued training	FP8 mixed precision; FP4 only with published recipe and fallback layers	Instability at scale [43, 116]
Sub-3-bit memory limits	Vector/trellis PTQ or QAT	Encoding cost; kernel support [10, 46, 144]

Table 19.4: Method selection given the target format and constraints (author's synthesis).

Target	Preferred methods	Reason
INT4 weight-only, no training data	AWQ, GPTQ (with act-order, damping), SignRound variants	Fast and mature; extensions (GPTAQ, Qronos) refine the objective [16, 71, 73, 142]
INT4 weight-only, small calibration budget	AWQ	No backpropagation or reconstruction [16]
W4A4 INT with outlier-heavy model	Rotation (QuaRot, SpinQuant, FlatQuant, OSTQuant)	Outlier removal; learned rotations better at the cost of optimisation [53, 123, 146, 173]
MXFP4 / NVFP4	MR-GPTQ, SOAR scale optimisation, 4/6 scaling	Format-specific scale handling [57, 110, 170]
2 to 3 bits weights	AQLM, QuIP#, QTIP, PV-tuning	Vector codes and fine-tuning [10, 132, 144, 149]
Post-trained model with RL / merges, NVFP4 target	Quantization-aware distillation	Avoids reproducing the multi-stage pipeline [133]
Mixed precision under a memory budget	EvoPress, HIGGS linearity-based allocation, GGUF k-quants	Principled per-layer allocation [49, 79, 194]
KV cache 2 to 4 bits	KIVI, KVQuant, rotation-based (RotateKV, OSCAR)	Per-channel keys and outlier handling [85, 87, 122, 157]

Chapter 20

Priority Assessment of Quantization Sub-Topics

Subjectivity statement

Every score, weight, rank and confidence label in this chapter is a **subjective judgement of the author**, informed by the evidence cited in earlier chapters. They are not measurements, not the output of a statistical model, and not the consensus of the community. Different readers with different workloads, hardware, risk tolerance or time horizons would legitimately score differently. The value of the exercise lies in making the criteria explicit so that a reader can substitute their own scores; Section 20.5 describes how.

20.1 What is being prioritised, and for whom

We rank fifteen sub-topics by *priority for a team that must run or improve LLM inference and training efficiency over the next 12 to 18 months (late 2026 to early 2028)*. "Priority" blends two questions: how much value does investing in this sub-topic deliver now, and how likely is it to remain relevant. The unit of assessment is the sub-topic as an engineering investment (for example, "adopt and validate W4A16 PTQ" or "build expertise in FP4 training"), not a single paper.

20.2 Rubric

Each sub-topic is scored from 1 (weak) to 5 (strong) on six criteria, with the following anchors. Scores are integers; half-points are not used so as not to suggest false precision.

The weights express the author's belief that practical impact and evidence maturity should dominate. Total score is $\sum_c w_c s_c$ and lies in $[1, 5]$.

20.3 Scored table

Justification of selected scores.

- *W4A16 weight-only PTQ* receives top marks on maturity and support: the GPTQ and AWQ papers date from 2022–2023, kernels reach close to the 4× bandwidth bound up to moderate batch sizes [16, 73, 100], and a large Llama-3.1 study finds INT4 weight-only competitive with 8-bit [20]. Its momentum score is only 3 because the technique is already standard.
- *FP8 W8A8 inference* is effectively lossless in the same study [20]; the support score is 4 rather than 5 because it requires Hopper-class or later hardware.

Table 20.1: Scoring rubric. Weights sum to 1. In all criteria a higher score is better for priority (for accuracy risk, 5 means *low* risk; for cost, 5 means *cheap* to adopt).

Criterion (code)	Weight	Score 5 means	Score 1 means
Evidence maturity (E)	0.20	Multiple independent large-scale evaluations or years of production use	Single-group, small-scale or unreplicated evidence
Practical impact (I)	0.25	Large reduction in memory, latency or cost for a common workload	Narrow or marginal benefit
Hardware and framework support (S)	0.15	Supported in mainstream hardware and serving stacks	Research code only; no native hardware support
Accuracy safety (A)	0.15	Typically near-lossless; failure modes well understood	Frequent large losses or poorly understood failures
Cost to adopt (C)	0.10	Hours of engineering; no training	Requires pretraining or custom kernels at scale
Momentum (M)	0.15	Rapidly growing adoption and research output in 2025 to 2026	Stagnant or narrowing

- *Evaluation protocols* score high on impact and cost because inexpensive additions (KL divergence, flip rates, long-context and multilingual tests) catch the failures documented in Chapter 18 [55, 96, 109].
- *Block-scaled FP4 inference* has the highest impact score (5) because it can raise compute throughput as well as lower memory, but modest maturity (3) and accuracy safety (3): MR-GPTQ reports that FP4 is not an automatic upgrade over INT4 and needs format-specific methods [110].
- *FP4 training* receives low evidence maturity (2) and accuracy safety (2): there is one large-scale report (12B parameters, 10T tokens) and several smaller-scale academic recipes [116, 147]; its momentum is nevertheless high.
- *Native ternary models* receive the lowest total: strong theoretical appeal but no frontier-scale evidence, no native hardware, and the need to pretrain from scratch [25, 26].
- *MoE-specific quantization* scores high on impact (4) and momentum (5) because frontier open models are MoE, but low on evidence (2): the literature is recent and mostly preprints [45, 104, 158].

20.4 Final ranking with confidence

To gauge how fragile the ranking is to the author's weights, we recompute it under four weightings: the base weights of Table 20.1; equal weights (1/6 each); an impact-heavy weighting $(E, I, S, A, C, M) = (0.15, 0.40, 0.10, 0.10, 0.10, 0.15)$; and a future-heavy weighting $(0.15, 0.15, 0.10, 0.10, 0.10, 0.40)$. The *confidence* label is assigned by a fixed rule: High if the maximum rank change across the four weightings is at most 2, Medium if it is 3 or 4, Low if it is 5 or more. This captures only sensitivity to *weights*; it does not capture error in the *scores* themselves, which is likely the larger uncertainty.

Table 20.2: Scores (1–5) on the six criteria of Table 20.1 and weighted total, sorted by total. Ties in total are marked "=". **All scores are the author's subjective judgements.**

Rank	Sub-topic	E	I	S	A	C	M	Total
1	W4A16 weight-only PTQ (AWQ, GPTQ)	5	5	5	4	5	3	4.55
2	FP8 W8A8 inference	5	4	4	5	4	3	4.20
3	Evaluation protocols for quantized models	3	4	4	5	4	4	3.95
4=	FP8 training	4	4	4	4	3	4	3.90
4=	Mixed-precision GEMM kernels and systems	4	4	3	5	3	4	3.90
6	Block-scaled FP4 inference (MXFP4, NVFP4)	3	5	3	3	3	5	3.80
7	QAT and quantization-aware distillation	3	4	3	4	2	5	3.60
8	Second-order PTQ refinements (GPTQ successors)	4	3	4	4	4	2	3.45
9	KV-cache quantization	3	4	3	3	3	4	3.40
10=	Rotation-based W4A4 (QuaRot, SpinQuant)	4	3	3	3	3	3	3.20
10=	MoE-specific quantization	2	4	2	3	3	5	3.20
12	Mixed-precision bit allocation	3	3	3	4	3	3	3.15
13	FP4 training	2	4	2	2	2	5	2.95
14	Vector and trellis sub-3-bit PTQ	3	3	2	3	2	3	2.75
15	Native ternary and binary models	2	2	1	2	1	3	1.90

20.4.1 Reading the ranking in tiers

Tier 1: adopt now (high confidence). W4A16 PTQ, FP8 W8A8, rigorous evaluation protocols, FP8 training and mixed-precision GEMM kernels. These have mature evidence and support; effort goes into engineering, validation and kernel integration rather than research. The ranking of FP8 inference is sensitive to the future-heavy weighting (it falls to seventh) simply because the momentum of newer topics rises, not because its value falls.

Tier 2: adopt selectively and validate heavily (medium confidence). Block-scaled FP4 inference, QAT and quantization-aware distillation, second-order PTQ refinements, KV-cache quantization. FP4 inference and QAT rise to the top three under the future-heavy weighting, signalling that they are where the field is moving; but accuracy safety is only 3 to 4, and the best recipes are format- and model-specific [110, 133]. KV quantization is a clear value proposition at long context and large batch, but long-context evidence is mixed [96].

Tier 3: invest if the use case demands (mixed confidence). Rotation-based W4A4, MoE-specific quantization, mixed-precision allocation, FP4 training. Rotation methods are the right tool when W4A4 on INT hardware is required, but block-scaled FP formats partially reduce their necessity [110, 146, 173]; MoE-specific work is high-momentum but young; FP4 training is high-risk and high-reward and mainly relevant to organisations that pretrain.

Table 20.3: Final ranking. "Ranks under alternatives" lists the rank under base, equal, impact-heavy and future-heavy weightings in that order. Spread is the maximum minus the minimum rank. Confidence follows the rule in the text. **Subjective.**

Rank	Sub-topic	Total	Ranks under alternatives	Spread	Confidence
1	W4A16 weight-only PTQ (AWQ, GPTQ)	4.55	1, 1, 1, 1	0	High
2	FP8 W8A8 inference	4.20	2, 2, 2, 7	5	Low
3	Evaluation protocols for quantized models	3.95	3, 3, 4, 3	1	High
4=	FP8 training	3.90	4, 4, 5, 5	1	High
4=	Mixed-precision GEMM kernels and systems	3.90	4, 4, 5, 5	1	High
6	Block-scaled FP4 inference (MXFP4, NVFP4)	3.80	6, 6, 2, 1	5	Low
7	QAT and quantization-aware distillation	3.60	7, 7, 7, 3	4	Medium
8	Second-order PTQ refinements (GPTQ successors)	3.45	8, 7, 10, 13	6	Low
9	KV-cache quantization	3.40	9, 9, 8, 9	1	High
10=	Rotation-based W4A4 (QuaRot, SpinQuant)	3.20	10, 10, 12, 11	2	High
10=	MoE-specific quantization	3.20	10, 10, 9, 8	2	High
12	Mixed-precision bit allocation	3.15	12, 10, 13, 12	3	Medium
13	FP4 training	2.95	13, 13, 11, 10	3	Medium
14	Vector and trellis sub-3-bit PTQ	2.75	14, 14, 14, 14	0	High
15	Native ternary and binary models	1.90	15, 15, 15, 15	0	High

Tier 4: monitor (high confidence). Vector and trellis sub-3-bit PTQ and native ternary or binary models. They deliver strong results in their niche (2–3 bit weights; maximum theoretical efficiency) [10, 25, 144], but kernel and hardware support, or the requirement to pretrain, restrict adoption.

20.5 Stability and what would change the ranking

- **FP4 hardware generalisation.** If NVFP4 and MXFP4 W4A4 recipes are shown to match FP8 accuracy across model families with mature open kernels, block-scaled FP4 inference would move into Tier 1 (accuracy safety and support scores up by one each would raise its total by 0.30 under the base weights).
- **Open evidence on FP4 pretraining at larger scale.** Independent replication at scale would raise the evidence score of FP4 training from 2 and could lift it by several ranks; a failure report would lower it.
- **Reasoning and agentic workloads.** If long-horizon agent tasks are found to be much more sensitive to quantization than current reasoning benchmarks suggest, evaluation protocols and 8-bit-first policies would gain priority and low-bit methods lose.
- **Custom hardware.** Native ternary or very-low-bit hardware would raise the support score of ternary models from 1 and change its rank.

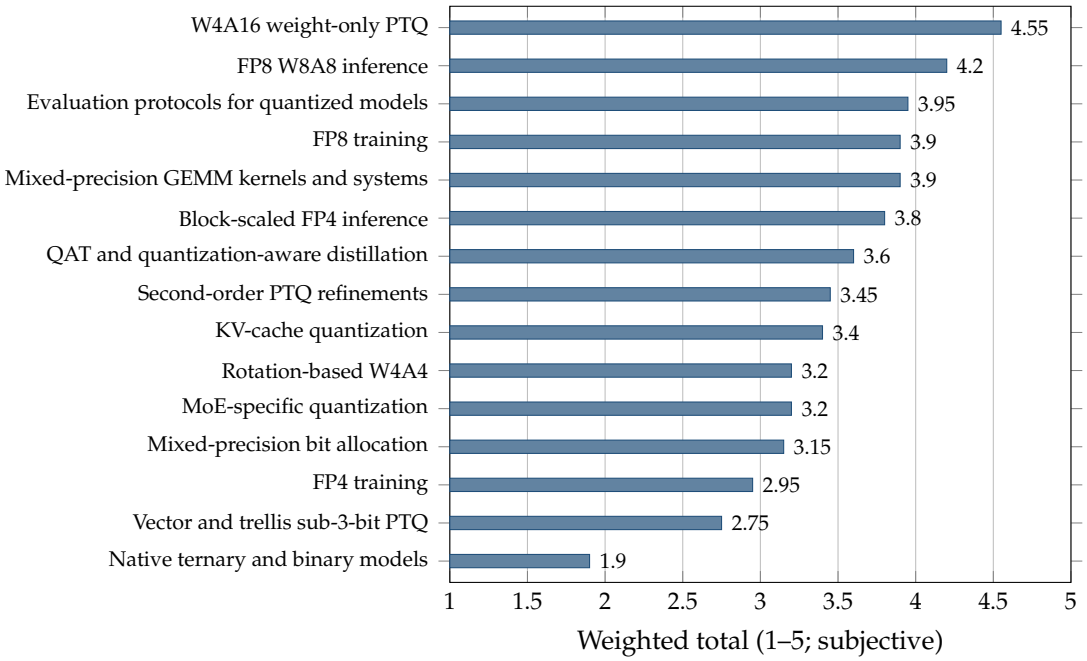


Figure 20.1: Weighted totals from Table 20.2. The scores are subjective judgements of the author; the plot only visualises them.

How to re-score for your context. Copy Table 20.2, change the six scores for each row to reflect your workload (for example, set the support score of FP4 to 1 if you have no Blackwell hardware, or the accuracy-safety score of everything below 8 bits to 1 for a safety-critical application), adjust the weights in Table 20.1, recompute $\sum_c w_c s_c$, and re-rank. The structure of the rubric, not the numerical outcome, is the transferable part of this chapter.

20.6 Limitations of the assessment

1. Integer scores on six criteria compress rich information; a score difference of one point is within plausible disagreement between informed reviewers.
2. The criteria are not independent (maturity correlates with support and momentum is partly anti-correlated with maturity), so the weighted sum double counts some effects.
3. Sub-topics overlap (for example, FP4 inference and rotation-based W4A4), so the ranking is a ranking of investment *areas*, not of mutually exclusive choices.
4. The assessment is made as of late October 2026 on the basis of public literature; unpublished industrial practice may differ.

Chapter 21

Open Problems and Conclusions

21.1 Open problems

The problems below are those that, in the author's judgement, most limit practice or understanding as of October 2026. Each is stated with the evidence that makes it open, a precise formulation where possible, and an indication of what a solution would look like. Statements labelled *conjecture* are the author's speculation, not results.

21.1.1 A theory of quantization error for end-to-end behaviour

Layerwise theory is strong: for a linear layer the proxy loss $\mathrm{tr}(\Delta \boldsymbol{W}\, \boldsymbol{H}\, \Delta \boldsymbol{W}^\top)$ is exact to second order (Chapter 5), and the high-resolution distortion of uniform quantizers is classical (Chapter 3). What is missing is a predictive theory of *downstream* behaviour. Empirically, the same average perplexity increase can correspond to very different flip rates, safety shifts, long-context losses or reasoning failures [41, 55, 96, 141]. A satisfactory theory would map a per-layer error budget and a task class to a predicted change in task metric, with calibrated uncertainty. Linearity results such as the HIGGS theorem (error adds across layers in a perplexity sense) are a step in this direction [79], but they are validated mostly for perplexity-like objectives and moderate perturbations.

21.1.2 Why do some models quantize better than others?

Llama-3 was reported to be more sensitive to quantization than Llama-2, an observation commonly attributed to longer training and denser information per weight [90, 179]; the undertrained-model analysis suggests that quantization degradation increases with the amount of training data per parameter [179]. A mechanism that predicts, from a checkpoint's weight and activation statistics alone, the expected degradation at a given format would let practitioners choose bit-widths without full evaluation. *Conjecture:* sensitivity is governed by a small number of spectral and outlier statistics of the layer Hessians that can be estimated from a few hundred tokens.

21.1.3 Robust 4-bit activation quantization without training

W4A4 is feasible with rotations and block-scaled formats, but the gap to FP8 is model-dependent and the best recipes are format-specific [110, 146, 173]. MR-GPTQ shows that NVFP4's small group size provably neutralizes traditional outlier-mitigation techniques, and that power-of-two scales hurt MXFP4 [110]. Open questions: (i) the optimal transform for each format and block size (random vs. learned, block vs. global); (ii) whether the INT4 versus FP4 conclusions [81] hold across model families and with optimal scale search [170]; (iii)

activation-side quantization for MoE experts with few tokens per expert.

21.1.4 Sub-3-bit weights at scale with fast kernels

Vector and trellis codes approach the rate–distortion bound at 2 to 3 bits (Chapter 8) but have slow encoding and narrow kernel support compared with scalar formats. Open: codes that combine trellis-level quality with kernels that stay bandwidth-bound at batch sizes of 16 to 128, plus encoding procedures whose cost is comparable to GPTQ. A related theoretical question is how close any *practical* decoder can get to the Gaussian rate–distortion function at 2 bits with a decoder cost of a few operations per weight.

21.1.5 Quantization-aware training for modern post-training pipelines

Frontier models are produced by multi-stage pipelines (SFT, RL, merging), and standard QAT is hard to engineer and less stable in such pipelines [133]. QAD recovers NVFP4 accuracy by distillation from a BF16 teacher, with near-BF16 results reported on five models [133]. Open: (i) quantization-aware *reinforcement learning* (should the policy gradient see the quantized policy?); (ii) how QAD compares with PTQ plus light fine-tuning at equal compute; (iii) how much data is enough, and whether synthetic data suffices; (iv) theoretical understanding of when the straight-through estimator misleads.

21.1.6 Scaling laws for quantization, including MoE and training-time formats

Precision scaling laws exist for pretraining and for QAT [83, 129, 136, 162], with notable conclusions: training larger models in lower precision may be compute-optimal, additional pretraining data can hurt post-training quantization, and the learning transition for QAT lies between 2 and 3 bits. Yet scaling laws for MoE models, for different formats (INT versus FP, block sizes), for KV-cache precision, and for reasoning performance are essentially absent. *Conjecture:* for MoE models, the effective memory-optimal bit-width for rarely activated experts is lower than for attention and shared experts, with the gap growing with sparsity; [104, 107] offer partial evidence.

21.1.7 Long-context and agentic robustness

The long-context study finds 8-bit nearly lossless and 4-bit sometimes catastrophic depending on method and model [96]. Agentic workloads (tool use, multi-turn, very long trajectories) have received little systematic study. Open: benchmarks that expose accumulation of small errors over long horizons, and quantizers that explicitly bound the *drift* of attention patterns (for example by protecting attention sinks and heavy-hitter tokens [101, 131, 165]).

21.1.8 Safety, security and verification

Quantization can change safety behaviour in ways that benign benchmarks miss [19, 41], and the many-to-one nature of rounding admits quantization-conditioned attacks [50]. Open: (i) defences that make the full-precision model's behaviour robust across the set of weights that round to the same quantized model; (ii) cheap certification that a quantized checkpoint's

behaviour on a safety suite matches the parent's within a tolerance; (iii) provenance and attestation of quantization pipelines.

21.1.9 Evaluation science

There is no agreed protocol for reporting quantization results. Recommended elements (Chapter 13) include KL divergence on held-out text, flip rate, task suites in several languages and context lengths, and free-form generation evaluation [48, 55, 109]. Open: a standardised, cheap, hard-to-game benchmark suite and reporting template with variance across calibration seeds [36]; statistics for detecting small behavioural shifts with limited evaluation budgets.

21.1.10 Hardware–algorithm co-design and portability

Format-specific recipes mean that quantization is increasingly co-designed with a hardware generation, raising portability and lock-in risk (Chapter 18). Open: format-agnostic quantizers that adapt to the target element format and block structure automatically, and open kernel libraries that cover the full space of formats and batch regimes on several vendors' hardware. Announced hardware claims (for example for the Rubin generation [193]) need independent measurement once available.

21.1.11 Low-precision training at frontier scale

FP8 training is established at 671B total parameters [43]; NVFP4 pretraining evidence includes a 12B model on 10T tokens [116], and DeepSeek V4 reportedly trains MoE experts in FP4 (secondary report [205]). Open: stability and quality of FP4 training at trillion-parameter, multi-trillion-token scale; whether unbiased gradient estimators with stochastic rounding [111, 147] are necessary or merely convenient; the best handling of the optimizer state and the master weights; and the interaction with reinforcement learning, where policy-gradient noise may compound with rounding noise.

21.1.12 Quantization together with other compression

Quantization composes with pruning/sparsity, low-rank approximation, distillation and KV-cache eviction (treated in companion papers). The interactions are poorly understood: errors from multiple compressions are not independent, and the ordering of operations matters. Open: a principled joint allocation of a total error budget across compression types, extending the bit-allocation theory of Chapter 9.

21.2 Conclusions

What is settled. At 8 bits (FP8, or well-tuned INT8) large models are effectively lossless, and the tools are widely supported [20, 169]. At 4 bits for weights only, post-training methods (GPTQ, AWQ and their descendants) combined with mature kernels deliver most of the theoretical bandwidth gain with small accuracy loss for large dense models [16, 20, 73, 100].

The rate–distortion view explains why: each bit buys about 6.02 dB of signal-to-quantization-noise ratio, an 8-bit model is far above the noise floor that matters, and the knee of the accuracy curve lies near 3 to 4 bits for current models.

What is progressing. Four-bit *activation* quantization depends on removing outliers by rotation or on block-scaled floating-point formats whose hardware support has arrived on recent accelerators [110, 116, 146, 173]. Sub-3-bit weights are feasible with vector and trellis codes or QAT but with higher engineering cost [10, 46, 144, 149]. KV-cache quantization to 2 to 4 bits is effective at short and moderate context [85, 87]. Quantization-aware training and distillation are becoming the means by which developers ship natively low-precision models [72, 133, 192, 201].

What is uncertain. Evidence on FP4 pretraining at frontier scale, on quantization of long-horizon agentic workloads, on robustness of MoE routing, and on safety shifts at the lowest bit-widths is thin or recent [41, 116, 158]. Vendor roadmaps are claims rather than measurements [193].

Practical message. (1) Decide by the binding resource: capacity, bandwidth, compute or KV memory (Chapter 19). (2) Begin with the conservative choice (FP8 or W4A16) and move to more aggressive formats only with task-level evidence. (3) Evaluate the deployed artifact on distributional metrics, not on average accuracy alone. (4) Record the complete quantization configuration for reproducibility. (5) Treat format-specific recipes as tied to hardware generations.

A note on this document's own evidence. Throughout, numbers from the literature are quoted as reported by the cited authors, and those numbers come from abstracts or model cards of arXiv preprints, vendor blogs and news reports, which were not independently reproduced; items labelled "own simulation" use synthetic data and demonstrate mechanisms rather than model quality; and all scores and rankings in Chapter 20 are subjective. Readers who intend to act on a specific number should consult the primary source and, ideally, replicate on their own model and task.

Appendix A

Notation

Table A.1: Symbols used throughout. Where a symbol has context-dependent meaning, the context is given.

Symbol	Meaning
$\boldsymbol{W} \in \mathbb{R}^{m \times n}$	Weight matrix of a linear layer (output dimension m, input dimension n); $\boldsymbol{W}_{i,:}$ is row i
$\boldsymbol{X} \in \mathbb{R}^{n \times T}$	Calibration activations (columns are tokens); $\boldsymbol{H} = \boldsymbol{X}\boldsymbol{X}^\top$ (sometimes $2\boldsymbol{X}\boldsymbol{X}^\top$) is the layer-wise Hessian of the proxy loss
$\boldsymbol{Y}$	Layer output $\boldsymbol{Y} = \boldsymbol{W}\boldsymbol{X}$
$\widehat{\boldsymbol{W}}$, $\Delta\boldsymbol{W}$	Quantized weights, $\Delta\boldsymbol{W} = \widehat{\boldsymbol{W}} - \boldsymbol{W}$
$\mathcal{Q}(\cdot)$	Quantization operator (round-to-grid, including scale)
b, B	Bit-width of an element; number of grid points 2^b
bpw	Effective bits per weight including scales and zero-points
s, z	Scale and zero-point of an affine quantizer, $\mathcal{Q}(x) = s\big(\text{clip}(\text{round}(x/s) + z, 0, 2^b - 1) - z\big)$
Δ	Step size of a uniform quantizer (equal to s for a symmetric uniform grid)
g	Group (block) size sharing a scale
α	SmoothQuant migration strength (Chapter 6); clip ratio in clipping analyses
$\boldsymbol{s}$ (bold)	Per-channel smoothing or AWQ scale vector
$\boldsymbol{R}$, $\boldsymbol{H}$	Orthogonal rotation matrix; (normalised) Hadamard matrix, $\boldsymbol{H}\boldsymbol{H}^\top = \boldsymbol{I}$
σ^2	Variance of a source; $D(R) = \sigma^2 2^{-2R}$ for a Gaussian source at rate R bits
D, R	Distortion (mean squared error) and rate (bits per sample)
SQNR	Signal-to-quantization-noise ratio $= 10\log_{10}(\sigma^2/D)$ in dB
$\text{KL}(p\|q)$	Kullback–Leibler divergence; used between full-precision and quantized next-token distributions
bpw_{eff}	Same as bpw when emphasis on effective rate is needed
E, k	Number of experts; number of experts activated per token (MoE)
B (context)	Batch size in serving analyses (distinct from the grid size above)
$n_{\text{layers}}, n_{\text{kv}}, d_h$	Number of layers, KV heads, and head dimension
AI	Arithmetic intensity (FLOPs per byte moved)
$\mathbb{E}[\cdot]$	Expectation; $\|\cdot\|$ Euclidean or Frobenius norm as context requires
tr, diag	Trace; diagonal matrix or vector of diagonal entries
Abbreviations for format names	
INTb	Signed or unsigned b-bit integer format with a scale
EaMm	Floating-point element with a exponent and m mantissa bits (E4M3, E5M2, E2M1)
MXFPb	OCP microscaling floating point, block of 32 with an E8M0 (power-of-two) shared scale
NVFP4	4-bit E2M1 elements, blocks of 16, E4M3 block scale, plus a tensor-level FP32 scale
WpAqKVr	Weights at p bits, activations at q bits, KV cache at r bits (A16 = 16-bit activations)

Appendix B

Supplementary Derivations

This appendix collects derivations that are used in the main text but would interrupt its flow, together with a few elementary results that are easy to state wrongly. All proofs are elementary and are the author's own derivations unless a citation is given.

B.1 Stochastic rounding: unbiasedness and variance

Let $\Delta > 0$ be the grid step and $x = k\Delta + r$ with $k \in \mathbb{Z}$ and $r \in [0, \Delta)$. Stochastic rounding returns $(k+1)\Delta$ with probability $p = r/\Delta$ and $k\Delta$ with probability $1 - p$.

Proposition B.1. *Stochastic rounding is unbiased,* $\mathbb{E}[\mathrm{SR}(x)] = x$*, with conditional variance* $\Delta^2 p(1-p) \le \Delta^2/4$*. If the fractional position* p *is uniform on* $[0,1]$*, the average variance is* $\Delta^2/6$*, which is twice the mean-squared error* $\Delta^2/12$ *of round-to-nearest.*

Proof. $\mathbb{E}[\mathrm{SR}(x)] = (1-p)k\Delta + p(k+1)\Delta = k\Delta + p\Delta = k\Delta + r = x$. The two-point variance is $\Delta^2 p(1-p)$, which is maximised at $p = 1/2$ with value $\Delta^2/4$. Averaging over uniform p gives $\Delta^2 \int_0^1 p(1-p)\,dp = \Delta^2/6$. For round-to-nearest the error is $\min(r, \Delta - r)$ which is uniform on $[0, \Delta/2]$ in magnitude, so its mean square is $\frac{2}{\Delta}\int_0^{\Delta/2} u^2 du = \Delta^2/12$. □

Remark B.1. Proposition B.1 explains the usage pattern in low-precision training: stochastic rounding is applied to *gradients*, where unbiasedness matters because errors accumulate over many steps, while round-to-nearest is used for *forward* operands, where lower variance matters more [111, 116, 147]. The cost of unbiasedness is a factor of two in mean-squared error (3 dB) for uniformly distributed fractional parts.

B.2 Optimal scale for fixed ternary pattern

Let $\boldsymbol{w} \in \mathbb{R}^n$ and $\boldsymbol{t} \in \{-1, 0, 1\}^n$. Minimising $\|\boldsymbol{w} - \alpha \boldsymbol{t}\|^2$ over $\alpha \in \mathbb{R}$ gives $\alpha^\star = \langle \boldsymbol{w}, \boldsymbol{t}\rangle / \langle \boldsymbol{t}, \boldsymbol{t}\rangle$. When $t_i = \mathrm{sign}(w_i)$ on the support $S = \{i : t_i \neq 0\}$, $\alpha^\star = |S|^{-1} \sum_{i \in S} |w_i|$, the mean magnitude of the retained entries, and the residual is $\sum_{i \in S}(|w_i| - \alpha^\star)^2 + \sum_{i \notin S} w_i^2$.

Proof. The objective is a convex quadratic in α; setting the derivative $-2\langle \boldsymbol{w} - \alpha \boldsymbol{t}, \boldsymbol{t}\rangle$ to zero gives the stated $\alpha^\star$. Substituting $t_i = \mathrm{sign}(w_i)$ on S gives $\langle \boldsymbol{w}, \boldsymbol{t}\rangle = \sum_S |w_i|$ and $\langle \boldsymbol{t}, \boldsymbol{t}\rangle = |S|$. □

The "absmean" quantizer of BitNet b1.58 takes $\gamma = \frac{1}{mn}\sum_{ij} |W_{ij}|$ and sets $\widehat{W}_{ij} = \gamma\, \mathrm{clip}(\mathrm{round}(W_{ij}/\gamma), -1, 1)$ [25]. It is the same construction with the support chosen by the threshold $|W_{ij}| > \gamma/2$ and the scale set to the mean over *all* entries instead of over the support; the latter would be the least-squares optimum for the chosen support, so absmean is a cheap, slightly suboptimal approximation (author's observation).

B.3 Effective bits per weight of common formats

For a block of g weights with b bits per element and a scale of b_s bits (and optionally a zero-point of b_z bits), the effective rate is

$$\text{bpw} = b + \frac{b_s + b_z}{g}.$$

For two-level formats (super-block of G elements containing G/g sub-blocks) the rate is $b + \left[(G/g)(b'_s + b'_z) + b_S + b_Z\right]/G$ where primes denote sub-block scale and offset bits and capital letters the super-block ones.

Table B.1: Effective bits per weight computed from the formula (analytic). GGUF layouts are as the author understands the public llama.cpp block structures [191, 194]; if a layout has changed, the number changes accordingly. The tensor-level scale of NVFP4 contributes a negligible amount.

Format	Layout	bpw
INT4, $g = 128$, FP16 scale	$4 + 16/128$	4.125
INT4, $g = 128$, FP16 scale and zero	$4 + 32/128$	4.25
INT4, $g = 64$, FP16 scale	$4 + 16/64$	4.25
MXFP4	$g = 32$, E8M0 scale: $4 + 8/32$	4.25
NVFP4	$g = 16$, E4M3 scale: $4 + 8/16$	4.5
MXFP8	$g = 32$, E8M0 scale: $8 + 8/32$	8.25
GGUF Q4_0	$g = 32$, FP16 scale: $4 + 16/32$	4.5
GGUF Q8_0	$g = 32$, FP16 scale: $8 + 16/32$	8.5
GGUF Q4_K	$G = 256$, 8 sub-blocks of 32 with 6-bit scale and 6-bit min, FP16 super-scale and super-min: $(1024 + 96 + 32)/256$	4.5
GGUF Q2_K	$G = 256$, 16 sub-blocks of 16 with 4-bit scale and 4-bit min, FP16 super-scale and super-min: $(512 + 128 + 32)/256$	2.625

Sanity check of the Q2_K entry. 256 elements at 2 bits give 512 bits; 16 sub-blocks each with an 8-bit pair give 128 bits; two FP16 super-block parameters give 32 bits; the total is 672 bits per 256 weights, i.e. 2.625 bits per weight.

B.4 Scale rounding in two-level (NVFP4-style) scaling

In NVFP4 each block of 16 E2M1 elements carries a scale stored in E4M3, and a per-tensor FP32 scale. Let $s = \max_i |x_i|/6$ be the ideal block scale (6 is the largest E2M1 magnitude). If the stored scale is $s_q = s(1 + \delta)$ then:

Proposition B.2. *For an E4M3 scale in the normal range, $|\delta| \le 2^{-4} = 0.0625$ under round-to-nearest; the block maximum, after division by the stored scale, satisfies $|x|_{\max}/s_q = 6/(1 + \delta) \in [5.647, 6.4]$.*

Proof. E4M3 has 3 stored mantissa bits, so within a binade $[2^e, 2^{e+1})$ the spacing is 2^{e-3} and the rounding error is at most 2^{e-4}; relative to a value $v \ge 2^e$ this is at most 2^{-4}. Hence $|\delta| \le 2^{-4}$. Then $6/(1 + 2^{-4}) = 6/1.0625 \approx 5.647$ and $6/(1 - 2^{-4}) = 6/0.9375 = 6.4$. □

When the normalised maximum exceeds 6 the element is clipped to 6, incurring an error of up to 0.4 in normalised units (6.25% of 6); when it falls below 6 the top of the grid is underused. This quantifies the type of error that scale-optimisation approaches target by choosing scales that minimise reconstruction error rather than matching the maximum [57, 170]. The E8M0 scale of MXFP4 is a power of two: the ratio between the ideal and the stored scale can be as large as $\sqrt{2}$ in either direction in log-space (or up to a factor 2 under floor rounding), which is a much coarser scale grid and is consistent with MR-GPTQ's observation that power-of-two scales induce high error [110].

B.5 E2M1 grid and rounding error

The E2M1 element has one sign bit, two exponent bits and one mantissa bit. Its non-negative values are $\{0, 0.5, 1, 1.5, 2, 3, 4, 6\}$, with consecutive spacings $0.5, 0.5, 0.5, 0.5, 1, 1, 2$. Under round-to-nearest the maximum absolute error is half the local spacing: 0.25 for values below 2, 0.5 on $[2, 4]$ and 1 on $[4, 6]$. The worst relative error occurs at the midpoint of each cell, where it equals half the spacing divided by the midpoint: 33% on $[0.5, 1]$, 20% on $[1, 1.5]$, 14% on $[1.5, 2]$, 20% on $[2, 3]$, 14% on $[3, 4]$ and 20% on $[4, 6]$. The *absolute* error near the block maximum is thus four times larger than near 1, so the grid is coarsest exactly where the largest (often most important) elements live. The "Four Over Six" proposal chooses between a scale that maps the block maximum to 6 or to 4 to reduce this effect [57].

B.6 Hadamard transforms: orthogonality and cost

The Sylvester construction defines $\boldsymbol{H}_1 = [1]$ and $\boldsymbol{H}_{2n} = \begin{pmatrix} \boldsymbol{H}_n & \boldsymbol{H}_n \\ \boldsymbol{H}_n & -\boldsymbol{H}_n \end{pmatrix}$ (unnormalised).

Proposition B.3. *$\boldsymbol{H}_n \boldsymbol{H}_n^\top = n\boldsymbol{I}_n$ for $n = 2^k$, hence $n^{-1/2}\boldsymbol{H}_n$ is orthogonal; the product $\boldsymbol{H}_n \boldsymbol{v}$ can be computed with $n \log_2 n$ additions and subtractions.*

Proof. By induction: $\boldsymbol{H}_1 \boldsymbol{H}_1^\top = 1$. If $\boldsymbol{H}_n \boldsymbol{H}_n^\top = n\boldsymbol{I}$ then $\boldsymbol{H}_{2n} \boldsymbol{H}_{2n}^\top = \begin{pmatrix} 2\boldsymbol{H}_n \boldsymbol{H}_n^\top & 0 \\ 0 & 2\boldsymbol{H}_n \boldsymbol{H}_n^\top \end{pmatrix} = 2n\boldsymbol{I}_{2n}$. For the cost, write $\boldsymbol{v} = (\boldsymbol{v}_1, \boldsymbol{v}_2)$; then $\boldsymbol{H}_{2n}\boldsymbol{v} = (\boldsymbol{H}_n(\boldsymbol{v}_1 + \boldsymbol{v}_2), \boldsymbol{H}_n(\boldsymbol{v}_1 - \boldsymbol{v}_2))$, requiring $2n$ additions plus two half-size transforms, so $T(2n) = 2T(n) + 2n$ and $T(n) = n \log_2 n$. □

Multiplying by a random diagonal sign matrix $\boldsymbol{D}$ yields the randomized Hadamard transform $n^{-1/2}\boldsymbol{H}_n \boldsymbol{D}$, which is orthogonal, cheap, and (by the argument of Chapter 7) spreads the energy of any fixed vector approximately uniformly over coordinates [146, 149, 150].

B.7 Error propagation through a quantized linear layer

Consider $\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x}$ with quantized weights $\boldsymbol{W} + \Delta\boldsymbol{W}$ and quantized input $\boldsymbol{x} + \Delta\boldsymbol{x}$. The output error is

$$\Delta\boldsymbol{y} = \Delta\boldsymbol{W}\,\boldsymbol{x} + \boldsymbol{W}\,\Delta\boldsymbol{x} + \Delta\boldsymbol{W}\,\Delta\boldsymbol{x}.$$

Suppose the entries of $\Delta \boldsymbol{W}$ and $\Delta \boldsymbol{x}$ are independent, zero-mean, with variances σ_w^2 and σ_x^2 and independent of each other. Then

$$\mathbb{E}\,\|\Delta \boldsymbol{W}\boldsymbol{x}\|^2 = m\sigma_w^2\,\|\boldsymbol{x}\|^2, \quad \mathbb{E}\,\|\boldsymbol{W}\Delta \boldsymbol{x}\|^2 = \sigma_x^2\,\|\boldsymbol{W}\|_F^2, \quad \mathbb{E}\,\|\Delta \boldsymbol{W}\Delta \boldsymbol{x}\|^2 = mn\,\sigma_w^2\sigma_x^2,$$

and the cross terms between the three vanish in expectation because each pair contains an independent zero-mean factor (the cross term of the first two is $\mathbb{E}\langle \Delta \boldsymbol{W}\boldsymbol{x}, \boldsymbol{W}\Delta \boldsymbol{x}\rangle = 0$ since $\Delta \boldsymbol{W}$ and $\Delta \boldsymbol{x}$ are independent and zero-mean; for the third term the cross-terms with the other two contain an odd count of zero-mean factors in at least one of the independent quantities).

Proof. Each identity follows from $\mathbb{E}\,\|\boldsymbol{A}\boldsymbol{z}\|^2 = \sigma^2\,\|\boldsymbol{z}\|^2 \cdot \#\text{rows}$ for a matrix $\boldsymbol{A}$ with i.i.d. zero-mean entries of variance σ^2 and a fixed vector $\boldsymbol{z}$, and from the symmetric statement for $\boldsymbol{W}\Delta \boldsymbol{x}$ with $\mathbb{E}\,\|\boldsymbol{W}\Delta \boldsymbol{x}\|^2 = \sigma_x^2 \sum_{ij} W_{ij}^2$. For the third term, $\mathbb{E}\,\|\Delta \boldsymbol{W}\Delta \boldsymbol{x}\|^2 = \sum_i \mathbb{E}(\sum_j \Delta W_{ij}\Delta x_j)^2 = m\,n\,\sigma_w^2\sigma_x^2$. □

The third (second-order) term has ratio $mn\sigma_w^2\sigma_x^2/(m\sigma_w^2\,\|\boldsymbol{x}\|^2) = n\sigma_x^2/\,\|\boldsymbol{x}\|^2 = 1/\mathrm{SQNR}_x$ relative to the first, so it is negligible when the activation SQNR is large and is the reason the harmonic composition of weight and activation SQNR of Chapter 13 is accurate to first order. The independence assumptions are idealisations: quantization errors of weights tuned by GPTQ are, by construction, correlated with the activations (that is the point of the method), so these formulas describe round-to-nearest, not error-compensated, quantization.

B.8 Lloyd–Max conditions

For a scalar source with density f and a quantizer with decision thresholds $t_0 < t_1 < \cdots < t_L$ and representatives $r_1, \ldots, r_L$ minimising $\sum_j \int_{t_{j-1}}^{t_j} (x - r_j)^2 f(x)\,dx$, the necessary conditions are that each threshold is the midpoint of adjacent representatives, $t_j = (r_j + r_{j+1})/2$, and each representative is the conditional mean (centroid) of its cell, $r_j = \mathbb{E}[X \mid t_{j-1} < X \le t_j]$ [196, 210]. These follow by setting derivatives with respect to t_j and r_j to zero. The conditions are not sufficient in general but characterise the optimum for log-concave densities (a result of Fleischer, which we quote without proof and have not re-verified). They are the basis of the non-uniform codebooks of NormalFloat and of sensitivity-weighted datatypes in LLM quantization [87, 138, 175].

Appendix C

Glossary

Activation-aware quantization Choosing scales or protecting channels using statistics of activations rather than weights alone (AWQ).

Affine quantizer Quantizer $x \mapsto s(\text{round}(x/s) + z)$ with scale s and zero-point z.

Arithmetic intensity FLOPs performed per byte moved from memory; compared with the hardware ridge point in the roofline model to decide whether an operation is memory- or compute-bound.

Attention sink Initial tokens that receive disproportionate attention mass and are associated with massive activations; relevant for KV quantization.

Block scaling Sharing one scale among a block of consecutive elements (for example 16 or 32), as in MX formats and NVFP4.

Calibration data Small unlabeled sample used to compute activation statistics or Hessians for post-training quantization.

Clipping Restricting the quantization range below the observed maximum to trade clipping error for finer resolution.

Codebook Finite set of representatives; vector quantization stores indices into a codebook.

Crest factor Ratio of the maximum absolute value to the root mean square of a block; high crest factors make uniform grids inefficient.

Dequantization Mapping stored low-precision values and scales back to higher precision for computation.

E4M3, E5M2 FP8 formats with 4 exponent and 3 mantissa bits, and 5 exponent and 2 mantissa bits.

E8M0 Eight-bit exponent-only scale format (power of two) used for MX shared scales.

Equivalent transformation A change of basis or scaling applied to weights and inverse-applied to activations such that the full-precision function is unchanged.

Fake quantization Simulating quantization in floating point (quantize then dequantize) during QAT or evaluation.

FP4 (E2M1) Four-bit floating-point element with grid $\{0, \pm 0.5, \pm 1, \pm 1.5, \pm 2, \pm 3, \pm 4, \pm 6\}$.

GGUF File format and quantization family of llama.cpp, including block-wise "k-quants".

Group size Number of weights sharing a scale along the input dimension.

Hadamard transform Orthogonal transform with ± 1 entries (normalised by $\sqrt{n}$) computable in $O(n \log n)$; used in incoherence processing.

Hessian (layer-wise)	$\boldsymbol{H} = \boldsymbol{X}\boldsymbol{X}^\top$ for the proxy loss $\left\|(\widehat{\boldsymbol{W}} - \boldsymbol{W})\boldsymbol{X}\right\|_F^2$.
Incoherence	Property that no weight entry or direction is much larger than typical, achieved by random rotations.
KV cache	Stored attention keys and values of past tokens, whose size grows with context length and batch.
KL divergence (as metric)	Divergence between the full-precision and quantized output distributions; more sensitive to behaviour change than accuracy.
Lloyd–Max quantizer	MSE-optimal scalar quantizer for a given source distribution.
Massive activation	Extremely large activation values at specific tokens and channels, associated with attention sinks.
Microscaling (MX)	OCP standard for block-scaled low-precision formats.
Mixed precision	Assigning different bit-widths to different layers, channels or tensors.
MoE	Mixture of Experts; sparse model in which a router selects a few experts per token.
NF4	NormalFloat 4-bit data type with quantiles of a normal distribution as levels (QLoRA).
Outlier	Channel or entry with magnitude far larger than typical; the principal difficulty for activation quantization.
Perplexity	Exponentiated average negative log-likelihood; standard but insufficient quality metric.
PTQ	Post-training quantization: quantizing a trained model without (or with minimal) training.
QAT	Quantization-aware training: training or fine-tuning with quantization in the loop.
QAD	Quantization-aware distillation: QAT with a KL loss against a full-precision teacher.
Rate–distortion function	Minimum rate for a given distortion; $D(R) = \sigma^2 2^{-2R}$ for a Gaussian source.
Roofline model	Performance model bounding throughput by the minimum of the compute peak and bandwidth times arithmetic intensity.
Round-to-nearest (RTN)	Baseline rounding to the closest grid point.
Smoothing	Migrating activation range into weights via per-channel scaling (SmoothQuant).
Stochastic rounding	Rounding up or down with probability proportional to proximity, giving an unbiased estimate.
STE	Straight-through estimator: gradient of rounding approximated as identity.
Ternary	Weights restricted to $\{-1, 0, +1\}$ (1.58 bits per weight when entropy-coded).
Trellis-coded quantization	Quantizer in which a stateful decoder steps through a trellis, allowing high effective dimension at fixed rate.
Vector quantization	Quantizing groups of weights jointly using a codebook.
WpAq	Notation for weights at p bits and activations at q bits.

Zero-point Integer offset aligning zero in the real domain with an integer grid point.

Bibliography

[1] A. Gersho and R. M. Gray. *Vector Quantization and Signal Compression*. Kluwer Academic, 1992.

[2] B. Hassibi and D. G. Stork. Second order derivatives for network pruning: Optimal Brain Surgeon. In *Advances in Neural Information Processing Systems 5*, 1993.

[3] B. Widrow and I. Kollár. *Quantization Noise: Roundoff Error in Digital Computation, Signal Processing, Control, and Communications*. Cambridge University Press, 2008.

[4] Ron Banner et al.. Post-training 4-bit quantization of convolution networks for rapid-deployment. *arXiv:1810.05723*, 2018.

[5] Itay Hubara et al.. Improving Post Training Neural Quantization: Layer-wise Calibration and Integer Programming. *arXiv:2006.10518*, 2020.

[6] Markus Nagel et al.. Up or Down? Adaptive Rounding for Post-Training Quantization. *arXiv:2004.10568*, 2020.

[7] Yuexiao Ma et al.. AffineQuant: Affine Transformation Quantization for Large Language Models. *arXiv:2403.12544*, 2024.

[8] Wanqi Yang et al.. AlphaQ: Calibration-Free Bit Allocation for Mixture-of-Experts Quantization. *arXiv:2606.04980*, 2026.

[9] Yeonhong Park et al.. Any-Precision LLM: Low-Cost Deployment of Multiple, Different-Sized LLMs. *arXiv:2402.10517*, 2024.

[10] Vage Egiazarian et al.. Extreme Compression of Large Language Models via Additive Quantization. *arXiv:2401.06118*, 2024.

[11] Alina Shutova et al.. Cache Me If You Must: Adaptive Key-Value Quantization for Large Language Models. *arXiv:2501.19392*, 2025.

[12] Zhiteng Li et al.. ARB-LLM: Alternating Refined Binarizations for Large Language Models. *arXiv:2410.03129*, 2024.

[13] Haoqian Meng et al.. ARCQuant: Boosting NVFP4 Quantization with Augmented Residual Channels for LLMs. *arXiv:2601.07475*, 2026.

[14] Yilong Zhao et al.. Atom: Low-bit Quantization for Efficient and Accurate LLM Serving. *arXiv:2310.19102*, 2023.

[15] Wenhua Cheng et al.. Optimize Weight Rounding via Signed Gradient Descent for the Quantization of LLMs. *arXiv:2309.05516*, 2023.

[16] Ji Lin et al.. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. *arXiv:2306.00978*, 2023.

[17] Yoshua Bengio, Nicholas Léonard and Aaron Courville. Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation. *arXiv:1308.3432*, 2013.

[18] Olga Kovaleva et al.. BERT Busters: Outlier Dimensions that Disrupt Transformers. *arXiv:2105.06990*, 2021.

[19] Zhichao Xu et al.. Beyond Perplexity: Multi-dimensional Safety Evaluation of LLM Compression. *arXiv:2407.04965*, 2024.

[20] Eldar Kurtic et al.. "Give Me BF16 or Give Me Death"? Accuracy-Performance Trade-Offs in LLM Quantization. *arXiv:2411.02355*, 2024.

[21] Dhiraj Kalamkar et al.. A Study of BFLOAT16 for Deep Learning Training. *arXiv:1905.12322*, 2019.

[22] Wei Huang et al.. BiLLM: Pushing the Limit of Post-Training Quantization for LLMs. *arXiv:2402.04291*, 2024.

[23] Matthieu Courbariaux, Yoshua Bengio and Jean-Pierre David. BinaryConnect: Training Deep Neural Networks with binary weights during propagations. *arXiv:1511.00363*, 2015.

[24] Dayou Du et al.. BitDistiller: Unleashing the Potential of Sub-4-Bit LLMs via Self-Distillation. *arXiv:2402.10631*, 2024.

[25] Shuming Ma et al.. The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits. *arXiv:2402.17764*, 2024.

[26] Shuming Ma et al.. BitNet b1.58 2B4T Technical Report. *arXiv:2504.12285*, 2025.

[27] Hongyu Wang, Shuming Ma and Furu Wei. BitNet a4.8: 4-bit Activations for 1-bit LLMs. *arXiv:2411.04965*, 2024.

[28] Jinheng Wang et al.. 1-bit AI Infra: Part 1.1, Fast and Lossless BitNet b1.58 Inference on CPUs. *arXiv:2410.16144*, 2024.

[29] Jacob Nielsen and Peter Schneider-Kamp. BitNet b1.58 Reloaded: State-of-the-art Performance Also on Smaller Networks. *arXiv:2407.09527*, 2024.

[30] Hongyu Wang, Shuming Ma and Furu Wei. BitNet v2: Native 4-bit Activations with Hadamard Transformation for 1-bit LLMs. *arXiv:2504.18415*, 2025.

[31] Hongyu Wang et al.. BitNet: Scaling 1-bit Transformers for Large Language Models. *arXiv:2310.11453*, 2023.

[32] Xinghao Wang et al.. BitStack: Any-Size Compression of Large Language Models in Variable Memory Environments. *arXiv:2410.23918*, 2024.

[33] Matthieu Courbariaux et al.. Binarized Neural Networks: Training Deep Neural Networks with Weights and Activations Constrained to +1 or -1. *arXiv:1602.02830*, 2016.

[34] Yelysei Bondarenko, Markus Nagel and Tijmen Blankevoort. Understanding and Overcoming the Challenges of Efficient Transformer Quantization. *arXiv:2109.12948*, 2021.

[35] Yuhang Li et al.. BRECQ: Pushing the Limit of Post-Training Quantization by Block Reconstruction. *arXiv:2102.05426*, 2021.

[36] Miles Williams and Nikolaos Aletras. On the Impact of Calibration Data in Post-training Quantization and Pruning. *arXiv:2311.09755*, 2023.

[37] Jun Li, Li Fuxin and Sinisa Todorovic. Efficient Riemannian Optimization on the Stiefel Manifold via the Cayley Transform. *arXiv:2002.01113*, 2020.

[38] Haocheng Xi et al.. COAT: Compressing Optimizer states and Activation for Memory-Efficient FP8 Training. *arXiv:2410.19313*, 2024.

[39] Peiyu Liu et al.. Unlocking Data-free Low-bit Quantization with Matrix Decomposition for KV Cache Compression. *arXiv:2405.12591*, 2024.

[40] Xianglong Yan et al.. D^2Quant: Accurate Low-bit Post-Training Weight Quantization for LLMs. *arXiv:2602.02546*, 2026.

[41] Junyuan Hong et al.. Decoding Compressed Trust: Scrutinizing the Trustworthiness of Efficient LLMs Under Compression. *arXiv:2403.15447*, 2024.

[42] Shuchang Zhou et al.. DoReFa-Net: Training Low Bitwidth Convolutional Neural Networks with Low Bitwidth Gradients. *arXiv:1606.06160*, 2016.

[43] DeepSeek-AI et al.. DeepSeek-V3 Technical Report. *arXiv:2412.19437*, 2024.

[44] Haokun Lin et al.. DuQuant: Distributing Outliers via Dual Transformation Makes Stronger Quantized LLMs. *arXiv:2406.01721*, 2024.

[45] Zhongqian Fu et al.. EAQuant: Enhancing Post-Training Quantization for MoE Models via Expert-Aware Optimization. *arXiv:2506.13329*, 2025.

[46] Mengzhao Chen et al.. EfficientQAT: Efficient Quantization-Aware Training for Large Language Models. *arXiv:2407.11062*, 2024.

[47] Reiner Pope et al.. Efficiently Scaling Transformer Inference. *arXiv:2211.05102*, 2022.

[48] Shiyao Li et al.. Evaluating Quantized Large Language Models. *arXiv:2402.18158*, 2024.

[49] Oliver Sieberling et al.. EvoPress: Accurate Dynamic Model Compression via Evolutionary Search. *arXiv:2410.14649*, 2024.

[50] Kazuki Egashira et al.. Exploiting LLM Quantization. *arXiv:2405.18137*, 2024.

[51] Jay Shah et al.. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision. *arXiv:2407.08608*, 2024.

[52] Liqun Ma, Mingjie Sun and Zhiqiang Shen. FBI-LLM: Scaling Up Fully Binarized LLMs from Scratch via Autoregressive Distillation. *arXiv:2407.07093*, 2024.

[53] Yuxuan Sun et al.. FlatQuant: Flatness Matters for LLM Quantization. *arXiv:2410.09426*, 2024.

[54] Ying Sheng et al.. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. *arXiv:2303.06865*, 2023.

[55] Abhinav Dutta et al.. Accuracy is Not All You Need. *arXiv:2407.09141*, 2024.

[56] Han Guo et al.. Fast Matrix Multiplications for Lookup Table-Quantized LLMs. *arXiv:2407.10960*, 2024.

[57] Jack Cook et al.. Four Over Six: More Accurate NVFP4 Quantization with Adaptive Block Scaling. *arXiv:2512.02010*, 2025.

[58] Brian Chmiel et al.. FP4 All the Way: Fully Quantized Training of LLMs. *arXiv:2505.19115*, 2025.

[59] Ruizhe Wang et al.. Optimizing Large Language Model Training Using FP4 Quantization. *arXiv:2501.17116*, 2025.

[60] Haojun Xia et al.. FP6-LLM: Efficiently Serving Large Language Models Through FP6-Centric Algorithm-System Co-Design. *arXiv:2401.14112*, 2024.

[61] Paulius Micikevicius et al.. FP8 Formats for Deep Learning. *arXiv:2209.05433*, 2022.

[62] Mart van Baalen et al.. FP8 versus INT8 for efficient deep learning inference. *arXiv:2303.17951*, 2023.

[63] Houwen Peng et al.. FP8-LM: Training FP8 Large Language Models. *arXiv:2310.18313*, 2023.

[64] Maxim Fishman et al.. Scaling FP8 training to trillion-token LLMs. *arXiv:2409.12517*, 2024.

[65] Xingwu Sun et al.. Scaling Laws for Floating Point Quantization Training. *arXiv:2501.02423*, 2025.

[66] Hao Kang et al.. GEAR: An Efficient KV Cache Compression Recipe for Near-Lossless Generative Inference of LLM. *arXiv:2403.05527*, 2024.

[67] Gemma Team et al.. Gemma 3 Technical Report. *arXiv:2503.19786*, 2025.

[68] Jianing Deng et al.. GEMQ: Global Expert-Level Mixed-Precision Quantization for MoE LLMs. *arXiv:2605.23078*, 2026.

[69] Amir Gholami et al.. A Survey of Quantization Methods for Efficient Neural Network Inference. *arXiv:2103.13630*, 2021.

[70] GLM-4. 5 Team et al.. GLM-4.5: Agentic, Reasoning, and Coding (ARC) Foundation Models. *arXiv:2508.06471*, 2025.

[71] Yuhang Li et al.. GPTAQ: Efficient Finetuning-Free Quantization for Asymmetric Calibration. *arXiv:2504.02692*, 2025.

[72] OpenAI et al.. gpt-oss-120b & gpt-oss-20b Model Card. *arXiv:2508.10925*, 2025.

[73] Elias Frantar et al.. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. *arXiv:2210.17323*, 2022.

[74] Mart van Baalen et al.. GPTVQ: The Blessing of Dimensionality for LLM Quantization. *arXiv:2402.15319*, 2024.

[75] Kuan Wang et al.. HAQ: Hardware-Aware Automated Quantization with Mixed Precision. *arXiv:1811.08886*, 2018.

[76] Zhen Dong et al.. HAWQ-V2: Hessian Aware trace-Weighted Quantization of Neural Networks. *arXiv:1911.03852*, 2019.

[77] Zhewei Yao et al.. HAWQV3: Dyadic Neural Network Quantization. *arXiv:2011.10680*, 2020.

[78] Zhen Dong et al.. HAWQ: Hessian AWare Quantization of Neural Networks with Mixed-Precision. *arXiv:1905.03696*, 2019.

[79] Vladimir Malinovskii et al.. Pushing the Limits of Large Language Model Quantization via the Linearity Theorem. *arXiv:2411.17525*, 2024.

[80] Ruikang Liu et al.. IntactKV: Improving Large Language Model Quantization by Keeping Pivot Tokens Intact. *arXiv:2403.01241*, 2024.

[81] Mengzhao Chen et al.. INT v.s. FP: A Comprehensive Study of Fine-Grained Low-bit Quantization Formats. *arXiv:2510.25602*, 2025.

[82] Benoit Jacob et al.. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. *arXiv:1712.05877*, 2017.

[83] Tim Dettmers and Luke Zettlemoyer. The case for 4-bit precision: k-bit Inference Scaling Laws. *arXiv:2212.09720*, 2022.

[84] Kimi Team et al.. Kimi K2: Open Agentic Intelligence. *arXiv:2507.20534*, 2025.

[85] Zirui Liu et al.. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. *arXiv:2402.02750*, 2024.

[86] Raghuraman Krishnamoorthi. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv:1806.08342*, 2018.

[87] Coleman Hooper et al.. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. *arXiv:2401.18079*, 2024.

[88] Xing Li et al.. KVTuner: Sensitivity-Aware Layer-Wise Mixed-Precision KV Cache Quantization for Efficient and Nearly Lossless LLM Inference. *arXiv:2502.04420*, 2025.

[89] Haoyu Wang et al.. LC-QAT: Data-Efficient 2-Bit QAT for LLMs via Linear-Constrained Vector Quantization. *arXiv:2606.10531*, 2026.

[90] Wei Huang et al.. An empirical study of LLaMA3 quantization: from LLMs to MLLMs. *arXiv:2404.14047*, 2024.

[91] Aaron Grattafiori et al.. The Llama 3 Herd of Models. *arXiv:2407.21783*, 2024.

[92] Shih-yang Liu et al.. LLM-FP4: 4-Bit Floating-Point Quantized Transformers. *arXiv:2310.16836*, 2023.

[93] Tim Dettmers et al.. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. *arXiv:2208.07339*, 2022.

[94] Zechun Liu et al.. LLM-QAT: Data-Free Quantization Aware Training for Large Language Models. *arXiv:2305.17888*, 2023.

[95] Yixiao Li et al.. LoftQ: LoRA-Fine-Tuning-Aware Quantization for Large Language Models. *arXiv:2310.08659*, 2023.

[96] Anmol Mekala et al.. Does quantization affect models' performance on long-context tasks?. *arXiv:2505.20276*, 2025.

[97] Steven K. Esser et al.. Learned Step Size Quantization. *arXiv:1902.08153*, 2019.

[98] Gunho Park et al.. LUT-GEMM: Quantized Matrix Multiplication based on LUTs for Efficient Inference in Large-Scale Generative Language Models. *arXiv:2206.09557*, 2022.

[99] Aozhong Zhang et al.. MagR: Weight Magnitude Reduction for Enhancing Post-Training Quantization. *arXiv:2406.00800*, 2024.

[100] Elias Frantar et al.. MARLIN: Mixed-Precision Auto-Regressive Parallel Inference on Large Language Models. *arXiv:2408.11743*, 2024.

[101] Mingjie Sun et al.. Massive Activations in Large Language Models. *arXiv:2402.17762*, 2024.

[102] Pranav Nair et al.. Matryoshka Quantization. *arXiv:2502.06786*, 2025.

[103] Zhixiong Zhao et al.. MoBiE: Efficient Inference of Mixture of Binary Experts under Post-Training Quantization. *arXiv:2604.06798*, 2026.

[104] Mohammed Nowaz Rabbani Chowdhury et al.. Efficient Quantization of Mixture-of-Experts with Theoretical Generalization Guarantees. *arXiv:2604.06515*, 2026.

[105] Xing Hu et al.. MoEQuant: Enhancing Quantization for Mixture-of-Experts Large Language Models via Expert-Balanced Sampling and Affinity Guidance. *arXiv:2505.03804*, 2025.

[106] Pingzhi Li et al.. QuantMoE-Bench: Examining Post-Training Quantization for Mixture-of-Experts. *arXiv:2406.08155*, 2024.

[107] Young Jin Kim, Raffy Fahim and Hany Hassan Awadalla. Mixture of Quantized Experts (MoQE): Complementary Effect of Low-bit Quantization and Robustness. *arXiv:2310.02410*, 2023.

[108] Paulius Micikevicius et al.. Mixed Precision Training. *arXiv:1710.03740*, 2017.

[109] Kelly Marchisio et al.. How Does Quantization Affect Multilingual LLMs?. *arXiv:2407.03211*, 2024.

[110] Vage Egiazarian et al.. Bridging the Gap Between Promise and Performance for Microscaling FP4 Quantization. *arXiv:2509.23202*, 2025.

[111] Albert Tseng, Tao Yu and Youngsuk Park. Training LLMs with MXFP4. *arXiv:2502.20586*, 2025.

[112] Haojie Duanmu et al.. MxMoE: Mixed-precision Quantization for MoE with Accuracy and Performance Co-Design. *arXiv:2505.05799*, 2025.

[113] Bita Rouhani et al.. With Shared Microexponents, A Little Shifting Goes a Long Way. *arXiv:2302.08007*, 2023.

[114] Bita Darvish Rouhani et al.. Microscaling Data Formats for Deep Learning. *arXiv:2310.10537*, 2023.

[115] Semyon Savkin et al.. NestQuant: Nested Lattice Quantization for Matrix Products and LLMs. *arXiv:2502.09720*, 2025.

[116] NVIDIA et al.. Pretraining Large Language Models with NVFP4. *arXiv:2509.25149*, 2025.

[117] Elias Frantar, Sidak Pal Singh and Dan Alistarh. Optimal Brain Compression: A Framework for Accurate Post-Training Quantization and Pruning. *arXiv:2208.11580*, 2022.

[118] Wenqi Shao et al.. OmniQuant: Omnidirectionally Calibrated Quantization for Large Language Models. *arXiv:2308.13137*, 2023.

[119] Yuzhuang Xu et al.. OneBit: Towards Extremely Low-bit Large Language Models. *arXiv:2402.11295*, 2024.

[120] Tim Dettmers et al.. 8-bit Optimizers via Block-wise Quantization. *arXiv:2110.02861*, 2021.

[121] Or Ordentlich and Yury Polyanskiy. Optimal Quantization for Matrix Multiplication. *arXiv:2410.13780*, 2024.

[122] Zhongzhu Zhou et al.. OSCAR: Offline Spectral Covariance-Aware Rotation for 2-bit KV Cache Quantization. *arXiv:2605.17757*, 2026.

[123] Xing Hu et al.. OstQuant: Refining Large Language Model Quantization with Orthogonal and Scaling Transformations for Better Distribution Fitting. *arXiv:2501.13987*, 2025.

[124] Xiuying Wei et al.. Outlier Suppression: Pushing the Limit of Low-bit Transformer Language Models. *arXiv:2209.13325*, 2022.

[125] Vincent-Daniel Yun et al.. Output-Aware Rotation for INT2 KV-Cache Quantization. *arXiv:2608.02691*, 2026.

[126] Changhun Lee et al.. OWQ: Outlier-Aware Weight Quantization for Efficient Fine-Tuning and Inference of Large Language Models. *arXiv:2306.02272*, 2023.

[127] Jungwook Choi et al.. PACT: Parameterized Clipping Activation for Quantized Neural Networks. *arXiv:1805.06085*, 2018.

[128] Woosuk Kwon et al.. Efficient Memory Management for Large Language Model Serving with PagedAttention. *arXiv:2309.06180*, 2023.

[129] Zechun Liu et al.. ParetoQ: Improving Scaling Laws in Extremely Low-bit LLM Quantization. *arXiv:2502.02631*, 2025.

[130] Yuzhang Shang et al.. PB-LLM: Partially Binarized Large Language Models. *arXiv:2310.00034*, 2023.

[131] Mengzhao Chen et al.. PrefixQuant: Eliminating Outliers by Prefixed Tokens for Large Language Models Quantization. *arXiv:2410.05265*, 2024.

[132] Vladimir Malinovskii et al.. PV-Tuning: Beyond Straight-Through Estimation for Extreme LLM Compression. *arXiv:2405.14852*, 2024.

[133] Meng Xin et al.. Quantization-Aware Distillation for NVFP4 Inference Accuracy Recovery. *arXiv:2601.20088*, 2026.

[134] Yuhui Xu et al.. QA-LoRA: Quantization-Aware Low-Rank Adaptation of Large Language Models. *arXiv:2309.14717*, 2023.

[135] Shichen Dong et al.. QAQ: Quality Adaptive Quantization for LLM KV Cache. *arXiv:2403.04643*, 2024.

[136] Mengzhao Chen et al.. Scaling Law for Quantization-Aware Training. *arXiv:2505.14302*, 2025.

[137] Amir Zandieh, Majid Daliri and Insu Han. QJL: 1-Bit Quantized JL Transform for KV Cache Quantization with Zero Overhead. *arXiv:2406.03482*, 2024.

[138] Tim Dettmers et al.. QLoRA: Efficient Finetuning of Quantized LLMs. *arXiv:2305.14314*, 2023.

[139] Elias Frantar and Dan Alistarh. QMoE: Practical Sub-1-Bit Compression of Trillion-Parameter Models. *arXiv:2310.16795*, 2023.

[140] Ying Zhang et al.. QQQ: Quality Quattuor-Bit Quantization for Large Language Models. *arXiv:2406.09904*, 2024.

[141] Ruikang Liu et al.. Quantization Hurts Reasoning? An Empirical Study on Quantized Reasoning Models. *arXiv:2504.04823*, 2025.

[142] Shihao Zhang et al.. Qronos: Correcting the Past by Shaping the Future... in Post-Training Quantization. *arXiv:2505.11695*, 2025.

[143] Yujun Lin et al.. QServe: W4A8KV4 Quantization and System Co-design for Efficient LLM Serving. *arXiv:2405.04532*, 2024.

[144] Albert Tseng et al.. QTIP: Quantization with Trellises and Incoherence Processing. *arXiv:2406.11235*, 2024.

[145] Yelysei Bondarenko, Markus Nagel and Tijmen Blankevoort. Quantizable Transformers: Removing Outliers by Helping Attention Heads Do Nothing. *arXiv:2306.12929*, 2023.

[146] Saleh Ashkboos et al.. QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs. *arXiv:2404.00456*, 2024.

[147] Andrei Panferov et al.. Quartet II: Accurate LLM Pre-Training in NVFP4 by Improved Unbiased Gradient Estimation. *arXiv:2601.22813*, 2026.

[148] Roberto L. Castro et al.. Quartet: Native FP4 Training Can Be Optimal for Large Language Models. *arXiv:2505.14669*, 2025.

[149] Albert Tseng et al.. QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks. *arXiv:2402.04396*, 2024.

[150] Jerry Chee et al.. QuIP: 2-Bit Quantization of Large Language Models With Guarantees. *arXiv:2307.13304*, 2023.

[151] Xingyu Zheng et al.. An Empirical Study of Qwen3 Quantization. *arXiv:2505.02214*, 2025.

[152] An Yang et al.. Qwen3 Technical Report. *arXiv:2505.09388*, 2025.

[153] DeepSeek-AI et al.. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv:2501.12948*, 2025.

[154] Wenhao Li et al.. RaBitQCache: Rotated Binary Quantization for KVCache in Long Context LLM Inference. *arXiv:2606.31519*, 2026.

[155] Weihao Gao et al.. Rate Distortion For Model Compression: From Theory To Practice. *arXiv:1810.06401*, 2018.

[156] Zhihang Yuan et al.. LLM Inference Unveiled: Survey and Roofline Model Insights. *arXiv:2402.16363*, 2024.

[157] Zunhai Su et al.. RotateKV: Accurate and Robust 2-Bit KV Cache Quantization for LLMs via Outlier-Aware Adaptive Rotations. *arXiv:2501.16383*, 2025.

[158] Hancheol Park et al.. Value-and-Structure Alignment for Routing-Consistent Quantization of Mixture-of-Experts Models. *arXiv:2606.05688*, 2026.

[159] Zhihang Yuan et al.. RPTQ: Reorder-based Post-training Quantization for Large Language Models. *arXiv:2304.01089*, 2023.

[160] Jintao Zhang et al.. SageAttention3: Microscaling FP4 Attention for Inference and An Exploration of 8-Bit Training. *arXiv:2505.11594*, 2025.

[161] Jintao Zhang et al.. SageAttention: Accurate 8-Bit Attention for Plug-and-play Inference Acceleration. *arXiv:2410.02367*, 2024.

[162] Tanishq Kumar et al.. Scaling Laws for Precision. *arXiv:2411.04330*, 2024.

[163] Lianmin Zheng et al.. SGLang: Efficient Execution of Structured Language Model Programs. *arXiv:2312.07104*, 2023.

[164] Wenhua Cheng et al.. SignRoundV2: Toward Closing the Performance Gap in Extremely Low-Bit Post-Training Quantization for LLMs. *arXiv:2512.04746*, 2025.

[165] Guangxuan Xiao et al.. Efficient Streaming Language Models with Attention Sinks. *arXiv:2309.17453*, 2023.

[166] Haojie Duanmu et al.. SKVQ: Sliding-window Key and Value Cache Quantization for Large Language Models. *arXiv:2405.06219*, 2024.

[167] Wei Huang et al.. SliM-LLM: Salience-Driven Mixed-Precision Quantization for Large Language Models. *arXiv:2405.14917*, 2024.

[168] Jiayi Pan et al.. SmoothQuant+: Accurate and Efficient 4-bit Post-Training WeightQuantization for LLM. *arXiv:2312.03788*, 2023.

[169] Guangxuan Xiao et al.. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. *arXiv:2211.10438*, 2022.

[170] Chengzhu Bao et al.. SOAR: Scale Optimization for Accurate Reconstruction in NVFP4 Quantization. *arXiv:2605.12245*, 2026.

[171] Elias Frantar and Dan Alistarh. SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot. *arXiv:2301.00774*, 2023.

[172] Ayush Kaushal et al.. Spectra: Surprising Effectiveness of Pretraining Ternary Language Models at Scale. *arXiv:2407.12327*, 2024.

[173] Zechun Liu et al.. SpinQuant: LLM quantization with learned rotations. *arXiv:2405.16406*, 2024.

[174] Tim Dettmers et al.. SpQR: A Sparse-Quantized Representation for Near-Lossless LLM Weight Compression. *arXiv:2306.03078*, 2023.

[175] Sehoon Kim et al.. SqueezeLLM: Dense-and-Sparse Quantization. *arXiv:2306.07629*, 2023.

[176] Muyang Li et al.. SVDQuant: Absorbing Outliers by Low-Rank Components for 4-Bit Diffusion Models. *arXiv:2411.05007*, 2024.

[177] Jianyu Wei et al.. T-MAC: CPU Renaissance via Table Lookup for Low-Bit LLM Deployment on Edge. *arXiv:2407.00088*, 2024.

[178] Amir Zandieh et al.. TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate. *arXiv:2504.19874*, 2025.

[179] Xu Ouyang et al.. Low-Bit Quantization Favors Undertrained LLMs: Scaling Laws for Quantized LLMs with 100T Training Tokens. *arXiv:2411.17691*, 2024.

[180] Charlie Blake, Douglas Orr and Carlo Luschi. Unit Scaling: Out-of-the-Box Low-Precision Training. *arXiv:2303.11257*, 2023.

[181] Yifei Liu et al.. VPTQ: Extreme Low-bit Vector Post-Training Quantization for Large Language Models. *arXiv:2409.17066*, 2024.

[182] Markus Nagel et al.. A White Paper on Neural Network Quantization. *arXiv:2106.08295*, 2021.

[183] Sidak Pal Singh and Dan Alistarh. WoodFisher: Efficient Second-Order Approximation for Neural Network Compression. *arXiv:2004.14340*, 2020.

[184] Mohammad Rastegari et al.. XNOR-Net: ImageNet Classification Using Binary Convolutional Neural Networks. *arXiv:1603.05279*, 2016.

[185] Penghang Yin et al.. Understanding Straight-Through Estimator in Training Activation Quantized Neural Nets. *arXiv:1903.05662*, 2019.

[186] Zhewei Yao et al.. ZeroQuant: Efficient and Affordable Post-Training Quantization for Large-Scale Transformers. *arXiv:2206.01861*, 2022.

[187] Xiaoxia Wu, Zhewei Yao and Yuxiong He. ZeroQuant-FP: A Leap Forward in LLMs Post-Training W4A8 Quantization Using Floating-Point Formats. *arXiv:2307.09782*, 2023.

[188] Zhewei Yao et al.. ZeroQuant-V2: Exploring Post-training Quantization in LLMs from Comprehensive Study to Low Rank Compensation. *arXiv:2303.08302*, 2023.

[189] C. E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27:379–423, 623–656, 1948.

[190] C. E. Shannon. Coding theorems for a discrete source with a fidelity criterion. *IRE National Convention Record*, 4:142–163, 1959.

[191] G. Gerganov and contributors. llama.cpp and the GGML/GGUF tensor library. Software repository, 2023–2026. https://github.com/ggml-org/llama.cpp.

[192] Google Developers Blog. Gemma 3 QAT models: bringing state-of-the-art AI to consumer GPUs. April 2025.

[193] HPCwire. NVIDIA says Rubin will deliver 5x AI inference boost over Blackwell. News article, 6 January 2026.

[194] I. Kawrakow. k-quants: block-wise quantization with super-block scales for llama.cpp. llama.cpp pull request #1684, June 2023.

[195] J. H. Conway and N. J. A. Sloane. *Sphere Packings, Lattices and Groups*. Springer, 3rd edition, 1999.

[196] J. Max. Quantizing for minimum distortion. *IRE Transactions on Information Theory*, 6(1):7–12, 1960.

[197] LMSYS Org. Fine-tune and deploy gpt-oss MXFP4: ModelOpt + SGLang. Blog post, 28 August 2025.

[198] M. Horowitz. Computing's energy problem (and what we can do about it). In *IEEE International Solid-State Circuits Conference (ISSCC)*, pages 10–14, 2014.

[199] M. W. Marcellin and T. R. Fischer. Trellis coded quantization of memoryless and Gauss–Markov sources. *IEEE Transactions on Communications*, 38(1):82–93, 1990.

[200] Meta AI. Introducing quantized Llama models with increased speed and a reduced memory footprint (Llama 3.2 1B/3B with SpinQuant and QLoRA-based QAT). Blog post, October 2024.

[201] Moonshot AI. Kimi K2 Thinking: model card (native INT4 quantization-aware training). Hugging Face, November 2025. https://huggingface.co/moonshotai/Kimi-K2-Thinking.

[202] N. Ailon and B. Chazelle. The fast Johnson–Lindenstrauss transform and approximate nearest neighbors. *SIAM Journal on Computing*, 39(1):302–322, 2009.

[203] NVIDIA Technical Blog. Introducing NVFP4 for efficient and accurate low-precision inference. 2025.

[204] NVIDIA. TensorRT-LLM and TensorRT Model Optimizer: documentation on FP8, INT4 AWQ and NVFP4 quantization. Software documentation, 2023–2026.

[205] NYU Shanghai RITS. DeepSeek releases V4: open-source 1.6T MoE with 1M context (secondary report of the 24 April 2026 release; FP4-trained experts, FP8 elsewhere). Web article, 2026. https://rits.shanghai.nyu.edu/ai/deepseek-releases-v4-open-source-1-6t-moe-with-1m-context.

[206] Open Compute Project. *OCP Microscaling Formats (MX) Specification, Version 1.0*. September 2023.

[207] R. M. Gray and D. L. Neuhoff. Quantization. *IEEE Transactions on Information Theory*, 44(6):2325–2383, 1998.

[208] R. Vershynin. *High-Dimensional Probability: An Introduction with Applications in Data Science*. Cambridge University Press, 2018.

[209] R. Zamir and M. Feder. On lattice quantization noise. *IEEE Transactions on Information Theory*, 42(4):1152–1159, 1996.

[210] S. P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.

[211] S. Williams, A. Waterman, and D. Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[212] T. Dettmers and contributors. bitsandbytes: 8-bit and 4-bit quantization library for PyTorch. Software repository, 2022–2026.

[213] T. M. Cover and J. A. Thomas. *Elements of Information Theory*. Wiley, 2nd edition, 2006.

[214] vLLM project. vLLM: documentation on quantization support (GPTQ, AWQ, FP8, compressed-tensors, Marlin kernels). Software documentation, 2023–2026. `https://docs.vllm.ai`.

[215] Y. LeCun, J. S. Denker, and S. A. Solla. Optimal brain damage. In *Advances in Neural Information Processing Systems 2*, 1990.